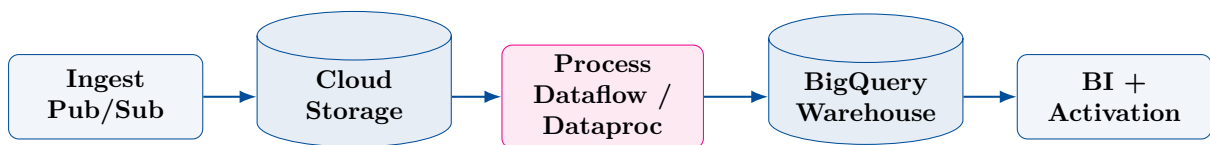

A COMPREHENSIVE, HANDS-ON ACADEMIC GUIDE

GCP Master Data Engineer



Inside this book



From durable object storage to governed, production-grade analytical platforms on Google Cloud.

PhD Aned Esquerra Arguelles

A 24-week training guide for cloud data engineering practice

August 31, 2026

Preface

Welcome to *GCP Master Data Engineer*. This book is a rigorous, hands-on academic guide for learning how modern data platforms are designed, implemented, secured, and operated on Google Cloud. It treats Google Cloud not as a menu of isolated products, but as an integrated data engineering system: storage, ingestion, processing, orchestration, warehousing, machine learning, governance, reliability, and cost control.

The training program behind this text is organized as twenty-four weeks of progressive practice. Each week introduces a cloud capability, a design lens, and a concrete implementation habit. The goal is not only to pass a certification exam or memorize product names; the goal is to reason like a professional data engineer who can defend architecture decisions under operational, financial, and security constraints.

Who This Book Is For

This book is written for data engineers, analytics engineers, cloud engineers, software engineers, and technical leads who want to build production-grade data systems on Google Cloud. It is also useful for candidates preparing for the Professional Data Engineer exam and for architects who need a structured map from storage fundamentals to governed analytical platforms. Readers should be comfortable with SQL, Python, command-line tools, and basic distributed-systems vocabulary. The early chapters review key cloud concepts, but the exercises assume that you will use the Google Cloud CLI and read official documentation as part of the workflow.

How This Book Is Organized

The book is divided into six parts, aligned to six training milestones and twenty-four weeks:

- **Part I: Core Data Infrastructure and Storage** establishes Cloud Storage, transactional sources, operational stores, identity, networking, and the durable landing zones that every data platform needs.
- **Part II: Data Warehousing and Analytics (BigQuery)** develops SQL, partitioning, clustering, dimensional modeling, performance tuning, and analytical cost governance.
- **Part III: Batch Processing and ETL Orchestration** covers Dataflow, Dataproc, Apache Beam, Spark migration, Cloud Composer, Dataform, and operational pipeline design.
- **Part IV: Streaming and Real-Time Analytics** focuses on Pub/Sub, event design, streaming windows, exactly-once trade-offs, real-time warehousing, observability, and resilience.
- **Part V: Machine Learning and MLOps** connects data engineering to Vertex AI, feature engineering, model pipelines, monitoring, and generative AI-assisted data work.

- **Part VI: Security, Governance, and Exam Prep** integrates IAM, CMEK, VPC Service Controls, Dataplex, lineage, data quality, FinOps, reliability reviews, and certification-style architecture decisions.

Appendices are intended to provide command references, exam checklists, solution sketches, and supplemental patterns as the course expands.

Reading Paths

- **New to Google Cloud data engineering:** read each week in order and complete the hands-on project before moving on.
- **Experienced warehouse practitioner:** skim Part I, then spend extra time in BigQuery performance, Dataflow, Composer, and governance chapters.
- **Platform or cloud engineer:** focus on storage architecture, IAM, CMEK, networking boundaries, lifecycle policy, operational reliability, and cost controls.
- **Certification candidate:** read the key takeaways, architecture trade-off tables, interview questions, and exercises, then return to the official product documentation cited in each chapter.

Conventions

Code identifiers, bucket names, CLI flags, and file names appear in **monospace**. Listings use the shared **pystyle** configuration from the preamble, with explicit languages on individual examples. Callout boxes flag **objectives**, **key terms**, **definitions**, **notes**, **tips**, and **pitfalls**. Citations point to primary papers and official Google Cloud documentation. Commands are written for PowerShell-friendly workflows where practical, but many Google Cloud CLI examples are portable shell commands that can be adapted to Cloud Shell.

Contents

0	Getting Started: Your Google Cloud Free Trial and Environment	1
0.1	Why Chapter 0 Matters	1
0.2	Creating a Google Cloud Account	2
0.2.1	Prerequisites	2
0.2.2	Sign-Up Walkthrough	2
0.2.3	Resource Hierarchy: Organization, Folders, Projects, Resources	3
0.3	Securing Identity Before Provisioning	4
0.3.1	Cloud IAM Role Types	4
0.3.2	Why Not Grant Owner Broadly	4
0.3.3	Service Accounts and Keys	4
0.3.4	Enable 2-Step Verification	5
0.4	Tooling for Windows and PowerShell	5
0.4.1	Install the Google Cloud CLI on Windows	5
0.4.2	Initialize Authentication and Defaults	5
0.4.3	Use <code>gcloud storage</code> and <code>gsutil</code>	6
0.4.4	Cloud Shell: No Local Install	6
0.5	Cost Guardrails Before Labs	6
0.5.1	Billing Account and Trial Protection	6
0.5.2	Create a Budget with Alert Thresholds	7
0.5.3	Billing Export and Quotas	8
0.6	Regions, Zones, Naming, and Labels	8
0.6.1	Regions and Zones	8
0.6.2	Naming Conventions Used by the Book	8
0.7	Hands-On Verification	9
0.7.1	Confirm Authentication and Configuration	9
0.7.2	Create, List, and Delete a Bucket	10
0.8	Cross-Cloud Setup Equivalence	10
0.9	Setup Checklist for the Rest of the Book	11
0.10	Further Reading	11
0.11	Exercises	12
0.12	Curated Community Resources	13
I	Core Data Infrastructure and Storage	16
1	Cloud Storage and Transfer Services	17
1.1	Week 1 Roadmap	18
1.2	Monday: Object Storage Fundamentals	19
1.2.1	Objects, Buckets, and Namespaces	19
1.2.2	Consistency, Durability, and Availability	19

1.2.3	Storage Classes	20
1.2.4	Location Types	20
1.2.5	Bucket Architecture for Data Lakes	21
1.3	Tuesday: Lifecycle, Versioning, Retention, and Soft Delete	21
1.3.1	Object Lifecycle Management	21
1.3.2	Object Versioning	22
1.3.3	Retention Policies and Bucket Lock	22
1.3.4	Soft Delete	22
1.4	Wednesday: Data Migration and Transfer Choices	23
1.4.1	Storage Transfer Service	23
1.4.2	Transfer Appliance	23
1.4.3	CLI Tools: <code>gcloud storage</code> and <code>gsutil</code>	23
1.4.4	Migration Decision Matrix	24
1.5	Thursday Hands-On Project: Bucket, Lifecycle, Transfer, and Client Code	24
1.5.1	PowerShell Preparation	24
1.5.2	Lifecycle JSON	25
1.5.3	Storage Transfer Job from Public S3	26
1.5.4	Python Client Snippet	26
1.5.5	Validation Checklist	27
1.6	Friday Use Case and Architecture: Multi-Region DR for Media Streaming	27
1.6.1	Scenario	27
1.6.2	Design	27
1.6.3	Cost and Operations	28
1.7	Governance and Security Design Notes	28
1.8	Operational Runbook for Week 1 Buckets	28
1.9	Interview Q&A	29
1.9.1	Concepts and Trade-offs	29
1.10	Further Reading	30
1.11	Exercises	30
1.12	Capstone Project: A Resilient Global Media Asset Store on Cloud Storage	31
2	Relational Databases: Cloud SQL and AlloyDB	37
2.1	Week 2 Roadmap	38
2.2	Monday: Cloud SQL Architecture, High Availability, and Read Replicas	38
2.2.1	Managed Architecture	38
2.2.2	Regional High Availability	39
2.2.3	Backups and Point-in-Time Recovery	39
2.2.4	Read Replicas	40
2.3	Tuesday: Database Migration Service and Private IP Connections	40
2.3.1	Migration Phases	40
2.3.2	Private IP and Private Services Access	41
2.3.3	DMS Design Checklist	41
2.4	Wednesday: AlloyDB for PostgreSQL, Columnar Engine, and ML Integration	41
2.4.1	Why AlloyDB Exists	41
2.4.2	Columnar Engine	42
2.4.3	ML Integration	42
2.5	Thursday Hands-On Project: HA Cloud SQL PostgreSQL, Read Replica, and Failover	43
2.5.1	PowerShell Preparation and HA Instance Creation	43
2.5.2	Read Replica and Failover Drill	44
2.5.3	Python Connectivity Check	44

2.5.4	Cleanup	45
2.6	Friday Use Case and Architecture: Zero-Downtime ERP Migration	45
2.6.1	Scenario	45
2.6.2	Migration Architecture	45
2.6.3	Cutover and Validation	45
2.7	Security, Operations, and Cost Notes	46
2.8	Interview Q&A	46
2.8.1	Concepts and Trade-offs	46
2.9	Further Reading	46
2.10	Exercises	47
2.11	Capstone Project: HA Cloud SQL and AlloyDB Analytics for ERP Migration . .	47
3	Global Relational Scaling: Cloud Spanner	53
3.1	Week 3 Roadmap	54
3.2	Monday: Spanner Concepts and Strong Consistency	55
3.2.1	Instances, Nodes, Processing Units, and Databases	55
3.2.2	Replicas, Leaders, and Paxos Quorums	55
3.2.3	TrueTime and External Consistency	56
3.3	Tuesday: Schema Design, Primary Keys, Interleaving, and Indexing	56
3.3.1	Primary Keys Define Physical Locality	56
3.3.2	Interleaved Tables	57
3.3.3	Secondary Indexes and Covering Reads	57
3.4	Wednesday: Performance Tuning and Plans	58
3.4.1	Reading Query Plans	58
3.4.2	Hot Spots and Split Health	58
3.4.3	Transactions, Mutations, and Latency	58
3.5	Thursday Hands-On Project: Multi-Region Cart Lab	59
3.5.1	PowerShell Preparation and Instance Creation	59
3.5.2	Interleaved Cart Schema	59
3.5.3	Python Synthetic Load	60
3.5.4	Query Plan Check and Cleanup	61
3.6	Friday Use Case: Global Payment Ledger	61
3.6.1	Scenario	61
3.6.2	Architecture Pattern	62
3.6.3	Correctness Controls	62
3.7	Security, Operations, and Cost Notes	62
3.8	Interview Q&A	63
3.8.1	Concepts and Trade-offs	63
3.9	Further Reading	63
3.10	Exercises	64
3.11	Capstone Project: Globally Consistent Payment Ledger on Cloud Spanner	64
4	Wide-Column NoSQL: Cloud Bigtable	70
4.1	Week 4 Roadmap	71
4.2	Monday: Bigtable Architecture, Tablets, Nodes, and App Profiles	72
4.2.1	Instances, Clusters, Nodes, Tables, and Column Families	72
4.2.2	Tablets and Node-Level Sharding	72
4.2.3	Replication and App Profiles	73
4.3	Tuesday: Row-Key Design, Salting, Hashing, and Hotspot Avoidance	73
4.3.1	Design from Access Patterns	73
4.3.2	Salting, Hashing, Bucketing, and Reverse Time	74
4.3.3	Column Families and Cell Versions	74

4.4	Wednesday: Performance Optimization, Garbage Collection, and Key Visualizer	74
4.4.1	Throughput, Nodes, and Workload Isolation	74
4.4.2	Garbage Collection Policies	75
4.4.3	Key Visualizer and Heatmap Evidence	75
4.5	Thursday Hands-On Project: IoT Temperature Sensor Ingestion	76
4.5.1	PowerShell Preparation and Cluster Creation	76
4.5.2	Python Ingestion Script	76
4.5.3	Read Check and Cleanup	77
4.6	Friday Use Case: 100,000 Stock Ticks per Second	78
4.6.1	Scenario	78
4.6.2	Architecture Pattern	78
4.6.3	Serving and Correctness Controls	78
4.7	Security, Operations, and Cost Notes	78
4.8	Interview Q&A	79
4.8.1	Concepts and Trade-offs	79
4.9	Further Reading	79
4.10	Exercises	80
4.11	Milestone 1 Capstone: E-Commerce Multi-Tier Data Backend	80
II	Data Warehousing and Analytics (BigQuery)	86
5	BigQuery Foundations and Core SQL	87
5.1	Week 5 Roadmap	88
5.2	Monday: Decoupled Architecture—Colossus, Jupiter, and Dremel	88
5.2.1	Separation of Storage and Compute	88
5.2.2	Dremel Execution and the Shuffle Tier	89
5.3	Tuesday: Pricing Models, Slots, and Editions	90
5.3.1	On-Demand versus Capacity Pricing	90
5.3.2	Reservations and Workload Isolation	90
5.4	Wednesday: Advanced SQL and Dimensional Modeling	91
5.4.1	Window Functions, Arrays, and Structs	91
5.4.2	Kimball Dimensional Modeling	91
5.4.3	Slowly Changing Dimensions	91
5.5	Thursday Hands-On Project: GitHub Public Dataset Analytics	92
5.5.1	Explore the Dataset	92
5.5.2	Unnest and Aggregate	92
5.5.3	Rolling Average with a Window Function	92
5.6	Friday Use Case and Architecture: Costing a 50 TB/Day Marketing Workload	93
5.6.1	Scenario	93
5.6.2	On-Demand Estimate	93
5.6.3	Edition Recommendation	94
5.7	Interview Q&A	94
5.7.1	Concepts and Trade-offs	94
5.8	Further Reading	94
5.9	Exercises	95
6	BigQuery Performance: Partitioning, Clustering, and Cost Optimization	96
6.1	Week 6 Roadmap	97
6.2	Monday: Partitioning Strategies	97
6.2.1	The Three Partitioning Models	97
6.2.2	Partition Design Rules	97

6.3	Tuesday: Clustering for Selective Filters	98
6.3.1	How Clustering Works	98
6.3.2	Choosing Clustering Columns	98
6.4	Wednesday: Materialized Views, BI Engine, and Plan Analysis	99
6.4.1	Materialized Views	99
6.4.2	BI Engine	99
6.4.3	Reading the Execution Plan	99
6.5	Thursday Hands-On Project: Measuring Scan Strategies	99
6.5.1	Build the Synthetic Table	99
6.5.2	Compare Scan Strategies	100
6.6	Friday Use Case: Re-Architecting a Multi-Petabyte Security Log Table	101
6.6.1	Scenario	101
6.6.2	Diagnosis	101
6.6.3	Re-Architecture	101
6.7	Interview Q&A	102
6.7.1	Concepts and Trade-offs	102
6.8	Further Reading	102
6.9	Exercises	103
7	Advanced Data Types and Federated Analytics	104
7.1	Week 7 Roadmap	105
7.2	Monday: BigQuery GIS and Search	105
7.2.1	The GEOGRAPHY Type	105
7.2.2	Search Indexes	105
7.3	Tuesday: External Tables over Cloud Storage	106
7.3.1	Creating and Querying External Tables	106
7.3.2	What Federation Gives Up	106
7.4	Wednesday: BigQuery Omni and Cross-Cloud Analytics	107
7.4.1	Architecture	107
7.4.2	When Omni Wins	107
7.5	Thursday Hands-On Project: External CSVs Joined with Native GIS Data	107
7.5.1	Stage the Files	107
7.5.2	Create the External Table and Join	108
7.5.3	Compare with a Native Load	108
7.6	Friday Use Case and Architecture: Retail History in S3, Analytics in GCP	109
7.6.1	Scenario	109
7.6.2	Options Analysis	109
7.7	Interview Q&A	110
7.7.1	Concepts and Trade-offs	110
7.8	Further Reading	110
7.9	Exercises	111
8	Data Lakehouse, Analytics Hub, and Cost Governance	112
8.1	Week 8 Roadmap	113
8.2	Monday: BigLake—Unifying Lake and Warehouse	113
8.2.1	Architecture	113
8.2.2	Fine-Grained Security on the Lake	113
8.3	Tuesday: Object Tables and Unstructured Data	114
8.3.1	Indexing Files as Rows	114
8.4	Wednesday: Analytics Hub and Data Clean Rooms	114
8.4.1	Sharing Without Copying	114
8.4.2	Data Clean Rooms	115

8.5	Thursday Hands-On Project: BigLake Security and an Analytics Hub Listing . .	115
8.5.1	Create the BigLake Table	115
8.5.2	Apply Row and Column Security	115
8.5.3	Publish via Analytics Hub	116
8.6	Friday Use Case and Architecture: An Advertising Data Clean Room	116
8.6.1	Scenario	116
8.6.2	Architecture	117
8.7	Interview Q&A	117
8.7.1	Concepts and Trade-offs	117
8.8	Further Reading	118
8.9	Exercises	118
8.10	Milestone 2 Capstone: Enterprise Data Lakehouse Build	119
 III Batch Processing and ETL Orchestration		122
9	Managed Hadoop and Spark with Cloud Dataproc	123
9.1	Week 9 Roadmap	124
9.2	Monday: Dataproc Architecture, Ephemeral Clusters, and Preemptible VMs . .	124
9.2.1	Cluster Anatomy	124
9.2.2	The Ephemeral Discipline	124
9.2.3	Cost Anatomy	125
9.3	Tuesday: Migrating from HDFS to Cloud Storage	125
9.3.1	Why the Move Matters	125
9.3.2	Migration Mechanics	125
9.4	Wednesday: Dataproc Serverless and Spark Tuning	125
9.4.1	Serverless Spark	125
9.4.2	The Tuning Shortlist	126
9.5	Thursday Hands-On Project: Ephemeral PySpark Sales Aggregation	126
9.5.1	Stage Data and Job Code	126
9.5.2	Create, Run, Self-Delete	127
9.6	Friday Use Case and Architecture: Migrating a 100-Node Cloudera Estate . . .	128
9.6.1	Scenario	128
9.6.2	Migration Waves	128
9.6.3	Cost and Risk	128
9.7	Interview Q&A	129
9.7.1	Concepts and Trade-offs	129
9.8	Further Reading	129
9.9	Exercises	130
10	Visual ETL with Cloud Data Fusion	131
10.1	Week 10 Roadmap	132
10.2	Monday: CDAP Concepts, Architecture, and Private IPs	132
10.2.1	The CDAP Foundation	132
10.2.2	Instances and Editions	132
10.2.3	Private Instances	132
10.3	Tuesday: Wrangler—Interactive Data Cleaning	133
10.3.1	The Wrangler Workflow	133
10.3.2	Directives in Practice	133
10.4	Wednesday: Plugins, Macros, and Scheduling	133
10.4.1	The Plugin Ecosystem	133
10.4.2	Macros and Parameterization	133

10.4.3	Scheduling and Triggers	134
10.5	Thursday Hands-On Project: No-Code Customer Cleansing to BigQuery	134
10.5.1	Stage the Messy Data	134
10.5.2	Build the Pipeline	134
10.6	Friday Use Case and Architecture: Data Fusion versus Dataflow for Analyst Teams	135
10.6.1	Scenario	135
10.6.2	Decision Framework	135
10.7	Interview Q&A	136
10.7.1	Concepts and Trade-offs	136
10.8	Further Reading	137
10.9	Exercises	137
11	Change Data Capture with Datastream	138
11.1	Week 11 Roadmap	139
11.2	Monday: CDC Concepts and Datastream Architecture	139
11.2.1	Why the Log, Not the Table	139
11.2.2	Datastream Architecture	139
11.3	Tuesday: Source Configurations	140
11.3.1	PostgreSQL	140
11.3.2	MySQL and Oracle	140
11.4	Wednesday: Destinations and Data Contracts	141
11.4.1	Cloud Storage versus BigQuery	141
11.4.2	Schema Enforcement as Contract	141
11.4.3	Breaking versus Non-Breaking Changes	141
11.5	Thursday Hands-On Project: PostgreSQL CDC to BigQuery with a Contract Test	141
11.5.1	Prepare the Source	141
11.5.2	Create the Stream	142
11.5.3	Test the Contract	142
11.6	Friday Use Case and Architecture: Zero-Downtime Oracle to BigQuery	143
11.6.1	Scenario	143
11.6.2	Architecture	143
11.7	Interview Q&A	144
11.7.1	Concepts and Trade-offs	144
11.8	Further Reading	144
11.9	Exercises	145
12	Pipeline Orchestration with Cloud Composer	146
12.1	Week 12 Roadmap	147
12.2	Monday: Airflow Concepts—DAGs, Operators, Tasks, XComs	147
12.2.1	The DAG Mental Model	147
12.2.2	Sensors and Deferrable Execution	147
12.2.3	XComs: Small Messages Only	147
12.3	Tuesday: Cloud Composer 2 Architecture	148
12.3.1	Under the Hood	148
12.3.2	Operational Surface	148
12.4	Wednesday: Cross-Service Orchestration	148
12.4.1	Google Cloud Operators	148
12.4.2	Design Rules for Orchestrated Pipelines	148
12.5	Thursday Hands-On Project: The File-Triggered ELT DAG	149
12.5.1	Prepare the Environment	149
12.5.2	Deploy and Test	150
12.6	Friday Use Case and Architecture: Fifteen-System Banking Reconciliation	150

12.6.1	Scenario	150
12.6.2	Design	151
12.7	Interview Q&A	151
12.7.1	Concepts and Trade-offs	151
12.8	Further Reading	152
12.9	Exercises	152
12.10	Milestone 3 Capstone: Automated Batch ELT Pipeline	152
IV	Streaming and Real-Time Analytics	155
13	Asynchronous Messaging with Pub/Sub	156
13.1	Week 13 Roadmap	157
13.2	Monday: Topics, Subscriptions, Acks, and Schema Enforcement	157
13.2.1	The Delivery Model	157
13.2.2	Acknowledgement Semantics	157
13.2.3	Schema Validation at the Edge	157
13.3	Tuesday: Dead Letters, Retries, Snapshots, and Seek	158
13.3.1	Dead-Letter Topics	158
13.3.2	Retry Policies and Backoff	158
13.3.3	Snapshots and Seek	158
13.4	Wednesday: Pub/Sub Lite versus Pub/Sub Standard	159
13.5	Thursday Hands-On Project: Financial Transactions with a Dead-Letter Net	159
13.5.1	Provision	159
13.5.2	Publish with Backoff	159
13.5.3	Consume, Fail, and Inspect the DLQ	160
13.6	Friday Use Case and Architecture: Telemetry for Five Million Mobile Games	160
13.6.1	Scenario	160
13.6.2	Decomposing Exactly-Once	161
13.6.3	Operational Reality	161
13.7	Interview Q&A	161
13.7.1	Concepts and Trade-offs	161
13.8	Further Reading	162
13.9	Exercises	162
14	Dataflow and Apache Beam Fundamentals	163
14.1	Week 14 Roadmap	164
14.2	Monday: The Beam Model—PCollections, PTransforms, ParDo, Runners	164
14.2.1	Pipelines as Graphs	164
14.2.2	ParDo and DoFn	164
14.2.3	Runners and Portability	165
14.3	Tuesday: Dataflow Architecture—Autoscaling, Shuffle, Streaming Engine	165
14.3.1	The Managed Service	165
14.3.2	Shuffle and Streaming Engine	165
14.4	Wednesday: Templates—Classic versus Flex	166
14.5	Thursday Hands-On Project: WordCount on the Dataflow Runner	166
14.5.1	Local Run and Unit Test	166
14.5.2	Submit to Dataflow	166
14.6	Friday Use Case and Architecture: Real-Time HL7 Anonymization	167
14.6.1	Scenario	167
14.6.2	Architecture	168
14.7	Interview Q&A	168

14.7.1 Concepts and Trade-offs	168
14.8 Further Reading	169
14.9 Exercises	169
15 Advanced Dataflow: Windowing, State, and Exactly-Once Semantics	170
15.1 Week 15 Roadmap	171
15.2 Monday: Event Time, Processing Time, and Watermarks	171
15.2.1 Two Clocks	171
15.2.2 Watermark Mechanics	171
15.3 Tuesday: Windowing Strategies—Fixed, Sliding, Session	172
15.3.1 Fixed and Sliding	172
15.3.2 Session Windows	172
15.4 Wednesday: Late Data, Triggers, and the State and Timers API	172
15.4.1 Triggers and Accumulation	172
15.4.2 Per-Key State and Timers	173
15.5 Thursday Hands-On Project: Sessionized Clicks with Late Data	173
15.5.1 The Streaming Pipeline	173
15.5.2 Session Variant and Unit Test	174
15.6 Friday: Stream-Stream Joins, Exactly-Once, and the Late-Mobile Dashboard	175
15.6.1 Joining Streams over Windows	175
15.6.2 End-to-End Exactly-Once	175
15.6.3 Use Case: The Dashboard That Lies	175
15.7 Interview Q&A	176
15.7.1 Concepts and Trade-offs	176
15.8 Further Reading	176
15.9 Exercises	177
16 Business Intelligence, Looker, and Real-Time Serving	178
16.1 Week 16 Roadmap	179
16.2 Monday: Looker Architecture and the Semantic Layer	179
16.2.1 How Looker Differs	179
16.2.2 Why a Semantic Layer	179
16.3 Tuesday: Dimensions, Measures, Explores, and Views	180
16.3.1 Modeling Vocabulary	180
16.4 Wednesday: Looker Studio versus Looker, and Reverse ETL	180
16.4.1 Self-Serve versus Governed	180
16.4.2 Reverse ETL	180
16.5 Thursday Hands-On Project: Streaming Dashboard and Reverse-ETL Sync	181
16.5.1 The Live Dashboard	181
16.5.2 The Reverse-ETL Sync	181
16.6 Friday Use Case and Architecture: Governed BI for Five Hundred Analysts	182
16.6.1 Scenario	182
16.6.2 Design	182
16.7 Interview Q&A	183
16.7.1 Concepts and Trade-offs	183
16.8 Further Reading	183
16.9 Exercises	184
16.10 Milestone 4 Capstone: Real-Time Fraud Detection	184

V	Machine Learning and MLOps	187
17	BigQuery ML: Machine Learning in SQL	188
17.1	Week 17 Roadmap	189
17.2	Monday: The BQML Catalog—Regression, Classification, Clustering	189
17.2.1	Model Types	189
17.2.2	Why Train Where Data Lives	190
17.3	Tuesday: Time-Series Forecasting with ARIMA_PLUS	190
17.3.1	Managed Forecasting	190
17.3.2	Anomaly Detection from Residuals	190
17.4	Wednesday: Evaluation and Hyperparameter Tuning	191
17.4.1	Reading ML.EVALUATE	191
17.4.2	Tuning and Overfitting	191
17.5	Thursday Hands-On Project: Churn Prediction on Google Analytics Data	191
17.5.1	Train	191
17.5.2	Evaluate and Predict	192
17.6	Friday Use Case and Architecture: SQL-Only Forecasting for 1,000 Stores	193
17.6.1	Scenario	193
17.6.2	Architecture	193
17.7	Interview Q&A	193
17.7.1	Concepts and Trade-offs	193
17.8	Further Reading	194
17.9	Exercises	194
18	Vertex AI Foundations: Training, Registry, and Serving	195
18.1	Week 18 Roadmap	196
18.2	Monday: The Ecosystem and AutoML versus Custom Training	196
18.2.1	The Platform Map	196
18.2.2	AutoML or Custom	196
18.3	Tuesday: Model Registry and Endpoint Deployment	197
18.3.1	Registry as the System of Record	197
18.3.2	Online versus Batch Serving	197
18.4	Wednesday: Explainable AI and Feature Attributions	197
18.5	Thursday Hands-On Project: AutoML Tabular from BigQuery to a Live Endpoint	198
18.5.1	Train via the Console or API	198
18.5.2	Deploy and Predict	198
18.6	Friday Use Case and Architecture: Secure GPU Training over Governed Data	199
18.6.1	Scenario	199
18.6.2	Architecture	199
18.7	Interview Q&A	200
18.7.1	Concepts and Trade-offs	200
18.8	Further Reading	201
18.9	Exercises	201
19	MLOps and Vertex AI Pipelines	202
19.1	Week 19 Roadmap	203
19.2	Monday: Vertex AI Feature Store—Online and Offline Serving	203
19.2.1	One Definition, Two Paths	203
19.2.2	Point-in-Time Correctness	203
19.3	Tuesday: Kubeflow Pipelines and Vertex AI Pipelines	204
19.3.1	Components and Composition	204
19.4	Wednesday: Continuous Training and Model Monitoring	204

19.4.1	Gates, Not Just Triggers	204
19.4.2	Skew versus Drift	205
19.5	Thursday Hands-On Project: Build and Run an Extract-Train-Deploy Pipeline	205
19.5.1	Compile and Submit	205
19.5.2	Inspect the Run	205
19.6	Friday Use Case and Architecture: Self-Retraining Recommendation Engine	206
19.6.1	Scenario	206
19.6.2	Architecture	206
19.7	Interview Q&A	207
19.7.1	Concepts and Trade-offs	207
19.8	Further Reading	208
19.9	Exercises	208
20	Unstructured Data and AI APIs	209
20.1	Week 20 Roadmap	210
20.2	Monday: Vision and Video Intelligence	210
20.2.1	Vision API	210
20.2.2	Video Intelligence	210
20.3	Tuesday: Natural Language and Translation APIs	211
20.3.1	Natural Language API	211
20.3.2	Translation	211
20.4	Wednesday: AI APIs Inside Dataflow Pipelines	211
20.4.1	The Integration Pattern	211
20.4.2	Choosing API versus BQML versus Custom	212
20.5	Thursday Hands-On Project: Streaming Review Sentiment to BigQuery	212
20.5.1	Provision and Publish	212
20.5.2	Run the Pipeline	213
20.6	Friday Use Case and Architecture: Terabytes of User Images, Searchable	213
20.6.1	Scenario	213
20.6.2	Architecture	214
20.7	Interview Q&A	214
20.7.1	Concepts and Trade-offs	214
20.8	Further Reading	215
20.9	Exercises	215
20.10	Milestone 5 Capstone: End-to-End MLOps Pipeline	216
VI	Security, Governance, and Exam Prep	218
21	Security, IAM, and Networking for Data Platforms	219
21.1	Week 21 Roadmap	220
21.2	Monday: VPC Service Controls and Private Connectivity	220
21.2.1	Perimeter Semantics	220
21.2.2	Perimeters and Data Engineering	220
21.2.3	Private Connectivity	220
21.3	Tuesday: IAM Deep Dive—Custom Roles, Service Accounts, Workload Identity	221
21.3.1	Roles and the Least-Privilege Discipline	221
21.3.2	Service Accounts as Workload Identity	221
21.4	Wednesday: Cloud KMS—CMEK versus CSEK in BigQuery and Cloud Storage	222
21.4.1	The Encryption Ladder	222
21.4.2	CMEK in Practice	222
21.5	Thursday Hands-On Project: Perimeter Around a BigQuery Dataset	222

21.5.1	Create the Perimeter	222
21.5.2	Negative Test and Enforcement	223
21.6	Friday Use Case and Architecture: The Healthcare Anti-Exfiltration Perimeter	223
21.6.1	Scenario	223
21.6.2	Layered Design	224
21.7	Interview Q&A	224
21.7.1	Concepts and Trade-offs	224
21.8	Further Reading	225
21.9	Exercises	225
22	Data Governance and Data Mesh with Dataplex	226
22.1	Week 22 Roadmap	227
22.2	Monday: Lakes, Zones, and Assets	227
22.2.1	The Organizational Vocabulary	227
22.2.2	Design Consequences	227
22.3	Tuesday: Data Catalog, Tag Templates, and Lineage	228
22.3.1	Searchable, Tagged Metadata	228
22.3.2	Policy Tags versus Descriptive Tags	228
22.3.3	Lineage	228
22.4	Wednesday: Auto Data Quality and Profiling	228
22.4.1	Profiling	228
22.4.2	AutoDQ	228
22.5	Thursday Hands-On Project: A Governed Lake with a Protected Salary Column	229
22.5.1	Create Lake, Zones, and Assets	229
22.5.2	Tag Template and Policy Tag	230
22.6	Friday Use Case and Architecture: A Five-Domain Data Mesh	230
22.6.1	Scenario	230
22.6.2	Mesh on GCP	230
22.7	Interview Q&A	231
22.7.1	Concepts and Trade-offs	231
22.8	Further Reading	232
22.9	Exercises	232
23	Compliance and Data Loss Prevention	233
23.1	Week 23 Roadmap	234
23.2	Monday: DLP Concepts—InfoTypes, Inspection, De-identification	234
23.2.1	Detection	234
23.2.2	Discovery and Inspection Modes	234
23.3	Tuesday: Masking, Tokenization, and Crypto-Based Hashing	235
23.3.1	The Transformation Menu	235
23.3.2	Who Can Reverse It	235
23.4	Wednesday: Automated Scanning and GDPR Erasure Propagation	235
23.4.1	Scheduled Discovery	235
23.4.2	Why DELETE Is Not Erasure	235
23.4.3	Crypto-Shredding as the Architecture	236
23.5	Thursday Hands-On Project: Inspect and De-identify Sensitive Files	236
23.5.1	Stage Sensitive Samples and Create a Key	236
23.5.2	Inspect and De-identify	237
23.6	Friday Use Case and Architecture: Third-Party PII Ingestion with an Erasure Map	238
23.6.1	Scenario	238
23.6.2	Architecture	238

23.7 Interview Q&A	239
23.7.1 Concepts and Trade-offs	239
23.8 Further Reading	239
23.9 Exercises	240
24 Production DataOps and the Professional Data Engineer Exam	241
24.1 Week 24 Roadmap	242
24.2 Monday: Infrastructure as Code with Terraform	242
24.2.1 The google Provider	242
24.2.2 State, Backends, and Workspaces	243
24.3 Tuesday: Dataform—SQL Pipelines as Code	243
24.3.1 SQLX Models	243
24.3.2 Assertions as Quality Gates	244
24.4 Wednesday: CI/CD and Zero-Downtime Schema Evolution	244
24.4.1 The Pipeline	244
24.4.2 Expand-Contract Migrations	244
24.5 Thursday Hands-On Project: Terraform + Dataform + CI with an Expand-Contract Drill	244
24.5.1 Repository Skeleton	245
24.5.2 The Expand-Contract Drill	245
24.6 Friday: Professional Data Engineer Exam Preparation	246
24.6.1 Exam Domains Mapped to This Book	246
24.6.2 The Case Studies	246
24.6.3 Readiness Method	246
24.7 Interview Q&A	247
24.7.1 Concepts and Trade-offs	247
24.8 Further Reading	247
24.9 Exercises	248
24.10 Milestone 6 Capstone: Secure, Governed Data Mesh	248
A Cross-Cloud Service Equivalence	251
A.1 Object and Data-Lake Storage	251
A.2 Managed Relational Databases	252
A.3 Elastic Compute and Analytical Warehouses	252
A.4 Data Organization and the Medallion Pattern	252
A.5 Document Data Modeling	253
A.6 Globally Distributed and NoSQL Databases	253
A.7 Managed Apache Spark	254
A.8 Query Acceleration and Materialized Views	254
A.9 In-Memory Caching and Hybrid Ingestion	254
A.10 Wide-Column and Key-Value NoSQL	255
A.11 Data Protection: Cloning, Time Travel, and Migration	255
A.12 Serverless Analytical Warehouses	255
A.13 Query Optimization and Physical Layout	256
A.14 Federated and Cross-Cloud Analytics	256
A.15 Lakehouse Governance, Sharing, and Clean Rooms	256
A.16 Managed Spark and Hadoop Modernization	257
A.17 Visual, Low-Code ETL	257
A.18 Change Data Capture	258
A.19 Workflow Orchestration	258
A.20 Messaging and Event Ingestion	259
A.21 Stream Processing Engines and Semantics	259

A.22 Governed BI and Semantic Layers	260
A.23 In-Warehouse Machine Learning and Feature Stores	260
A.24 ML Platforms, Pipelines, and Serving	261
A.25 Pretrained AI APIs and Generative AI	261
A.26 Security: Perimeters, Keys, and Identity	262
A.27 Governance Fabrics, Lineage, and Data Quality	262
A.28 Sensitive-Data Discovery and De-identification	263
A.29 Infrastructure as Code, SQL Pipelines, and CI/CD	263
A.30 Databricks Equivalents at a Glance	264
B Environment Setup Reference	266
B.1 Project and Billing Foundation	266
B.2 The gcloud CLI	266
B.3 Client Libraries and Local Development	267
B.4 Terraform	267
B.5 Cost Guardrails	268
B.6 Per-Chapter Tool Requirements	269
C CLI and SQL Quick Reference	270
C.1 Core gcloud	270
C.2 Cloud Storage (gcloud storage / gsutil)	270
C.3 BigQuery (bq and SQL)	271
C.4 Dataflow and Apache Beam	271
C.5 Dataproc	272
C.6 Cloud Composer (Airflow)	272
C.7 Pub/Sub	272
C.8 Datastream	273
C.9 Cloud Data Fusion	273
C.10 Vertex AI	273
C.11 Dataplex	274
C.12 Sensitive Data Protection (DLP)	274
D Production-Readiness and Governance Checklists	275
D.1 Storage and Data Lake (Chapters 1, 8)	275
D.2 Warehousing and Analytics (Chapters 5–8)	275
D.3 Batch ETL and Orchestration (Chapters 9–12, 24)	276
D.4 Streaming and Real-Time (Chapters 13–16)	276
D.5 Machine Learning and MLOps (Chapters 17–20)	277
D.6 Security, Governance, and Compliance (Chapters 21–23)	277
D.7 Reliability and FinOps (Chapter 23 and throughout)	278
E Lab Solutions and Solution Sketches	279
E.1 Chapter 0: Environment Verification	279
E.2 Chapter 1: Bucket, Lifecycle, Transfer, and Client Code	279
E.3 Chapter 2: HA Cloud SQL PostgreSQL, Read Replica, and Failover	279
E.4 Chapter 3: Multi-Region Cart Lab	280
E.5 Chapter 4: IoT Temperature Sensor Ingestion	280
E.6 Chapter 5: GitHub Public Dataset Analytics	280
E.7 Chapter 6: Measuring Scan Strategies	281
E.8 Chapter 7: External CSVs Joined with Native GIS Data	281
E.9 Chapter 8: BigLake Security and an Analytics Hub Listing	281
E.10 Chapter 9: Ephemeral PySpark Sales Aggregation	281

E.11 Chapter 10: No-Code Customer Cleansing to BigQuery	282
E.12 Chapter 11: PostgreSQL CDC to BigQuery with a Contract Test	282
E.13 Chapter 12: The File-Triggered ELT DAG	282
E.14 Chapter 13: Financial Transactions with a Dead-Letter Net	283
E.15 Chapter 14: WordCount on the Dataflow Runner	283
E.16 Chapter 15: Sessionized Clicks with Late Data	283
E.17 Chapter 16: Streaming Dashboard and Reverse-ETL Sync	283
E.18 Chapter 17: Churn Prediction on Google Analytics Data	284
E.19 Chapter 18: AutoML Tabular from BigQuery to a Live Endpoint	284
E.20 Chapter 19: Extract–Train–Deploy Pipeline	284
E.21 Chapter 20: Streaming Review Sentiment to BigQuery	284
E.22 Chapter 21: Perimeter Around a BigQuery Dataset	285
E.23 Chapter 22: A Governed Lake with a Protected Salary Column	285
E.24 Chapter 23: Inspect and De-identify Sensitive Files	285
E.25 Chapter 24: Terraform + Dataform + CI with an Expand-Contract Drill	286
E.26 Selected Exercises	286
F Professional Data Engineer Exam Mapping	287
F.1 Exam Domains Mapped to Chapters	287
F.2 The Case Studies	288
F.3 Study Strategy	288
F.4 Self-Assessment Checklist	288
G Interview Question Bank	290
G.1 Storage and Databases (Chapters 1–4)	290
G.2 Warehousing and Analytics (Chapters 5–8, 16)	291
G.3 Batch Processing and Orchestration (Chapters 9–12)	292
G.4 Streaming and Real-Time (Chapters 13–15)	294
G.5 Machine Learning and MLOps (Chapters 17–20)	294
G.6 Security, Governance, and Compliance (Chapters 21–23)	296
G.7 DataOps and Career (Chapter 24)	296
G.8 Study Method for the Bank	297
Glossary	298

List of Figures

1	Google Cloud’s hierarchy places hands-on resources inside Projects, which may sit under Folders and an Organization.	3
1.1	A storage-class lifecycle for a lakehouse-style Cloud Storage landing zone.	21
1.2	A multi-region disaster recovery architecture for ten petabytes of media assets.	27
1.3	Capstone reference architecture for a resilient global media asset store on Cloud Storage.	32
2.1	Cloud SQL regional HA places a standby in another zone and requires client reconnection after failover.	39
2.2	AlloyDB can serve transactions from PostgreSQL-compatible row storage while accelerating selected analytical scans with a columnar engine.	42
2.3	Zero-downtime ERP migration using DMS, private connectivity, Cloud SQL HA, and AlloyDB analytics offload.	45
2.4	Capstone reference architecture for HA Cloud SQL with AlloyDB analytics offload during ERP migration.	49
3.1	Spanner combines Paxos replication with TrueTime commit timestamps to provide globally ordered transactions.	56
3.2	Interleaving stores child rows near parent rows while secondary indexes support alternate access paths.	57
3.3	A multi-continent payment ledger uses Spanner for globally consistent OLTP and streams committed facts to analytics systems.	62
3.4	Capstone reference architecture for a globally consistent payment ledger on multi-region Spanner.	65
4.1	Bigtable splits a table into tablets, then serves those row-key ranges across cluster nodes.	73
4.2	A timestamp-leading key hot spots current writes, while a salted reversed-timestamp key spreads ingestion and preserves recent-event scans.	74
4.3	Trading ingestion uses Pub/Sub and Dataflow for burst absorption, Bigtable for low-latency serving, and analytical stores for archive and surveillance.	78
4.4	Milestone 1 reference architecture integrating Cloud Storage, Cloud SQL, Spanner, and Bigtable behind private application connectivity.	82
5.1	BigQuery’s decoupled stack: stateless Dremel compute reads columnar data from Colossus over Jupiter.	89
6.1	Re-architecting the security log table: partition, cluster, pre-aggregate, and expire.	101
7.1	Hybrid multi-cloud retail analytics: Omni for cold federated reads, native load for the hot window.	109

8.1	Data clean room: both parties contribute governed tables; only thresholded aggregates leave.	117
8.2	Milestone 2 reference architecture: a governed lakehouse from Cloud Storage to shared analytics.	120
9.1	Cloudera-to-GCP migration waves: assess, move data, move compute, retire what BigQuery absorbs.	128
11.1	Datastream reads the source transaction log once and projects it to BigQuery or Cloud Storage.	140
12.1	Banking reconciliation: sensed, validated inputs; idempotent reconciliation; signed output and alerting.	151
12.2	Milestone 3 reference architecture: scheduled batch ELT with CDC verification gates.	153
13.1	Global game telemetry: edge schema enforcement, exactly-once shuffle, idempotent sinks.	161
14.1	HL7 anonymization: parse and tokenize in flight; analysts see only the pseudonymous projection.	168
16.1	Governed BI: certified explores for shared truth, self-serve Studio at the edges. .	183
16.2	Milestone 4 reference architecture: real-time fraud detection, serverless end to end.	185
17.1	SQL-only inventory forecasting: CDC replica to nightly per-series forecasts to the ERP.	193
18.1	Secure GPU workflow: governed reads, scoped training identities, gated production promotion.	200
19.1	Closed-loop MLOps: monitoring detects drift, a preconditioned trigger retrains, gates protect production.	207
20.1	Image enrichment at scale: Object Tables enumerate, Dataflow batches API calls, BigQuery serves search.	214
20.2	Milestone 5 reference architecture: the full MLOps lifecycle orchestrated by Composer.	217
21.1	Healthcare perimeter: valid credentials from an unauthorized network are denied by VPC SC.	224
22.1	Data mesh on Dataplex: domain lakes publish governed products into one catalog under shared standards.	231
23.1	Third-party PII ingestion: quarantine, tokenize for analytics, encrypt per subject for provable erasure.	238
24.1	Milestone 6 reference architecture: three projects, one perimeter, centralized governance.	249

List of Tables

1	Book naming and labeling conventions.	9
1.1	Cloud Storage classes and common data engineering uses.	20
1.2	Storage protection and governance controls.	23
1.3	Choosing a Cloud Storage migration mechanism.	24
1.4	Operational signals for Cloud Storage runbooks.	29
2.1	Cloud SQL reliability and scale patterns.	40
2.2	Database Migration Service readiness checklist.	41
3.1	Spanner schema design trade-offs.	58
4.1	Bigtable performance diagnosis patterns.	75
5.1	BigQuery pricing models and their trade-offs [11].	90
6.1	When clustering pays off beyond partitioning [42].	98
7.1	External versus native BigQuery tables [52].	106
9.1	Common Spark-on-Dataproc symptoms and first responses [68].	126
10.1	Data Fusion versus Dataflow decision matrix.	136
11.1	CDC source prerequisites and classic failures [33].	140
13.1	Pub/Sub Standard versus Lite [51].	159
14.1	Reading Dataflow job metrics during tuning.	165
15.1	Beam windowing strategies and their canonical workloads [65].	172
17.1	Classifier evaluation metrics and their churn-prediction reading.	191
18.1	Online versus batch serving on Vertex AI [57].	197
21.1	Identity anti-patterns and their least-privilege alternatives [38].	221
22.1	AutoDQ rule types and what their failures usually mean.	229
23.1	Erasure propagation across platform layers.	236
24.1	Exam domains mapped to book chapters, with the trap each domain sets.	246

Getting Started: Your Google Cloud Free Trial and Environment

Learning Objectives

- Create a Google Cloud account, start the \$300 / 90-day free trial, and explain how the Always Free tier differs from trial credits.
- Navigate the Organization, Folder, Project, and Resource hierarchy and use Projects as isolation, ownership, and billing boundaries.
- Secure a learner environment with Cloud IAM least privilege, service-account hygiene, and 2-Step Verification.
- Install and initialize the Google Cloud CLI on Windows, or use Cloud Shell when a browser-only environment is preferable.
- Configure cost guardrails with budgets, alert thresholds, labels, quotas, and billing export before creating data resources.
- Apply the book's region, zone, naming, and labeling conventions consistently across every hands-on chapter.
- Verify the setup by authenticating, selecting a project, creating a temporary Cloud Storage bucket, listing it, and deleting it.

Key Terms

Google Cloud account: a billing-enabled cloud identity associated with a Google account and payment verification. **Free trial:** time-limited introductory credit, currently \$300 for 90 days for eligible new customers. **Always Free:** ongoing monthly no-cost usage limits for selected services in eligible regions. **Project:** the main Google Cloud isolation, API, quota, IAM, and billing unit used throughout this book. **Cloud IAM:** Google Cloud's identity and access management system for granting roles on resources. **Service account:** a non-human identity used by applications, pipelines, and managed services. **Budget:** a billing guardrail that sends alerts when actual or forecasted spend crosses thresholds.

0.1 Why Chapter 0 Matters

Chapter 1 begins with Cloud Storage, but successful cloud data engineering starts before the first bucket is created. A learner needs an account, a project, a billing boundary, a secure identity model, a working command line, and cost alerts. These foundations are not administrative trivia. They determine who can create resources, where data can live, how charges are attributed, and how quickly an experiment can be cleaned up.

This chapter is written for a Windows workstation using PowerShell. All commands are safe to run from a normal PowerShell session unless explicitly marked as an administrative install step. Cloud Shell is included as a browser-based alternative when local installation is unavailable. The goal is a small, well-labeled, disposable learning environment that can be used for every hands-on exercise in this book.

Definition

Google Cloud setup is the combination of identity, billing, project selection, regional defaults, CLI authentication, labels, and cost controls required before provisioning technical services. A good setup makes later labs repeatable, auditable, and inexpensive.

0.2 Creating a Google Cloud Account

0.2.1 Prerequisites

To start the labs, you need three things. First, you need a Google account. This can be a personal Google account for individual study or an account supplied by an organization or school. Second, eligible new customers need a card or accepted payment method for identity and payment verification. Third, you need a browser, a phone or authenticator app for stronger sign-in security, and either a Windows PowerShell terminal or access to Cloud Shell.

Google Cloud's introductory offer gives eligible new customers a free trial with \$300 in credits for 90 days. The credits can be used across many Google Cloud services, subject to product limits and trial restrictions. Google also offers an Always Free tier for selected services, which provides limited no-cost usage each month in eligible regions. The trial and Always Free tier are related but not identical: trial credits expire after the trial period, while Always Free limits can continue after the trial if the account remains in good standing and usage stays within eligible limits [37].

Note

Cloud offers change over time. Treat the exact trial amount, duration, eligible countries, and Always Free product limits as values to verify on the official free program page before beginning a cohort or workshop [37].

0.2.2 Sign-Up Walkthrough

Use the Google Cloud Console sign-up flow rather than an unofficial link. The process is normally:

1. Open the Google Cloud free program page and choose the option to start for free.
2. Sign in with the Google account you will use for the book.
3. Select country, account type, and required profile information.
4. Accept the terms after reading the trial and billing conditions.
5. Provide a card or accepted payment method for verification.
6. Create or confirm the first billing account.
7. Land in the Google Cloud Console and open the project selector.
8. Create a dedicated learning project instead of using a default or shared project.

The card protects Google from abuse and verifies payment readiness, but the trial is designed to prevent automatic charges during the free trial unless you explicitly upgrade to a paid account. When credits are exhausted or the trial ends, resources may stop, and you must choose whether to upgrade. You should still configure budgets and alerts immediately, because cost awareness is a professional habit, not only a payment-protection mechanism.

0.2.3 Resource Hierarchy: Organization, Folders, Projects, Resources

Google Cloud organizes resources hierarchically. At the top, an **Organization** represents a company, school, or domain-managed tenant. Under the Organization, **Folders** can group departments, environments, applications, or compliance zones. Under folders, **Projects** host resources. Resources are the services you create: buckets, BigQuery datasets, Pub/Sub topics, service accounts, Dataflow jobs, Dataproc clusters, and more.

Personal learners often start without an Organization resource. In that case, projects sit directly under the account's project list. Enterprise learners usually work inside an Organization with folder-level policies, shared networking, centralized billing, and security controls. The same project-centered mental model still applies.

Definition

A **Project** is the primary boundary for APIs, quotas, IAM grants, billing attribution, audit logs, and many operational policies. In this book, every hands-on resource belongs to one learner project unless a chapter explicitly says otherwise.

Projects are the unit of isolation because they collect related resources and make deletion practical. Deleting a lab project is safer than manually deleting dozens of scattered resources. Projects are also the billing attribution boundary. A billing account can fund many projects, but charges are reported by project, service, SKU, label, and time. Finally, projects are an IAM boundary. You can grant a user permission on one project without granting access to unrelated projects.

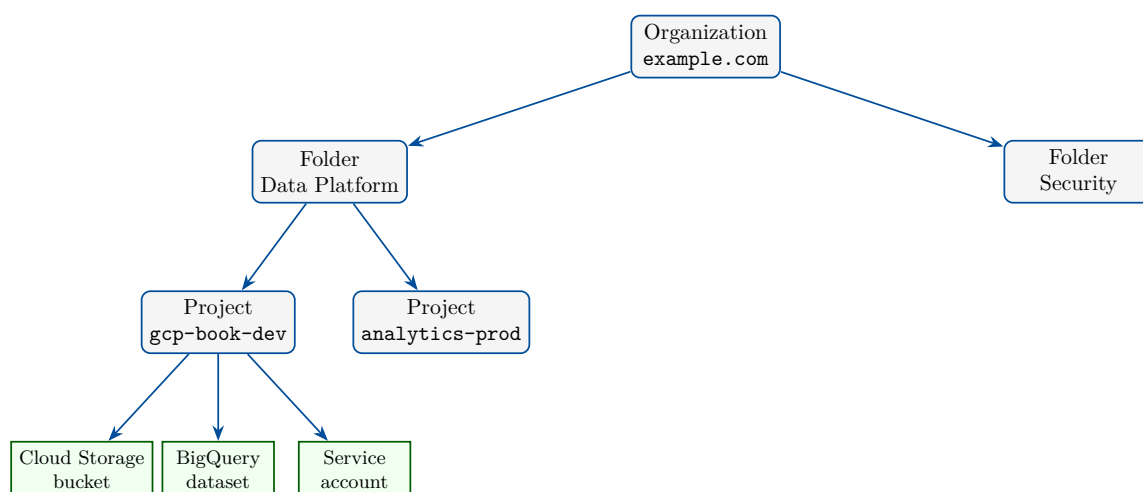


Figure 1: Google Cloud's hierarchy places hands-on resources inside Projects, which may sit under Folders and an Organization.

0.3 Securing Identity Before Provisioning

0.3.1 Cloud IAM Role Types

Cloud IAM grants permissions by binding a principal to a role on a resource. A principal can be a user, group, service account, domain, or workload identity. A role is a collection of permissions. A binding says, in effect, “this principal receives this role on this resource.” Permissions inherited from an Organization or Folder can flow down to projects, so always inspect the effective access path when troubleshooting.

Cloud IAM roles come in three major types [38]:

- **Basic roles:** Owner, Editor, and Viewer. These are broad legacy roles. They are easy to understand but too coarse for most production use.
- **Predefined roles:** Google-managed roles scoped to services and job functions, such as `roles/storage.admin`, `roles/bigquery.dataViewer`, or `roles/pubsub.publisher`.
- **Custom roles:** organization- or project-level roles containing a specific set of permissions selected by your team.

The principle of least privilege says each identity should receive only the permissions needed, at the narrowest practical scope, for the shortest practical time. In a learner project, you may temporarily act as an administrator because you own the environment. In a professional environment, day-to-day data engineers should not hold broad Owner access. They should use groups, predefined roles, approval workflows, break-glass procedures, and audited elevation.

0.3.2 Why Not Grant Owner Broadly

The Owner role can change IAM policy, enable services, manage resources, and link or modify billing settings. It is powerful enough to create security, cost, and data-loss incidents. Owner should be limited to a small number of accountable administrators and emergency paths. Avoid granting Owner to service accounts, lab partners, automation jobs, or broad groups. If a person only needs to create buckets, grant a storage-specific role. If a pipeline only needs to write objects to one bucket, grant an object-writing role on that bucket rather than Owner on the project.

Common Pitfall

Leaked service-account keys and Owner sprawl are two of the fastest ways to lose control of a cloud project. A JSON key committed to source control can be copied instantly, and an over-broad Owner grant can disable guardrails or create expensive resources. Prefer short-lived credentials, workload identity patterns, service-specific predefined roles, and routine access reviews.

0.3.3 Service Accounts and Keys

Service accounts are identities for workloads, not people. A Dataflow job, VM, Cloud Run service, Composer environment, or local automation script may run as a service account. Treat service accounts as production identities with owners, descriptions, minimal roles, and rotation or replacement plans.

Avoid user-managed service-account keys whenever possible. A downloaded key file is a long-lived bearer credential: whoever has the file can authenticate as the service account until the key is revoked or expires. Prefer managed service identities, Workload Identity Federation, attached service accounts on Google Cloud runtimes, and short-lived credentials. When a key

is unavoidable for a lab or legacy integration, store it outside source control, restrict the service account, set a rotation date, and delete the key as soon as the exercise ends [38].

0.3.4 Enable 2-Step Verification

Before creating resources, enable 2-Step Verification on the Google account used for the book. Use an authenticator app, security key, or another supported second factor. Store backup codes securely. If the account is managed by an organization, follow its identity policy and do not bypass centralized controls. Strong sign-in is especially important because the same account can access billing, IAM, audit logs, and project deletion.

0.4 Tooling for Windows and PowerShell

0.4.1 Install the Google Cloud CLI on Windows

The Google Cloud CLI provides the `gcloud`, `bq`, `gsutil`, and modern `gcloud storage` command groups used throughout this book [36]. On Windows, install it with the official Google Cloud CLI installer, Windows package management approved by your organization, or a managed software center. Use the official documentation rather than copying binaries from a third party.

After installation, open a new PowerShell window so that the updated `PATH` is loaded. Confirm that the command is available:

```
1 Get-Command gcloud
2
3 gcloud --version
```

Listing 1: Verifying the Google Cloud CLI from Windows PowerShell.

Expected output includes the path to `gcloud.cmd` and a version block similar to this:

```
1 Google Cloud SDK 528.0.0
2 bq 2.1.18
3 core 2026.07.15
4 gcloud-crc32c 1.0.0
5 gsutil 5.35
```

Listing 2: Representative Google Cloud CLI version output.

Version numbers will differ. The important result is that `gcloud` runs from PowerShell without requiring a special terminal.

0.4.2 Initialize Authentication and Defaults

Run `gcloud init` to authenticate, select or create a configuration, choose the default project, and optionally set a default Compute Engine region and zone. The command opens a browser login flow. Use the same Google account that owns or has access to the learner project.

```
1 gcloud init
2
3 gcloud auth login
4
5 gcloud config set project PROJECT_ID
6 gcloud config set compute/region us-central1
7 gcloud config set compute/zone us-central1-a
```

Listing 3: Initial Google Cloud CLI setup in PowerShell.

Replace `PROJECT_ID` with your real project ID. The project ID is globally unique, lower case, and different from the display name. A display name can be changed; the project ID is the stable identifier used by APIs, logs, and commands.

Note

`gcloud init` and `gcloud auth login` both authenticate a user account for CLI use. If you already initialized the CLI, `gcloud auth login` can refresh or add an account without redoing all configuration prompts.

0.4.3 Use `gcloud storage` and `gsutil`

Google now emphasizes `gcloud storage` for Cloud Storage command-line operations, while `gsutil` remains common in older scripts, documentation, and installed environments. This book prefers `gcloud storage` for new examples and mentions `gsutil` when it appears in production patterns or migration materials. Learn both names so that existing runbooks are readable.

For example, these commands express similar intent:

```
1 # Preferred for new examples in this book
2 gcloud storage buckets list
3
4 # Legacy command still seen in many environments
5 gsutil ls
```

Listing 4: Modern and legacy Cloud Storage command examples.

0.4.4 Cloud Shell: No Local Install

Cloud Shell is a browser-based shell in the Google Cloud Console with the Google Cloud CLI preinstalled, temporary compute, and persistent home storage within documented limits. It is useful for classrooms, locked-down laptops, and quick console-adjacent troubleshooting. Open it from the Cloud Shell icon in the Google Cloud Console. The shell already knows the console account, but you should still confirm the active project and configuration before running labs.

```
1 gcloud config get-value project
2 gcloud auth list
```

Listing 5: Checking the active project from Cloud Shell.

Cloud Shell is convenient, but local PowerShell remains valuable. Local development lets you use an editor, source control, local sample data, and the same scripts you will later automate. Use Cloud Shell when installation is blocked; use local PowerShell when building repeatable project workflows.

0.5 Cost Guardrails Before Labs

0.5.1 Billing Account and Trial Protection

A project must be linked to a billing account before most billable resources can run. The billing account is the payment and invoicing container. The project is where charges originate. The free trial credits apply to eligible usage while the trial is active. Google states that you are not automatically charged after the free trial ends unless you upgrade to a paid account [37]. This protection is helpful, but it does not replace cost design. If you upgrade, remove forgotten resources, or move to an organizational billing account, spend can become real.

Tip

Measure it. Every lab should have a way to answer three questions: what did we create, who owns it, and how much can it cost if forgotten for a week? Budgets, labels, quotas, and billing export turn those questions into observable signals.

0.5.2 Create a Budget with Alert Thresholds

Create a budget immediately after linking billing. In the Console, open **Billing** → **Budgets & alerts**, choose the billing account, create a budget scoped to the learner project, and set alert thresholds such as 50%, 90%, and 100% of a small monthly amount. For an individual learner, a budget of \$10 or \$25 is often enough to reveal mistakes. For a cohort, set budgets per project or per folder according to the teaching plan.

Budget alerts notify selected users or billing administrators. They do not automatically stop all resources. Think of a budget as an alarm, not a circuit breaker. Pair budgets with quotas, labels, lifecycle rules, and cleanup habits.

The exact CLI support for budgets can vary by installed components and API availability. The following PowerShell example shows a practical pattern: create a project, link billing, enable billing APIs, and create a budget from a JSON file through `gcloud billing budgets create`. If your organization restricts billing APIs, perform the budget step in the Console.

```

1 $ProjectId = "gcp-book-$(New-Guid.Guid.Substring(0,8))"
2 $BillingAccount = "BILLING_ACCOUNT_ID"
3 $Region = "us-central1"
4
5 # Create a dedicated project for the book.
6 gcloud projects create $ProjectId --name="GCP Book Labs"
7
8 # Link the project to a billing account.
9 gcloud billing projects link $ProjectId --billing-account=$BillingAccount
10
11 # Set defaults for later commands.
12 gcloud config set project $ProjectId
13 gcloud config set compute/region $Region
14 gcloud config set compute/zone "$Region-a"
15
16 # Enable the services used by setup and early chapters.
17 gcloud services enable cloudbilling.googleapis.com billingbudgets.googleapis.com storage.googleapis.com
18
19 # Create budget.json in the current directory, then create the budget.
20 @'
21 {
22   "displayName": "gcp-book-lab-budget",
23   "budgetFilter": {
24     "projects": ["projects/PROJECT_NUMBER"]
25   },
26   "amount": {
27     "specifiedAmount": {
28       "currencyCode": "USD",
29       "units": "25"
30     }
31   },
32   "thresholdRules": [
33     {"thresholdPercent": 0.5},
34     {"thresholdPercent": 0.9},
35     {"thresholdPercent": 1.0}
36   ]
37 }
21

```

```

37 }
38 '@ | Set-Content -Path .\budget.json -Encoding utf8
39
40 $ProjectNumber = gcloud projects describe $ProjectId --format="value(
    projectNumber)"
41 (Get-Content .\budget.json) -replace "PROJECT_NUMBER", $ProjectNumber |
42     Set-Content -Path .\budget.json -Encoding utf8
43
44 gcloud billing budgets create --billing-account=$BillingAccount --budget-from-
    file=.\budget.json
45 Remove-Item .\budget.json

```

Listing 6: Creating a learner project and budget guardrail from PowerShell.

0.5.3 Billing Export and Quotas

Billing export sends detailed cost and usage data to BigQuery. Enable it early in serious learning environments so that future chapters can query real spend patterns. In a single-person trial, billing export may feel unnecessary, but it teaches the same FinOps loop used in production: export, label, query, visualize, and review.

Quotas limit how much of a resource class a project can consume. They protect Google Cloud and can protect you from runaway experiments. Review quotas for expensive or scalable services before labs that create compute, streaming, or data processing resources. A quota error during a lab is not always bad; it may be a guardrail doing its job.

0.6 Regions, Zones, Naming, and Labels

0.6.1 Regions and Zones

A Google Cloud region is a specific geographic area, such as `us-central1`. A zone is an isolated deployment area within a region, such as `us-central1-a`. Some resources are zonal, such as many virtual machine placements. Some are regional, such as many managed service locations. Some are multi-region or global. Data engineering resources often have location compatibility requirements. For example, moving data between a Cloud Storage bucket and BigQuery dataset can involve location and egress considerations.

This book uses `us-central1` and `us-central1-a` as defaults unless a chapter explicitly chooses another region to demonstrate disaster recovery, data residency, or multi-region design. If you are outside the United States or under organizational policy, choose an approved region and use it consistently. Do not mix regions casually; inconsistent location choices complicate cost, latency, and governance.

0.6.2 Naming Conventions Used by the Book

Names should be readable, unique, and disposable. Use lower-case kebab-case for projects, buckets, topics, datasets when allowed, and service accounts where the product permits it. Include platform, purpose, environment, and a short random suffix when global uniqueness is required.

Resource	Pattern	Example
Project ID	<code>gcp-book-<name>-<suffix></code>	<code>gcp-book-labs-a1b2c3d4</code>
Bucket	<code><project>-<zone>-<purpose></code>	<code>gcp-book-labs-a1b2c3d4-raw</code>
Service account	<code><workload>-sa</code>	<code>dataflow-lab-sa</code>
BigQuery dataset	<code><domain>_<layer></code>	<code>retail_raw</code>
Pub/Sub topic	<code><domain>-<event>-topic</code>	<code>orders-created-topic</code>
Labels	<code>owner, env, book, chapter</code>	<code>env=lab, chapter=00</code>

Table 1: Book naming and labeling conventions.

Labels are key-value metadata used for cost allocation, inventory, and operations. Apply labels at creation time whenever a service supports them. Recommended labels for this book are `book=gcp-data-engineer`, `env=lab`, `owner=<your-alias>`, and `chapter=00`. Avoid personal data in labels because labels may appear in billing exports, logs, and reports.

Common Pitfall

A globally unique bucket name can accidentally reveal project names, customer names, or internal programs. Use neutral lab names and random suffixes. Never include secrets, email addresses, or regulated data categories in resource names.

0.7 Hands-On Verification

This section confirms that authentication, project selection, APIs, region defaults, and Cloud Storage access work. It creates one temporary bucket, lists it, and deletes it. The commands assume PowerShell and a configured project.

0.7.1 Confirm Authentication and Configuration

Run the following commands:

```
1 gcloud auth list
2 gcloud config list
```

Listing 7: Verifying authentication and active configuration.

Expected output resembles the following. Your account, project, and zone will differ.

```
1 Credentialed Accounts
2 ACTIVE ACCOUNT
3 * learner@example.com
4
5 [compute]
6 region = us-central1
7 zone = us-central1-a
8 [core]
9 account = learner@example.com
10 project = gcp-book-labs-a1b2c3d4
```

Listing 8: Representative authentication and configuration output.

If no active account appears, run `gcloud auth login`. If no project appears, run `gcloud config set project PROJECT_ID`. If the storage API is disabled, enable it with `gcloud services enable storage.googleapis.com`.

0.7.2 Create, List, and Delete a Bucket

Cloud Storage bucket names are globally unique. The following script builds a unique name from your project ID and a timestamp, creates the bucket in the default region, lists it, and then deletes it. The bucket is empty, so deletion should succeed immediately.

```

1 $ProjectId = gcloud config get-value project
2 $Region = gcloud config get-value compute/region
3 $Suffix = Get-Date -Format "yyyyMMddHHmmss"
4 $Bucket = "${ProjectId}-ch00-${Suffix}"
5
6 # Create a temporary bucket with labels.
7 gcloud storage buckets create "gs://${Bucket}" --location=$Region --uniform-
   bucket-level-access --project=$ProjectId
8
9 gcloud storage buckets update "gs://${Bucket}" --update-labels=book=gcp-data-
   engineer,env=lab,chapter=00
10
11 # List the bucket and inspect basic metadata.
12 gcloud storage buckets list --filter="name:$Bucket"
13 gcloud storage buckets describe "gs://${Bucket}" --format="value(name,location,
   iamConfiguration.uniformBucketLevelAccess.enabled)"
14
15 # Clean up immediately.
16 gcloud storage rm --recursive "gs://${Bucket}"

```

Listing 9: Creating, listing, and deleting a temporary verification bucket.

Representative output:

```

1 Creating gs://gcp-book-labs-a1b2c3d4-ch00-20260731010203/...
2 ---
3 creation_time: 2026-07-31T07:02:05+0000
4 name: gcp-book-labs-a1b2c3d4-ch00-20260731010203
5 storage_url: gs://gcp-book-labs-a1b2c3d4-ch00-20260731010203/
6
7 gcp-book-labs-a1b2c3d4-ch00-20260731010203 US-CENTRAL1 True
8 Removing gs://gcp-book-labs-a1b2c3d4-ch00-20260731010203/...

```

Listing 10: Representative output from the bucket verification script.

Note

A successful verification proves only that the basic learner path works. Later chapters may require additional APIs, roles, quotas, or service accounts. Return to this setup checklist whenever a lab fails before debugging the service itself.

0.8 Cross-Cloud Setup Equivalence

Cross-Cloud Equivalence

Setup step	AWS	Azure	GCP
Free offer	Free Tier (12 mo + always free)	Free account (\$200/30 days + always free)	Free trial (\$300/90 days + Always Free)
Sign-up guard	Card + root email	Card + Microsoft account	Card + Google account
Identity model	IAM / IAM Identity Center	Microsoft Entra ID + RBAC	Cloud IAM
Admin isolation	Never use root daily	Avoid Global Admin daily	Avoid Owner; least privilege
CLI	AWS CLI v2 (<code>aws configure</code>)	Azure CLI (<code>az login</code>)	<code>gcloud CLI</code> (<code>gcloud init</code>)
Browser shell	AWS CloudShell	Azure Cloud Shell	Cloud Shell
Cost guardrail	AWS Budgets + alerts	Cost Management budgets	Budgets & alerts
Console	AWS Management Console	Azure Portal	Google Cloud Console

Microsoft Fabric uses a Microsoft 365/Power BI work account plus a Fabric trial capacity (no card, org tenant); Snowflake uses a 30-day/\$400 trial (no card) guarded by Resource Monitors; MongoDB Atlas offers a forever-free M0 cluster (no card) secured by database users and an IP allowlist.

0.9 Setup Checklist for the Rest of the Book

Before starting Chapter 1, confirm the following:

1. The Google account used for labs has 2-Step Verification enabled.
2. The free trial is active, or the project is intentionally attached to an approved paid billing account.
3. A dedicated learner project exists and is selected in `gcloud config list`.
4. The project has a budget with alerts at 50%, 90%, and 100% of the chosen threshold.
5. The default region and zone are set to `us-central1` and `us-central1-a`, or to approved local alternatives.
6. The Cloud Storage API is enabled.
7. `gcloud auth list` shows the intended account as active.
8. The temporary bucket verification succeeded and the bucket was deleted.
9. You know how to open Cloud Shell if local PowerShell stops working.
10. You have recorded the project ID, billing account owner, region, and cleanup plan.

0.10 Further Reading

Read the Google Cloud free program page for the current free trial and Always Free terms [37]. Review the resource hierarchy and project documentation before designing multi-team environments [53]. Study Cloud IAM role types, service accounts, and least-privilege guidance before granting access [38]. Use the Google Cloud CLI documentation for installation, initialization, and command reference details [36]. Review Cloud Billing budgets, alerts, and billing

export before running long-lived labs [15]. The Google Cloud Architecture Framework provides broader operational, security, reliability, and cost review guidance [35].

Key Takeaways

- Google Cloud’s free trial supplies temporary credits, while Always Free provides ongoing limited usage for eligible services.
- Projects are the practical unit of lab isolation, billing attribution, quota management, IAM scope, and cleanup.
- Least privilege, limited Owner grants, secure service accounts, and 2-Step Verification should be in place before data work begins.
- Windows learners can use the Google Cloud CLI from PowerShell; Cloud Shell is the no-install browser alternative.
- Budgets alert you, labels explain ownership, quotas constrain scale, and billing export makes costs queryable.
- Consistent regions, names, and labels prevent avoidable troubleshooting and cost-allocation problems.
- A setup is not complete until authentication, configuration, bucket creation, listing, and deletion have been verified.

0.11 Exercises

1. **(Checklist)** Complete every item in the setup checklist above. Record the project ID, active account, default region, and budget amount in your private lab notes.
2. **(Conceptual)** Explain the difference between a billing account and a project. Why can one billing account fund many projects while each project remains a useful isolation boundary?
3. **(Security)** Identify three reasons the Owner role should not be granted broadly. For each reason, name a safer predefined role or process.
4. **(Service accounts)** Describe why a downloaded JSON service-account key is riskier than an attached service account on a managed Google Cloud runtime.
5. **(PowerShell)** Run `gcloud auth list` and `gcloud config list`. Paste the output into private notes after removing email addresses if your notes may be shared.
6. **(Cost)** Create or verify a project-scoped budget with at least three alert thresholds. Explain why an alert is not the same as automatic shutdown.
7. **(Naming)** Propose names and labels for a raw bucket, a curated bucket, a Pub/Sub topic, and a Dataflow service account for a retail orders lab.
8. **(Hands-on)** Re-run the temporary bucket verification, then prove cleanup by listing buckets with the same prefix and confirming the verification bucket is gone.
9. **(Design)** Your organization requires data to remain in Canada. Rewrite this chapter’s region and naming assumptions for an approved Canadian region and explain which later labs may need location checks.

10. **(Reflection)** Write a five-sentence runbook for what you will do when a budget alert fires during the book.

0.12 Curated Community Resources

Beyond this book and the vendor documentation, the following community-curated resources are worth your time. Do not try to consume everything at once: pick one resource per category, practice it with real data, and build something small before moving on.

Practical Case Studies (Video Walkthroughs)

Fifteen end-to-end builds that show how the tools work together:

1. SQL Data Warehouse from Scratch — SQL, ETL, data modeling, star schemas (<https://lnkd.in/gcrJ2qde>)
2. End-to-End Data Analytics Project — Python, Pandas, SQL Server, data cleaning (<https://lnkd.in/gHxTN-B2>)
3. Netflix Data Engineering Project — Python, SQL, ELT, exploratory analysis (<https://lnkd.in/gMkbpWmh>)
4. DuckDB and Python Data Engineering Project — API ingestion, pipeline development (<https://lnkd.in/gEPnubGA>)
5. Airflow Podcast Data Pipeline — Airflow, APIs, scheduling (<https://lnkd.in/gmy5rS4Z>)
6. Automated Python ETL Pipeline with Airflow — SQL Server, DAG development (<https://lnkd.in/gHrBc8BE>)
7. PySpark and dbt Data Engineering Project — incremental loading, dimensional modeling (<https://lnkd.in/gVZjvyR7>)
8. Apache Spark Data Engineering Project — PySpark, Databricks (<https://lnkd.in/gVNkqk5A>)
9. Airbnb Data Engineering Project — dbt, Snowflake, AWS, testing (<https://lnkd.in/g2f4TxPW>)
10. AWS ETL Pipeline Project — S3, Lambda, Glue, Athena, serverless ETL (<https://lnkd.in/gFC3NkNs>)
11. AWS Data Warehouse Project — PySpark, Glue, Redshift (<https://lnkd.in/gyZtHMEF>)
12. End-to-End Azure Data Engineering Project — Data Factory, Databricks, ADLS (<https://lnkd.in/gUpbRjzE>)
13. Spotify Azure Data Engineering Project — streaming, Delta Lake (<https://lnkd.in/gUGKuZ8N>)
14. Uber Real-Time Data Engineering Project — Event Hubs, structured streaming (<https://lnkd.in/gu7Tpg7D>)
15. End-to-End GCP Data Engineering Project — Cloud Storage, Dataproc, BigQuery (<https://lnkd.in/gv8Yedfu>)

Free Video Courses

Twenty videos covering the essential skill path — recommended learning order: Python, SQL, Linux, Git, Docker, data modeling, data warehousing, Spark, Kafka, Airflow, dbt, cloud, portfolio projects.

- **Foundations:** Data Engineering Course for Beginners (<https://lnkd.in/gprfkqnp>); Big Data Engineer Masterclass (<https://lnkd.in/gqPkrG63>)
- **Core tools:** Python for Data Engineers (<https://lnkd.in/gZeKJEi4>); SQL for Big Data Engineering (<https://lnkd.in/gt5DZXwQ>); PostgreSQL Full Course (<https://lnkd.in/gAU5n7gM>); Linux Command Line (<https://lnkd.in/gA-7zFaQ>); Git and GitHub Tutorial (<https://lnkd.in/gs73kHau>); Docker Tutorial (<https://lnkd.in/g59TCWPD>)
- **Warehousing and modeling:** Data Warehousing Beginner's Guide (https://lnkd.in/gS_-iJAb); Data Modeling, Star Schema and Kimball (<https://lnkd.in/g4it9vpy>)
- **Batch processing:** PySpark Tutorial for Beginners (<https://lnkd.in/gre86dH6>); PySpark Full Course (<https://lnkd.in/gZXg38hk>)
- **Streaming:** Apache Kafka Full Course (https://lnkd.in/gnnXc9_d); Kafka Crash Course with Project (<https://lnkd.in/gP6DFCem>)
- **Pipelines:** Apache Airflow Tutorial (<https://lnkd.in/gbFzzReZ>); dbt Tutorial with Netflix Project (<https://lnkd.in/gHvmdjNX>); ELT with dbt, Snowflake and Airflow (https://lnkd.in/g5ZP_xC3)
- **Cloud:** AWS for Data Engineers (<https://lnkd.in/g8vRZmkp>)
- **End-to-end projects:** Airbnb Project with Snowflake, dbt and AWS (<https://lnkd.in/g2f4TxPw>); Real-Time E-Commerce with Kafka and Snowflake (<https://lnkd.in/g58V6A3P>)

GitHub Repositories

Twenty free repositories to learn from, practice with, and build upon. Choose one roadmap, complete one project, study the code structure, then break it, fix it, and document what you learned.

- **Roadmaps and guides:** Data Engineering Zoomcamp (<https://lnkd.in/gxKmbbHc>); Data Engineer Handbook (<https://lnkd.in/gstev9z7>); Data Engineering Cookbook (<https://lnkd.in/gABzJ-vE>); Complete Data Engineer Roadmap (<https://lnkd.in/gmwQrmvm>)
- **Curated lists:** Awesome Data Engineering (<https://lnkd.in/gh4Ky273>); Awesome Big Data (https://lnkd.in/g_y-n3zm); Awesome Open-Source Data Engineering (<https://lnkd.in/ggDTw66U>)
- **Portfolio and analytics engineering:** Data Engineering Project Template (<https://lnkd.in/g-QKrMXV>); Jaffle Shop dbt Practice Project (<https://lnkd.in/gmXVhYKX>); dbt Core (<https://lnkd.in/gnCXxNbB>)
- **Ingestion and orchestration:** Apache Airflow (<https://lnkd.in/gqvG5F6N>); Airbyte (<https://lnkd.in/g2B9AVwM>); dlt (<https://lnkd.in/g3A8BMp3>)
- **Processing:** Apache Spark (<https://lnkd.in/g5CPE3hh>); Apache Kafka (<https://lnkd.in/gdH5T4xX>)

- **Storage and lakehouses:** DuckDB (<https://lnkd.in/gehQd42W>); Apache Iceberg (<https://lnkd.in/gVweVV8y>)
- **Quality, lineage and governance:** Great Expectations (<https://lnkd.in/g8TPyAy8>); OpenLineage (https://lnkd.in/gz3_MiAP); DataHub (<https://lnkd.in/gEcNMRkJ>)

Part I

Core Data Infrastructure and Storage

Cloud Storage and Transfer Services

Learning Objectives

- Explain the object storage model and why Cloud Storage is the default durable landing zone for many Google Cloud data platforms.
- Compare Standard, Nearline, Coldline, and Archive storage classes using access frequency, recovery expectations, and total cost.
- Choose among regional, dual-region, and multi-region bucket locations for analytics, availability, sovereignty, and disaster recovery requirements.
- Design lifecycle, versioning, soft delete, retention, and Bucket Lock controls that protect data without creating uncontrolled storage growth.
- Evaluate Storage Transfer Service, Transfer Appliance, `gcloud storage`, and `gsutil` for online and offline migrations.
- Implement a hands-on Cloud Storage bucket, lifecycle rule, transfer job, and Python client workflow.
- Design a multi-region disaster recovery architecture for ten petabytes of media assets with encryption, lifecycle policy, and cost governance.

Cross-Cloud Equivalence

Although this book teaches **GCP**, the Week 1 storage foundations map almost one-to-one onto the other clouds—learn one mental model and carry it everywhere.

Capability	AWS	Azure	GCP
Object storage	Amazon S3	ADLS Gen2 / Blob	Cloud Storage
Hot tier	S3 Standard	Hot	Standard
Infrequent access	S3 Standard-IA	Cool / Cold	Nearline
Archive (instant)	Glacier Instant Retrieval	Cold	Coldline
Deep archive	Glacier Deep Archive	Archive	Archive
Lifecycle rules	Lifecycle policies	Lifecycle management	Object Lifecycle Mgmt
Versioning	S3 Versioning	Blob versioning	Object Versioning
Immutability (WORM)	Object Lock	Immutable blob storage	Bucket Lock
Cross-region copy	CRR / SRR	Object replication (GRS)	Dual/multi-region + Transfer
Online transfer	DataSync	AzCopy	Storage Transfer Service
Offline transfer	Snow Family	Data Box	Transfer Appliance
Design durability	11 nines	11 nines (LRS) / 16 (GRS)	11 nines
Availability SLA	99.9% (Standard)	99.9% (RA-GRS read 99.99%)	99.95% (multi-region)
Encryption at rest	SSE-S3 / SSE-KMS / SSE-C	SSE + CMK via Key Vault	Google-managed / CMEK / CSEK
Access control	IAM & bucket policies, ACLs	Azure RBAC, SAS, POSIX ACLs	IAM, ACLs, signed URLs
Pricing levers	Storage + requests + egress + retrieval	Storage + operations + egress + retrieval	Storage + operations + egress + retrieval
Consistency	Strong read-after-write	Strong read-after-write	Strong (global)

Beyond raw object storage, the same layer reappears as a managed abstraction: **Fabric OneLake** is a tenant-wide lake built on ADLS Gen2 (Delta–Parquet), **Snowflake** persists data as immutable micro-partitions on the provider’s object store, and **MongoDB’s** WiredTiger is a node-local engine rather than object storage—the contrast is itself instructive.

Key Terms

Object: immutable byte payload plus metadata addressed by bucket and object name. **Bucket:** globally unique namespace, location boundary, and policy container for objects. **Storage class:** pricing and retrieval behavior profile such as Standard, Nearline, Coldline, or Archive. **Lifecycle rule:** policy that transitions, deletes, or otherwise manages objects based on age, version, prefix, or state. **Retention policy:** minimum retention period that prevents deletion or replacement before a compliance deadline. **Storage Transfer Service:** managed service for scheduled, online transfers among clouds, URLs, file systems, and Cloud Storage.

1.1 Week 1 Roadmap

This chapter is the first week of the GCP Master Data Engineer program. It begins with object storage fundamentals, then adds governance and protection controls, then compares migration mechanisms, and closes with hands-on deployment and an architecture case. Cloud Storage appears simple because its core API is small: create a bucket, write objects, read objects, list objects, apply metadata, and delete objects. In production data engineering, however, that small API becomes the root of ingestion, data lake zoning, recovery, retention, compliance, and cost management. A poorly designed bucket hierarchy can leak data, strand objects in

expensive classes, slow downstream pipelines, or make disaster recovery impossible to test.

Cloud Storage is also the service where many engineers first encounter the design style of Google Cloud: global control plane, strong durability claims, location choices, uniform IAM, customer-managed encryption keys, event integration, and policy automation. The same architectural concerns reappear later in BigQuery, Dataflow, Dataproc, Pub/Sub, Dataplex, and Vertex AI. Therefore, the storage decisions made in Week 1 are not introductory trivia; they are the foundation of the rest of the book.

Definition

Cloud Storage is Google Cloud’s managed object storage service for durable, highly available storage of immutable objects. It is commonly used for data lakes, backups, media assets, application artifacts, analytical staging, model artifacts, and cross-system exchange [28].

1.2 Monday: Object Storage Fundamentals

1.2.1 Objects, Buckets, and Namespaces

Object storage differs from block storage and file storage. A block device gives an operating system addressable blocks. A file system gives applications hierarchical directories, permissions, and partial file operations. Object storage gives applications objects in buckets. Each object has a name, bytes, metadata, generation, checksum information, and access controls inherited or assigned through the surrounding policy model. The name may contain slashes, but those slashes are part of a flat object key rather than true directories.

This distinction matters in data engineering. Data lake paths such as `raw/events/date=2026-07-30/hour=21/` are conventions over object names. Query engines and pipeline code interpret prefixes as partitions, but Cloud Storage itself stores named objects. A rename is usually implemented as copy plus delete. A large recursive move is not a cheap metadata operation; it can be a large data movement.

Note

A bucket is both a namespace and an administrative boundary. Location, uniform bucket-level access, public access prevention, retention policy, lifecycle policy, default encryption behavior, and labels are bucket-level design choices. Object prefixes can organize data, but they do not replace careful bucket design.

The object model is well suited to analytical systems because files are normally written once and read many times. Pipeline tasks write immutable outputs, downstream engines scan them, and governance services catalog them. The model is less suited to workloads that require low-latency random updates to small records. Later chapters use Spanner and Bigtable for those patterns, reflecting lessons from Google’s distributed storage systems [3, 2]. The broader history of distributed storage includes the Google File System, which framed large-file storage for data processing at scale [4].

1.2.2 Consistency, Durability, and Availability

Cloud Storage is designed for very high durability through redundant storage of object data and metadata. Durability is the probability that stored data will not be lost. Availability is the probability that the service can successfully serve requests when clients need it. A system can

be durable but temporarily unavailable; data still exists, but an outage or network problem can prevent access. A data engineer must design for both.

Definition

Durability protects against data loss. **Availability** protects access to data during a time window. **Recovery time objective** (RTO) describes how quickly service must be restored; **recovery point objective** (RPO) describes how much data loss is acceptable.

Cloud Storage supports strong read-after-write and list consistency for object operations. That consistency simplifies ingestion workflows: once a writer receives success for a new object, downstream processes can discover and read it. Still, correctness requires idempotent pipeline design. A producer can retry after a network timeout and create duplicate names if names are not deterministic. A consumer can process the same object twice if its checkpointing is weak. Storage guarantees are necessary but not sufficient for exactly-once data engineering.

1.2.3 Storage Classes

Cloud Storage classes express access frequency and recovery economics. They are not merely labels. A class controls storage price, retrieval cost, minimum storage duration, and the operational expectation for how often objects are accessed. The four general-purpose classes are Standard, Nearline, Coldline, and Archive [28].

Class	Typical access	Design intent	Data engineering examples
Standard	Frequent or low-latency	Active data and hot analytical staging	Current raw zone, active feature data, recent media assets
Nearline	About monthly	Infrequently accessed but occasionally restored	Monthly audit extracts, older raw partitions, warm backups
Coldline	About quarterly	Disaster recovery and rarely read archives	Quarterly compliance snapshots, DR copies
Archive	About yearly	Lowest-cost long-term preservation	Legal hold exports, historical video masters, deep archive

Table 1.1: Cloud Storage classes and common data engineering uses.

Common Pitfall

Choosing a colder class too early can increase total cost. Storage price may fall, but retrieval charges, operation charges, minimum storage duration, and pipeline delay can dominate if data is unexpectedly reprocessed. Model total cost using realistic replay scenarios, not only steady-state storage volume.

1.2.4 Location Types

A bucket location controls where object data is stored. Region buckets keep data in one Google Cloud region. Dual-region buckets store data in two specific regions. Multi-region buckets place data across a broad geographic area. The correct choice depends on the consuming compute location, regulatory requirements, disaster recovery targets, and data egress patterns.

Regional buckets are often best for pipelines whose compute is regional: a Dataflow job in `us-central1`, a Dataproc cluster in `us-central1`, or a BigQuery dataset in a compatible loca-

tion. Dual-region buckets are useful when an organization wants explicit geographic redundancy and control over the participating regions. Multi-region buckets simplify global availability for content distribution and cross-region analytics, but they can be the wrong fit when strict locality or predictable co-location with compute matters.

Tip

Pick the narrowest location that meets durability, availability, sovereignty, and recovery requirements. Wider geography can improve resilience and read locality for global consumers, but it may complicate compliance, downstream dataset location choices, and cost attribution.

1.2.5 Bucket Architecture for Data Lakes

A common pattern separates zones by trust and processing state: **raw**, **validated**, **curated**, and **sandbox**. The raw zone preserves source fidelity. The validated zone enforces schema and quality checks. The curated zone exposes analytics-ready data. The sandbox zone supports experiments with stricter expiration. In small projects, these zones can be prefixes in a bucket. In regulated or multi-team platforms, they often become separate buckets with separate IAM, retention, encryption, and lifecycle controls.

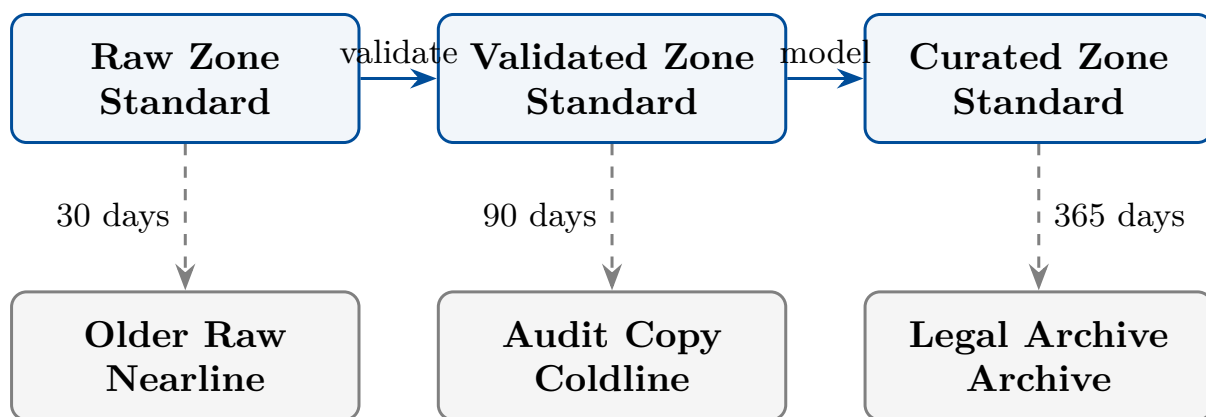


Figure 1.1: A storage-class lifecycle for a lakehouse-style Cloud Storage landing zone.

1.3 Tuesday: Lifecycle, Versioning, Retention, and Soft Delete

1.3.1 Object Lifecycle Management

Object Lifecycle Management automates storage hygiene. A lifecycle policy contains rules. Each rule has a condition and an action. Conditions can match age, creation date, storage class, object prefix or suffix, number of newer versions, and live or noncurrent state. Actions can change storage class or delete objects. Lifecycle policies are essential because manual cleanup does not scale to billions of objects.

For data platforms, lifecycle rules should be tied to data semantics. Raw event files may transition to Archive after they are outside the normal replay window. Temporary Dataflow staging files may be deleted after a few days. Curated analytical extracts may remain Standard for a quarter because business users frequently run year-to-date reports. A single global rule such as “archive everything after thirty days” is rarely correct for all prefixes.

Note

Lifecycle management is policy-as-code for storage economics. Review lifecycle rules with the same rigor as pipeline code: version them, test them in a non-production bucket, and connect each rule to a retention, recovery, or cost requirement.

1.3.2 Object Versioning

Object Versioning keeps noncurrent generations when a live object is overwritten or deleted. This protects against accidental replacement and application bugs, but it also increases storage consumption. Versioning is especially valuable for configuration files, small reference datasets, model artifacts, and high-value manifests. It is less attractive for high-volume append-only event files where object names are unique and overwrites should never happen.

Common Pitfall

Versioning without lifecycle controls can create hidden cost growth. Noncurrent versions are easy to forget because normal list operations emphasize live objects. Pair versioning with lifecycle rules that delete old noncurrent versions after the recovery window.

1.3.3 Retention Policies and Bucket Lock

A retention policy sets a minimum age before objects can be deleted or replaced. It can support compliance rules, audit preservation, and ransomware-resistant backups. Bucket Lock makes a retention policy permanent for the bucket. Once locked, the policy cannot be shortened or removed. This is intentionally irreversible because it is designed for compliance use cases.

Definition

Bucket Lock is an irreversible action that locks a retention policy on a bucket. It should be used only after legal, compliance, platform, and operations teams agree on the retention period and test the policy in a non-production environment.

Retention and lifecycle policies can interact. A lifecycle rule may attempt to delete an object, but retention can prevent deletion until the minimum period has passed. This is desirable when retention expresses a higher-priority compliance control. Engineers should document the expected precedence so that cost forecasts and cleanup jobs do not assume deletion will occur earlier.

1.3.4 Soft Delete

Soft delete provides a recovery window for deleted Cloud Storage objects. It is a safety mechanism against accidental or malicious deletion, distinct from object versioning and retention. Versioning preserves generations when objects change. Retention prevents deletion before a deadline. Soft delete allows recovery of deleted objects within a configured interval. The right combination depends on threat model and operational goals.

Control	Protects against	Primary caution
Lifecycle policy	Cost drift and unmanaged old data	Incorrect rules can move or delete data before business users expect it
Object Versioning	Accidental overwrite or delete of important objects	Noncurrent versions create hidden storage growth
Retention policy	Premature deletion before compliance deadline	Can block cleanup and increase storage cost
Bucket Lock	Tamper-resistant compliance retention	Irreversible; test before locking
Soft delete	Recent accidental or malicious deletion	Recovery window is not a substitute for full backup and DR design

Table 1.2: Storage protection and governance controls.

1.4 Wednesday: Data Migration and Transfer Choices

1.4.1 Storage Transfer Service

Storage Transfer Service is a managed service for online transfers. It supports scheduled and one-time movement from sources such as Amazon S3, Azure Blob Storage, HTTP or HTTPS lists, POSIX file systems through agents, and Cloud Storage buckets [55]. It is preferable when data can move over the network within the migration window, when repeat synchronization is needed, or when the team wants managed retries, scheduling, logging, and incremental behavior.

For a data engineer, the important design questions are not only source and destination. Ask whether object metadata must be preserved, whether deletes should propagate, whether prefixes must be remapped, how credentials are scoped, how transfer logs are monitored, and what happens if the source changes during migration. A transfer job should have a reconciliation plan: expected object count, expected byte count, sample checksums, and a sign-off method.

1.4.2 Transfer Appliance

Transfer Appliance is for offline migration of large data volumes when network transfer is too slow, too costly, or operationally impossible [56]. Google ships a secure appliance, the customer loads data locally, and the appliance is shipped back for ingestion. Offline transfer changes the risk profile. It reduces dependence on wide-area bandwidth, but it introduces logistics, physical chain of custody, export scheduling, and a cutover plan for changed data.

Definition

Online transfer moves data over a network connection. **Offline transfer** moves data by physically shipping encrypted storage hardware. Online transfer favors repeatability and lower logistics; offline transfer favors very large initial loads when network time is unacceptable.

1.4.3 CLI Tools: `gcloud storage` and `gsutil`

Google Cloud provides both `gcloud storage` and `gsutil` workflows. `gcloud storage` is the modern Google Cloud CLI surface for storage operations. `gsutil` remains widely used in scripts and documentation, especially for advanced or legacy workflows. A professional data engineer should be able to read both, but new automation should prefer the supported direction for the organization and be explicit about tool versions.

Tip

For migrations, do not benchmark with a tiny file. Use representative object sizes, compression, count distribution, and network path. Millions of small objects often stress listing and per-object overhead more than raw bytes.

1.4.4 Migration Decision Matrix

Option	Best fit	Watch for
gcloud storage cp	Small to medium ad hoc copies, scripted deployment, developer workflows	Local bandwidth, retries, local credentials, shell portability
gsutil rsync	Legacy sync scripts and operational parity checks	Semantics of delete propagation and metadata differences
Storage Transfer Service	Managed cloud-to-cloud or scheduled transfers	Credential scope, source mutation during transfer, monitoring
Transfer Appliance	Very large initial loads with poor network feasibility	Logistics, chain of custody, final delta sync
Partner tools	Specialized enterprise migrations	Licensing, support boundaries, auditability

Table 1.3: Choosing a Cloud Storage migration mechanism.

1.5 Thursday Hands-On Project: Bucket, Lifecycle, Transfer, and Client Code

This project creates a Cloud Storage bucket, applies a lifecycle policy that transitions objects to Archive after thirty days, and demonstrates a Storage Transfer Service job from a public Amazon S3 bucket. The commands are realistic, but names and project identifiers must be changed before execution. Use a sandbox project, confirm billing expectations, and delete test resources after the lab.

Note

The placeholders `PROJECT_ID`, `BUCKET_NAME`, `REGION`, and `TRANSFER_JOB_NAME` are intentionally explicit. Replace them with values that satisfy your organization's naming and location standards.

1.5.1 PowerShell Preparation

The following commands use environment variables in a PowerShell-friendly form. The `gcloud storage` commands themselves can also be run in Cloud Shell with minor syntax changes.

```

1 # Replace these values before running.
2 $env:PROJECT_ID = "my-gcp-data-engineering-lab"
3 $env:BUCKET_NAME = "my-gcp-data-engineering-lab-raw"
4 $env:REGION = "US"
5
6 gcloud config set project $env:PROJECT_ID
7
8 gcloud storage buckets create gs://$env:BUCKET_NAME '
9   --location=$env:REGION '
10  --uniform-bucket-level-access
11
12 "event_id,event_ts,customer_id" | Set-Content -Path .\sample-events.csv

```

```

13 "1,2026-07-30T21:35:44Z,c001" | Add-Content -Path .\sample-events.csv
14
15 gcloud storage cp .\sample-events.csv gs://$env:BUCKET_NAME/raw/events/sample-
   events.csv
16
17 gcloud storage ls --long gs://$env:BUCKET_NAME/raw/events/

```

Listing 1.1: Creating a bucket and uploading a sample object with the Google Cloud CLI.

1.5.2 Lifecycle JSON

The lifecycle rule below transitions objects under selected prefixes to Archive after thirty days. In real platforms, use separate rules for raw, curated, temporary, and compliance prefixes.

```

1 {
2   "rule": [
3     {
4       "action": {
5         "type": "SetStorageClass",
6         "storageClass": "ARCHIVE"
7       },
8       "condition": {
9         "age": 30,
10        "matchesPrefix": [
11          "raw/",
12          "landing/"
13        ]
14      }
15    }
16  ]
17 }

```

Listing 1.2: Lifecycle policy that transitions matching objects to Archive after thirty days.

```

1 @'
2 {
3   "rule": [
4     {
5       "action": {
6         "type": "SetStorageClass",
7         "storageClass": "ARCHIVE"
8       },
9       "condition": {
10        "age": 30,
11        "matchesPrefix": ["raw/", "landing/"]
12      }
13    }
14  ]
15 }
16 '@ | Set-Content -Path .\lifecycle-archive-30-days.json
17
18 gcloud storage buckets update gs://$env:BUCKET_NAME '
19   --lifecycle-file=.\lifecycle-archive-30-days.json
20
21 gcloud storage buckets describe gs://$env:BUCKET_NAME '
22   --format="yaml(location,storageClass,uniformBucketLevelAccess,
   lifecycle_config)"

```

Listing 1.3: Applying and verifying the lifecycle policy.

1.5.3 Storage Transfer Job from Public S3

Public S3 sources change over time. Treat the bucket name and prefix below as lab placeholders and confirm that the source permits public reads before running. In production, use scoped credentials and a dedicated service account for the transfer job.

```

1 $env:TRANSFER_JOB_NAME = "public-s3-to-gcs-lab"
2 $env:SOURCE_S3_BUCKET = "commoncrawl"
3 $env:SOURCE_PREFIX = "crawl-data/CC-MAIN-2024-10/warc.paths.gz"
4
5 gcloud transfer jobs create '
6   s3://$env:SOURCE_S3_BUCKET/$env:SOURCE_PREFIX '
7   gs://$env:BUCKET_NAME/transfers/commoncrawl/ '
8   --name=$env:TRANSFER_JOB_NAME '
9   --description="Lab transfer from public S3 object to Cloud Storage" '
10  --source-creds-file="" '
11  --schedule-starts=$(Get-Date -Format yyyy-MM-dd) '
12  --schedule-repeats-every=1d
13
14 gcloud transfer jobs list --filter="name:$env:TRANSFER_JOB_NAME"

```

Listing 1.4: Creating a Storage Transfer Service job from a public Amazon S3 bucket.

If the CLI version in your environment does not allow an empty credentials file for a public S3 source, create the job through the Google Cloud console or use the Storage Transfer Service API with an empty AWS credentials block according to the current documentation. The architectural lesson is the same: use a managed transfer job when the source, schedule, retry behavior, and monitoring should be controlled by a service rather than a laptop shell.

1.5.4 Python Client Snippet

The Python client library is useful for applications and tests that need to create objects, read metadata, or assert bucket policy. Install `google-cloud-storage` in an isolated environment and authenticate with Application Default Credentials.

```

1 from google.cloud import storage
2
3
4 def upload_text(project_id: str, bucket_name: str, object_name: str, text: str)
5     -> None:
6     client = storage.Client(project=project_id)
7     bucket = client.bucket(bucket_name)
8     blob = bucket.blob(object_name)
9     blob.upload_from_string(text, content_type="text/csv")
10    blob.reload()
11    print({
12        "bucket": bucket.name,
13        "object": blob.name,
14        "generation": blob.generation,
15        "storage_class": blob.storage_class,
16        "size": blob.size,
17    })
18
19 if __name__ == "__main__":
20     upload_text(
21         project_id="my-gcp-data-engineering-lab",
22         bucket_name="my-gcp-data-engineering-lab-raw",
23         object_name="raw/events/python-client-sample.csv",
24         text="event_id,event_ts\n2,2026-07-30T21:40:00Z\n",
25     )

```

Listing 1.5: Uploading an object and reading metadata with the Python Cloud Storage client.

1.5.5 Validation Checklist

A completed lab should show the bucket exists, uniform bucket-level access is enabled, the lifecycle configuration is attached, the sample object exists, the transfer job is visible, and the Python client can upload or inspect an object. For production change control, add labels such as `env=lab`, `owner=data-engineering`, and `cost-center=training`. Labels make cost reports and cleanup safer.

1.6 Friday Use Case and Architecture: Multi-Region DR for Media Streaming

1.6.1 Scenario

A media streaming company stores ten petabytes of video assets. The library contains raw mezzanine files, encoded renditions, thumbnails, subtitles, manifests, and analytical logs. Customers stream globally. Editors and encoding pipelines are regionally concentrated. The company wants low risk of data loss, fast recovery from regional disruption, encryption with customer-managed keys, lifecycle-based cost control, and evidence that disaster recovery works.

The architecture must separate content serving from archival preservation. Hot renditions used by active customers should remain in Standard class and close to content delivery paths. Original masters may transition to colder classes after quality control and contractual windows. Temporary transcoding outputs should expire quickly. Compliance and rights-management records may require retention policies. Analytics logs should land in Cloud Storage and flow to BigQuery, whose Dremel roots explain its analytical execution model [63, 9].

1.6.2 Design

Use a multi-region or dual-region bucket strategy for serving assets, with explicit replication requirements for masters. For example, active assets can live in a US multi-region bucket when customer demand is broad and regulatory constraints permit it. A dual-region bucket pairing two selected regions can provide clearer DR placement. Use customer-managed encryption keys (CMEK) with Cloud KMS keys in appropriate locations, and define key rotation and incident procedures. Use separate buckets for masters, encoded renditions, temporary work, logs, and public delivery integration rather than one universal bucket.

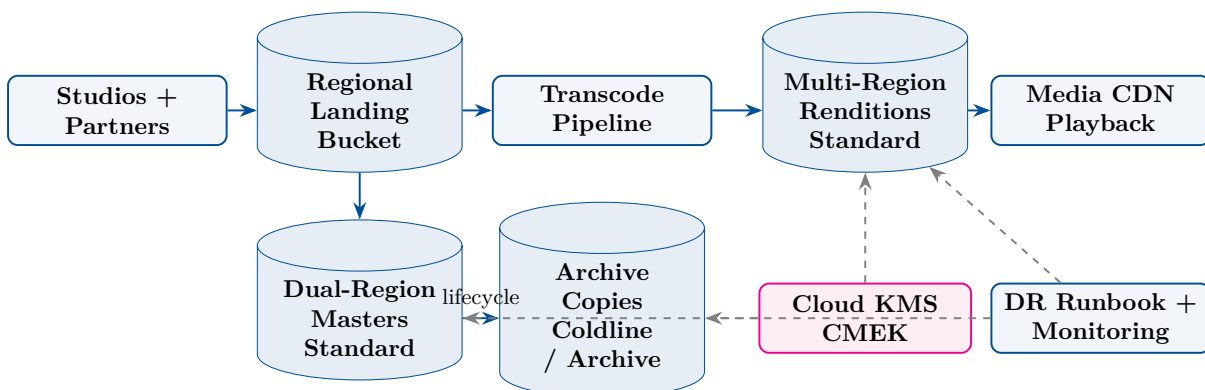


Figure 1.2: A multi-region disaster recovery architecture for ten petabytes of media assets.

1.6.3 Cost and Operations

Ten petabytes changes every decision. A one percent mistake is one hundred terabytes. Lifecycle rules should be simulated against inventory reports before rollout. Archive transitions should consider contractual replay requirements, re-encoding probability, content popularity, and legal hold. For active renditions, cache and CDN strategy may reduce object read pressure, but it does not remove the need for source-of-truth storage durability. For masters, DR tests should restore a statistically meaningful sample and verify checksums, metadata, KMS permissions, and downstream playback or transcoding compatibility.

Tip

Use labels, bucket inventory, access logs, and billing export to BigQuery for storage FinOps. A weekly report should identify top buckets by bytes, operation count, retrieval fees, old temporary prefixes, noncurrent versions, and lifecycle exceptions.

Common Pitfall

A multi-region bucket is not a complete disaster recovery plan. DR also requires IAM continuity, key availability, tested restore procedures, pipeline redeployment, DNS or delivery failover, monitoring, and people who know the runbook.

1.7 Governance and Security Design Notes

Cloud Storage security begins with identity and access management. Prefer uniform bucket-level access for most data engineering buckets so that IAM is the policy model. Avoid object ACLs unless a legacy integration requires them. Enable public access prevention for private data. Use service accounts with least privilege for Dataflow jobs, Dataproc clusters, Composer environments, transfer jobs, and application writers. Separate writer, reader, operator, and auditor roles. For sensitive data, combine CMEK, restricted networks, audit logs, and governance with Dataplex where appropriate [32].

Data classification should be visible in bucket names, labels, and catalog metadata. A bucket named for a raw ingestion zone should not also host public assets. A bucket containing regulated data should have explicit retention, access review cadence, and incident response owner. The Google Cloud Architecture Framework provides a useful structure for reviewing reliability, security, cost, performance, and operations across these decisions [35].

Note

Storage architecture is a socio-technical system. The bucket policy may be correct, but if naming, ownership, documentation, and monitoring are unclear, teams will create unsafe workarounds.

1.8 Operational Runbook for Week 1 Buckets

Every production bucket should have an operational runbook before it receives critical data. The runbook does not need to be long, but it must answer who owns the bucket, what data classification it contains, which systems write to it, which systems read from it, which lifecycle rules are expected to fire, and how incidents are handled. A bucket with no owner becomes a landfill. A bucket with no deletion policy becomes a cost risk. A bucket with no restore test becomes a false sense of safety.

Tip

For each important bucket, keep a one-page record: purpose, owner, location, storage class strategy, CMEK status, lifecycle rules, retention controls, service accounts, expected monthly growth, recovery procedure, and last restore-test date.

The simplest useful monitoring pattern exports Cloud Audit Logs and billing data to BigQuery, then builds a weekly review. The review should flag sudden growth, unusual write principals, public-access changes, lifecycle failures, objects stuck in temporary prefixes, high retrieval fees from cold classes, and buckets with no labels. Later chapters will connect these signals to Cloud Monitoring, Dataplex data quality, and FinOps dashboards.

Signal	Question it answers	Possible response
Bytes by bucket and prefix	Is storage growing as forecast?	Investigate source volume, lifecycle rules, or duplicate writes
Operations by principal	Who is reading or writing data?	Review IAM, service accounts, and unexpected clients
Cold-class retrieval fees	Are archived objects being re-stored too often?	Reconsider class transition timing or replay design
Lifecycle action counts	Are policies firing as expected?	Test rule conditions and retention interactions
Unlabeled buckets	Can costs and ownership be assigned?	Enforce labels through provisioning policy

Table 1.4: Operational signals for Cloud Storage runbooks.

1.9 Interview Q&A

1.9.1 Concepts and Trade-offs

1. **Q: When would you choose a regional bucket over a multi-region bucket?**

A: Choose a regional bucket when compute and analytics are regional, data residency requires a specific region, or lower locality complexity matters more than broad geographic availability. Choose wider geography only when it satisfies a concrete availability or access requirement.

2. **Q: Why can Archive storage be more expensive than Standard for some workloads?**

A: Archive reduces storage price but adds retrieval and minimum-duration considerations. If data is frequently restored, reprocessed, or migrated, the total cost can exceed Standard or Nearline.

3. **Q: How do versioning, retention, and soft delete differ?**

A: Versioning preserves older generations, retention prevents premature deletion or overwrite, and soft delete provides a recovery window after deletion. They solve related but distinct failure modes.

4. **Q: How would you validate a petabyte-scale transfer?**

A: Compare expected and actual bytes, object counts, sampled checksums, metadata, prefix coverage, transfer logs, and downstream readability. Also test incremental synchronization for changed source data.

1.10 Further Reading

Read the Cloud Storage overview for the current behavior of buckets, locations, classes, lifecycle, retention, and soft delete [28]. Review the Google Cloud Architecture Framework for reliability, security, cost, performance, and operations review prompts [35]. Study Storage Transfer Service and Transfer Appliance documentation before designing migrations [55, 56]. For distributed-storage context, read the Google File System paper [4]. For the analytical systems that often consume Cloud Storage data, read Dremel and BigQuery documentation [63, 9]. For governance direction, consult Dataplex documentation [32]. Dataflow documentation is useful when object storage becomes the source or sink for Apache Beam pipelines [30].

Key Takeaways

- Cloud Storage is the durable object foundation for many GCP data platforms, but bucket design is an architectural decision, not a default checkbox.
- Storage class selection must account for retrieval, operations, minimum duration, and replay probability, not only monthly storage price.
- Location type should be chosen from compute locality, sovereignty, availability, and disaster recovery requirements.
- Lifecycle policies, versioning, retention, Bucket Lock, and soft delete each protect against different risks and can interact.
- Storage Transfer Service is the default managed option for online transfer; Transfer Appliance is for large offline migrations where network transfer is not feasible.
- A working DR architecture includes IAM, KMS, lifecycle rules, monitoring, restore tests, and runbooks in addition to replicated storage.

1.11 Exercises

1. **(Conceptual)** Explain the difference between durability and availability. Give one example of a system that is durable but temporarily unavailable.
2. **(Conceptual)** A team wants to store all raw data in Archive immediately after ingestion. Identify three questions you would ask before approving the design.
3. **(Hands-on)** Create a lab bucket with uniform bucket-level access and upload three sample objects under `raw/`, `validated/`, and `tmp/`. Add labels for owner, environment, and cost center.
4. **(Hands-on)** Write a lifecycle policy that deletes `tmp/` objects after seven days, transitions `raw/` objects to Nearline after thirty days, and deletes noncurrent versions after fourteen days.
5. **(Hands-on)** Use `gcloud storage` or `gsutil` to compare object listings before and after enabling versioning. Document what changes in the output.
6. **(Design)** Choose regional, dual-region, or multi-region locations for a healthcare analytics platform with strict residency requirements and explain the trade-off.
7. **(Design)** Create a migration plan for two hundred terabytes in S3 and ten petabytes in an on-premises NAS. Decide which transfer mechanisms you would use and how you would validate completion.

8. **(Architecture)** Extend the media streaming DR design with BigQuery billing export, alerting, and a quarterly restore test. Define the evidence you would present in an architecture review.

1.12 Capstone Project: A Resilient Global Media Asset Store on Cloud Storage

Capstone Project

Mission: Design and prototype a resilient Cloud Storage foundation for a global media-streaming company. Your solution must ingest source assets from another cloud, protect irreplaceable video masters, serve globally distributed consumers, and control the cost of a ten-petabyte library without weakening disaster recovery.

Estimated time: 4–6 hours.

What you'll practice:

- Choosing a multi-region location and storage classes for hot, warm, cold, and archive media assets.
- Implementing lifecycle transitions from Standard to Nearline, Coldline, and Archive.
- Enabling Object Versioning, a retention policy, and Bucket Lock for WORM-style protection.
- Applying CMEK encryption and least-privilege IAM for operators, transfer jobs, and viewers.
- Creating a Storage Transfer Service design that imports from a public Amazon S3 source into Cloud Storage.
- Producing an architecture diagram, runbook evidence, and acceptance criteria that an architecture review board can evaluate.

Scenario

StreamForge Media operates a global streaming platform with roughly 10 PB of video assets. The library includes mezzanine source files from studios, encoded renditions for adaptive bitrate playback, captions, thumbnails, manifests, quality-control reports, and rights-management metadata. Customer playback is global, while encoding teams and editorial workflows are concentrated in North America. The company wants a storage design that keeps popular assets close to serving paths, preserves masters against accidental deletion and ransomware, and provides a tested disaster recovery story for regional disruption.

The business problem is not simply “store 10 PB cheaply.” The active catalog must remain available during high-traffic premieres. Older content must transition to colder classes only when retrieval cost and restore time remain acceptable. Legal and licensing records must be immutable for the required period. Source assets arrive from partners, including a public AWS S3 bucket used for this lab. Security leadership also requires customer-managed encryption keys, public access prevention, auditability, and service accounts with narrow permissions.

Requirements

Your capstone design must include the following controls and explain the trade-offs behind each one:

- Create a Cloud Storage bucket in a multi-region location for globally resilient media assets.
- Use a lifecycle policy that transitions objects from Standard to Nearline, then Coldline, then Archive based on age and prefix.

- Enable Object Versioning so accidental overwrites and deletes can be recovered within an operational window.
- Configure a retention policy and lock it with Bucket Lock after validation, creating a WORM control for protected masters.
- Use CMEK encryption with a Cloud KMS key whose location is compatible with the bucket location.
- Define a Storage Transfer Service job that imports selected objects from a public AWS S3 bucket into the destination bucket.
- Apply least-privilege IAM: separate transfer, media-operator, auditor, and serving identities rather than granting broad project roles.
- Document cost controls for 10 PB, including labels, lifecycle review, noncurrent-version cleanup, and cold-class retrieval assumptions.

Reference Architecture

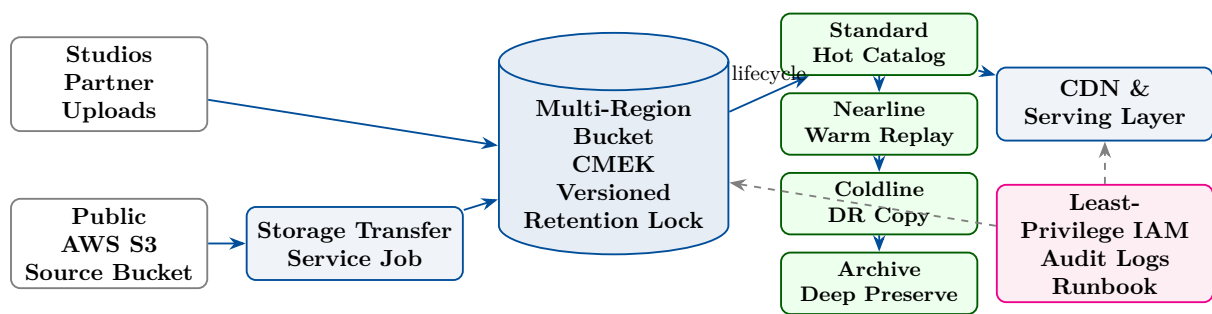


Figure 1.3: Capstone reference architecture for a resilient global media asset store on Cloud Storage.

Implementation Steps

1. **Prepare project variables and create the bucket.** Use a sandbox project first. The examples are PowerShell-friendly and use a multi-region location. Replace every placeholder before execution.

```

1 $env:PROJECT_ID = "my-gcp-media-capstone"
2 $env:BUCKET_NAME = "my-gcp-media-capstone-assets"
3 $env:LOCATION = "US"
4 $env:KMS_RING = "media-assets-ring"
5 $env:KMS_KEY = "media-assets-key"
6 $env:KMS_LOCATION = "us"
7 $env:TRANSFER_SA = "media-transfer-sa"
8 $env:OPS_GROUP = "gcs-media-operators@example.com"
9 $env:AUDIT_GROUP = "gcs-media-auditors@example.com"
10
11 gcloud config set project $env:PROJECT_ID
12
13 gcloud kms keyrings create $env:KMS_RING '
14   --location=$env:KMS_LOCATION
15
16 gcloud kms keys create $env:KMS_KEY '
17   --location=$env:KMS_LOCATION '
18   --keyring=$env:KMS_RING '

```

```

19  --purpose=encryption
20
21  gcloud storage buckets create gs://$env:BUCKET_NAME '
22    --location=$env:LOCATION '
23    --uniform-bucket-level-access '
24    --public-access-prevention '
25    --default-encryption-key=projects/$env:PROJECT_ID/locations/$env:
      KMS_LOCATION/keyRings/$env:KMS_RING/cryptoKeys/$env:KMS_KEY '
26    --labels=env=capstone,workload=media-assets,owner=data-engineering
27
28  gcloud iam service-accounts create $env:TRANSFER_SA '
29    --display-name="Media transfer service account"
30
31  gcloud storage buckets add-iam-policy-binding gs://$env:BUCKET_NAME '
32    --member="serviceAccount:$env:TRANSFER_SA@$env:PROJECT_ID.iam.
      gserviceaccount.com" '
33    --role="roles/storage.objectCreator"
34
35  gcloud storage buckets add-iam-policy-binding gs://$env:BUCKET_NAME '
36    --member="group:$env:OPS_GROUP" '
37    --role="roles/storage.objectAdmin"
38
39  gcloud storage buckets add-iam-policy-binding gs://$env:BUCKET_NAME '
40    --member="group:$env:AUDIT_GROUP" '
41    --role="roles/storage.objectViewer"

```

Listing 1.6: PowerShell setup, CMEK key, bucket creation, and least-privilege IAM.

2. **Attach lifecycle policy and enable versioning.** Keep current active renditions in Standard, move warm objects to Nearline after 30 days, cold DR copies to Coldline after 90 days, and preserved masters to Archive after 365 days. Delete old noncurrent versions after the recovery period so versioning does not create unbounded cost.

```

1  {
2    "rule": [
3      {
4        "action": {"type": "SetStorageClass", "storageClass": "NEARLINE"},
5        "condition": {"age": 30, "matchesPrefix": ["renditions/", "masters/"]}
6      },
7      {
8        "action": {"type": "SetStorageClass", "storageClass": "COLDLINE"},
9        "condition": {"age": 90, "matchesPrefix": ["masters/", "dr/"]}
10     },
11     {
12       "action": {"type": "SetStorageClass", "storageClass": "ARCHIVE"},
13       "condition": {"age": 365, "matchesPrefix": ["masters/", "legal/"]}
14     },
15     {
16       "action": {"type": "Delete"},
17       "condition": {"isLive": false, "numNewerVersions": 3, "age": 45}
18     },
19     {
20       "action": {"type": "Delete"},
21       "condition": {"age": 7, "matchesPrefix": ["tmp/", "transcode-staging/"]}
22     }
23   ]
24 }

```

Listing 1.7: Lifecycle policy for hot, warm, cold, archive, and noncurrent media versions.

```

2 {
3   "rule": [
4     {
5       "action": {"type": "SetStorageClass", "storageClass": "NEARLINE"},
6       "condition": {"age": 30, "matchesPrefix": ["renditions/", "masters
7     ]},
8     {
9       "action": {"type": "SetStorageClass", "storageClass": "COLDLINE"},
10      "condition": {"age": 90, "matchesPrefix": ["masters/", "dr/"]}
11    },
12    {
13      "action": {"type": "SetStorageClass", "storageClass": "ARCHIVE"},
14      "condition": {"age": 365, "matchesPrefix": ["masters/", "legal/"]}
15    },
16    {
17      "action": {"type": "Delete"},
18      "condition": {"isLive": false, "numNewerVersions": 3, "age": 45}
19    },
20    {
21      "action": {"type": "Delete"},
22      "condition": {"age": 7, "matchesPrefix": ["tmp/", "transcode-staging
23    ]}
24  ]
25 }
26 '@ | Set-Content -Path .\media-lifecycle.json
27
28 gcloud storage buckets update gs://$env:BUCKET_NAME '
29   --lifecycle-file=.\media-lifecycle.json
30
31 gsutil versioning set on gs://$env:BUCKET_NAME
32
33 gsutil retention set 365d gs://$env:BUCKET_NAME
34
35 gsutil retention get gs://$env:BUCKET_NAME
36
37 # Irreversible: run only after policy review and non-production testing.
38 gsutil retention lock gs://$env:BUCKET_NAME
39
40 gcloud storage buckets describe gs://$env:BUCKET_NAME '
41   --format="yaml(location,defaultKmsKeyName,versioning,retentionPolicy,
42     lifecycle_config)"

```

Listing 1.8: Applying lifecycle policy, versioning, retention policy, and Bucket Lock.

3. **Define the transfer job from public AWS S3.** In a production migration, credentials should be scoped to the minimum readable prefix. This lab models a public source, imports to a restricted prefix, and avoids delete propagation from the source.

```

1 from google.cloud import storage_transfer
2
3 project_id = "my-gcp-media-capstone"
4 destination_bucket = "my-gcp-media-capstone-assets"
5
6 client = storage_transfer.StorageTransferServiceClient()
7
8 transfer_job = {
9   "description": "Capstone public S3 media seed import",
10  "project_id": project_id,
11  "status": storage_transfer.TransferJob.Status.ENABLED,
12  "transfer_spec": {
13    "aws_s3_data_source": {

```



```

14         "bucket_name": "commoncrawl",
15         "path": "crawl-data/CC-MAIN-2024-10/"
16     },
17     "gcs_data_sink": {
18         "bucket_name": destination_bucket,
19         "path": "imports/public-s3/commoncrawl/"
20     },
21     "transfer_options": {
22         "overwrite_objects_already_existing_in_sink": False,
23         "delete_objects_unique_in_sink": False
24     }
25 },
26 "schedule": {
27     "schedule_start_date": {"year": 2026, "month": 7, "day": 31},
28     "schedule_end_date": {"year": 2026, "month": 7, "day": 31}
29 }
30 }
31
32 response = client.create_transfer_job(request={"transfer_job": transfer_job
33                                     })
34 print(response.name)

```

Listing 1.9: Storage Transfer Service job body for a public S3 source.

4. **Validate and document operations.** Upload a small marker object under `masters/`, overwrite it once to prove versioning, verify retention status, inspect lifecycle configuration, and record the transfer job name. Do not test deletion in the locked production bucket; use a disposable bucket for destructive validation.

Deliverables

- A one-page architecture summary explaining the bucket location, lifecycle design, retention period, CMEK key location, and IAM model.
- The completed reference diagram or an equivalent diagram with sources, transfer service, Cloud Storage bucket controls, lifecycle classes, and CDN or serving layer.
- Command transcript or screenshots proving bucket creation, versioning, lifecycle policy, retention policy, CMEK, IAM bindings, and transfer job configuration.
- A 10 PB cost model with assumptions for active catalog size, monthly growth, retrieval frequency, noncurrent versions, and Archive restore scenarios.
- A DR runbook excerpt that states RTO, RPO, restore test cadence, sample-checksum process, KMS dependency, and escalation owners.

Acceptance Criteria

- The design uses a multi-region Cloud Storage bucket and explains why it is preferred over regional or dual-region for this scenario.
- Lifecycle rules cover Standard to Nearline to Coldline to Archive transitions and include cleanup for temporary prefixes or noncurrent versions.
- Object Versioning is enabled and paired with a bounded noncurrent-version policy.
- The retention policy and Bucket Lock decision are documented as irreversible, reviewed, and tested before production use.
- CMEK is configured with a compatible Cloud KMS location, and the operational impact of key disablement is documented.

- Storage Transfer Service imports from a public AWS S3 source or equivalent lab source and writes to a controlled destination prefix.
- IAM grants are least privilege and avoid broad project Owner, Editor, or Viewer roles for routine storage operations.
- The cost model addresses 10 PB scale, retrieval charges, lifecycle minimum durations, and at least one DR restore event.
- The runbook includes validation evidence: object counts, byte counts, sampled checksums, transfer logs, audit logs, and restore-test results.

Stretch Goals

- Evaluate Autoclass for prefixes where access patterns are unpredictable, and compare it with explicit lifecycle rules.
- Replace the multi-region bucket with a dual-region design that uses turbo replication, then compare RPO, cost, and sovereignty trade-offs.
- Build a BigQuery billing-export dashboard that estimates monthly storage, operation, retrieval, and replication costs for 10 PB.
- Add a Cloud DLP or Sensitive Data Protection scan for captions, metadata, and support files that might contain regulated information.
- Create an inventory-based reconciliation job that compares expected and actual object counts, byte totals, generations, and storage classes.
- Add a quarterly game-day plan that disables a non-production KMS key, restores sample media from Archive, and verifies playback through the serving layer.

Relational Databases: Cloud SQL and AlloyDB

Learning Objectives

- Explain Cloud SQL architecture for managed PostgreSQL, MySQL, and SQL Server workloads in Google Cloud.
- Design regional high availability, automatic failover, backups, point-in-time recovery, and read replicas for relational systems.
- Use Database Migration Service and private IP connectivity to move source databases into Cloud SQL with minimal downtime.
- Describe AlloyDB for PostgreSQL, including its disaggregated storage, columnar engine, and machine learning integrations.
- Implement an HA Cloud SQL for PostgreSQL instance, create a read replica, and simulate failover using PowerShell-friendly `gcloud` commands.
- Build Python client checks that validate primary and replica connectivity during a hands-on relational database lab.
- Design a zero-downtime ERP migration to Cloud SQL with an AlloyDB analytics replica for operational reporting.

Cross-Cloud Equivalence

Managed relational services use different product names, but the architectural vocabulary is portable: engine compatibility, HA placement, replicas, migration tooling, and analytics acceleration.

Capability	AWS	Azure	GCP
Managed relational	RDS / Aurora	Azure SQL / Azure Database for PostgreSQL	Cloud SQL / AlloyDB
Scale-out or accelerated	Aurora	Hyperscale	AlloyDB columnar engine
High availability	Multi-AZ	Zone-redundant HA	Regional HA
Read replicas	RDS replicas / Aurora readers	Read replicas / geo-replicas	Cloud SQL read replicas / AlloyDB read pools
Migration	DMS + SCT	Azure DMS	Database Migration Service

MongoDB Atlas, Microsoft Fabric, and Snowflake are useful analogs but not direct replacements. MongoDB Atlas targets document workloads, Fabric combines analytics services

over OneLake, and Snowflake separates compute from storage for warehousing. Cloud SQL and AlloyDB remain the default GCP choices when the source of truth is relational and transactional.

Key Terms

Cloud SQL: Google Cloud's managed relational database service for PostgreSQL, MySQL, and SQL Server. **Regional HA:** Cloud SQL configuration with a standby instance in another zone of the same region. **Read replica:** asynchronously replicated instance used for read scaling, reporting, or regional read locality. **Database Migration Service:** managed service for migrating databases to Cloud SQL and AlloyDB with continuous replication. **Private IP:** connectivity pattern that keeps database traffic on private Google Cloud networking rather than public addresses. **AlloyDB:** PostgreSQL-compatible managed database designed for high performance, availability, and analytics acceleration.

2.1 Week 2 Roadmap

Week 2 moves from object storage into the transactional systems that feed analytical platforms. Most data engineering projects begin with operational data in relational databases: customers, orders, invoices, inventory, account balances, workflow states, and audit events. A professional data engineer must understand how those systems stay available, how replicas lag, how migrations preserve consistency, and how analytical offloading prevents reporting from hurting online transactions.

This chapter uses Cloud SQL for managed relational operations and AlloyDB for PostgreSQL-compatible workloads that need higher throughput or embedded analytical acceleration. Monday establishes Cloud SQL architecture, high availability, and replicas. Tuesday focuses on Database Migration Service and private IP networking. Wednesday introduces AlloyDB, its columnar engine, and machine learning integration. Thursday builds an HA PostgreSQL lab with a replica and failover drill. Friday applies the patterns to a zero-downtime ERP migration.

Definition

Cloud SQL is a fully managed relational database service that handles provisioning, maintenance, backups, replication, and high availability for supported database engines. It reduces undifferentiated administration, but schema design, transaction boundaries, connection management, and migration correctness remain engineering responsibilities [26].

2.2 Monday: Cloud SQL Architecture, High Availability, and Read Replicas

2.2.1 Managed Architecture

Cloud SQL wraps familiar relational engines with a managed control plane. You still choose an engine version, machine shape, storage size, region, network access, backup window, maintenance window, and database flags. Google Cloud manages host provisioning, patch orchestration, storage growth when configured, monitoring integration, backups, and failover mechanics. Applications connect through standard database protocols, the Cloud SQL Auth Proxy, private IP, public IP with authorized networks, or language connectors.

Note

Managed does not mean ownerless. Application teams still own schema migrations, query efficiency, index strategy, connection pooling, transaction isolation choices, failover testing, and business recovery procedures.

For data engineers, Cloud SQL often appears in three roles. First, it is an operational source that exports change data into pipelines. Second, it is a serving store for metadata, workflow state, and small application datasets. Third, it is a migration target when an on-premises database must move quickly without replatforming. In all three roles, reliability comes from combining database features with network design, IAM, monitoring, and runbooks.

2.2.2 Regional High Availability

Cloud SQL regional high availability creates a primary instance and a standby instance in different zones of the same region. The standby is not a read target. It exists to take over when the primary zone or host becomes unhealthy. Persistent disk replication and managed failover are designed to reduce recovery time compared with restoring from backup, but applications must still reconnect and retry safely.

Definition

High availability improves service continuity after infrastructure failure. It is not the same as disaster recovery. Regional HA protects against zonal failure inside one region; cross-region DR requires backups, replicas, export plans, or a separate architecture.

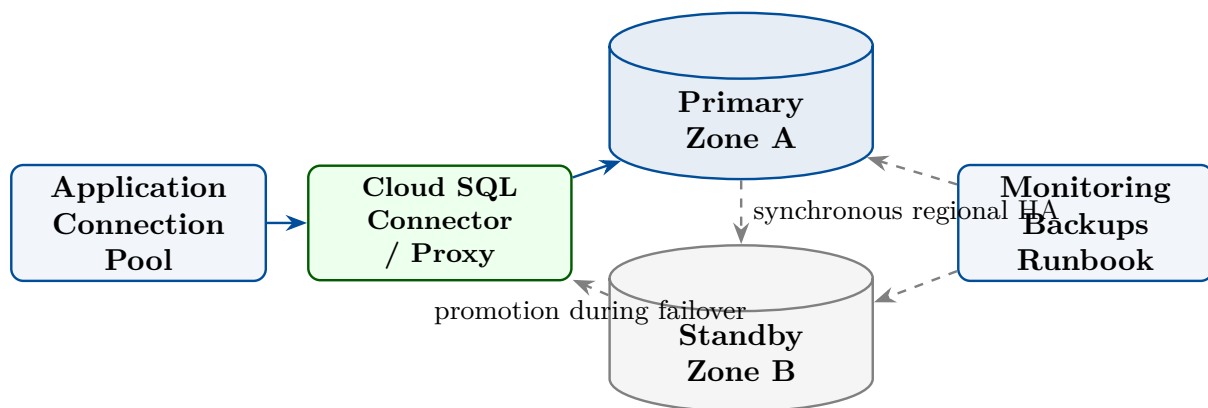


Figure 2.1: Cloud SQL regional HA places a standby in another zone and requires client reconnection after failover.

Common Pitfall

Failover testing often reveals application bugs, not database bugs. Hard-coded IP addresses, missing retry logic, oversized connection pools, long transactions, and non-idempotent writes can turn a managed failover into an application outage.

2.2.3 Backups and Point-in-Time Recovery

Backups protect against logical failure: accidental deletes, destructive deployments, corrupt data loads, or ransomware. High availability keeps the service running after infrastructure failure, but it faithfully replicates bad transactions. Point-in-time recovery uses write-ahead

logs or binary logs to restore to a selected moment inside the retention window. Production designs should document backup retention, restore targets, restore testing cadence, encryption, and who can initiate recovery.

Tip

Treat restore tests as a production control. A backup that has never been restored is an assumption. Schedule a non-production restore, run data quality checks, and record elapsed time against the recovery time objective.

2.2.4 Read Replicas

Cloud SQL read replicas use asynchronous replication. They improve read scale, isolate analytical queries, and can provide regional read locality. They do not guarantee zero-lag reads. A reporting dashboard that reads from a replica may show slightly stale results, which is usually acceptable for analytics but dangerous for payment confirmation, inventory decrement, or idempotency checks.

Pattern		Best fit	Caution
Regional standby	HA	Zonal failure protection for primary database	Standby is not readable and does not replace backups
Same-region replica	read	Reporting, read scale, operational dashboards	Replica lag and extra cost
Cross-region replica	read	Geographic read locality or DR seed	Network cost, lag, and promotion runbook
Backup restore		Logical recovery or environment cloning	Restore time and validation effort

Table 2.1: Cloud SQL reliability and scale patterns.

2.3 Tuesday: Database Migration Service and Private IP Connections

2.3.1 Migration Phases

Database Migration Service (DMS) supports migrations into Cloud SQL and AlloyDB by combining initial load with change replication for compatible sources [29]. A typical migration has five phases: assessment, connectivity, full load, continuous replication, and cutover. Assessment verifies engine version, extensions, character sets, unsupported objects, stored procedures, large tables, and downtime constraints. Connectivity establishes secure paths from the source to Google Cloud. Full load copies existing data. Continuous replication applies changes while applications still write to the source. Cutover stops writes, drains replication lag, validates data, and redirects applications.

Note

Migration success is measured by business continuity, not only bytes copied. The cutover plan must include freeze windows, rollback criteria, DNS or configuration changes, application smoke tests, and stakeholder sign-off.

2.3.2 Private IP and Private Services Access

Private IP connections keep database traffic inside private networking. For Cloud SQL, private IP requires VPC configuration and private services access. Applications running on GCE, GKE, Cloud Run with appropriate VPC access, Dataflow, or Composer can connect without exposing the database through a public address. Private connectivity reduces attack surface and simplifies compliance narratives, but it introduces routing, DNS, firewall, and peering considerations.

Definition

Private services access connects a consumer VPC to Google-managed services by using allocated internal address ranges. Cloud SQL private IP depends on this pattern so that instances receive private addresses reachable from selected VPC clients [27].

Common Pitfall

A private IP database is not automatically secure. Poor IAM, broad network reachability, weak passwords, missing SSL requirements, or overprivileged service accounts can still expose data. Private IP is one layer in a defense-in-depth design.

2.3.3 DMS Design Checklist

Before creating a migration job, document source credentials, network path, replication user privileges, schema conversion needs, table exclusions, large object handling, trigger behavior, sequence synchronization, replication slot cleanup, and performance limits. For PostgreSQL migrations, verify logical replication settings and extensions. For MySQL, verify binary logging and row-based replication settings. For SQL Server, confirm the supported path and downtime expectation.

Area	Question	Evidence
Compatibility	Are engine version, extensions, and schema objects supported?	Assessment report and remediation list
Connectivity	Can DMS reach the source and destination privately?	VPC, VPN, firewall, and connection profile tests
Replication	Can changes be captured continuously?	Source replication settings and lag dashboard
Validation	Do row counts, checksums, and application tests pass?	Reconciliation report and sign-off
Cutover	How are writes frozen and clients redirected?	Runbook with rollback criteria

Table 2.2: Database Migration Service readiness checklist.

2.4 Wednesday: AlloyDB for PostgreSQL, Columnar Engine, and ML Integration

2.4.1 Why AlloyDB Exists

AlloyDB is a PostgreSQL-compatible managed database designed for demanding enterprise workloads [7]. It separates compute from a distributed storage layer, supports automated backups and high availability, and adds Google-engineered performance features. For data engineers, AlloyDB is attractive when teams want PostgreSQL compatibility but need stronger

performance, read scaling, or mixed operational and analytical capability than a conventional managed instance can provide.

Tip

Use Cloud SQL when the workload needs managed relational simplicity and broad engine options. Evaluate AlloyDB when PostgreSQL compatibility is required and the workload is performance-sensitive, analytically active, or expected to grow beyond a conventional single-instance pattern.

2.4.2 Columnar Engine

Traditional row storage is efficient for transactional lookups and updates because all columns of a row are near each other. Analytical queries often scan a few columns across many rows. AlloyDB's columnar engine accelerates suitable analytical queries by storing and processing selected data in a column-oriented format while preserving PostgreSQL compatibility for applications. This can reduce the need to offload every operational report into a separate warehouse immediately, though BigQuery remains the preferred platform for large-scale analytical warehousing.

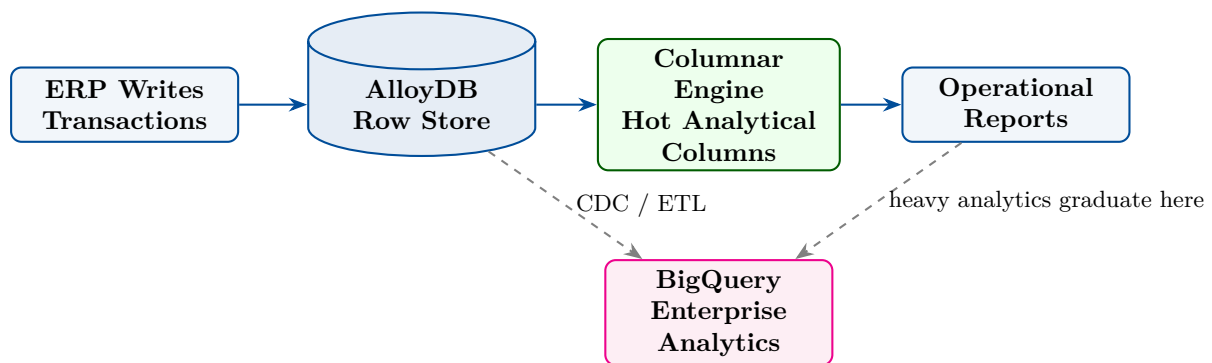


Figure 2.2: AlloyDB can serve transactions from PostgreSQL-compatible row storage while accelerating selected analytical scans with a columnar engine.

2.4.3 ML Integration

AlloyDB supports integrations that bring machine learning closer to PostgreSQL data, including vector search and model invocation patterns documented by Google Cloud [5]. These capabilities are useful when an application needs low-latency retrieval, semantic search, or lightweight predictive features near operational data. They should not turn the transactional database into an uncontrolled feature store. Training pipelines, large-scale feature computation, and governed model deployment still belong in services such as BigQuery, Dataflow, and Vertex AI.

Common Pitfall

Do not run every dashboard, vector search, and batch feature job against the primary transactional database. Even accelerated systems have concurrency, cost, and operational limits. Isolate workloads with read pools, replicas, BigQuery exports, and clear service-level objectives.

2.5 Thursday Hands-On Project: HA Cloud SQL PostgreSQL, Read Replica, and Failover

This lab deploys a regional HA Cloud SQL for PostgreSQL instance, creates a read replica, runs a simple Python connectivity test, and invokes a managed failover. Use a sandbox project, replace every placeholder, and delete resources afterward to avoid charges. Commands are written for Windows PowerShell.

Note

Cloud SQL HA instances and replicas incur cost. Use a small machine type in a training project, choose a short lab window, and record cleanup commands before creating resources.

2.5.1 PowerShell Preparation and HA Instance Creation

```

1 $env:PROJECT_ID = "my-gcp-db-lab"
2 $env:REGION = "us-central1"
3 $env:NETWORK = "default"
4 $env:INSTANCE = "erp-pg-ha-lab"
5 $env:REPLICA = "erp-pg-ha-lab-replica"
6 $env:DB_NAME = "erp"
7 $env:DB_USER = "erp_app"
8 $env:DB_PASSWORD = "ChangeMe-Use-Secret-Manager"
9
10 gcloud config set project $env:PROJECT_ID
11
12 gcloud services enable sqladmin.googleapis.com servicenetworking.googleapis.com
13
14 gcloud compute addresses create google-managed-services-$env:NETWORK '
15   --global '
16   --purpose=VPC_PEERING '
17   --prefix-length=16 '
18   --network=$env:NETWORK
19
20 gcloud services vpc-peerings connect '
21   --service=servicenetworking.googleapis.com '
22   --ranges=google-managed-services-$env:NETWORK '
23   --network=$env:NETWORK
24
25 gcloud sql instances create $env:INSTANCE '
26   --database-version=POSTGRES_15 '
27   --region=$env:REGION '
28   --availability-type=REGIONAL '
29   --tier=db-custom-2-7680 '
30   --storage-type=SSD '
31   --storage-size=20GB '
32   --network=$env:NETWORK '
33   --no-assign-ip '
34   --backup-start-time=03:00 '
35   --enable-point-in-time-recovery '
36   --maintenance-window-day=SUN '
37   --maintenance-window-hour=4
38
39 gcloud sql databases create $env:DB_NAME --instance=$env:INSTANCE
40
41 gcloud sql users create $env:DB_USER '
42   --instance=$env:INSTANCE '
43   --password=$env:DB_PASSWORD

```

Listing 2.1: Creating an HA Cloud SQL for PostgreSQL instance with private IP placeholders.

2.5.2 Read Replica and Failover Drill

```

1 gcloud sql instances create $env:REPLICA '
2   --master-instance-name=$env:INSTANCE '
3   --region=$env:REGION '
4   --tier=db-custom-2-7680
5
6 gcloud sql instances describe $env:INSTANCE '
7   --format="yaml(name,region,gceZone,secondaryGceZone,availabilityType,state,
8     ipAddressess)"
9
10 gcloud sql instances describe $env:REPLICA '
11   --format="yaml(name,masterInstanceName,replicaNames,state,ipAddressess)"
12
13 # Run during an approved lab window. Applications must reconnect after failover
14
15 gcloud sql instances failover $env:INSTANCE
16
17 gcloud sql operations list '
18   --instance=$env:INSTANCE '
19   --limit=5

```

Listing 2.2: Creating a read replica, checking status, and simulating failover.

2.5.3 Python Connectivity Check

The following snippet assumes the Cloud SQL Auth Proxy or connector path provides a local PostgreSQL endpoint. In production, store credentials in Secret Manager and use IAM-authenticated connectors where possible.

```

1 import os
2 import time
3
4 import psycopg
5
6
7 def connect_once(label: str) -> None:
8     conninfo = {
9         "host": os.environ.get("PGHOST", "127.0.0.1"),
10        "port": os.environ.get("PGPORT", "5432"),
11        "dbname": os.environ.get("PGDATABASE", "erp"),
12        "user": os.environ.get("PGUSER", "erp_app"),
13        "password": os.environ["PGPASSWORD"],
14        "connect_timeout": 10,
15    }
16    with psycopg.connect(**conninfo) as conn:
17        with conn.cursor() as cur:
18            cur.execute("select current_database(), now()")
19            database_name, current_time = cur.fetchone()
20            print({"label": label, "database": database_name, "time": str(
21                current_time)})
22
23 if __name__ == "__main__":
24     for attempt in range(1, 6):
25         try:
26             connect_once(label=f"attempt-{attempt}")
27             break
28         except psycopg.OperationalError as exc:
29             print({"attempt": attempt, "status": "retrying", "error": str(exc)})
30             time.sleep(10)

```

Listing 2.3: Validating PostgreSQL connectivity before and after a Cloud SQL failover.

2.5.4 Cleanup

```
1 gcloud sql instances delete $env:REPLICA --quiet
2 gcloud sql instances delete $env:INSTANCE --quiet
```

Listing 2.4: Deleting lab Cloud SQL resources after validation.

2.6 Friday Use Case and Architecture: Zero-Downtime ERP Migration

2.6.1 Scenario

A manufacturing company runs a monolithic on-premises ERP on PostgreSQL. The ERP manages purchasing, inventory, production orders, finance, and customer shipments. Executives want the platform moved to Google Cloud without a disruptive weekend outage. Reporting queries currently run on the same database as transactions and regularly slow month-end close. The target architecture must use Cloud SQL for transactional continuity, Database Migration Service for continuous replication, private connectivity for security, and AlloyDB for accelerated operational analytics.

2.6.2 Migration Architecture

The smallest safe plan is not a big-bang rewrite. Keep the ERP application behavior stable, migrate the database first, and offload heavy reads. Establish VPN or Interconnect from the data center to Google Cloud, create DMS connection profiles, perform initial load, enable continuous replication, validate row counts and checksums, rehearse cutover, freeze writes, drain lag, promote Cloud SQL, and redirect the ERP application. After stabilization, replicate selected tables or change streams into AlloyDB and BigQuery for analytics.

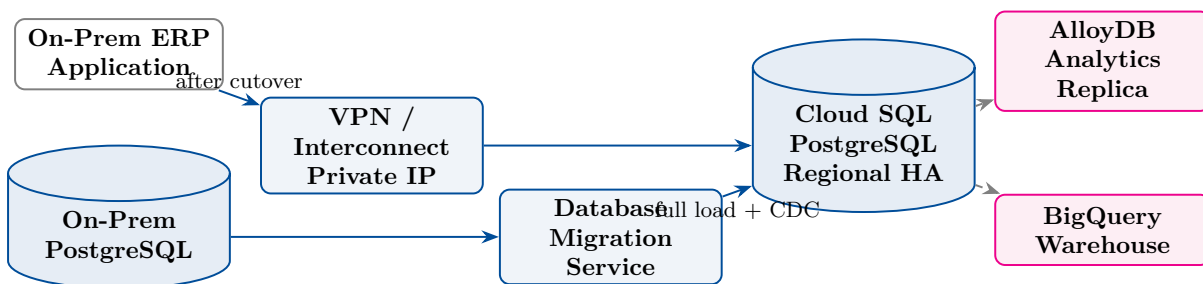


Figure 2.3: Zero-downtime ERP migration using DMS, private connectivity, Cloud SQL HA, and AlloyDB analytics offload.

2.6.3 Cutover and Validation

Zero downtime is an aspiration that must be translated into measurable tolerances. The ERP may tolerate a short write freeze while replication lag drains and the application restarts. Define acceptable lag, maximum cutover duration, rollback point, and business validation. Validate table counts, financial totals, open order counts, sequence values, permissions, stored procedure behavior, and representative application transactions. Keep the source database read-only until the rollback window closes.

Tip

Use business invariants for validation: trial balance totals, inventory quantities by warehouse, open invoice counts, and shipment status counts. Row counts are necessary but not sufficient.

2.7 Security, Operations, and Cost Notes

Relational databases concentrate high-value data. Use private IP, least-privilege IAM, strong database roles, Secret Manager, audit logging, and SSL where required. Avoid embedding passwords in scripts or application configuration. Use connection pooling because serverless and autoscaled clients can overwhelm PostgreSQL with thousands of short-lived connections. Monitor CPU, memory, storage growth, lock waits, slow queries, replication lag, connection count, backup status, and failover events.

Note

Database cost is driven by always-on compute, storage, backups, replicas, HA standby capacity, network traffic, and operational mistakes such as unbounded reporting queries. Label instances and export billing data so database spend is visible by workload.

2.8 Interview Q&A

2.8.1 Concepts and Trade-offs

1. **Q: How does Cloud SQL regional HA differ from a read replica?**

A: Regional HA uses a standby in another zone for failover and is not a read endpoint. A read replica is asynchronously replicated and can serve reads, but it may lag and does not replace HA.

2. **Q: Why should migrations use private IP when possible?**

A: Private IP keeps database traffic off public addresses, reduces exposed attack surface, and fits compliance expectations, while still requiring IAM, firewall, credential, and database-role controls.

3. **Q: What is the main risk of reading from replicas?**

A: Replica lag. Reporting can usually tolerate slight staleness, but workflows that require read-after-write consistency should use the primary or design explicit consistency checks.

4. **Q: When would you choose AlloyDB over Cloud SQL for PostgreSQL?**

A: Choose AlloyDB when PostgreSQL compatibility is required and the workload needs higher performance, read pools, improved availability patterns, or columnar acceleration for operational analytics.

2.9 Further Reading

Read the Cloud SQL documentation for current PostgreSQL, MySQL, SQL Server, HA, backup, replica, and connectivity behavior [26]. Review the Cloud SQL private IP guidance before designing production networks [27]. Study Database Migration Service documentation for supported sources, continuous replication, connection profiles, and cutover practices [29]. Review AlloyDB documentation for clusters, instances, high availability, read pools, columnar engine behavior, and AI integrations [7, 6, 5]. Use the Google Cloud Architecture Framework to evaluate reliability, security, cost, performance, and operations [35].

Key Takeaways

- Cloud SQL simplifies database administration, but engineers still own schema quality, connection behavior, migration correctness, and recovery testing.
- Regional HA protects against zonal infrastructure failure; backups and point-in-time recovery protect against logical failure.
- Read replicas scale reads and isolate reporting, but asynchronous lag must be visible and acceptable.
- Database Migration Service migrations need assessment, private connectivity, continuous replication, validation, and a rehearsed cutover runbook.
- AlloyDB extends PostgreSQL-compatible operations with performance features, columnar acceleration, and AI-oriented integrations.
- ERP migrations should stabilize the transactional target first, then offload analytics to replicas, AlloyDB, or BigQuery.

2.10 Exercises

1. **(Conceptual)** Explain why high availability does not protect against an accidental `delete from orders` statement.
2. **(Conceptual)** Compare a Cloud SQL regional HA standby, same-region read replica, and cross-region read replica.
3. **(Hands-on)** Create a Cloud SQL PostgreSQL instance in a sandbox project with backups and point-in-time recovery enabled.
4. **(Hands-on)** Create a read replica and measure replication lag after inserting test rows into the primary.
5. **(Hands-on)** Run a failover drill and record application reconnect time using a small Python script.
6. **(Design)** Draft a DMS migration checklist for a 2 TB on-premises PostgreSQL database with a one-hour cutover window.
7. **(Design)** Choose private IP, public IP with authorized networks, or connector-based access for Cloud Run, GKE, and on-premises clients.
8. **(Architecture)** Extend the ERP migration design with monitoring, rollback criteria, business validation metrics, and an AlloyDB reporting workload.

2.11 Capstone Project: HA Cloud SQL and AlloyDB Analytics for ERP Migration

Capstone Project

Mission: Design and prototype a zero-downtime ERP database migration from on-premises PostgreSQL to HA Cloud SQL, then offload operational analytics to AlloyDB. Your solution must preserve transactional correctness, use private connectivity, provide a tested failover plan, and give reporting users a path that does not overload the ERP primary.

Estimated time: 5–7 hours.

What you'll practice:

- Creating a regional HA Cloud SQL for PostgreSQL target with backups and point-in-time recovery.
- Designing DMS connection profiles, migration jobs, validation checks, and cutover steps.
- Using private IP and least-privilege access for application and migration traffic.
- Creating a read replica and defining promotion or failover runbooks.
- Adding AlloyDB as an analytics-serving replica pattern for operational reporting.
- Producing a reference architecture, implementation transcript, acceptance criteria, and rollback plan.

Scenario

Northwind Precision Manufacturing runs a monolithic ERP backed by a 4 TB on-premises PostgreSQL database. The database receives order, procurement, inventory, finance, and shipping transactions around the clock. Month-end reporting joins large invoice, payment, and inventory tables and regularly slows application users. Leadership wants the database moved to Google Cloud with no data loss, only a brief planned write freeze, private connectivity, and a clear path for faster reporting.

Requirements

- Deploy a Cloud SQL for PostgreSQL target with regional HA, automated backups, and point-in-time recovery.
- Use private IP connectivity through a VPC design appropriate for application clients and DMS.
- Define DMS source and destination connection profiles, full load, continuous replication, and cutover criteria.
- Create at least one Cloud SQL read replica for reporting isolation or DR rehearsal.
- Design an AlloyDB analytics replica pattern for month-end reporting tables and selected operational dashboards.
- Document database roles, Secret Manager usage, audit logging, monitoring, and alert thresholds.
- Provide validation evidence for row counts, checksums, sequence values, business totals, and application smoke tests.
- Define rollback criteria and source database handling during the post-cutover stabilization window.

Reference Architecture

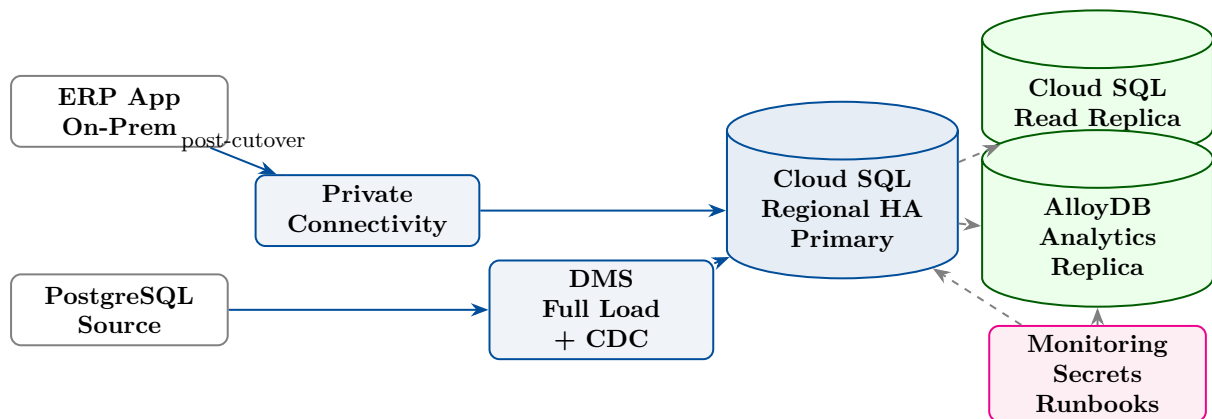


Figure 2.4: Capstone reference architecture for HA Cloud SQL with AlloyDB analytics offload during ERP migration.

Implementation Steps

1. **Create the HA Cloud SQL target and read replica.** Replace all placeholders, use a sandbox project first, and do not store production passwords in shell history.

```

1 $env:PROJECT_ID = "my-gcp-erp-capstone"
2 $env:REGION = "us-central1"
3 $env:NETWORK = "default"
4 $env:INSTANCE = "erp-prod-pg-ha"
5 $env:REPLICA = "erp-prod-pg-reporting"
6 $env:DB_NAME = "erp"
7 $env:DB_USER = "erp_app"
8 $env:DB_PASSWORD = "Replace-With-Secret-Manager-Value"
9
10 gcloud config set project $env:PROJECT_ID
11
12 gcloud sql instances create $env:INSTANCE '
13   --database-version=POSTGRES_15 '
14   --region=$env:REGION '
15   --availability-type=REGIONAL '
16   --tier=db-custom-4-15360 '
17   --storage-type=SSD '
18   --storage-size=500GB '
19   --storage-auto-increase '
20   --network=$env:NETWORK '
21   --no-assign-ip '
22   --backup-start-time=02:00 '
23   --enable-point-in-time-recovery
24
25 gcloud sql databases create $env:DB_NAME --instance=$env:INSTANCE
26
27 gcloud sql users create $env:DB_USER '
28   --instance=$env:INSTANCE '
29   --password=$env:DB_PASSWORD
30
31 gcloud sql instances create $env:REPLICA '
32   --master-instance-name=$env:INSTANCE '
33   --region=$env:REGION '
34   --tier=db-custom-4-15360

```

Listing 2.5: Capstone Cloud SQL target, database, user, and replica creation.

2. **Create DMS placeholders and validation artifacts.** The exact flags vary by source engine and network path, so the listing emphasizes the repeatable evidence each migration must capture.

```

1 $env:MIGRATION_JOB = "erp-postgres-to-cloudsql"
2 $env:SOURCE_PROFILE = "erp-onprem-postgres"
3 $env:DEST_PROFILE = "erp-cloudsql-postgres"
4
5 gcloud database-migration connection-profiles list --region=$env:REGION
6
7 gcloud database-migration migration-jobs list --region=$env:REGION
8
9 gcloud database-migration migration-jobs describe $env:MIGRATION_JOB '
10 --region=$env:REGION '
11 --format="yaml(state,phase,source,destination,dumpPath,duration)"
12
13 @'
14 validation_item,status,evidence
15 row_counts,pending,Compare source and target counts for critical tables
16 checksums,pending,Sample deterministic hashes by primary key range
17 sequences,pending,Verify next values after cutover rehearsal
18 business_totals,pending,Compare trial balance and inventory by warehouse
19 smoke_tests,pending,Run login order invoice and shipment workflows
20 '@ | Set-Content -Path .\erp-migration-validation.csv

```

Listing 2.6: DMS planning commands and migration validation evidence placeholders.

3. **Prototype application retry validation.** Run this check before and after a planned failover or cutover rehearsal.

```

1 import os
2 import time
3
4 import psycopg
5
6
7 SQL = """
8 select
9     count(*) as open_orders
10 from erp_orders
11 where status in ('OPEN', 'ALLOCATED')
12 """
13
14
15 def query_open_orders() -> int:
16     with psycopg.connect(
17         host=os.environ["PGHOST"],
18         port=os.environ.get("PGPORT", "5432"),
19         dbname=os.environ["PGDATABASE"],
20         user=os.environ["PGUSER"],
21         password=os.environ["PGPASSWORD"],
22         connect_timeout=10,
23     ) as conn:
24         with conn.cursor() as cur:
25             cur.execute(SQL)
26             return cur.fetchone()[0]
27
28
29 for attempt in range(1, 8):
30     try:
31         print({"attempt": attempt, "open_orders": query_open_orders()})
32         break
33     except psycopg.OperationalError as exc:
34         print({"attempt": attempt, "retry": True, "error": str(exc)})

```

```
35      time.sleep(15)
```

Listing 2.7: Capstone retry loop for ERP database smoke testing.

Deliverables

- A one-page architecture summary explaining HA Cloud SQL, private IP, DMS, read replica, AlloyDB analytics offload, and rollback boundaries.
- The completed reference diagram or an equivalent diagram showing on-premises ERP, DMS, private connectivity, Cloud SQL HA, read replica, AlloyDB, monitoring, and secrets.
- Command transcript or screenshots proving Cloud SQL creation, backup settings, PITR, private IP, read replica, and failover rehearsal.
- A migration runbook with assessment findings, replication setup, validation queries, cutover timeline, rollback triggers, and owner assignments.
- A reporting offload plan that identifies which ERP reports stay on Cloud SQL replicas, which move to AlloyDB, and which graduate to BigQuery.

Acceptance Criteria

- The target Cloud SQL instance uses regional HA, backups, point-in-time recovery, private IP, and least-privilege database roles.
- DMS migration phases are documented from source assessment through continuous replication and cutover.
- Validation includes row counts, sampled checksums, sequence values, business invariants, and application smoke tests.
- The cutover plan defines write freeze, lag drain, DNS or configuration switch, rollback criteria, and source read-only handling.
- A read replica or AlloyDB offload path protects the transactional primary from heavy reporting.
- Monitoring covers CPU, memory, connections, storage growth, slow queries, locks, replica lag, backup status, and failover events.
- Secrets are not hardcoded in application code, Terraform variables, notebooks, or command transcripts.
- The design states residual risks, including replica lag, extension compatibility, stored procedure behavior, and application reconnect behavior.

Stretch Goals

- Add a Cloud SQL Auth Proxy or language connector deployment pattern for Cloud Run, GKE, and on-premises migration workers.
- Build a BigQuery dashboard from exported validation CSV files that tracks migration readiness by table and business process.
- Add Datastream or another CDC path from Cloud SQL into BigQuery for long-term analytical history.

- Prototype AlloyDB vector search for ERP support-ticket retrieval using sanitized, non-sensitive text.
- Create a failover game day that measures application reconnect time, replica lag, alert delivery, and runbook accuracy.
- Estimate monthly cost for Cloud SQL HA, replica, backups, AlloyDB, network transfer, and operational monitoring.

Global Relational Scaling: Cloud Spanner

Learning Objectives

- Explain Cloud Spanner architecture, including instances, nodes, processing units, splits, replicas, leaders, Paxos, and TrueTime.
- Describe how Cloud Spanner provides strong global consistency and external consistency for relational transactions.
- Design schemas with primary keys, interleaved tables, secondary indexes, and foreign keys while avoiding hot spots.
- Use query execution plans, Key Visualizer, statistics, and indexing strategy to tune Spanner performance.
- Create a multi-region Spanner instance, build an interleaved e-commerce cart schema, and load 10,000 synthetic records.
- Evaluate a globally distributed payment-processing ledger that requires 99.999% availability and zero data loss across continents.
- Build a capstone architecture for a globally consistent payment ledger on multi-region Spanner with operations, validation, and recovery evidence.

Cross-Cloud Equivalence

Globally distributed transactional databases are marketed differently across clouds, but the design questions are portable: consistency model, write placement, replica quorum, horizontal scale, latency, and relational compatibility.

Capability	GCP	AWS	Azure	Traditional sharded RDBMS
Globally consistent OLTP	Cloud Spanner with external consistency and SQL	Aurora Database for low-latency global reads; Aurora DSQL for distributed SQL	Cosmos DB with strong consistency for NoSQL APIs	Usually application-managed two-phase commit or eventual reconciliation
Horizontal scale	Automatic splits across nodes and replicas	Cluster and regional replicas; DSQL partitions distributed data	Partitioned containers and global distribution	Manual shards, re-sharding, and routing layer
Synchronous replication	Paxos quorum across replicas	Region-aware replication varies by product and mode	Configurable consistency and multi-region writes	Often limited to one region or expensive cross-shard coordination
External consistency / time	TrueTime commit timestamps order transactions globally	Service-specific transaction ordering; no direct TrueTime equivalent	Strong consistency can linearize reads within configured regions	Application timestamps rarely prove external consistency
Schema modeling	Relational tables, primary keys, secondary indexes, interleaving, foreign keys	Relational models in Aurora; distributed SQL model in DSQL	NoSQL containers, partition keys, and consistency levels	Normalized tables inside shards; cross-shard foreign keys are hard

MongoDB sharded clusters can distribute document workloads and Snowflake can replicate analytical data, but neither is a direct drop-in for globally consistent relational OLTP. MongoDB usually trades relational constraints for document flexibility, while Snowflake is analytical rather than a low-latency transaction processor.

Key Terms

Cloud Spanner: Google Cloud’s fully managed, horizontally scalable relational database with strong consistency and high availability. **TrueTime:** Google’s bounded-time API that lets Spanner assign commit timestamps with known uncertainty. **External consistency:** guarantee that if transaction A commits before transaction B starts, B receives a later timestamp and observes A where appropriate. **Split:** contiguous key range that Spanner can move or replicate independently for scale. **Interleaved table:** child table whose rows are physically colocated with a parent row by primary key prefix. **Leader replica:** replica that coordinates writes for a Paxos group.

3.1 Week 3 Roadmap

Week 3 moves from regional relational services to a database built for global transactional scale. Cloud Spanner is the GCP service to evaluate when the business needs relational semantics, SQL, horizontal scaling, synchronous replication, and high availability beyond a single region [23]. It is not merely a bigger Cloud SQL instance. It changes how engineers think about primary keys, transactions, locality, indexes, and operational evidence.

Monday introduces nodes, processing units, splits, replicas, Paxos, TrueTime, and strong global consistency. Tuesday focuses on schema design with primary keys, interleaved tables, foreign keys, and secondary indexes. Wednesday studies performance tuning, query plans, leader placement, statistics, and hot-spot avoidance. Thursday builds a multi-region Spanner lab with an

interleaved shopping-cart schema and 10,000 synthetic rows. Friday applies the patterns to a payment-processing ledger that requires 99.999% availability and zero data loss across continents.

Definition

Cloud Spanner is a managed relational database that combines SQL, transactions, schema, and strong consistency with horizontal scaling and synchronous replication. It was designed from the Spanner research system described by Google, where TrueTime and Paxos make globally ordered commits practical at large scale [3].

3.2 Monday: Spanner Concepts and Strong Consistency

3.2.1 Instances, Nodes, Processing Units, and Databases

A Spanner instance is the administrative container for one or more databases. The instance configuration chooses a regional or multi-region topology, which determines where replicas live and which regions can serve reads and writes. Capacity is allocated in nodes or processing units. A node is a larger capacity unit; processing units allow finer increments for smaller workloads. Capacity affects CPU, storage throughput, and the amount of work Spanner can process concurrently, but it does not change the fundamental consistency model.

Spanner stores data in splits. A split is a contiguous range of primary keys that can be moved, replicated, and served independently. As data grows or traffic patterns change, Spanner can split ranges and distribute them across servers. This is why primary-key design matters: the primary key defines the physical ordering of rows, and the ordering determines whether writes spread evenly or concentrate on a narrow hot range.

Note

Spanner removes many database administration tasks, but it does not remove data modeling. Poor primary keys, unbounded transactions, and inefficient secondary indexes can still create latency and cost problems.

3.2.2 Replicas, Leaders, and Paxos Quorums

Every split is replicated. For each Paxos group, one replica acts as leader and coordinates writes. A write commits only after a quorum agrees, which protects against losing committed data when a replica or zone fails. Regional configurations keep replicas inside one region for lower write latency. Multi-region configurations place replicas across multiple regions to improve availability and global read locality, at the cost of higher write latency because consensus crosses longer network distances.

Definition

Paxos is a consensus protocol family that lets distributed replicas agree on a value even when some replicas fail. In Spanner, Paxos helps ensure that committed writes survive replica failures and that a majority quorum decides the ordered history for each key range.

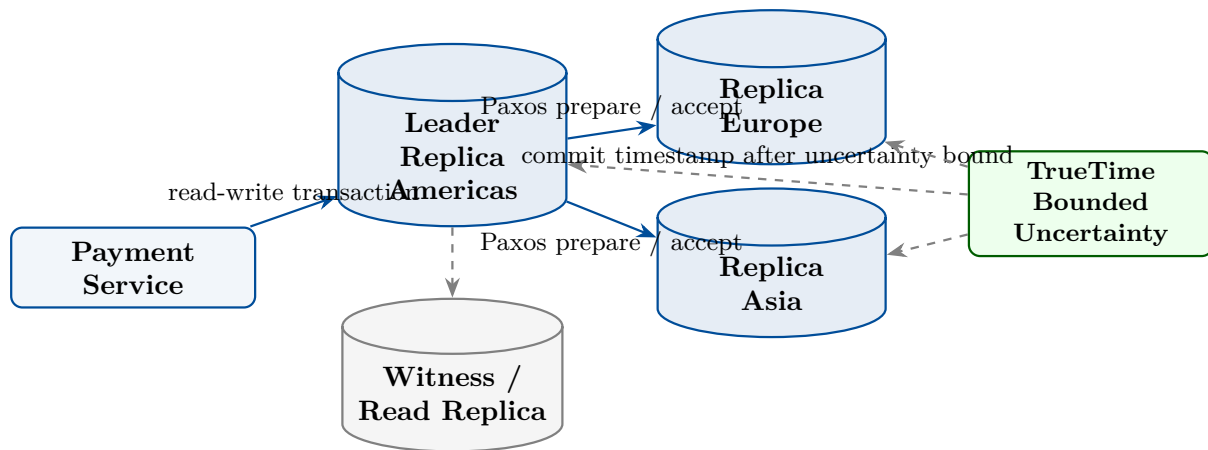


Figure 3.1: Spanner combines Paxos replication with TrueTime commit timestamps to provide globally ordered transactions.

3.2.3 TrueTime and External Consistency

TrueTime exposes a time interval rather than a single perfect clock reading. The interval contains the real current time with bounded uncertainty. Spanner uses this bound when assigning commit timestamps. Before a transaction is externally visible, Spanner waits long enough to ensure that later transactions receive later timestamps. This commit-wait technique is one reason Spanner can provide external consistency across distributed replicas.

Definition

External consistency is stronger than ordinary serializability. It preserves real-time ordering: if one transaction commits before another transaction begins, the second transaction is ordered after the first.

Strong global consistency is especially valuable for ledgers, entitlements, inventory allocation, identity state, and idempotency records. Without it, engineers often add application-level locks, reconciliation jobs, and regional conflict resolution. Spanner does not make latency disappear, but it gives a precise consistency contract that simplifies correctness-sensitive designs.

Common Pitfall

Do not choose Spanner only because a database is large. Choose it when the workload needs horizontal relational scale, high availability, strong consistency, or global distribution. For one-region departmental databases, Cloud SQL or AlloyDB may be simpler and cheaper.

3.3 Tuesday: Schema Design, Primary Keys, Interleaving, and Indexing

3.3.1 Primary Keys Define Physical Locality

In Spanner, primary-key order is physical design. Rows with nearby primary-key values tend to live near each other until the range splits. This helps range scans and parent-child access patterns, but it can hurt write scalability when the leading key is monotonically increasing. For example, using a timestamp as the first primary-key column can concentrate all new writes into the newest range. Better designs place a high-cardinality business identifier, hash bucket,

tenant identifier, or region-aware prefix before time when write distribution matters.

Tip

Start primary-key design from access patterns and write distribution. Ask: Which queries must be fast? Which rows are read together? Which column changes monotonically? Which tenant, account, or merchant could become hot?

3.3.2 Interleaved Tables

Interleaved tables physically colocate child rows with a parent row by sharing the parent primary-key prefix. An order table can be interleaved under a customer table; order lines can be interleaved under orders. This is useful when child rows are usually read with their parent and when the child lifecycle follows the parent lifecycle. It is less useful for independent child records that are frequently scanned by a different access path.

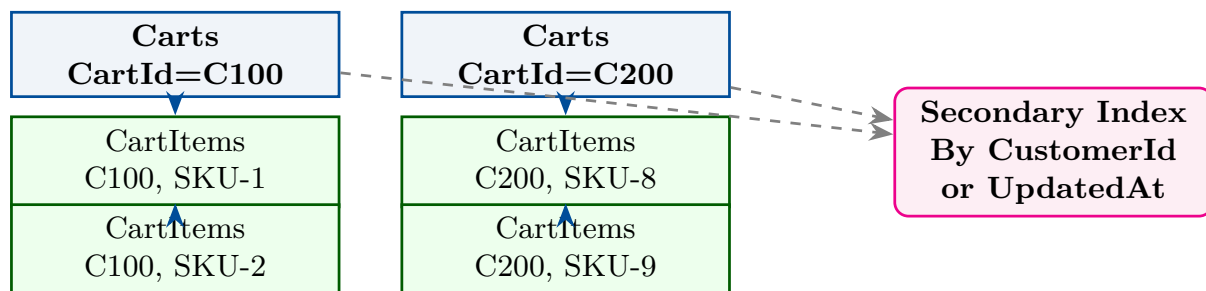


Figure 3.2: Interleaving stores child rows near parent rows while secondary indexes support alternate access paths.

Note

Interleaving is not the same as a foreign key. Foreign keys enforce referential integrity. Interleaving influences physical locality and optional cascade behavior. A design can use one, both, or neither depending on access and integrity requirements.

3.3.3 Secondary Indexes and Covering Reads

Secondary indexes provide alternate sort orders and lookup paths. They are essential when queries filter by columns that are not the leading primary-key columns. However, every index is another structure that must be maintained synchronously during writes. Too few indexes cause full scans; too many indexes increase write cost and storage. A covering index can store additional columns so Spanner can answer a query from the index without joining back to the base table.

Design choice	Best fit	Caution
Random or hashed leading key	High write distribution for large tenants or accounts	Range queries may need a secondary index or bucket fan-out
Tenant then entity key	Multi-tenant isolation and tenant-scoped reads	One very large tenant can still become hot
Timestamp suffix	Recent-history scans inside a parent or tenant	Timestamp as leading key can hot spot writes
Interleaved child table	Parent-child reads and lifecycle coupling	Deep hierarchies and large parents can concentrate traffic
Covering secondary index	Low-latency reads by alternate predicate	Extra write latency, storage, and consistency cost

Table 3.1: Spanner schema design trade-offs.

3.4 Wednesday: Performance Tuning and Plans

3.4.1 Reading Query Plans

Spanner query plans show scans, joins, distributed unions, filters, sorting, and row counts. A healthy point lookup uses the primary key or a selective index. A suspicious plan scans far more rows than it returns, sorts large intermediate results, or performs distributed work for a query that should be tenant-local. Use the Google Cloud console, query statistics tables, and the command-line tools to compare the plan with the intended access path [24].

Definition

Selectivity describes how much a predicate narrows the candidate rows. A highly selective predicate, such as a unique payment identifier, should use a key or index. A low-selectivity predicate, such as status equals open across all merchants, may still scan many rows.

3.4.2 Hot Spots and Split Health

Hot spots happen when a small key range receives a disproportionate share of reads or writes. Common causes include monotonically increasing primary keys, a single global counter row, a popular tenant with no distribution strategy, or an index whose leading column has low cardinality. Key Visualizer helps detect hot key ranges, while metrics expose CPU, latency, lock waits, and transaction aborts [47].

Common Pitfall

A secondary index can hot spot even when the base table is well distributed. If every new transaction writes an index entry with the same leading status or current timestamp range, the index becomes the bottleneck.

3.4.3 Transactions, Mutations, and Latency

Read-write transactions acquire locks and coordinate commits through the leader. Keep them short, deterministic, and limited to rows that must be atomically changed together. Large analytical reads, export jobs, and backfills should use partitioned reads, Dataflow connectors, or BigQuery exports rather than long transactions. For idempotent ingestion, store request identifiers and business keys so retries can safely detect already committed work.

Tip

Tune from evidence. Capture the SQL text, parameters, query plan, latency percentile, rows scanned, rows returned, CPU, lock wait, and index choices before changing schema. Then compare the same evidence after the change.

3.5 Thursday Hands-On Project: Multi-Region Cart Lab

This lab creates a multi-region Spanner instance, defines an interleaved cart schema, and loads 10,000 synthetic cart-item records. Use a sandbox project, replace placeholders, and delete resources when finished. Commands are written for Windows PowerShell.

Note

Multi-region Spanner instances can incur meaningful cost. Use the smallest capacity suitable for the lab, set a cleanup reminder, and verify that no production project or billing account is used.

3.5.1 PowerShell Preparation and Instance Creation

```

1 $env:PROJECT_ID = "my-gcp-spanner-lab"
2 $env:INSTANCE_ID = "cart-global-lab"
3 $env:DATABASE_ID = "cartdb"
4 $env:INSTANCE_CONFIG = "nam3"
5 $env:PROCESSING_UNITS = "1000"
6
7 gcloud config set project $env:PROJECT_ID
8
9 gcloud services enable spanner.googleapis.com
10
11 gcloud spanner instances create $env:INSTANCE_ID `
12   --config=$env:INSTANCE_CONFIG `
13   --description="Global cart lab" `
14   --processing-units=$env:PROCESSING_UNITS
15
16 gcloud spanner databases create $env:DATABASE_ID `
17   --instance=$env:INSTANCE_ID

```

Listing 3.1: Creating a multi-region Cloud Spanner instance from PowerShell.

3.5.2 Interleaved Cart Schema

```

1 $ddl = @'
2 CREATE TABLE Carts (
3   CartId STRING(36) NOT NULL,
4   CustomerId STRING(36) NOT NULL,
5   HomeRegion STRING(16) NOT NULL,
6   Status STRING(16) NOT NULL,
7   CreatedAt TIMESTAMP NOT NULL OPTIONS (allow_commit_timestamp=true),
8   UpdatedAt TIMESTAMP NOT NULL OPTIONS (allow_commit_timestamp=true),
9 ) PRIMARY KEY (CartId);
10
11 CREATE TABLE CartItems (
12   CartId STRING(36) NOT NULL,
13   LineId STRING(36) NOT NULL,
14   Sku STRING(64) NOT NULL,
15   Quantity INT64 NOT NULL,
16   UnitPrice NUMERIC NOT NULL,
17   AddedAt TIMESTAMP NOT NULL OPTIONS (allow_commit_timestamp=true),

```



```

18 ) PRIMARY KEY (CartId, LineId),
19     INTERLEAVE IN PARENT Carts ON DELETE CASCADE;
20
21 CREATE INDEX CartsByCustomerUpdated
22 ON Carts(CustomerId, UpdatedAt DESC)
23 STORING (Status, HomeRegion);
24 '@
25
26 $ddl | gcloud spanner databases ddl update $env:DATABASE_ID '
27     --instance=$env:INSTANCE_ID '
28     --ddl-file=-

```

Listing 3.2: Creating an interleaved e-commerce cart schema with DDL.

3.5.3 Python Synthetic Load

The Python client uses batches to insert 1,000 carts and 10,000 cart items. In production, prefer service-account credentials, workload identity, and controlled batch sizes matched to latency and mutation limits.

```

1 import os
2 import random
3 import uuid
4 from decimal import Decimal
5
6 from google.cloud import spanner
7 from google.cloud.spanner_v1 import COMMIT_TIMESTAMP
8
9
10 PROJECT_ID = os.environ["PROJECT_ID"]
11 INSTANCE_ID = os.environ.get("INSTANCE_ID", "cart-global-lab")
12 DATABASE_ID = os.environ.get("DATABASE_ID", "cartdb")
13 CART_COUNT = 1000
14 ITEMS_PER_CART = 10
15 REGIONS = ["americas", "europe", "asia"]
16 STATUSES = ["ACTIVE", "ABANDONED", "CHECKED_OUT"]
17
18
19 def chunked(values, size):
20     for start in range(0, len(values), size):
21         yield values[start:start + size]
22
23
24 def main() -> None:
25     client = spanner.Client(project=PROJECT_ID)
26     instance = client.instance(INSTANCE_ID)
27     database = instance.database(DATABASE_ID)
28
29     carts = []
30     items = []
31     for cart_number in range(CART_COUNT):
32         cart_id = str(uuid.uuid4())
33         customer_id = str(uuid.uuid4())
34         carts.append((
35             cart_id,
36             customer_id,
37             random.choice(REGIONS),
38             random.choice(STATUSES),
39             COMMIT_TIMESTAMP,
40             COMMIT_TIMESTAMP,
41         ))
42         for line_number in range(ITEMS_PER_CART):
43             items.append((

```

```

44         cart_id,
45         f"line-{{line_number:02d}}",
46         f"SKU-{{random.randint(1, 5000):05d}}",
47         random.randint(1, 5),
48         Decimal(random.randrange(100, 25000)) / Decimal("100"),
49         COMMIT_TIMESTAMP,
50     ))
51
52     with database.batch() as batch:
53         batch.insert(
54             table="Carts",
55             columns=("CartId", "CustomerId", "HomeRegion", "Status", "CreatedAt",
56                     "UpdatedAt"),
57             values=carts,
58         )
59
60     for item_chunk in chunked(items, 1000):
61         with database.batch() as batch:
62             batch.insert(
63                 table="CartItems",
64                 columns=("CartId", "LineId", "Sku", "Quantity", "UnitPrice", "
65                         AddedAt"),
66                 values=item_chunk,
67             )
68
69     print({"carts": len(carts), "cart_items": len(items)})
70
71 if __name__ == "__main__":
72     main()

```

Listing 3.3: Loading 10,000 synthetic records into Cloud Spanner with google-cloud-spanner.

3.5.4 Query Plan Check and Cleanup

```

1 gcloud spanner databases execute-sql $env:DATABASE_ID '
2     --instance=$env:INSTANCE_ID '
3     --sql="SELECT CartId, Status FROM Carts WHERE CustomerId = 'replace-customer-
4         id'"
5
6 gcloud spanner databases execute-sql $env:DATABASE_ID '
7     --instance=$env:INSTANCE_ID '
8     --query-mode=PLAN '
9     --sql="SELECT c.CartId, i.Sku, i.Quantity FROM Carts c JOIN CartItems i USING
10         (CartId) WHERE c.CartId = 'replace-cart-id'"

```

Listing 3.4: Checking sample query plans and deleting lab resources.

3.6 Friday Use Case: Global Payment Ledger

3.6.1 Scenario

A payment processor authorizes purchases in North America, Europe, and Asia. Merchants require low-latency authorization, regulators require immutable ledger evidence, and the business requires 99.999% availability with zero data loss across continents. A regional outage must not lose committed debits, credits, holds, refunds, or idempotency records. The ledger must support strict account balance correctness, globally unique transaction identifiers, and audit reads by compliance teams.

3.6.2 Architecture Pattern

The core ledger belongs in a multi-region Spanner instance. Payment APIs write idempotency records, ledger entries, and account-balance deltas in read-write transactions. Regional service tiers route requests to nearby endpoints, but the database maintains one globally consistent transaction history. Pub/Sub and Dataflow can stream committed ledger facts to BigQuery for analytics, while Cloud Monitoring and audit logs provide operational evidence.

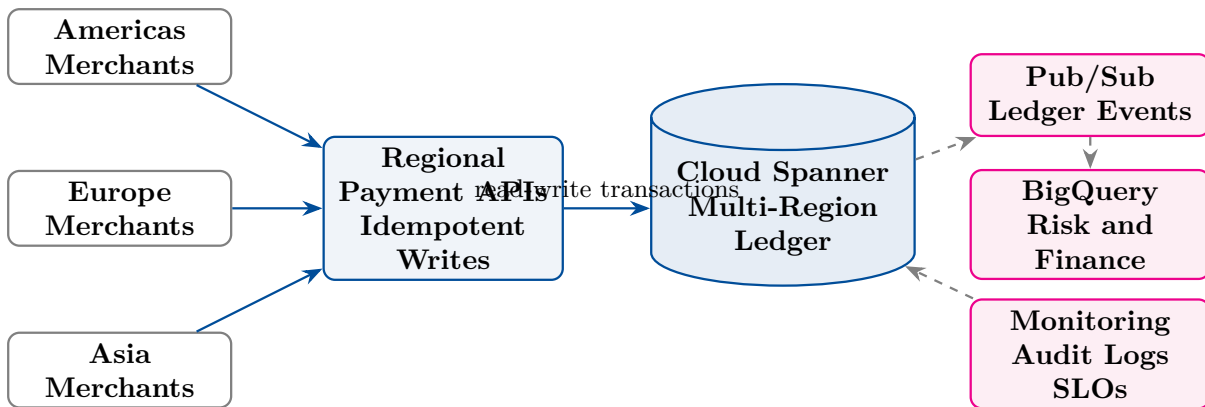


Figure 3.3: A multi-continent payment ledger uses Spanner for globally consistent OLTP and streams committed facts to analytics systems.

3.6.3 Correctness Controls

Ledger transactions should be append-only where possible. Instead of updating historical entries, write reversing entries and preserve immutable facts. Use a unique payment request identifier for idempotency, a ledger-entry table keyed by account and commit timestamp or sequence, and balance rows updated transactionally only when the invariant must be enforced immediately. For analytics, replicate committed facts downstream rather than running heavy reports against the OLTP leader.

Tip

For payment systems, availability is not enough. The design must prove no committed transaction is lost, no duplicate request is double-applied, and every balance can be reconstructed from the ledger.

3.7 Security, Operations, and Cost Notes

Use least-privilege IAM roles, database roles, audit logging, CMEK where required, VPC Service Controls for sensitive perimeters, and Secret Manager for application configuration. Monitor CPU utilization by priority, storage, leader regions, request latency, transaction aborts, lock wait, query latency, rows scanned, and backup status. Use change streams or exports for downstream analytics rather than broad scans on the transactional database.

Note

Spanner cost is driven by provisioned capacity, storage, backups, change streams, network traffic, and inefficient queries. Because capacity is always on, label instances and export billing data so global OLTP cost is visible by product, environment, and tenant.

3.8 Interview Q&A

3.8.1 Concepts and Trade-offs

1. **Q: What problem does TrueTime solve in Spanner?**

A: TrueTime gives Spanner a bounded uncertainty interval for real time. Spanner uses commit wait so transaction timestamps respect real-time ordering, enabling external consistency.

2. **Q: Why can a monotonically increasing primary key hurt Spanner performance?**

A: New writes concentrate at the end of one key range, creating a hot split. A higher-cardinality leading key, hash bucket, or tenant-aware design distributes writes better.

3. **Q: When should you use an interleaved table?**

A: Use interleaving when child rows are usually read with the parent and share the parent lifecycle. Avoid it when children are independently scanned or one parent can become extremely large and hot.

4. **Q: How is Spanner different from Cloud SQL read replicas?**

A: Cloud SQL replicas are asynchronous read copies of a primary database. Spanner synchronously replicates data with consensus and exposes strong consistency across distributed replicas.

5. **Q: What evidence would you review before adding an index?**

A: Query text, parameters, execution plan, rows scanned, rows returned, latency, CPU, lock wait, and current index usage. Add the smallest index that improves the measured access path.

3.9 Further Reading

Read the Cloud Spanner documentation for current instance, database, schema, query, backup, and operations guidance [23]. Review schema design best practices before choosing primary keys, interleaving, and indexes [25]. Study query execution plans and optimizer documentation for performance tuning [24]. Use Key Visualizer and monitoring documentation to identify hot spots and operational anomalies [47]. Read the original Spanner paper to understand the research foundations behind TrueTime, Paxos, and external consistency [3].

Key Takeaways

- Spanner is a globally consistent relational database, not a larger single-node database.
- TrueTime and Paxos let Spanner provide externally consistent transactions across distributed replicas.
- Primary keys define physical locality, so schema design directly affects scalability and latency.
- Interleaved tables colocate parent-child rows, while secondary indexes provide alternate access paths at synchronous write cost.
- Query plans, Key Visualizer, metrics, and transaction statistics should guide performance tuning.
- Multi-region Spanner is a strong fit for ledgers, entitlements, and other global systems that require zero data loss and strong consistency.

3.10 Exercises

1. **(Conceptual)** Explain external consistency and how it differs from ordinary eventual consistency.
2. **(Conceptual)** Compare regional Spanner, multi-region Spanner, Cloud SQL HA, and a traditional sharded relational database.
3. **(Design)** Propose a primary key for a global payments table and explain how it avoids write hot spots.
4. **(Design)** Decide whether carts and cart items should be interleaved for three different e-commerce access patterns.
5. **(Hands-on)** Create a sandbox multi-region Spanner instance and verify its instance configuration.
6. **(Hands-on)** Load 10,000 synthetic cart records and inspect a query plan for a cart lookup.
7. **(Performance)** Identify two indexes that could hot spot and redesign their leading columns.
8. **(Architecture)** Extend the payment-ledger use case with monitoring, audit logging, backup, restore, and analytics export requirements.

3.11 Capstone Project: Globally Consistent Payment Ledger on Cloud Spanner

Capstone Project

Mission: Design and prototype a globally consistent payment ledger on multi-region Cloud Spanner. Your solution must preserve zero data loss for committed transactions, support idempotent payment requests, survive a continental outage scenario, and produce auditable evidence for finance, risk, and compliance teams.

Estimated time: 6–8 hours.

What you'll practice:

- Creating a multi-region Spanner instance and payment-ledger database.
- Designing account, payment request, ledger entry, and balance tables with scalable primary keys.
- Implementing idempotent read-write transactions in Python.
- Creating secondary indexes for operational lookup without hot-spotting writes.
- Defining monitoring, backup, restore, and analytics export evidence for zero data loss.
- Producing a reference architecture, implementation transcript, acceptance criteria, and residual-risk statement.

Scenario

Globex Payments processes merchant card and wallet transactions in the Americas, Europe, and Asia. The current platform keeps one primary relational database per continent and reconciles conflicts overnight. Duplicate authorizations, delayed refunds, and inconsistent account holds have become unacceptable. Leadership wants one globally consistent ledger that can prove every committed debit and credit survives regional failures.

Requirements

- Deploy a multi-region Cloud Spanner instance appropriate for a production payment ledger.
- Design tables for accounts, payment requests, ledger entries, account balances, and audit checkpoints.
- Enforce idempotency so retried authorization requests cannot double-post.
- Preserve zero data loss for committed transactions through synchronous replication and tested backups.
- Support merchant, account, request, and time-range lookups with measured indexes and query plans.
- Export committed ledger facts to BigQuery or another analytical system without overloading OLTP transactions.
- Document IAM, database roles, CMEK, audit logging, monitoring, alert thresholds, and incident runbooks.
- Provide validation evidence for balance reconstruction, duplicate suppression, transaction ordering, and restore testing.

Reference Architecture

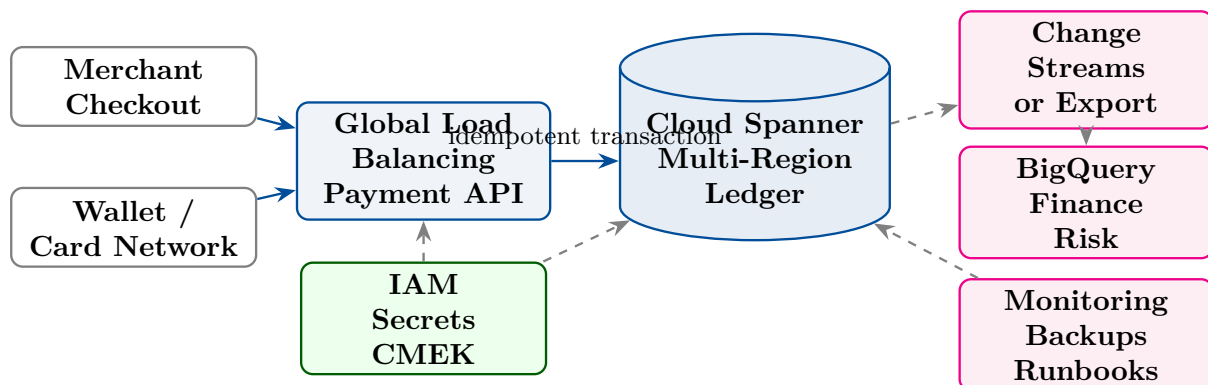


Figure 3.4: Capstone reference architecture for a globally consistent payment ledger on multi-region Spanner.

Implementation Steps

1. **Create the Spanner instance, database, and ledger schema.** Replace all placeholders and use a sandbox project first.

```

1 $env:PROJECT_ID = "my-gcp-ledger-capstone"
2 $env:INSTANCE_ID = "global-ledger-prod"
3 $env:DATABASE_ID = "ledgerdb"
4 $env:INSTANCE_CONFIG = "nam-eur-asia1"
5 $env:PROCESSING_UNITS = "1000"
6
7 gcloud config set project $env:PROJECT_ID
8 gcloud services enable spanner.googleapis.com
9
10 gcloud spanner instances create $env:INSTANCE_ID '

```

```

11  --config=$env:INSTANCE_CONFIG '
12  --description="Globally consistent payment ledger" '
13  --processing-units=$env:PROCESSING_UNITS
14
15  gcloud spanner databases create $env:DATABASE_ID '
16  --instance=$env:INSTANCE_ID
17
18  $ddl = @'
19  CREATE TABLE Accounts (
20    AccountId STRING(36) NOT NULL,
21    MerchantId STRING(36) NOT NULL,
22    HomeRegion STRING(16) NOT NULL,
23    Currency STRING(3) NOT NULL,
24    CreatedAt TIMESTAMP NOT NULL OPTIONS (allow_commit_timestamp=true),
25  ) PRIMARY KEY (AccountId);
26
27  CREATE TABLE PaymentRequests (
28    RequestId STRING(64) NOT NULL,
29    AccountId STRING(36) NOT NULL,
30    MerchantId STRING(36) NOT NULL,
31    Amount NUMERIC NOT NULL,
32    Currency STRING(3) NOT NULL,
33    Status STRING(16) NOT NULL,
34    CreatedAt TIMESTAMP NOT NULL OPTIONS (allow_commit_timestamp=true),
35  ) PRIMARY KEY (RequestId);
36
37  CREATE TABLE LedgerEntries (
38    AccountId STRING(36) NOT NULL,
39    EntryId STRING(36) NOT NULL,
40    RequestId STRING(64) NOT NULL,
41    Direction STRING(6) NOT NULL,
42    Amount NUMERIC NOT NULL,
43    Currency STRING(3) NOT NULL,
44    CommittedAt TIMESTAMP NOT NULL OPTIONS (allow_commit_timestamp=true),
45  ) PRIMARY KEY (AccountId, EntryId),
46    INTERLEAVE IN PARENT Accounts ON DELETE NO ACTION;
47
48  CREATE TABLE AccountBalances (
49    AccountId STRING(36) NOT NULL,
50    Currency STRING(3) NOT NULL,
51    AvailableBalance NUMERIC NOT NULL,
52    UpdatedAt TIMESTAMP NOT NULL OPTIONS (allow_commit_timestamp=true),
53  ) PRIMARY KEY (AccountId, Currency);
54
55  CREATE INDEX PaymentRequestsByMerchant
56  ON PaymentRequests(MerchantId, CreatedAt DESC)
57  STORING (Status, Amount, Currency);
58  '@
59
60  $ddl | gcloud spanner databases ddl update $env:DATABASE_ID '
61  --instance=$env:INSTANCE_ID '
62  --ddl-file=-

```

Listing 3.5: Capstone Spanner payment-ledger database and schema creation.

2. **Prototype idempotent posting logic.** The transaction first checks the request identifier, then writes the request, ledger entry, and balance update atomically.

```

1  import os
2  import uuid
3  from decimal import Decimal
4
5  from google.cloud import spanner
6  from google.cloud.spanner_v1 import COMMIT_TIMESTAMP

```

```

7
8
9 PROJECT_ID = os.environ["PROJECT_ID"]
10 INSTANCE_ID = os.environ.get("INSTANCE_ID", "global-ledger-prod")
11 DATABASE_ID = os.environ.get("DATABASE_ID", "ledgerdb")
12
13
14 def post_payment(transaction, request_id, account_id, merchant_id, amount,
15 currency):
16     existing = list(transaction.execute_sql(
17         "SELECT Status FROM PaymentRequests WHERE RequestId = @request_id",
18         params={"request_id": request_id},
19         param_types={"request_id": spanner.param_types.STRING},
20     ))
21     if existing:
22         return {"request_id": request_id, "status": existing[0][0], "
23 duplicate": True}
24
25     entry_id = str(uuid.uuid4())
26     balance_rows = list(transaction.execute_sql(
27         """
28         SELECT AvailableBalance
29         FROM AccountBalances
30         WHERE AccountId = @account_id AND Currency = @currency
31         """,
32         params={"account_id": account_id, "currency": currency},
33         param_types={
34             "account_id": spanner.param_types.STRING,
35             "currency": spanner.param_types.STRING,
36         },
37     ))
38     current_balance = balance_rows[0][0] if balance_rows else Decimal("0")
39     new_balance = current_balance + amount
40
41     transaction.insert(
42         table="PaymentRequests",
43         columns=("RequestId", "AccountId", "MerchantId", "Amount", "
44 Currency", "Status", "CreatedAt"),
45         values=[(request_id, account_id, merchant_id, amount, currency, "
46 POSTED", COMMIT_TIMESTAMP)],
47     )
48     transaction.insert(
49         table="LedgerEntries",
50         columns=("AccountId", "EntryId", "RequestId", "Direction", "Amount"
51 , "Currency", "CommittedAt"),
52         values=[(account_id, entry_id, request_id, "CREDIT", amount,
53 currency, COMMIT_TIMESTAMP)],
54     )
55     transaction.insert_or_update(
56         table="AccountBalances",
57         columns=("AccountId", "Currency", "AvailableBalance", "UpdatedAt"),
58         values=[(account_id, currency, new_balance, COMMIT_TIMESTAMP)],
59     )
60     return {"request_id": request_id, "status": "POSTED", "duplicate":
61 False}
62
63
64 def main() -> None:
65     client = spanner.Client(project=PROJECT_ID)
66     database = client.instance(INSTANCE_ID).database(DATABASE_ID)
67     result = database.run_in_transaction(
68         post_payment,
69         request_id="merchant-123-order-456",

```



```

63     account_id="acct-001",
64     merchant_id="merchant-123",
65     amount=Decimal("42.50"),
66     currency="USD",
67 )
68 print(result)
69
70
71 if __name__ == "__main__":
72     main()

```

Listing 3.6: Idempotent payment posting with a Spanner read-write transaction.

3. **Capture operations evidence.** Record query plans, backup configuration, and restore-test notes as part of the project transcript.

```

1 gcloud spanner databases execute-sql $env:DATABASE_ID '
2   --instance=$env:INSTANCE_ID '
3   --query-mode=PLAN '
4   --sql="SELECT RequestId, Status FROM PaymentRequests WHERE MerchantId = '
5       merchant-123' ORDER BY CreatedAt DESC LIMIT 20"
6
6 gcloud spanner backups create ledgerdb-backup-$(Get-Date -Format
7   yyyyMMddHHmm) '
8   --instance=$env:INSTANCE_ID '
9   --database=$env:DATABASE_ID '
10  --retention-period=7d
11
11 gcloud spanner backups list --instance=$env:INSTANCE_ID

```

Listing 3.7: Operational evidence commands for the payment-ledger capstone.

Deliverables

- A one-page architecture summary explaining why Spanner is required instead of Cloud SQL, AlloyDB, or a sharded RDBMS.
- The completed reference diagram or an equivalent diagram showing merchants, payment APIs, Spanner, security controls, analytics export, monitoring, and backups.
- DDL and command transcript proving instance creation, schema deployment, indexes, backup configuration, and cleanup plan.
- Python or pseudocode evidence for idempotent posting and duplicate-request suppression.
- Query-plan evidence for merchant lookup, account ledger lookup, and balance reconstruction.
- A zero-data-loss runbook covering regional failure, restore testing, alert response, and post-incident reconciliation.

Acceptance Criteria

- The Spanner instance uses a multi-region configuration and an explicit capacity plan.
- Primary keys distribute writes while preserving efficient account-ledger and merchant-request access.
- Payment posting is idempotent and writes request, ledger, and balance records in one read-write transaction.
- The design documents how committed transactions survive replica or regional failures.

- Validation reconstructs balances from ledger entries and proves duplicate requests are not double-applied.
- Monitoring covers latency percentiles, abort rates, CPU, lock wait, storage, hot splits, backup status, and failed exports.
- Analytics uses change streams, export, or controlled replication rather than unbounded OLTP scans.
- Secrets, credentials, payment identifiers, and customer data are protected in commands, logs, and screenshots.

Stretch Goals

- Add change streams from Spanner into Dataflow and BigQuery for near-real-time risk scoring.
- Implement double-entry accounting with debit and credit entries that must balance within each transaction.
- Add a reconciliation notebook that rebuilds account balances from immutable ledger entries.
- Create a synthetic load test that measures p50, p95, and p99 authorization latency by region.
- Add a backup restore rehearsal into a separate Spanner instance and validate row counts and ledger totals.
- Estimate monthly cost for capacity, storage, backups, change streams, network transfer, monitoring, and analytics export.

Wide-Column NoSQL: Cloud Bigtable

Learning Objectives

- Explain Cloud Bigtable architecture, including instances, clusters, nodes, tablets, column families, and app profiles.
- Design row keys that distribute high-throughput reads and writes while preserving efficient access patterns.
- Apply salting, hashing, reversed timestamps, and entity-prefix layouts to avoid hotspotting.
- Tune Bigtable performance with node sizing, schema choices, garbage collection policies, and Key Visualizer.
- Deploy a Bigtable cluster and ingest IoT temperature-sensor readings with a Python script and reversed-timestamp row key.
- Design an ingestion and serving layer for a financial trading platform processing 100,000 stock ticks per second.
- Complete the Part I capstone by integrating Cloud SQL, Cloud Spanner, Cloud Bigtable, and Cloud Storage behind secure private connectivity.

Cross-Cloud Equivalence

Wide-column and high-throughput NoSQL systems expose different APIs, but the core design questions are consistent: partition key, row-key order, hot-range prevention, throughput scaling, consistency, and operational visibility.

Capability	GCP Bigtable	AWS	Azure	Apache HBase / Cassandra
Wide-column model	Sparse rows, column families, cells, timestamps, Bigtable API	DynamoDB items and attributes; Keyspaces uses Cassandra tables	Cosmos DB Cassandra API; Table Storage entities with partition and row keys	HBase tables and column families; Cassandra wide rows and clustering columns
Row-key / partition design	Row key controls tablet locality and scan order	DynamoDB partition key and sort key; Keyspaces partition key	Cosmos partition key and clustering; Table Storage partition key plus row key	HBase row key regions; Cassandra partition key and clustering order
Hotspotting mitigation	Salting, hashing, bucketing, reversed timestamps, app-profile routing	Write sharding, adaptive capacity, random prefixes, capacity planning	Synthetic partition keys, partition spread, RU planning	Region pre-splitting, token distribution, bucketing, compaction-aware modeling
Tablets vs partitions	Tablets are contiguous row-key ranges served by nodes and split automatically	DynamoDB partitions and Keyspaces storage partitions distribute load	Logical partitions map to physical partitions	HBase regions and Cassandra token ranges distribute ownership
Throughput scaling	Add nodes per cluster; replicate clusters; route with app profiles	Provisioned or on-demand capacity; table and global secondary index limits	Request units and partition throughput limits	Add region servers or Cassandra nodes and rebalance tokens

MongoDB can serve high-volume document workloads, but it is not a direct wide-column replacement because its shard key, document model, and secondary-index behavior differ. Snowflake is excellent for analytical serving, yet it is not designed as a single-digit-millisecond operational key-value store.

Key Terms

Cloud Bigtable: Google Cloud’s managed, horizontally scalable wide-column database for low-latency, high-throughput operational and analytical workloads. **Tablet:** a contiguous range of row keys assigned to servers and split as data or traffic grows. **Row key:** the byte string that determines physical ordering, locality, and scan behavior in a Bigtable table. **Column family:** a grouping of columns that share storage, compression, and garbage collection policy. **App profile:** a routing and transaction policy that sends client traffic to one or more clusters in a Bigtable instance. **Hotspot:** a narrow row-key range or node that receives disproportionate reads or writes.

4.1 Week 4 Roadmap

Week 4 closes Part I by moving from relational infrastructure to high-throughput wide-column storage. Cloud Bigtable is the GCP service to evaluate when the workload needs predictable low latency at large scale, sparse data, very high write rates, and key-range access patterns rather than ad hoc relational joins [12]. It descends from the Bigtable system that influenced HBase, Cassandra, and many distributed storage systems [2].

Monday introduces Bigtable architecture: instances, clusters, nodes, tablets, column families, timestamps, replication, and app profiles. Tuesday focuses on row-key design, salting, hashing, and hotspot avoidance. Wednesday studies performance tuning, garbage collection, Key Visualizer, and operational evidence. Thursday builds a hands-on lab that ingests IoT temperature-

sensor data with reversed-timestamp row keys. Friday applies the same principles to a financial trading platform that must ingest and serve 100,000 stock ticks per second.

Definition

Cloud Bigtable is a managed wide-column store that sorts all rows lexicographically by row key. It is optimized for high-throughput point lookups, prefix scans, time-series patterns, feature stores, personalization, telemetry, and serving layers where query access is known before the schema is designed.

4.2 Monday: Bigtable Architecture, Tablets, Nodes, and App Profiles

4.2.1 Instances, Clusters, Nodes, Tables, and Column Families

A Bigtable instance is an administrative boundary that can contain one or more clusters. A cluster is a set of Bigtable nodes in one zone. Nodes provide CPU, memory, and network capacity for serving reads and writes. Tables live inside the instance, and each table stores rows ordered by row key. Columns are grouped into column families, and each cell can have multiple timestamped versions.

Unlike Cloud SQL or Spanner, Bigtable does not enforce relational constraints, joins, or multi-row SQL transactions. Its strength is serving known access patterns at massive scale. You model the table from the query first: identify the entity, time range, prefix scan, latest-state lookup, or reverse-time feed that the application must read quickly. Then choose a row key and column families that make that read path local and predictable.

Note

Bigtable is sparse. A row can have thousands of possible columns, but only stored cells consume space. This is useful for telemetry, feature vectors, metrics, and event attributes where different entities report different measurements.

4.2.2 Tablets and Node-Level Sharding

Bigtable stores each table as tablets. A tablet is a contiguous row-key range such as `sensor#03#...` through `sensor#04#...`. Tablets can split as the table grows, and Bigtable assigns tablets across nodes in a cluster. The row key therefore controls both scan locality and write distribution. If every new write lands at the same end of the keyspace, only a small number of tablets and nodes are busy even when the cluster has many nodes.

Definition

Lexicographic row-key order means that row keys are sorted byte by byte. Prefixes are powerful because they make range scans efficient, but low-cardinality or monotonically increasing prefixes can create hot tablets.

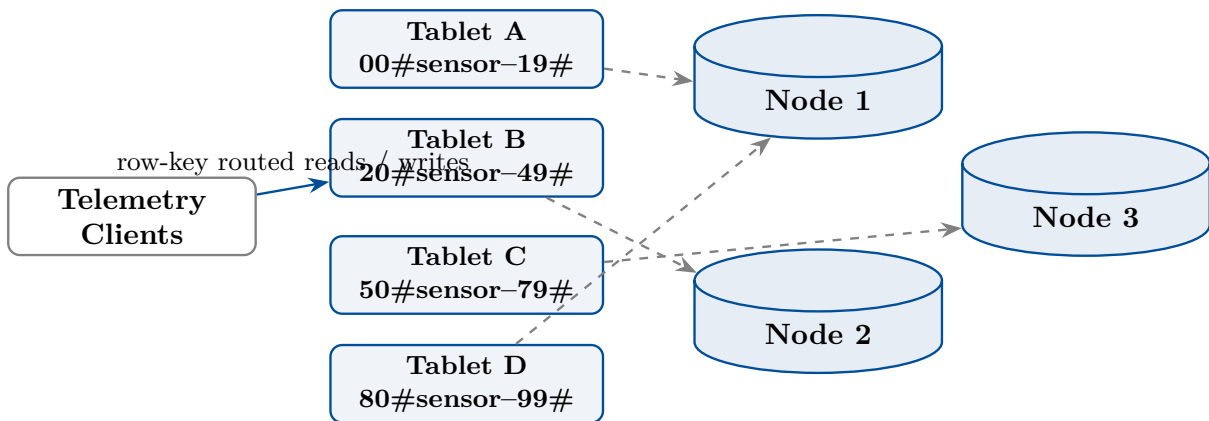


Figure 4.1: Bigtable splits a table into tablets, then serves those row-key ranges across cluster nodes.

4.2.3 Replication and App Profiles

A Bigtable instance can replicate data across clusters. Replication supports availability, locality, and workload isolation, but it also requires careful routing decisions. App profiles define whether a client routes to one cluster or multiple clusters and whether single-row transactions are allowed. For example, an ingestion app profile might send writes to a regional cluster near devices, while an analytics app profile might route batch reads to a separate cluster so scans do not compete with serving traffic.

Tip

Use app profiles as part of application design, not as an afterthought. Separate ingestion, serving, analytics, and backfill traffic when their latency and retry behavior differ.

4.3 Tuesday: Row-Key Design, Salting, Hashing, and Hotspot Avoidance

4.3.1 Design from Access Patterns

A Bigtable schema is successful when the row key makes the critical query a short point lookup or a bounded range scan. A row key such as `device#bucket#reverse_timestamp` works for recent events by device. A row key such as `symbol#bucket#reverse_timestamp` works for recent market ticks by symbol. A row key such as `tenant#hash#entity` works for multi-tenant entity lookup when tenants need isolation but large tenants must spread traffic.

The dangerous row key is often the obvious one. A key beginning with the current timestamp makes chronological scans simple, but it concentrates all new writes at the newest range. A key beginning with a single popular symbol, merchant, or device can also become hot. A good design balances three pressures: distributing writes, preserving read locality, and keeping the key understandable enough to operate.

Common Pitfall

Do not use an unbounded timestamp as the leading row-key component for high-volume ingestion. It usually creates a moving hot spot at the end of the table, even when the cluster has enough total nodes.

4.3.2 Salting, Hashing, Bucketing, and Reverse Time

Salting adds a small prefix such as 00 through 63 before the natural key. Hashing derives the prefix from stable fields such as device identifier or account identifier. Bucketing groups time into coarse windows such as hour or day. Reversed timestamps invert chronological order so the newest data appears first inside an entity prefix. These techniques are often combined: `hash#symbol#reversed_millis#trade_id` distributes writes while supporting recent-tick scans by symbol and bucket fan-out.

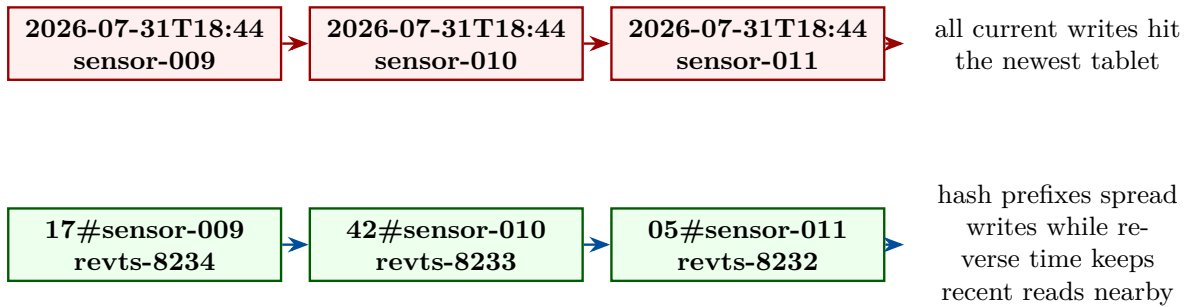


Figure 4.2: A timestamp-leading key hot spots current writes, while a salted reversed-timestamp key spreads ingestion and preserves recent-event scans.

4.3.3 Column Families and Cell Versions

Column families should group data with similar lifecycle, compression, and access behavior. A time-series table might place measurements in a `metrics` family and metadata in a `meta` family. A trading table might keep raw ticks in `tick`, derived indicators in `signal`, and audit ingestion markers in `audit`. Avoid creating many column families without a reason; each family has configuration and storage implications.

Note

Bigtable cells are timestamped. You can store multiple versions of a value, but garbage collection policy determines how many versions or how much history remains. Versioning is not a substitute for a deliberate event-history schema.

4.4 Wednesday: Performance Optimization, Garbage Collection, and Key Visualizer

4.4.1 Throughput, Nodes, and Workload Isolation

Bigtable performance depends on schema, cluster size, storage type, request mix, replication, and client behavior [13]. Adding nodes increases serving capacity, but it cannot fully compensate for a hot row key. Before scaling up, inspect CPU, latency, request count, error rate, storage utilization, and Key Visualizer heatmaps. If one range is hot while other nodes are idle, fix the row-key distribution first.

Workload isolation is equally important. A batch export or backfill that scans broad key ranges can affect latency for online serving. Use separate clusters, app profiles, scheduled windows, Dataflow connector throttling, and bounded scans. For high-throughput writes, batch mutations, reuse clients, use retry settings intentionally, and keep row mutations reasonably sized.

Tip

Tune in this order: verify the row key, verify the access path, isolate workload classes, tune client batching and retries, then add nodes. Scaling a poor key often hides the problem temporarily and raises cost permanently.

4.4.2 Garbage Collection Policies

Garbage collection policies control old cell versions. A max-age policy keeps cells younger than a specified duration. A max-versions policy keeps a fixed number of versions. Intersection and union policies combine those rules. Use them to align storage cost with retention requirements. For IoT readings, raw high-frequency telemetry may need only 30 days in Bigtable before being archived to Cloud Storage or BigQuery. For financial ticks, compliance may require immutable downstream storage even if the serving table retains only recent windows.

Definition

Bigtable garbage collection removes cells according to configured family policies during compaction. It is not an immediate delete guarantee for compliance workflows; design retention, export, and legal hold requirements explicitly.

4.4.3 Key Visualizer and Heatmap Evidence

Key Visualizer displays access patterns by row-key range over time [46]. Bright diagonal lines often indicate a moving timestamp hot spot. Vertical bands can indicate one hot prefix or tenant. Broad evenly distributed color suggests healthy spread. Use the heatmap together with Cloud Monitoring metrics because a quiet heatmap can still hide client errors, failed retries, or inefficient downstream processing.

Common Pitfall

A salted key requires read fan-out. If an application needs the latest 100 rows for a symbol and the table has 64 salt buckets, the service may need to query 64 prefixes and merge results. Use only as many buckets as the write rate requires.

Signal	Likely meaning	Response
High latency and one hot key range	Row-key hotspotting	Add hash or salt prefix, pre-split when appropriate, or re-design leading key
High CPU on all nodes	Cluster capacity pressure	Add nodes, reduce broad scans, batch requests, or isolate workloads
Storage growing faster than expected	Retention or version policy mismatch	Review garbage collection, archive raw history, and compress family design
Many failed retries	Client backoff or quota issue	Tune deadlines, retry policy, batching, and app-profile routing
Analytical scans hurting serving	Workload contention	Use a separate replicated cluster or route batch jobs with a different app profile

Table 4.1: Bigtable performance diagnosis patterns.

4.5 Thursday Hands-On Project: IoT Temperature Sensor Ingestion

This lab creates a single-cluster Bigtable instance, configures a table for temperature telemetry, and writes synthetic sensor events with reversed-timestamp row keys. Use a sandbox project, replace placeholders, and delete resources when finished. Commands are written for Windows PowerShell.

Note

Bigtable clusters can incur cost while running. Use a small development cluster, label resources, and delete the instance immediately after the lab unless you are intentionally keeping it for later exercises.

4.5.1 PowerShell Preparation and Cluster Creation

```

1 $env:PROJECT_ID = "my-gcp-bigtable-lab"
2 $env:INSTANCE_ID = "iot-telemetry-lab"
3 $env:CLUSTER_ID = "iot-telemetry-c1"
4 $env:ZONE = "us-central1-b"
5 $env:TABLE_ID = "temperature_readings"
6
7 gcloud config set project $env:PROJECT_ID
8 gcloud services enable bigtable.googleapis.com bigtableadmin.googleapis.com
9
10 gcloud bigtable instances create $env:INSTANCE_ID '
11   --display-name="IoT telemetry lab" '
12   --cluster=$env:CLUSTER_ID '
13   --cluster-zone=$env:ZONE '
14   --cluster-num-nodes=1 '
15   --cluster-storage-type=SSD
16
17 cbt -project $env:PROJECT_ID -instance $env:INSTANCE_ID createtable $env:
18   TABLE_ID
19 cbt -project $env:PROJECT_ID -instance $env:INSTANCE_ID createfamily $env:
20   TABLE_ID metrics
21 cbt -project $env:PROJECT_ID -instance $env:INSTANCE_ID createfamily $env:
22   TABLE_ID meta
23 cbt -project $env:PROJECT_ID -instance $env:INSTANCE_ID setgcpcpolicy $env:
24   TABLE_ID metrics maxage=30d

```

Listing 4.1: Creating a Bigtable instance, cluster, and telemetry table from PowerShell.

4.5.2 Python Ingestion Script

The script uses a two-digit hash bucket, sensor identifier, and reversed millisecond timestamp. Recent readings for one sensor can be found by scanning that sensor's bucketed prefixes, while writes from many sensors spread across the keyspace.

```

1 import hashlib
2 import os
3 import random
4 import time
5 from datetime import datetime, timezone
6
7 from google.cloud import bigtable
8
9
10 PROJECT_ID = os.environ["PROJECT_ID"]
11 INSTANCE_ID = os.environ.get("INSTANCE_ID", "iot-telemetry-lab")

```

```

12 TABLE_ID = os.environ.get("TABLE_ID", "temperature_readings")
13 MAX_TS = 9999999999999999
14 SENSOR_COUNT = 200
15 READINGS_PER_SENSOR = 25
16
17
18 def bucket_for(sensor_id: str, buckets: int = 64) -> str:
19     digest = hashlib.sha256(sensor_id.encode("utf-8")).hexdigest()
20     return f"{int(digest[:4], 16) % buckets:02d}"
21
22
23 def make_row_key(sensor_id: str, event_ms: int) -> bytes:
24     reverse_ms = MAX_TS - event_ms
25     return f"{bucket_for(sensor_id)}#{sensor_id}#{reverse_ms:013d}".encode("utf-8")
26
27
28 def main() -> None:
29     client = bigtable.Client(project=PROJECT_ID, admin=True)
30     instance = client.instance(INSTANCE_ID)
31     table = instance.table(TABLE_ID)
32     mutations = []
33
34     now_ms = int(time.time() * 1000)
35     for sensor_number in range(SENSOR_COUNT):
36         sensor_id = f"sensor-{sensor_number:04d}"
37         for offset in range(READINGS_PER_SENSOR):
38             event_ms = now_ms - offset * 60000 - random.randint(0, 5000)
39             row = table.direct_row(make_row_key(sensor_id, event_ms))
40             temperature = 18.0 + random.random() * 12.0
41             event_time = datetime.fromtimestamp(event_ms / 1000, tz=timezone.
42             utc)
43             row.set_cell("metrics", "temp_c", f"{temperature:.2f}")
44             row.set_cell("metrics", "humidity_pct", f"{random.uniform(30, 80)
45             :.1f}")
46             row.set_cell("meta", "sensor_type", "warehouse-thermistor")
47             row.set_cell("meta", "event_time", event_time.isoformat())
48             mutations.append(row)
49
50     table.mutate_rows(mutations)
51     print({"rows_written": len(mutations), "table": TABLE_ID})
52
53 if __name__ == "__main__":
54     main()

```

Listing 4.2: Ingesting IoT temperature readings with google-cloud-bigtable and reversed-timestamp row keys.

4.5.3 Read Check and Cleanup

```

1 cbt -project $env:PROJECT_ID -instance $env:INSTANCE_ID read $env:TABLE_ID
  count=10
2
3 python .\ingest-iot-bigtable.py
4
5 cbt -project $env:PROJECT_ID -instance $env:INSTANCE_ID read $env:TABLE_ID
  count=10
6
7 gcloud bigtable instances delete $env:INSTANCE_ID --quiet

```

Listing 4.3: Reading sample rows and deleting the Bigtable lab instance.

4.6 Friday Use Case: 100,000 Stock Ticks per Second

4.6.1 Scenario

A trading platform receives quotes and trades from multiple exchanges. The target rate is 100,000 stock ticks per second during market peaks. Downstream systems need immediate lookup of the latest ticks by symbol, short historical windows for charting, anomaly detection features, and replay into analytics. The platform must absorb bursts without putting the serving path or compliance archive at risk.

4.6.2 Architecture Pattern

Pub/Sub absorbs exchange events and decouples producers from consumers. Dataflow validates, normalizes, enriches, and writes tick rows to Bigtable using a salted symbol and reversed-timestamp key. Bigtable serves recent market data to APIs with single-digit-millisecond lookups and bounded prefix scans. Cloud Storage stores immutable raw feed files, while BigQuery receives curated analytical facts for surveillance, reporting, and model training.

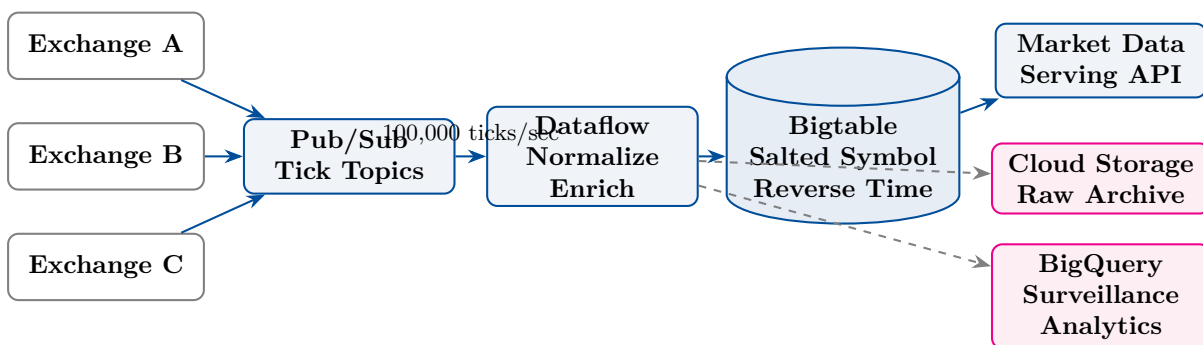


Figure 4.3: Trading ingestion uses Pub/Sub and Dataflow for burst absorption, Bigtable for low-latency serving, and analytical stores for archive and surveillance.

4.6.3 Serving and Correctness Controls

A practical tick row key is `bucket#symbol#reverse_event_nanos#exchange#sequence`. The bucket spreads writes, the symbol supports bounded reads, reverse time returns latest ticks first, and exchange sequence numbers support de-duplication. Use separate column families for raw tick fields, normalized fields, and derived indicators. Keep the serving table focused on recent windows; archive immutable raw feeds and replay checkpoints to Cloud Storage for audit and recovery.

Tip

For trading systems, prove both speed and replayability. Bigtable serves the current operational window, while Cloud Storage and BigQuery preserve raw history, reconstructed facts, and compliance evidence.

4.7 Security, Operations, and Cost Notes

Use least-privilege IAM, service accounts per workload, CMEK where required, VPC Service Controls for sensitive projects, audit logging, and private application paths where possible. Bigtable access is usually from application code, Dataflow, or GKE/GCE workloads using

IAM rather than database usernames. Monitor node CPU, latency percentiles, request errors, storage, replication lag, hot ranges, garbage collection policy, and client retry behavior.

Note

Bigtable cost is driven by nodes, storage type, stored bytes, backups, replication, network, and inefficient scans. A table that retains every raw version forever can become more expensive than the serving workload itself.

4.8 Interview Q&A

4.8.1 Concepts and Trade-offs

1. **Q: Why is row-key design the most important Bigtable schema decision?**

A: The row key determines physical sort order, tablet ranges, scan locality, and whether reads and writes distribute across nodes.

2. **Q: What is a tablet in Bigtable?**

A: A tablet is a contiguous row-key range that Bigtable can split, move, and serve independently across cluster nodes.

3. **Q: Why use a reversed timestamp in a time-series key?**

A: It places the newest events first inside an entity prefix, so recent-history reads can stop after a small range scan.

4. **Q: What is the downside of salting row keys?**

A: Salting spreads writes but can require read fan-out across buckets and result merging for entity-level queries.

5. **Q: How do app profiles help production systems?**

A: They route client traffic to specific clusters or multi-cluster policies, allowing workload isolation and locality-aware access.

6. **Q: When is Bigtable a poor fit?**

A: It is a poor fit for ad hoc joins, relational constraints, small departmental OLTP, and workloads without predictable key-based access paths.

4.9 Further Reading

Read the Cloud Bigtable documentation for current table, cluster, replication, backup, and administration guidance [12]. Study schema design guidance before committing to row keys and column families [14]. Review performance guidance for node sizing, workload isolation, and request patterns [13]. Use Key Visualizer documentation to interpret heatmaps and diagnose hot ranges [46]. Read the original Bigtable paper to understand the storage model and design lineage [2].

Key Takeaways

- Bigtable is a managed wide-column database for predictable, key-based access at very high throughput.
- Row keys determine locality and distribution; poor leading keys create hot tablets regardless of total cluster capacity.

- Salting, hashing, bucketing, and reversed timestamps trade read simplicity for write distribution and recent-event access.
- Column families should reflect access and retention patterns because garbage collection is configured at the family level.
- Key Visualizer, Cloud Monitoring, and client metrics should guide tuning before adding nodes.
- Bigtable often pairs with Pub/Sub, Dataflow, Cloud Storage, and BigQuery to separate ingestion, serving, archive, and analytics responsibilities.

4.10 Exercises

1. **(Conceptual)** Explain how a Bigtable tablet differs from a Spanner split and from a DynamoDB partition.
2. **(Conceptual)** Describe why Bigtable is not a relational database even though it stores tables, rows, and columns.
3. **(Design)** Propose a row key for a fleet of 2 million vehicles reporting location every five seconds.
4. **(Design)** Redesign a timestamp-leading key that is creating a moving hotspot during ingestion.
5. **(Hands-on)** Create the IoT lab table and write at least 1,000 readings using the Python ingestion script.
6. **(Performance)** Interpret a Key Visualizer heatmap that shows one vertical red band for three hours.
7. **(Operations)** Define garbage collection policies for raw telemetry, latest device state, and derived alert flags.
8. **(Architecture)** Extend the trading use case with disaster recovery, replay, monitoring, and cost controls.

4.11 Milestone 1 Capstone: E-Commerce Multi-Tier Data Backend

Capstone Project

Mission: Design and prototype the Part I e-commerce data backend. Integrate Cloud SQL for user accounts, Cloud Spanner for global inventory and checkout transactions, Cloud Bigtable for clickstream tracking, and Cloud Storage for product images with lifecycle management and CMEK. Secure the tiers with VPC peering, private services access, private IP, service accounts, and least-privilege IAM.

Estimated time: 8–10 hours.

What you'll practice:

- Mapping Week 1 Cloud Storage image buckets with lifecycle rules, uniform access, and CMEK.
- Using Week 2 Cloud SQL private IP for user-account and profile data.
- Using Week 3 Cloud Spanner for globally consistent inventory reservations and checkout transactions.

- Using Week 4 Cloud Bigtable for clickstream events, product impressions, and recent behavior serving.
- Designing VPC peering and private connectivity so application tiers do not depend on public database endpoints.
- Producing acceptance evidence across security, data modeling, operations, and cleanup.

Scenario

Mercury Retail runs an international e-commerce platform. User profile updates are regional and relational. Inventory reservations and checkout must remain globally consistent. Product images need durable object storage with lifecycle transitions and customer-managed encryption keys. Clickstream events arrive at high volume and must support recent-session personalization. Leadership wants a single reference design that proves Part I services can work together securely.

Requirements

- Store user accounts, hashed credentials, profile settings, and consent flags in Cloud SQL with private IP.
- Store global inventory, carts, checkout orders, and reservation transactions in Cloud Spanner.
- Store clickstream events in Cloud Bigtable using a salted user or session prefix and reversed timestamp.
- Store product images in Cloud Storage with uniform bucket-level access, lifecycle rules, versioning where appropriate, and CMEK.
- Use a shared application VPC with private services access or required service networking peering for Cloud SQL and other private managed-service paths.
- Use least-privilege service accounts for web API, checkout worker, image pipeline, and clickstream ingester.
- Document IAM, KMS keys, audit logs, monitoring dashboards, backup policies, and cleanup commands.
- Provide validation evidence for private connectivity, row-key distribution, Spanner transaction correctness, Cloud SQL backups, and Storage lifecycle configuration.

Reference Architecture

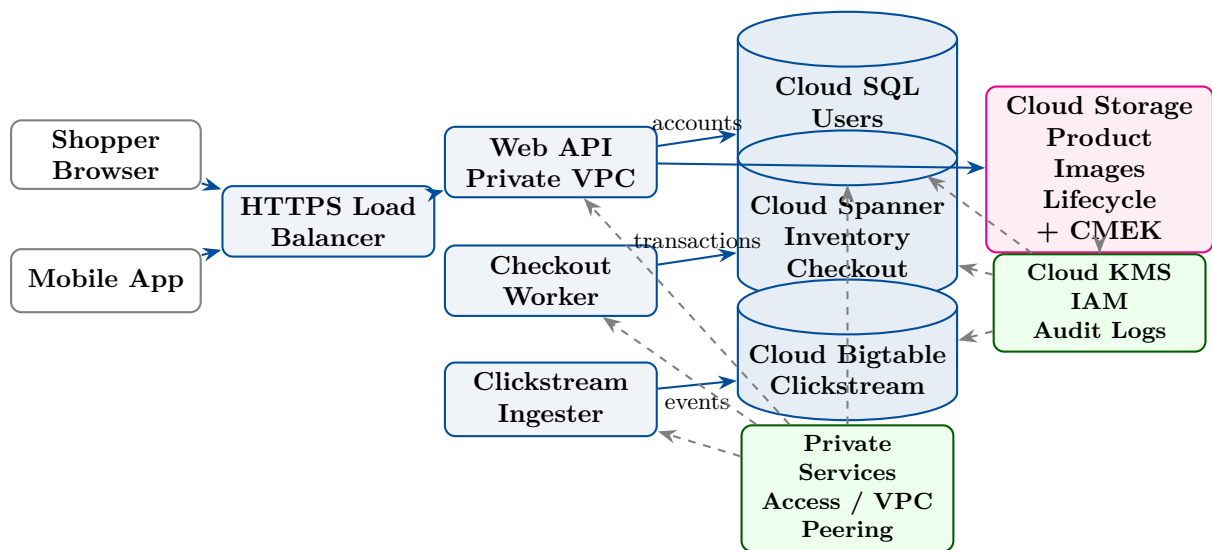


Figure 4.4: Milestone 1 reference architecture integrating Cloud Storage, Cloud SQL, Spanner, and Bigtable behind private application connectivity.

Implementation Steps

1. Create the shared network, private services access range, KMS key, image bucket, and managed data services. Replace placeholders and use a sandbox project.

```

1 $env:PROJECT_ID = "my-gcp-ecommerce-capstone"
2 $env:REGION = "us-central1"
3 $env:ZONE = "us-central1-b"
4 $env:VPC = "ecommerce-private-vpc"
5 $env:RANGE = "google-managed-services-range"
6 $env:KEYRING = "ecommerce-keys"
7 $env:KEY = "data-backend-key"
8 $env:BUCKET = "my-ecommerce-product-images"
9
10 gcloud config set project $env:PROJECT_ID
11 gcloud services enable compute.googleapis.com servicenetworking.googleapis.com '
12   sqladmin.googleapis.com spanner.googleapis.com bigtable.googleapis.com '
13   bigtableadmin.googleapis.com storage.googleapis.com cloudkms.googleapis.com
14
15 gcloud compute networks create $env:VPC --subnet-mode=custom
16 gcloud compute networks subnets create app-subnet '
17   --network=$env:VPC --range=10.20.0.0/24 --region=$env:REGION
18
19 gcloud compute addresses create $env:RANGE '
20   --global --purpose=VPC_PEERING --prefix-length=20 --network=$env:VPC
21
22 gcloud services vpc-peerings connect '
23   --service=servicenetworking.googleapis.com '
24   --ranges=$env:RANGE '
25   --network=$env:VPC
26
27 gcloud kms keyrings create $env:KEYRING --location=$env:REGION
28 gcloud kms keys create $env:KEY --location=$env:REGION '
29   --keyring=$env:KEYRING --purpose=encryption

```

```

30
31 gcloud storage buckets create gs://$env:BUCKET '
32   --location=$env:REGION '
33   --uniform-bucket-level-access '
34   --default-encryption-key=projects/$env:PROJECT_ID/locations/$env:REGION/
    keyRings/$env:KEYRING/cryptoKeys/$env:KEY
35
36 gcloud storage buckets update gs://$env:BUCKET --versioning

```

Listing 4.4: Creating the Milestone 1 network, storage, and managed-service foundations.

2. **Deploy representative schemas and write one clickstream event.** This prototype shows the service boundaries; production code should add migrations, secret management, and CI validation.

```

1 import hashlib
2 import os
3 import time
4 import uuid
5
6 from google.cloud import bigtable
7
8
9 PROJECT_ID = os.environ["PROJECT_ID"]
10 INSTANCE_ID = os.environ.get("BIGTABLE_INSTANCE", "ecommerce-clicks")
11 TABLE_ID = os.environ.get("BIGTABLE_TABLE", "clickstream")
12 MAX_TS = 9999999999999999
13
14
15 def bucket_for(user_id: str, buckets: int = 32) -> str:
16     digest = hashlib.sha256(user_id.encode("utf-8")).hexdigest()
17     return f"{int(digest[:4], 16) % buckets:02d}"
18
19
20 def row_key(user_id: str, session_id: str, event_ms: int) -> bytes:
21     reverse_ms = MAX_TS - event_ms
22     return f"{bucket_for(user_id)}#{user_id}#{session_id}#{reverse_ms:013d}"
    ".encode("utf-8")
23
24
25 def main() -> None:
26     client = bigtable.Client(project=PROJECT_ID, admin=True)
27     table = client.instance(INSTANCE_ID).table(TABLE_ID)
28     user_id = "user-123"
29     session_id = "session-456"
30     event_ms = int(time.time() * 1000)
31     row = table.direct_row(row_key(user_id, session_id, event_ms))
32     row.set_cell("event", "event_id", str(uuid.uuid4()))
33     row.set_cell("event", "type", "product_view")
34     row.set_cell("event", "product_id", "sku-00042")
35     row.set_cell("event", "event_ms", str(event_ms))
36     row.commit()
37     print({"status": "written", "user_id": user_id, "session_id":
    session_id})
38
39
40 if __name__ == "__main__":
41     main()

```

Listing 4.5: Writing a Bigtable clickstream event with a salted reversed-timestamp row key.

3. **Add database-specific setup.** Create a private-IP Cloud SQL instance for users, a Spanner instance for inventory and checkout, and a Bigtable instance for clickstream. Use

the same VPC and service accounts where applicable.

```

1 gcloud sql instances create ecommerce-users '
2   --database-version=POSTGRES_16 '
3   --region=$env:REGION '
4   --network=$env:VPC '
5   --no-assign-ip '
6   --backup-start-time=03:00
7
8 gcloud spanner instances create ecommerce-global '
9   --config=nam3 '
10  --description="E-commerce global inventory and checkout" '
11  --processing-units=1000
12
13 gcloud spanner databases create commerce --instance=ecommerce-global
14
15 gcloud bigtable instances create ecommerce-clicks '
16   --display-name="E-commerce clickstream" '
17   --cluster=ecommerce-clicks-c1 '
18   --cluster-zone=$env:ZONE '
19   --cluster-num-nodes=3 '
20   --cluster-storage-type=SSD
21
22 cbt -project $env:PROJECT_ID -instance ecommerce-clicks createtable
   clickstream
23 cbt -project $env:PROJECT_ID -instance ecommerce-clicks createfamily
   clickstream event
24 cbt -project $env:PROJECT_ID -instance ecommerce-clicks setgcpolicy
   clickstream event maxage=90d

```

Listing 4.6: Representative Cloud SQL, Spanner, and Bigtable creation commands for the capstone.

Deliverables

- A one-page architecture summary explaining why each Part I service is used and what data it owns.
- A completed reference diagram showing browser, application VPC, private connectivity, Cloud SQL, Spanner, Bigtable, Storage, KMS, IAM, logging, and monitoring.
- Command transcripts for network peering, KMS, bucket creation, lifecycle configuration, Cloud SQL private IP, Spanner database, and Bigtable table setup.
- DDL or schema notes for user accounts, inventory reservations, checkout orders, and clickstream rows.
- Python or pseudocode evidence for one user lookup, one checkout transaction, one image object operation, and one clickstream write.
- A security and operations checklist covering IAM, service accounts, CMEK, backups, lifecycle, audit logs, alerting, and cleanup.

Acceptance Criteria

- Cloud SQL is reachable through private IP from the application VPC and has automated backups enabled.
- Spanner schema supports globally consistent inventory reservation and checkout without duplicate order application.

- Bigtable row-key design distributes clickstream writes and supports recent-session reads with bounded scans.
- Cloud Storage product-image bucket uses uniform bucket-level access, CMEK, lifecycle policy, and appropriate versioning.
- VPC peering or private services access is documented with reserved ranges, routes, firewall assumptions, and service producer connectivity.
- IAM grants each workload only the roles required for its owned data service and no broad owner/editor permissions.
- Monitoring covers database latency, errors, backups, storage growth, hot ranges, lifecycle actions, and KMS key usage.
- The final report includes cleanup steps and a residual-risk statement for production hardening.

Stretch Goals

- Add Pub/Sub and Dataflow between the web tier and Bigtable so clickstream ingestion can absorb traffic bursts.
- Export Spanner order facts and Bigtable clickstream summaries to BigQuery for product analytics.
- Add Cloud Armor, Identity-Aware Proxy, Secret Manager, and Binary Authorization to strengthen the application tier.
- Create Terraform or Deployment Manager equivalents for the manual commands.
- Add synthetic load tests for checkout latency, image-object throughput, and clickstream row-key distribution.
- Estimate monthly cost by service, environment, and workload owner using labels and billing export.

Part II

Data Warehousing and Analytics (BigQuery)

BigQuery Foundations and Core SQL

Learning Objectives

- Explain BigQuery’s decoupled architecture built on Colossus storage, the Jupiter network, and the Dremel execution engine.
- Compare on-demand pricing with capacity-based Editions and slot reservations, and select a model for a given workload shape.
- Apply window functions, arrays, and structs in GoogleSQL to analytical problems that are painful in flat relational SQL.
- Model a star schema with declared grain, fact table types, conformed dimensions, and SCD Type 2 change tracking.
- Query the GitHub public dataset, unnest nested fields, and compute a seven-day rolling average with window functions.
- Estimate the monthly cost of a workload scanning fifty terabytes per day and recommend an Edition tier with justification.

Cross-Cloud Equivalence

Serverless analytical warehouses differ in pricing mechanics and tuning surface, but the core ideas—decoupled storage and compute, columnar execution, and pay-for-what-you-scan or pay-for-capacity—are portable across platforms.

Capability	GCP BigQuery	AWS Redshift	Azure
Warehouse engine	Dremel over Colossus	RA3 managed storage	Synapse / Fabric warehouse
Unit of compute	Slots	RPUs / node hours	DWU / capacity units
On-demand pricing	Per TiB scanned	Redshift Serverless RPU	Serverless SQL per TB
Capacity pricing	Editions (Standard, Enterprise, Enterprise Plus)	Provisioned clusters	Provisioned pools / Fabric capacities
Result acceleration	BI Engine	Materialized views, AQUA	Fabric caching / result-set cache
Nested data	ARRAY and STRUCT first-class	SUPER (semi-structured)	OPENJSON over nvarchar
Cross-cloud analytics	BigQuery Omni	Redshift data sharing	Fabric OneLake shortcuts

Snowflake is the closest philosophical cousin: fully managed, storage and compute separated, and priced by credit consumption on virtual warehouses. **MongoDB Atlas** is an

operational document store with an aggregation framework, not a columnar warehouse; it pairs with analytical stores rather than replacing them. **Microsoft Fabric** lands its warehouse and lakehouse workloads on OneLake, making the storage layer the unifying concept rather than the engine.

Key Terms

Slot: BigQuery’s unit of computational capacity; queries consume slots for the duration of their execution stages. **On-demand pricing:** per-TiB-scanned billing where each query pays for the bytes it reads. **Editions:** capacity pricing tiers (Standard, Enterprise, Enterprise Plus) billed in slot-hours with autoscaling. **Colossus:** Google’s distributed file system that durably stores BigQuery’s columnar data. **Star schema:** a dimensional model with fact tables surrounded by conformed dimension tables. **SCD Type 2:** a slowly changing dimension strategy that keeps full history by versioning rows with validity intervals.

5.1 Week 5 Roadmap

Week 5 opens Part II by moving from storage primitives to the analytical engine at the center of most GCP data platforms. BigQuery is where raw files from Chapter 1’s buckets, relational extracts from Chapter 2, and streams from later chapters converge into governed, queryable assets. Understanding its architecture is not academic trivia: the separation of storage and compute explains why BigQuery scales without cluster management, why it charges per byte scanned or per slot, and why table design decisions echo directly in the monthly bill.

Monday dissects the decoupled architecture: Colossus, Jupiter, and Dremel. Tuesday covers pricing models, slots, and Editions. Wednesday develops advanced GoogleSQL—window functions, arrays, structs—and Kimball dimensional modeling with SCD Type 2. Thursday is a hands-on lab against the GitHub public dataset. Friday estimates and governs the cost of a fifty-terabyte-per-day marketing analytics workload.

Definition

BigQuery is Google Cloud’s serverless, petabyte-scale data warehouse. Storage and compute are fully decoupled: data lives in columnar format on Colossus, and queries are executed by Dremel workers that shuffle intermediate results over the Jupiter network [9, 63].

5.2 Monday: Decoupled Architecture—Colossus, Jupiter, and Dremel

5.2.1 Separation of Storage and Compute

Traditional data warehouses couple storage and compute: a node owns disks, holds a slice of the data, and executes the part of the query touching that slice. Scaling storage means adding compute, and scaling compute means redistributing data. BigQuery breaks this coupling. Data resides in Colossus, the same planet-scale distributed file system lineage as the Google File System [4], encoded in a columnar format with rich metadata. Compute is stateless: Dremel workers read from Colossus over the network, process query stages, and terminate. Because workers hold no durable state, BigQuery can allocate thousands of slots to one query and zero to the same project a second later.

Note

The decoupled design is why BigQuery has no indexes to build, no vacuum to schedule, and no cluster to size. The engineering levers move to table layout (partitioning, clustering, Chapter 6), query shape, and pricing-model choice.

5.2.2 Dremel Execution and the Shuffle Tier

Dremel executes a query as a directed acyclic graph of stages. Leaf stages scan columnar blocks from Colossus; intermediate stages aggregate, join, or repartition rows through an in-memory shuffle service; a final stage assembles results. The Jupiter petabit-scale network makes reading remote storage cheap enough that locality is not the bottleneck it was for classic Hadoop clusters [63]. Each stage consumes **slots**, and the query's slot-milliseconds and bytes-billed are visible after execution—two numbers you will use constantly in cost governance.

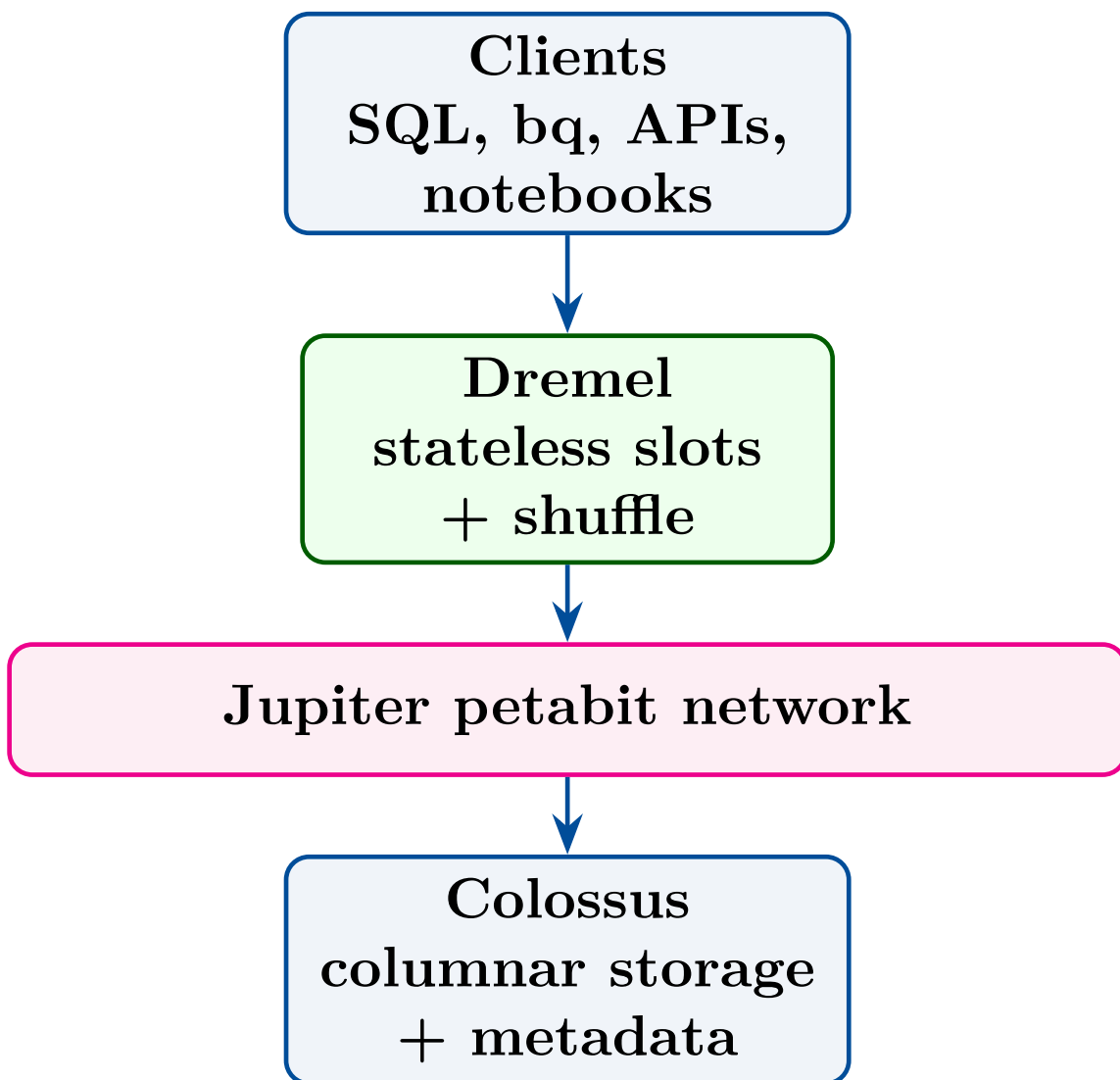


Figure 5.1: BigQuery's decoupled stack: stateless Dremel compute reads columnar data from Colossus over Jupiter.

Common Pitfall

“Serverless” does not mean “unbounded.” A badly written query—an unfiltered scan of a petabyte table, a cross join, a skewed key—will happily consume enormous slot capacity and budget. Serverless removes cluster administration, not engineering discipline.

5.3 Tuesday: Pricing Models, Slots, and Editions

5.3.1 On-Demand versus Capacity Pricing

On-demand pricing charges per TiB of data scanned by each query, with the first tier free monthly. It is the right default for spiky, experimental, or low-volume workloads: no commitments, no idle cost. Its weakness is variance—one analyst’s missing **WHERE** clause can scan petabytes.

Capacity pricing buys **slots** in an Edition (Standard, Enterprise, or Enterprise Plus) and bills slot-hours. Editions add autoscaling: a baseline of slots is always available, and additional slots spin up under load and spin down afterward. Enterprise and Enterprise Plus unlock features such as longer time-travel windows, advanced security, and higher concurrent-query limits [11].

Option	Billing model	Best for	Watch-outs
On-demand	Per TiB scanned	Spiky, unpredictable, low-volume workloads	One bad query can scan petabytes; set custom query cost quotas
Editions (slots)	Slot-hours per tier	Steady production workloads needing predictable spend	Commitments are per region; idle baseline slots still bill
Flat-rate (legacy)	Fixed monthly slots	Legacy contracts migrating to Editions	Being phased out; no new features
BigQuery Omni	Per TiB (or slots) on cross-cloud data	Analytics over data that stays in AWS or Azure	Source-cloud egress plus cross-region latency

Table 5.1: BigQuery pricing models and their trade-offs [11].

5.3.2 Reservations and Workload Isolation

Capacity purchased in an Edition is assigned to **reservations**, which map slot pools to projects, folders, or jobs. A common pattern separates **etl**, **bi**, and **ad-hoc** reservations so a dashboard refresh storm cannot starve a critical nightly load. Idle-slot sharing lets one reservation borrow unused capacity from another, which keeps utilization high without manual rebalancing.

Tip

Start on-demand, measure four weeks of real usage (bytes scanned per day, concurrency, peak slot demand from `INFORMATION_SCHEMA.JOBS`), then model Editions. Buying a commitment before you know your slot-hours profile usually wastes money.

5.4 Wednesday: Advanced SQL and Dimensional Modeling

5.4.1 Window Functions, Arrays, and Structs

GoogleSQL treats nested and repeated data as first-class. A **STRUCT** groups related fields into one column; an **ARRAY** holds repeated values. Public datasets such as the GitHub archive use them heavily: one row per commit, with an array of touched files. **UNNEST** flattens arrays into rows for aggregation, and window functions compute running and rolling measures without self-joins.

```

1 WITH daily AS (
2   SELECT repo_name,
3          DATE(committer.date) AS commit_date,
4          COUNT(*) AS commits
5   FROM `bigquery-public-data.github_repos.sample_commits`,
6        UNNEST(repo_name) AS repo_name
7   GROUP BY repo_name, commit_date
8 )
9 SELECT repo_name,
10        commit_date,
11        commits,
12        AVG(commits) OVER (
13          PARTITION BY repo_name
14          ORDER BY commit_date
15          ROWS BETWEEN 6 PRECEDING AND CURRENT ROW
16        ) AS rolling_7d_avg
17 FROM daily
18 ORDER BY repo_name, commit_date;
```

Listing 5.1: Seven-day rolling average of daily commits per repository with window functions.

Window frames (**ROWS BETWEEN ...**), ranking (**ROW_NUMBER**, **RANK**), and navigation (**LAG**, **LEAD**) replace correlated subqueries and self-joins, and they execute in a single pass per partition. When you catch yourself writing a self-join on a date range, reach for a window frame first.

5.4.2 Kimball Dimensional Modeling

Dimensional modeling organizes analytical data into **fact tables** (measurable events) and **dimension tables** (descriptive context) [62]. The discipline starts with **declaring the grain**: a fact table row means exactly one thing—one order line, one daily account snapshot, one shipment leg. The grain commits you to what the table can answer and protects every downstream aggregation from double counting.

Fact tables come in three classic types. **Transaction facts** record discrete events at their natural grain. **Periodic snapshot facts** summarize state at regular intervals, such as daily inventory levels. **Accumulating snapshot facts** track a workflow through milestones—order placed, paid, shipped, delivered—updating one row as the process advances. **Conformed dimensions** (a shared **dim_date**, **dim_customer**, **dim_product**) allow facts from different processes to be combined in one report.

5.4.3 Slowly Changing Dimensions

A Type 1 dimension overwrites changed attributes; a Type 2 dimension preserves history by closing the current row and inserting a new version. Type 2 is required when reports must reconstruct “what did we believe then,” such as the customer’s segment at the moment of each purchase.

```

1 -- SCD Type 2 upsert into a customer dimension on BigQuery
2 MERGE analytics.dim_customer T
```



```

3 USING staging.customers_daily S
4 ON T.customer_id = S.customer_id AND T.is_current = TRUE
5 WHEN MATCHED AND (T.email <> S.email OR T.segment <> S.segment) THEN
6   UPDATE SET valid_to = CURRENT_DATE(), is_current = FALSE
7 WHEN NOT MATCHED THEN
8   INSERT (customer_id, email, segment, valid_from, valid_to, is_current)
9   VALUES (S.customer_id, S.email, S.segment, CURRENT_DATE(), DATE '9999-12-31',
10           TRUE);
-- Follow with a second INSERT for the new versions of the matched rows.

```

Listing 5.2: SCD Type 2 upsert into a customer dimension using MERGE.

Common Pitfall

The MERGE must match on both the natural key and `is_current = TRUE`. Matching on the key alone updates every historical version of the customer, silently corrupting the timeline. The second pass—inserting the new current version of each closed row—is equally easy to forget.

5.5 Thursday Hands-On Project: GitHub Public Dataset Analytics

This lab queries the GitHub public dataset, unnests nested commit fields, and computes a seven-day rolling average of commits per repository. Work in the BigQuery console or with the bq CLI; public datasets cost nothing to store, and queries draw from the free monthly on-demand tier.

5.5.1 Explore the Dataset

```

1 bq show bigquery-public-data:github_repos.sample_commits
2
3 bq query --nouse_legacy_sql '
4   "SELECT repo_name, commit, author.date.sec AS authored_sec
5   FROM 'bigquery-public-data.github_repos.sample_commits'
6   LIMIT 5"

```

Listing 5.3: Inspecting the GitHub public dataset schema from the bq CLI.

5.5.2 Unnest and Aggregate

```

1 SELECT repo_name,
2        DATE(committer.date) AS commit_date,
3        COUNT(*) AS commits
4 FROM 'bigquery-public-data.github_repos.sample_commits',
5      UNNEST(repo_name) AS repo_name
6 WHERE DATE(committer.date) >= DATE_SUB(CURRENT_DATE(), INTERVAL 90 DAY)
7 GROUP BY repo_name, commit_date
8 ORDER BY commits DESC
9 LIMIT 100;

```

Listing 5.4: Unnesting repository arrays and counting daily commits.

5.5.3 Rolling Average with a Window Function

Reuse the rolling-average query from Wednesday. Verify the frame by hand for one repository: pick eight consecutive days, average the first seven, and compare with the `rolling_7d_avg`

value on the seventh day. Manual verification of one window catches frame-boundary errors that aggregates hide.

Note

Before every query in the console, glance at the bytes-processed estimate in the top-right corner. Make this a reflex: estimate first, run second.

Deliverables

- The rolling-average query saved in a repository file with a one-paragraph explanation of the frame choice.
- A screenshot or CLI output of the bytes-processed estimate for each query run.
- A short table of the top ten repositories by commits in the window, with their rolling averages.

Acceptance Criteria

- The query uses `UNNEST` on a repeated field and a window function with an explicit frame.
- The rolling average is computed per repository with `PARTITION BY` and a seven-row frame.
- Results are reproducible: a teammate can paste the SQL and get identical output.

Stretch Goals

- Rewrite the aggregation using `STRUCTs` to keep per-repo daily counts nested in one row per repository.
- Compare bytes billed for the sample table versus the full `commits` table and explain the difference.
- Add a `RANK()` window to flag each repository's single busiest day in the period.

5.6 Friday Use Case and Architecture: Costing a 50 TB/Day Marketing Workload

5.6.1 Scenario

A marketing analytics team scans roughly fifty terabytes of event data daily across attribution, campaign, and audience queries. Leadership asks two questions: what will BigQuery cost per month, and should the workload move to an Edition?

5.6.2 On-Demand Estimate

On-demand analysis pricing is per TiB scanned; using a representative list price of \$6.25 per TiB, fifty terabytes per day is $50 \times 30 = 1,500$ TiB per month, or roughly \$9,375 monthly—before storage, streaming inserts, and any queries that blow past the fifty-terabyte norm [11]. The number itself is less important than the method: volume times frequency times price, plus a sensitivity range for query-growth scenarios.

5.6.3 Edition Recommendation

Steady daily volume with predictable working hours argues for capacity pricing. The recommendation pattern is: measure slot demand from `INFORMATION_SCHEMA.JOBS_BY_PROJECT` for two to four weeks, choose the smallest Edition whose autoscaling ceiling covers the p95 peak, isolate the workload in its own reservation, and keep on-demand for genuinely spiky experimentation. If measured sustained usage is, for example, 200–400 slots during business hours, an Enterprise baseline with autoscaling is typically cheaper and far more predictable than 1,500 TiB of on-demand scanning—and it unlocks longer time travel and advanced governance features.

Tip

Whichever model you choose, set guardrails: custom query cost quotas per user and per project, dry-run checks in CI for expensive SQL, and a weekly bytes-billed report grouped by user, table, and query pattern [21].

Common Pitfall

Do not compare “on-demand cost of current queries” with “cost of a commitment” without checking waste first. Partition filters and clustering (Chapter 6) routinely cut bytes scanned by ten to one hundred times; the cheapest slot is the one a well-designed table never needs.

5.7 Interview Q&A

5.7.1 Concepts and Trade-offs

1. **Q: Which BigQuery components make storage and compute independently scalable?**
A: Colossus stores columnar data durably; stateless Dremel workers compute over it; Jupiter connects them with enough bandwidth that workers need not be co-located with data.
2. **Q: When do Editions beat on-demand pricing?**
A: When sustained slot usage is steady enough that slot-hours cost less than per-TiB scanning—typically continuous production ELT and BI workloads with measurable daily volume.
3. **Q: What does “declaring the grain” of a fact table commit you to?**
A: Every row means exactly one thing at one level of detail; all measures, keys, and aggregations must respect that declaration or results double-count.
4. **Q: When is SCD Type 2 required over Type 1?**
A: When history must be reconstructed—regulatory reporting, point-in-time joins, or any analysis of “what we knew when.”
5. **Q: Why must an SCD2 MERGE match on both key and is_current?**
A: Matching on the key alone would update every historical version row, destroying the timeline the Type 2 design exists to preserve.

5.8 Further Reading

Read the BigQuery documentation for architecture, datasets, and query reference [9], and the pricing page for current per-TiB and Edition rates [11]. Study the Dremel paper for the execution model’s origins [63] and the GFS paper for the storage lineage [4]. Kimball and Ross remain

the definitive dimensional-modeling reference [62]. Review Cloud Monitoring documentation for the slot-utilization and bytes-billed dashboards used in the Friday use case [21].

Key Takeaways

- BigQuery’s decoupled architecture—Colossus storage, Jupiter network, Dremel compute—eliminates cluster management but not design responsibility.
- Choose on-demand for spiky work and Editions for steady workloads; measure slot demand before committing.
- Window functions, arrays, and structs replace self-joins and staging tables for most rolling and nested analytics.
- Declaring the grain is the foundational dimensional-modeling act; conformed dimensions make facts composable.
- SCD Type 2 preserves history with validity intervals and demands a disciplined two-step MERGE.
- Cost governance is a design input: bytes-processed estimates, quotas, and reservation isolation belong in the platform, not in a quarterly surprise.

5.9 Exercises

1. **(Conceptual)** Explain to a skeptical DBA why BigQuery needs no indexes, vacuum, or cluster sizing—and what disciplines replace them.
2. **(Conceptual)** A query scans the same 2 TB table fifty times per day. Compare on-demand and Edition costs qualitatively and state what you would measure to decide.
3. **(Hands-on)** Run the rolling-average query on the GitHub dataset, then rewrite it with `ROWS BETWEEN 6 PRECEDING AND 3 PRECEDING` and explain what the new frame means.
4. **(Hands-on)** Create a `dim_customer` table, load three versions of one customer with the SCD2 MERGE pattern, and write a point-in-time query returning the customer’s segment on a past date.
5. **(Design)** Model an order-management star schema: declare the grain of each fact table, classify each as transaction, periodic snapshot, or accumulating snapshot, and list the conformed dimensions.
6. **(Architecture)** Extend the Friday cost estimate with three scenarios—flat volume, 20% monthly growth, and a Black-Friday spike—and recommend a pricing strategy for each.

BigQuery Performance: Partitioning, Clustering, and Cost Optimization

Learning Objectives

- Design partitioned tables using ingestion-time, time-unit column, and integer-range partitioning.
- Apply clustering to sort data for selective filters and explain how it complements partitioning.
- Decide when materialized views and BI Engine justify their cost, and recognize their restrictions.
- Read a BigQuery query execution plan to find skewed stages, broadcast joins, and expensive scans.
- Measure the bytes-billed difference between full, partitioned, and clustered scans on a large synthetic table.
- Re-architect a multi-petabyte security-log table that is generating uncontrolled query costs.

Cross-Cloud Equivalence

Every analytical platform offers the same two levers—eliminating data before it is read, and pre-computing what is read often—under different names.

Capability	GCP BigQuery	AWS Redshift	Snowflake
Coarse pruning	Partitioning	Sort keys / dist keys	Micro-partition pruning
Fine-grained ordering	Clustering (up to 4 columns)	Compound/interleaved sort keys	Clustering keys, auto recluster
Pre-aggregation	Materialized views	Materialized views	Materialized views
In-memory acceleration	BI Engine	AQUA / result cache	Result and warehouse cache
Cost lever	Bytes billed / slots	RPU-seconds / nodes	Credits per warehouse-second

Microsoft Fabric applies the same logic to OneLake Delta tables (partitioning plus file-level statistics and V-Order optimization), and **MongoDB** answers the same question with indexes and covered queries rather than partition pruning—different mechanism, identical goal: touch less data per query.

Key Terms

Partitioning: dividing a table into date-, time-, or integer-range segments so queries prune unneeded segments. **Clustering:** co-locating rows with similar values in up to four columns inside each partition for fine-grained block skipping. **Materialized view:** a precomputed query result that BigQuery maintains incrementally and transparently substitutes into queries. **BI Engine:** an in-memory analysis layer that accelerates dashboard queries against BigQuery tables. **Bytes billed:** the column bytes a query reads; the fundamental on-demand cost and the best proxy for compute effort. **Skew:** uneven distribution of work across execution slots or shuffle partitions, visible in query plans.

6.1 Week 6 Roadmap

Week 6 turns BigQuery from a correct tool into an economical one. The levers are few but powerful: partitioning prunes whole segments, clustering skips blocks inside segments, materialized views precompute repeated aggregations, BI Engine serves dashboards from memory, and execution plans reveal where the bytes and slots actually went. Monday covers partitioning strategies. Tuesday covers clustering. Wednesday covers materialized views, BI Engine, and plan analysis. Thursday measures the bytes-billed difference between scan strategies on a synthetic billion-row table. Friday re-architects a runaway security-log workload.

Definition

Pruning is the act of proving, before execution, that some portion of a table cannot contain relevant rows and therefore need not be read. Partitioning prunes by segment; clustering prunes by sorted block within segments; both reduce bytes billed directly [45, 42].

6.2 Monday: Partitioning Strategies

6.2.1 The Three Partitioning Models

BigQuery supports three partitioning schemes. **Ingestion-time partitioning** segments data by arrival date (or hour), exposed through the `_PARTITIONTIME` pseudo-column—ideal for logs and events where arrival order is the query axis. **Time-unit column partitioning** segments by a `DATE`, `TIMESTAMP`, or `DATETIME` column you control, at hourly, daily, monthly, or yearly granularity—ideal when the business time differs from arrival time, such as late-arriving financial events keyed by trade date. **Integer-range partitioning** buckets an integer column into named ranges—useful for sharded identifiers such as `customer_id` ranges.

```

1 CREATE TABLE analytics.events
2 PARTITION BY DATE(event_ts)
3 CLUSTER BY customer_id, event_type
4 OPTIONS (
5   partition_expiration_days = 395,
6   require_partition_filter = TRUE,
7   description = 'Customer events, partitioned by event date')
8 AS SELECT * FROM staging.raw_events WHERE FALSE;

```

Listing 6.1: Creating a daily time-partitioned, clustered table with an expiration.

6.2.2 Partition Design Rules

Partition on the column your selective filters actually use—almost always a time column. Keep partitions reasonably sized: thousands of tiny partitions degrade metadata performance more

than a handful of giant ones help pruning. Set `require_partition_filter = TRUE` on large tables; it converts the silent full-scan mistake into a loud error. Use partition expiration as lifecycle management: expiring old partitions is free and immediate compared to `DELETE` statements that scan and rewrite.

Common Pitfall

Partitioning by a high-cardinality or wrong-axis column (for example, `customer_id` when every query filters by date) prunes nothing and complicates every load. Pruning only works when query predicates align with the partition column.

Note

Ingestion-time and column-time partitioning diverge when data arrives late. An event from Monday arriving Wednesday lands in Wednesday’s ingestion partition but belongs to Monday’s business partition. Choose the axis your analysts filter by, and monitor the gap.

6.3 Tuesday: Clustering for Selective Filters

6.3.1 How Clustering Works

Within each partition, clustering sorts rows by up to four columns and tracks min/max statistics per block. A query filtering `WHERE customer_id = 42` on a table clustered by `customer_id` skips every block whose statistics exclude 42. Unlike partition pruning, clustering works on high-cardinality columns and on queries that filter multiple columns—and its cost estimates improve automatically as data settles, because BigQuery re-clusters in the background for free.

6.3.2 Choosing Clustering Columns

Choose columns that appear in selective equality or range filters, ordered from coarse to fine granularity. Clustering on a low-cardinality column (a boolean `is_active`) yields no pruning because nearly every block contains both values. Combine partitioning and clustering: partition by date to bound the scan in time, cluster by the tenant or entity column to bound it within each date.

Query pattern	Partitioning alone	Partitioning + clustering
Last 7 days, all tenants	Excellent pruning	Same; clustering adds little
One tenant, last 7 days	Reads whole 7 partitions	Reads only that tenant’s blocks
High-cardinality ID lookup	No help unless ID is partition column	Strong block skipping
Low-cardinality flag filter	No help	No help—do not cluster on it

Table 6.1: When clustering pays off beyond partitioning [42].

Tip

Clustering costs nothing extra at query time and its maintenance is automatic, so for large fact tables the question is rarely “whether” but “which columns.” Profile your top twenty queries’ `WHERE` clauses and let them choose the clustering key order.

6.4 Wednesday: Materialized Views, BI Engine, and Plan Analysis

6.4.1 Materialized Views

A materialized view precomputes a query result and keeps it fresh incrementally as base tables change. BigQuery’s smart tuning transparently rewrites eligible queries against the view even when the query text names the base table [44]. The restrictions matter: a limited SQL subset (no arbitrary joins, no UDFs, no nested/repeated aggregation), so treat materialized views as simple, high-value pre-aggregations—daily KPIs per region, not general-purpose caching.

6.4.2 BI Engine

BI Engine is a reserved, in-memory analysis layer that accelerates interactive dashboard queries, especially from Looker and Looker Studio [8]. It is wasted on batch ELT and one-off forensic queries; it pays off when hundreds of analysts hit the same sub-second latency expectations against the same tables. Size the reservation to the working set you want resident, and measure cache-hit rates before expanding it.

6.4.3 Reading the Execution Plan

The query plan graph shows stages with rows read, rows shuffled, slot time, and wait time. Three signatures dominate real troubleshooting: a **huge first stage** means a missing partition filter or a full scan; a **stage where one slot dwarfs the others** means skew—one key value holds a disproportionate share of a join or group-by; a **broadcast** annotation on a large input means the engine replicated a table it expected to be small. Fix skew by filtering the hot key separately, pre-aggregating before the join, or salting the skewed key and aggregating twice.

Common Pitfall

A query can be “fast” and ruinously expensive at the same time—slots are elastic, so a petabyte scan finishes quickly while the bill records every byte. Optimize for bytes billed and slot-milliseconds, not wall-clock speed.

6.5 Thursday Hands-On Project: Measuring Scan Strategies

This lab creates a large synthetic table, then compares bytes billed and execution time across a full scan, a partition-pruned scan, and a clustered scan. Use dry runs to record bytes before executing anything expensive.

6.5.1 Build the Synthetic Table

```

1 CREATE OR REPLACE TABLE lab.events
2 PARTITION BY DATE(event_ts)
3 CLUSTER BY customer_id
4 OPTIONS(require_partition_filter = TRUE)
5 AS
6 SELECT
7   TIMESTAMP_ADD('2026-01-01 00:00:00 UTC',
8     INTERVAL CAST(FLOOR(RAND() * 180 * 86400) AS INT64) SECOND) AS
9     event_ts,
10    CAST(FLOOR(RAND() * 100000) AS INT64) AS customer_id,
11    CAST(FLOOR(RAND() * 50) AS INT64) AS event_type,
12    RAND() * 1000 AS amount

```



```
12 FROM UNNEST(GENERATE_ARRAY(1, 100000000)) AS n;
```

Listing 6.2: Generating a partitioned, clustered synthetic events table (100M rows).

6.5.2 Compare Scan Strategies

Run each query first with `-dry_run` to capture bytes billed, then execute and record slot time from the plan.

```
1 # Full-range scan (all partitions): expect bytes ~= whole table
2 bq query --nouse_legacy_sql --dry_run '
3   "SELECT event_type, SUM(amount)
4     FROM lab.events
5     GROUP BY event_type"
6
7 # Partition-pruned scan: 7 of 180 days
8 bq query --nouse_legacy_sql --dry_run '
9   "SELECT event_type, SUM(amount)
10    FROM lab.events
11    WHERE DATE(event_ts) BETWEEN '2026-06-01' AND '2026-06-07'
12    GROUP BY event_type"
13
14 # Partition + cluster pruned: 7 days, one customer
15 bq query --nouse_legacy_sql --dry_run '
16   "SELECT SUM(amount)
17    FROM lab.events
18    WHERE DATE(event_ts) BETWEEN '2026-06-01' AND '2026-06-07'
19         AND customer_id = 4242"
```

Listing 6.3: Dry-run comparison of full, partitioned, and clustered scans.

Tabulate the three dry-run byte counts and the executed slot-milliseconds. A healthy result shows pruning cutting bytes by roughly the date ratio (7/180) and clustering cutting the second query by another one to two orders of magnitude.

Deliverables

- The three dry-run byte counts and executed slot-milliseconds in a small results table.
- A screenshot of the execution plan for the slowest query, annotated with the dominant stage.
- A one-paragraph explanation of why clustering did or did not help each query shape.

Acceptance Criteria

- The synthetic table is partitioned with `require_partition_filter = TRUE` and clustered by a selective column.
- Bytes billed are captured via dry run before any expensive execution.
- The partitioned scan reads materially fewer bytes than the full scan, and the clustered scan fewer still.

Stretch Goals

- Recreate the table without clustering and quantify the clustering delta on the third query.
- Add a materialized view for `event_type` daily totals and verify transparent rewrite in the plan.

- Introduce a deliberately skewed `customer_id` (90% of rows one value), rerun, and diagnose the skewed stage in the plan.

6.6 Friday Use Case: Re-Architecting a Multi-Petabyte Security Log Table

6.6.1 Scenario

A security team queries a 4 PB log-analysis table for incident investigations. The table is unpartitioned, analysts routinely scan months of history, and the monthly BigQuery bill has tripled in a year. Queries are interactive: investigators filter by time window, source IP, host, and severity.

6.6.2 Diagnosis

The plan review shows full scans on every query, a 90-day hot path that accounts for 95% of investigations, and heavy filters on `source_ip` and `host_name`—both high-cardinality, both ignored by any physical layout. One forensic query pattern joins the log table to a small asset inventory, and the plan shows the inventory broadcasting correctly while the log side scans entirely.

6.6.3 Re-Architecture

Partition by `DATE(event_ts)` with `require_partition_filter = TRUE`; cluster by `source_ip`, `host_name`; set partition expiration at the compliance boundary (for example 400 days) and export older partitions to Archive-class Cloud Storage before expiry for legal-hold retention at a fraction of warehouse cost. Add a materialized view for the daily severity-by-host summary that feeds the SOC dashboard, and put the dashboard behind a BI Engine reservation sized to the 90-day hot window. Set per-user custom query quotas so one unbounded forensic query cannot consume the day's budget.

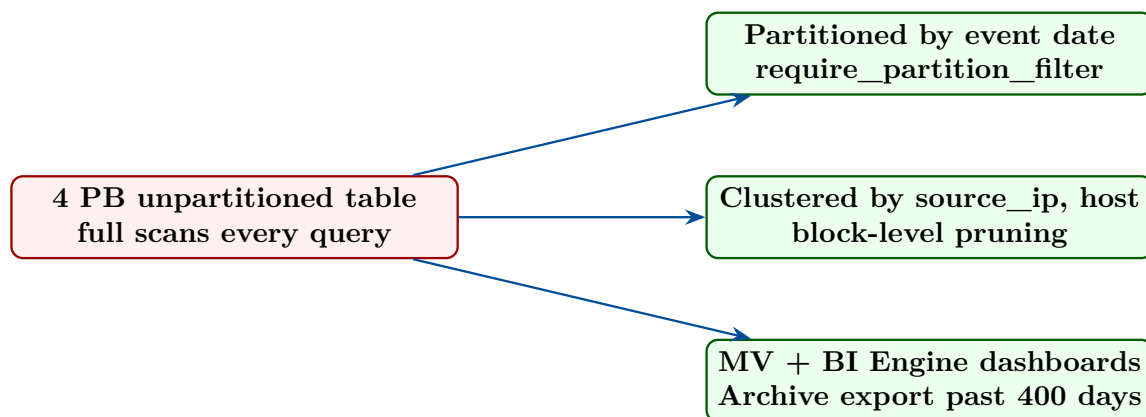


Figure 6.1: Re-architecting the security log table: partition, cluster, pre-aggregate, and expire.

Tip

Govern the change with numbers, not vibes: capture the pre-change p50/p95 bytes billed and slot time per query pattern, deploy the new layout behind a view with the old name, and publish the delta. Cost reviews fund the next optimization when the last one has a receipt.

6.7 Interview Q&A

6.7.1 Concepts and Trade-offs

1. **Q: How do partitioning and clustering reduce bytes billed differently?**
 A: Partitioning skips entire segments when the filter matches the partition column; clustering skips blocks inside partitions using sorted-column statistics, even on high-cardinality columns.
2. **Q: When does clustering beat partitioning for selective filters?**
 A: When the filter column is high-cardinality or non-temporal (customer ID, IP address), or when the table is already partitioned by date and queries need a second selective axis.
3. **Q: What does a materialized view precompute, and what does it cost?**
 A: A fixed aggregation over a base table, maintained incrementally; it costs stored bytes plus background refresh slots, and it restricts the SQL you can use.
4. **Q: Where in the execution plan do you spot a skewed stage?**
 A: A stage whose per-slot output rows or slot time is dominated by one or few workers, usually on a join or group-by over a hot key.
5. **Q: What does BI Engine accelerate, and when is it wasted?**
 A: Interactive, repeated dashboard queries over hot tables; it is wasted on batch ELT and one-off ad hoc forensics.

6.8 Further Reading

Read the partitioned and clustered table guides for syntax, limits, and best practices [45, 42]. Review materialized-view restrictions before designing around them [44], and the BI Engine documentation for reservation sizing [8]. The performance best-practices guide ties these levers together [9], and Cloud Monitoring documentation covers the slot-utilization alerting referenced throughout [21].

Key Takeaways

- Partition by the column selective filters actually use—usually time—and enforce it with `require_partition_filter`.
- Cluster on selective, high-cardinality filter columns ordered coarse to fine; skip low-cardinality flags.
- Materialized views are simple, transparently-rewritten pre-aggregations, not general caches; BI Engine serves interactive dashboards only.
- Optimize for bytes billed and slot-milliseconds; wall-clock speed on a serverless engine hides cost.
- Diagnose with the execution plan: big first stage means missing pruning, lopsided stage means skew.
- Re-architecture is an evidence exercise: measure before, change the layout, publish the delta.

6.9 Exercises

1. **(Conceptual)** A table is partitioned by `customer_id` range but all queries filter by date. Explain the cost consequence and the migration plan to fix it without downtime.
2. **(Conceptual)** Why does clustering on a boolean column provide no pruning, and what would you cluster on instead for a “flag plus customer” filter pattern?
3. **(Hands-on)** Reproduce the Thursday lab, then add integer-range partitioning experiments on a separate table and compare dry-run bytes.
4. **(Hands-on)** Create a materialized view over your synthetic table and confirm in the query plan that a base-table query is rewritten to use it.
5. **(Analysis)** Given an execution plan showing stage 2 with one slot writing 80% of shuffle output, name the likely cause and two fixes.
6. **(Design)** Produce a one-page cost-reduction proposal for the 4 PB security table, including estimated bytes-billed deltas, risks, and a rollback plan.

Advanced Data Types and Federated Analytics

Learning Objectives

- Query geospatial data with the `GEOGRAPHY` type, spatial joins, and BigQuery Search indexes.
- Create external tables over Cloud Storage files and explain their performance and governance limits.
- Evaluate BigQuery Omni for cross-cloud analytics, including egress cost and latency trade-offs.
- Decide when federated querying beats ingestion and when data should be loaded natively.
- Build an external table over CSV files and join it with native geospatial taxi-dropoff data.
- Design a multi-cloud analytics architecture for a retailer whose history lives in Amazon S3.

Cross-Cloud Equivalence

Every warehouse now answers the same two questions: “can I query data I have not loaded?” and “can I query data in another cloud?” The mechanics and bills differ.

Capability	GCP BigQuery	AWS	Azure / Fabric
Query files in place	External tables / BigLake	Redshift Spectrum / Athena	Synapse serverless / OneLake shortcuts
Cross-cloud analytics	BigQuery Omni	Athena federated queries	Fabric shortcuts to S3/ADLS
Geospatial type	<code>GEOGRAPHY</code> (S2-based)	<code>GEOMETRY</code> / <code>GEOGRAPHY</code> (Athena, Redshift)	<code>GEOMETRY</code> (SQL) / KQL geo
Full-text search	<code>SEARCH</code> indexes	OpenSearch integration	Azure AI Search / KQL full-text
Egress consideration	Omni reads billed; source egress applies	Cross-region/egress per GB	Cross-cloud egress per GB

Snowflake queries external tables and Iceberg tables in customer storage, and **MongoDB Atlas Data Federation** reaches into S3 and Atlas clusters. The universal lesson: federation moves compute to data only when the engine supports it; otherwise bytes travel, and the egress bill follows.

Key Terms

GEOGRAPHY: BigQuery’s spherical geospatial type, with functions for distance, containment, and spatial joins. **External table**: a BigQuery table whose data remains in Cloud Storage (or another source) and is read at query time. **BigQuery Omni**: cross-cloud analytics that runs BigQuery compute adjacent to data in AWS or Azure. **Federated query**: a query executed against data outside BigQuery’s managed storage. **Search index**: a BigQuery index that accelerates **SEARCH** and text-filter predicates on large tables. **Data contract**: the agreed schema and semantics a data source guarantees to its consumers.

7.1 Week 7 Roadmap

Week 7 widens BigQuery’s aperture. Monday adds the **GEOGRAPHY** type and search indexes for spatial and text analytics. Tuesday creates external tables that query Cloud Storage files in place. Wednesday introduces BigQuery Omni for analytics over data that must remain in AWS or Azure. Thursday joins an external CSV table to native geospatial taxi data in one query. Friday designs a multi-cloud architecture for a retailer with history in S3 and analytics in GCP.

Definition

Federation means querying data where it lives instead of loading it first. The trade is constant: federation avoids pipeline build-out and storage duplication, but gives up clustering, predictable latency, and (often) cost predictability that native managed storage provides [52].

7.2 Monday: BigQuery GIS and Search

7.2.1 The GEOGRAPHY Type

BigQuery’s **GEOGRAPHY** type models points, lines, and polygons on a spherical Earth, built on the S2 geometry library. The workhorse functions are **ST_GEO_GPOINT** (construct a point from longitude, latitude), **ST_DISTANCE** (meters between geographies), **ST_WITHIN** and **ST_CONTAINS** (point-in-polygon tests), and **ST_INTERSECTS** (any overlap). Spatial joins—“which zone contains each taxi drop-off”—express naturally as **JOIN ... ON ST_WITHIN(point, polygon)**.

```

1 SELECT z.zone_name, COUNT(*) AS dropoffs
2 FROM 'project.analytics.taxi_dropoffs' t
3 JOIN 'project.reference.city_zones' z
4   ON ST_WITHIN(t.dropoff_point, z.boundary)
5 GROUP BY z.zone_name
6 ORDER BY dropoffs DESC;
```

Listing 7.1: Point-in-polygon spatial join between taxi drop-offs and zones.

Common Pitfall

ST_GEOGPOINT takes **longitude first, latitude second**. Swapping them silently relocates your fleet into the ocean. Validate with a known landmark whenever you construct points from raw coordinates.

7.2.2 Search Indexes

A BigQuery **search index** accelerates **SEARCH** queries and some filter patterns over text-heavy columns—log forensics, error-message hunting, free-text ticket fields [9]. Create one with **CREATE**

SEARCH INDEX ... ON table(ALL COLUMNS) or a named column list. Indexes consume storage and take time to build; the payoff appears on selective text predicates over large tables, not on dashboards that scan whole partitions anyway.

Tip

Geospatial predicates benefit from clustering by a location-derived key (such as an S2 cell or geohash prefix) on very large tables. The index-free spatial join is correct but reads everything; a clustered pre-filter keeps spatial analytics economical.

7.3 Tuesday: External Tables over Cloud Storage

7.3.1 Creating and Querying External Tables

An external table registers a schema and a set of Cloud Storage URIs (CSV, JSON, Avro, Parquet) without copying data into BigQuery managed storage. Queries read the files at execution time.

```

1 CREATE OR REPLACE EXTERNAL TABLE ext.partner_returns (
2   return_id   STRING,
3   order_id    STRING,
4   reason_code STRING,
5   returned_at TIMESTAMP
6 )
7 OPTIONS (
8   format = 'CSV',
9   skip_leading_rows = 1,
10  uris = ['gs://partner-exports/returns/*.csv']);

```

Listing 7.2: External table over CSV exports in Cloud Storage.

7.3.2 What Federation Gives Up

External tables cannot be clustered, do not track table statistics for the optimizer the same way, re-read files on every query, and depend on the source bucket's availability and your IAM to it. They shine for exploratory analysis of data you may never load, for low-frequency reference data owned elsewhere, and as a staging step before a curated load. They are wrong for hot dashboards and for anything whose latency or cost must be predictable.

Property	External table	Native (managed) table
Storage	Files in Cloud Storage	BigQuery managed columnar
Clustering/pruning stats	None	Partition + cluster pruning
Read pattern	Files re-read each query	Columnar blocks, cached metadata
Best use	Exploration, one-off joins, staging	Repeated analytics, dashboards, SLA-bound
Cost predictability	Low (full file scans)	High (pruned scans)

Table 7.1: External versus native BigQuery tables [52].

Note

External tables do not enforce your schema on the files; a malformed CSV row fails or corrupts the query at read time. Treat the source prefix as a data contract: version it, validate it, and agree on who may change it.

7.4 Wednesday: BigQuery Omni and Cross-Cloud Analytics

7.4.1 Architecture

BigQuery Omni runs BigQuery compute in an AWS or Azure region, adjacent to data stored in S3 or ADLS. You create a connection to the foreign bucket, define external tables over it, and query with the same GoogleSQL. Compute happens near the data; only query results—not raw data—cross clouds unless you explicitly `CREATE TABLE AS` or export.

7.4.2 When Omni Wins

Omni wins when data gravity, sovereignty, or egress cost forbids moving bulk data, and when the analysis is intermittent enough that duplicated pipelines would cost more than federated reads. It loses when analytics are daily and heavy (load the data natively instead), when sub-second latency matters (cross-cloud round trips accumulate), and when the source cloud's egress pricing turns large result exports into a budget event.

Common Pitfall

Omni removes *bulk* data movement, not *all* data movement. Results, cross-cloud joins, and exports still generate source-cloud egress and cross-region latency. Model egress as a first-class line item before committing to the architecture.

Definition

Data gravity is the tendency of applications and analytics to be pulled toward where large datasets already reside, because moving petabytes is slower and more expensive than moving compute.

7.5 Thursday Hands-On Project: External CSVs Joined with Native GIS Data

This lab creates an external table over CSV files in Cloud Storage and joins it to native geospatial taxi-dropoff data, comparing the federated result with a native load.

7.5.1 Stage the Files

```

1 $env:BUCKET = "my-gcp-data-engineering-lab-ext"
2
3 @'
4 zone_id,zone_name,min_lng,min_lat,max_lng,max_lat
5 Z1,Downtown,-73.999,40.700,-73.950,40.760
6 Z2,Midtown,-74.010,40.740,-73.960,40.790
7 '@ | Set-Content -Path .\zones.csv
8
9 gcloud storage cp .\zones.csv gs://$env:BUCKET/ext/zones/zones.csv

```

Listing 7.3: Creating sample zone CSVs and uploading them to Cloud Storage.

For the demo, replace the bounding-box CSV with real polygon boundaries (GeoJSON converted to WKT) when available; the query below uses a simplified rectangular polygon built with `ST_GEOGFROMTEXT`.

7.5.2 Create the External Table and Join

```

1 CREATE OR REPLACE EXTERNAL TABLE ext.city_zones (
2   zone_id STRING, zone_name STRING,
3   min_lng FLOAT64, min_lat FLOAT64,
4   max_lng FLOAT64, max_lat FLOAT64)
5 OPTIONS (format = 'CSV', skip_leading_rows = 1,
6          uris = ['gs://my-gcp-data-engineering-lab-ext/ext/zones/*.csv']);
7
8 SELECT z.zone_name, COUNT(*) AS dropoffs
9 FROM `bigquery-public-data.new_york_taxi_trips.tlc_yellow_trips_2022` t
10 JOIN ext.city_zones z
11   ON ST_WITHIN(
12     ST_GEOGPOINT(t.dropoff_longitude, t.dropoff_latitude),
13     ST_GEOGFROMTEXT(CONCAT('POLYGON(',
14       z.min_lng, ' ', z.min_lat, ', ', z.max_lng, ' ', z.min_lat, ', ',
15       z.max_lng, ' ', z.max_lat, ', ', z.min_lng, ' ', z.max_lat, ', ',
16       z.min_lng, ' ', z.min_lat, ')))'))
17 GROUP BY z.zone_name
18 ORDER BY dropoffs DESC;

```

Listing 7.4: External zones table joined to native taxi drop-off points.

7.5.3 Compare with a Native Load

Load the same CSV into a native table with `bq load`, rerun the join, and compare bytes billed and elapsed time. The native version should prune and cache better; the external version required zero pipeline work. That comparison—speed of answer versus cost per answer—is the entire federation calculus in miniature.

Deliverables

- The external table DDL and the spatial join query, committed to the course repository.
- A results table of drop-offs per zone, plus the native-versus-external comparison of bytes billed.
- A short note on what would break if the partner changed the CSV column order.

Acceptance Criteria

- The external table reads CSVs directly from Cloud Storage with no intermediate load.
- The join uses `ST_WITHIN` with correctly ordered longitude/latitude coordinates.
- The native-versus-external comparison cites measured bytes billed from dry runs or job metadata.

Stretch Goals

- Replace rectangles with real neighborhood polygons in WKT and rerun the join.
- Cluster a native copy of the taxi table by a geohash prefix and measure the spatial-join speedup.

- Add a search index to a text-heavy column in the taxi dataset and benchmark a **SEARCH** query.

7.6 Friday Use Case and Architecture: Retail History in S3, Analytics in GCP

7.6.1 Scenario

A retailer runs its new platform on GCP, but five years of sales history—two petabytes of Parquet—sit in Amazon S3 under an active AWS workload. Executives want unified analytics in BigQuery without disrupting the AWS systems that still write to that bucket.

7.6.2 Options Analysis

Three architectures compete. **Full migration**: transfer history with Storage Transfer Service into Cloud Storage and load natively—best long-term cost and performance, highest up-front effort, and it creates a dual-write or cutover problem while AWS systems keep writing. **Pure Omni**: query S3 in place—fastest to value, no migration, but every daily dashboard pays federated-scan costs and AWS egress on exports. **Hybrid**: Omni for the cold historical tail queried rarely, plus a one-time load of the hot recent window into native partitioned tables, plus an incremental feed for new data. Most real programs land on the hybrid.

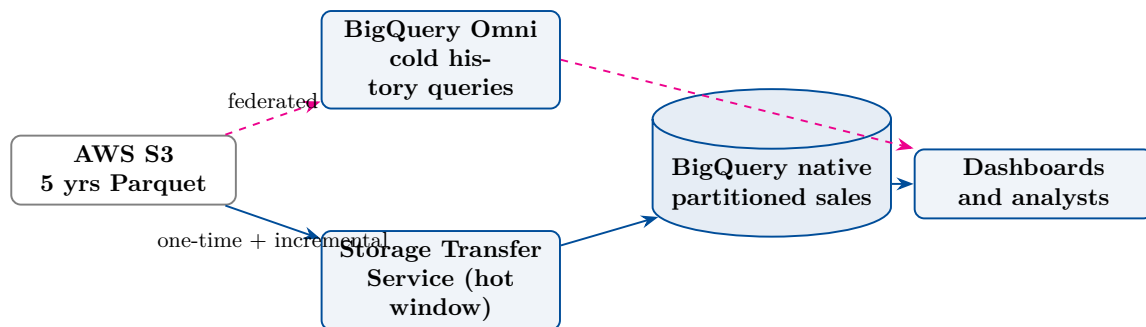


Figure 7.1: Hybrid multi-cloud retail analytics: Omni for cold federated reads, native load for the hot window.

Tip

Write the exit criteria into the design on day one: at what monthly federated-scan cost does full migration win? Track Omni bytes and egress in billing export; when the line crosses the migration amortization curve, execute the migration you already designed.

Common Pitfall

Do not federate a workload and then pretend it is free of pipelines. Someone still owns schema drift in S3, connection credentials, egress budget alerts, and the failure mode where the AWS bucket policy changes and every GCP dashboard breaks at once.

7.7 Interview Q&A

7.7.1 Concepts and Trade-offs

1. **Q: What is the difference between an external table and a native BigQuery table?**
A: External tables read source files at query time with no clustering or managed statistics; native tables use BigQuery’s columnar storage with pruning, caching, and predictable performance.
2. **Q: Which GEOGRAPHY function tests whether a point falls inside a polygon?**
A: `ST_WITHIN(point, polygon)` (equivalently `ST_CONTAINS(polygon, point)`).
3. **Q: What extra cost dimension does BigQuery Omni introduce?**
A: Source-cloud egress and cross-cloud latency whenever results, joins, or exports move bytes out of AWS or Azure.
4. **Q: Why can external tables not benefit from clustering?**
A: BigQuery does not manage the file layout; rows are not rewritten into sorted blocks, so no block-level statistics exist to prune with.
5. **Q: When is federated querying preferable to ingestion?**
A: For exploratory, low-frequency, or short-lived questions over data you do not own or may never need again—when pipeline cost would exceed query cost.

7.8 Further Reading

Read the geospatial analytics introduction for the full GEOGRAPHY function set [43], the external-data documentation for format support and limits [52], and the Omni documentation for connection setup and cross-cloud pricing [10]. Review Storage Transfer Service documentation for the migration leg of the Friday architecture [55], and Apache Iceberg for the open table format increasingly used to make lake files portable across engines [66].

Key Takeaways

- The GEOGRAPHY type makes spatial joins first-class SQL; remember longitude precedes latitude.
- External tables trade pipeline effort for prunable, predictable performance—use them for exploration and staging.
- BigQuery Omni moves compute to foreign data but not the egress bill; model it before committing.
- Federation versus ingestion is an economic decision about query frequency, latency, and data gravity.
- Multi-cloud analytics almost always lands on a hybrid: federate the cold tail, load the hot window.
- Any federated source is a data contract: version it, monitor it, and assign an owner.

7.9 Exercises

1. **(Conceptual)** Explain why “we will just query S3 from BigQuery forever” usually fails economically at daily-dashboard scale.
2. **(Conceptual)** Describe two failure modes unique to external tables that native tables do not have, and a mitigation for each.
3. **(Hands-on)** Reproduce the Thursday lab, then deliberately swap longitude and latitude and describe how you would detect the error systematically.
4. **(Hands-on)** Create a search index on a text column of a public dataset and compare a `SEARCH` query’s bytes billed before and after.
5. **(Design)** Write the decision memo for the Friday retailer: hybrid versus full migration, with numbers, risks, and exit criteria.
6. **(Architecture)** Add a data-contract enforcement layer (schema validation on new files, quarantine prefix) to the external-table design.

Data Lakehouse, Analytics Hub, and Cost Governance

Learning Objectives

- Build BigLake tables that give Cloud Storage files warehouse-grade, fine-grained security.
- Index unstructured files—PDFs and images—with Object Tables and route them to AI and SQL processing.
- Publish governed datasets through Analytics Hub listings without copying data.
- Apply row access policies and column-level policy tags, and verify them with negative tests.
- Architect a data clean room that merges CRM and advertising data without exposing raw PII.
- Complete the Milestone 2 capstone: a governed enterprise data lakehouse over five years of retail data.

Cross-Cloud Equivalence

The lakehouse pattern—open files in object storage with warehouse-grade security and governance layered on top—exists on every platform, with different names for the unification layer.

Capability	GCP	AWS	Azure / Fabric
Lakehouse tables	BigLake (+ Iceberg engine support)	Lake Formation governed tables / Athena	OneLake lakehouse (Delta)
Unstructured indexing	Object Tables	Athena + external metadata	OneLake files + AI skills
Row-level security	Row access policies	Lake Formation row filters	Row-level security (SQL)
Column-level security	Policy tags	Lake Formation column grants	Column-level security / Purview
Data sharing	Analytics Hub	Data Exchange / Redshift sharing	Fabric external data sharing
Data clean rooms	Analytics Hub clean rooms	AWS Clean Rooms	Confidential clean rooms

Snowflake delivers the same story through external tables, Iceberg tables, row access policies, masking policies, and listings on its Marketplace. **MongoDB**’s analog is field-

level encryption and role-based views—the shared principle is that governance belongs on the data asset itself, not on each downstream copy.

Key Terms

BigLake table: a table over Cloud Storage files that supports BigQuery’s fine-grained security and is readable by multiple engines through one connection. **Object table:** a read-only BigQuery table that indexes unstructured Cloud Storage objects (images, PDFs, audio) with metadata and signed URLs. **Analytics Hub:** a BigQuery exchange for publishing and subscribing to governed data listings across organizational boundaries. **Policy tag:** a taxonomy-based label attached to columns that enforces access control at query time. **Row access policy:** a filter predicate enforced on a table per principal, regardless of the query written. **Data clean room:** a governed environment where parties analyze combined data without either side seeing the other’s raw records.

8.1 Week 8 Roadmap

Week 8 completes Part II by turning the warehouse outward: governed lakes, unstructured data, and cross-organization sharing. Monday introduces BigLake and the lakehouse unification. Tuesday covers Object Tables for unstructured files. Wednesday covers Analytics Hub, listings, and data clean rooms. Thursday builds a BigLake table with column-level security and publishes a dataset to a partner. Friday architects an advertising data clean room. The week closes with the Milestone 2 capstone.

Definition

A **lakehouse** combines the open, low-cost file storage of a data lake with the schema enforcement, performance features, and fine-grained security of a warehouse. On GCP the pattern is Cloud Storage files exposed through BigLake tables, governed by the same IAM, policy tags, and row access policies as native BigQuery tables [40].

8.2 Monday: BigLake—Unifying Lake and Warehouse

8.2.1 Architecture

A BigLake table points at files in Cloud Storage (Parquet, Avro, CSV, or Iceberg metadata) through a **Cloud resource connection**—a service identity the files’ bucket trusts. Queries execute in BigQuery but read governed files; the same tables are readable by Spark on Dataproc through the BigLake connector, so one copy of the data serves SQL analysts and data scientists without exports. Because access flows through the connection rather than per-user bucket permissions, users query files they could never read directly—the warehouse becomes the security boundary for the lake.

8.2.2 Fine-Grained Security on the Lake

BigLake’s central value is that lake files inherit warehouse security primitives: **column-level security** via policy tags and **row-level security** via row access policies, enforced at query time for every engine reading through the table. Without BigLake, file-level IAM is all-or-nothing per object; with it, an analyst sees only permitted rows and columns of the very Parquet files a Spark job writes.

Note

BigLake also supports Apache Iceberg table metadata, letting BigQuery query Iceberg tables that other engines (Spark, Trino, Snowflake) read and write—a practical hedge against format lock-in [66].

Common Pitfall

The Cloud resource connection’s service account needs read access to the bucket—and nothing more. Granting it broad `storage.objectAdmin` on production buckets converts a query-time read path into a data-management backdoor. Least privilege applies to connections too.

8.3 Tuesday: Object Tables and Unstructured Data

8.3.1 Indexing Files as Rows

An Object Table is a read-only BigQuery table whose rows are the unstructured objects under a Cloud Storage prefix: PDFs, images, audio, video. Columns expose object metadata—URI, content type, size, generation—plus signed URLs for retrieval. That single abstraction lets SQL enumerate, filter, and join millions of files against structured tables, and it is the entry point for Document AI and Vision API enrichment: select the URIs matching a filter, send them through an ML transform, land the extracted fields in a structured table.

```

1 CREATE OR REPLACE EXTERNAL TABLE claims.docs
2 WITH CONNECTION 'us.doc-processor-connection'
3 OPTIONS (
4   object_metadata = 'SIMPLE',
5   metadata_cache_mode = 'AUTOMATIC',
6   uris = ['gs://claims-documents/incoming/*.pdf']);
7
8 -- Enumerate recent large PDFs for downstream OCR
9 SELECT uri, content_type, size
10 FROM claims.docs
11 WHERE content_type = 'application/pdf'
12 AND size > 1048576;
```

Listing 8.1: Object table indexing a claims document prefix.

Tip

Keep the object-metadata cache mode automatic for prefixes with steady churn; stale object listings are the most common Object Table surprise when upstream processes write and delete files continuously.

8.4 Wednesday: Analytics Hub and Data Clean Rooms

8.4.1 Sharing Without Copying

Analytics Hub is an exchange built into BigQuery. A **publisher** creates a listing over a dataset; a **subscriber** links it into their own project as a read-only, always-current **linked dataset**—no export, no copy, no stale extract. Governance travels with the listing: policy tags and row access policies on the source tables apply to subscribers, and revoking the listing revokes access everywhere. This replaces the classic anti-pattern of nightly CSV drops to an SFTP server with unknown downstream copies.

8.4.2 Data Clean Rooms

An Analytics Hub **data clean room** adds analysis rules to shared data: contributors publish data where queries must aggregate over a minimum number of distinct individuals, columns can be designated join-only or unqueryable, and results cannot be exported below threshold. Two parties—say, an advertiser and a publisher—can measure campaign overlap without either side reading the other’s row-level PII.

Definition

A **data clean room** is a governed sharing environment in which parties run approved analyses over combined data while analysis rules prevent any party from observing another party’s raw, row-level records [39].

Common Pitfall

Aggregation thresholds stop naive snooping, not clever snooping: differencing two carefully crafted aggregations can isolate an individual. Treat clean-room query governance as an ongoing review discipline, not a one-time configuration.

8.5 Thursday Hands-On Project: BigLake Security and an Analytics Hub Listing

This lab creates a BigLake table over Parquet files, applies column-level and row-level security, verifies both with negative tests, and publishes a curated dataset through an Analytics Hub listing.

8.5.1 Create the BigLake Table

```

1 $env:PROJECT_ID = "my-gcp-lakehouse-lab"
2 $env:BUCKET = "my-gcp-lakehouse-lab-retail"
3
4 bq mk --connection --location=US --connection_type=CLOUD_RESOURCE lakehouse-
   conn
5
6 # Grant the connection's service account read access to the bucket only:
7 # gcloud storage buckets add-iam-policy-binding gs://$env:BUCKET '
8 #   --member=serviceAccount:<connection-sa> --role=roles/storage.objectViewer
9
10 bq query --nouse_legacy_sql '
11   "CREATE OR REPLACE EXTERNAL TABLE lakehouse.fact_sales
12   WITH CONNECTION \\'us.lakehouse-conn\'
13   OPTIONS(format='PARQUET',
14            uris=['gs://$env:BUCKET/curated/fact_sales/*.parquet'])"
```

Listing 8.2: Creating a Cloud resource connection and BigLake table.

8.5.2 Apply Row and Column Security

```

1 CREATE ROW ACCESS POLICY region_filter
2 ON lakehouse.fact_sales
3 GRANT TO ("group:analysts-eu@example.com")
4 FILTER USING (region = 'EU');
5 -- Negative test: query as an analysts-us member must return zero EU rows.
6
7 -- Policy tags: Data Catalog taxonomy -> tag on the customer_email column.
```



```
8 -- Analysts without the Fine-Grained Reader role on the tag see NULLs.
```

Listing 8.3: Row access policy and policy tag enforcement on the lakehouse table.

8.5.3 Publish via Analytics Hub

In the BigQuery console (or API), create a **data exchange partner-exchange**, then a **listing** over a curated, already-secured dataset. Subscribe from a second project representing the partner and confirm the linked dataset appears read-only and reflects source updates within seconds. Finally, run the negative tests: a US analyst queries the row-policed table and sees no EU rows; an untagged user sees NULL (or an access error) in the tagged column.

Note

Policy tags deny by default. A forgotten group grant blocks dashboards silently and indistinguishably from a data outage—always pair every policy tag with a negative test and a positive access test for the intended group.

Deliverables

- DDL for the BigLake table, row access policy, and policy-tag assignments.
- Evidence of the negative tests: query output (or error) for the unauthorized principal.
- A screenshot of the Analytics Hub listing and the partner project's linked dataset.

Acceptance Criteria

- The BigLake table reads Cloud Storage files through a least-privilege connection.
- Row access policy filters rows per group and is verified with a negative test.
- The listing shares a live, read-only dataset with no data copy.

Stretch Goals

- Recreate the curated layer as Iceberg-format BigLake tables and read them from a Spark job.
- Add a second row access policy for another region and prove the filters compose per principal.
- Convert the listing into a data clean room with an aggregation-threshold analysis rule.

8.6 Friday Use Case and Architecture: An Advertising Data Clean Room

8.6.1 Scenario

An ad agency wants to merge its client's CRM segments with Google Ads exposure data to measure campaign reach and conversion overlap. Both datasets contain PII; neither party may see the other's raw records; regulators require documented controls.

8.6.2 Architecture

Both parties contribute governed tables to an Analytics Hub clean room. The CRM side publishes a table where direct identifiers are pre-hashed and designated join-only; the ads side does the same with its identifiers. An analysis rule requires every query to aggregate over at least fifty distinct individuals; overlap measurement queries run inside the clean room and only aggregated results—never row-level data—leave it. Policy tags on quasi-identifiers (postal code, birth date) restrict them further, and all queries are audit-logged for the compliance file.

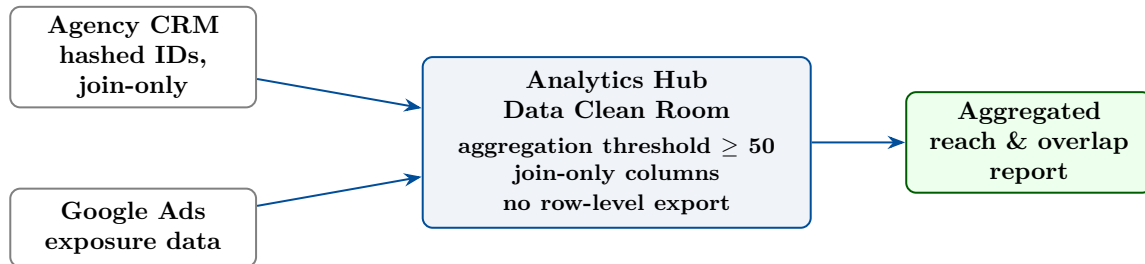


Figure 8.1: Data clean room: both parties contribute governed tables; only thresholded aggregates leave.

Tip

Document the re-identification analysis alongside the clean-room configuration: which quasi-identifiers remain, what differencing attacks the analysis rules do and do not prevent, and who approved the residual risk. Chapter 23 covers DLP risk analysis for exactly this purpose.

8.7 Interview Q&A

8.7.1 Concepts and Trade-offs

1. **Q: How does BigLake give Cloud Storage files warehouse-grade security?**
A: Access flows through a governed connection and BigQuery’s query engine, so policy tags and row access policies are enforced at query time on lake files just as on native tables.
2. **Q: What can an Object Table index that a regular external table cannot?**
A: Unstructured objects—PDFs, images, audio—as rows with metadata and signed URLs, with no requirement that the files share a tabular schema.
3. **Q: How does Analytics Hub share data without copying it?**
A: Subscribers receive a read-only linked dataset that references the publisher’s live tables; revoking the listing revokes access instantly.
4. **Q: What does a data clean room prevent each party from seeing?**
A: The other party’s row-level records; only analyses satisfying the configured rules (aggregation thresholds, join-only columns) may run or export.
5. **Q: Why use policy tags instead of separate views per role?**
A: Tags attach enforcement to the column itself across every table and query path, whereas per-role views multiply, drift, and are bypassed by direct table access.

8.8 Further Reading

Read the BigLake documentation for connection setup, Iceberg support, and security semantics [40], and the Analytics Hub documentation for exchanges, listings, and clean-room analysis rules [39]. Review Apache Iceberg for the open table format underlying portable lakehouses [66]. BigQuery’s main documentation covers policy tags, row access policies, and authorized views in depth [9], and the Architecture Framework supplies the governance review prompts [35].

Key Takeaways

- BigLake turns lake files into governed tables: one copy of data, warehouse-grade security, multiple engines.
- Object Tables make unstructured files enumerable and joinable from SQL, and are the front door to AI enrichment.
- Analytics Hub replaces extract-based sharing with live, revocable, governed listings.
- Row access policies and policy tags must be verified with negative tests—deny-by-default fails silently.
- Clean rooms protect raw records, not against every inference attack; document residual re-identification risk.
- Cost governance at the lakehouse layer means prunable layouts, monitored scans, and explicit sharing boundaries.

8.9 Exercises

1. **(Conceptual)** Explain why per-user bucket IAM cannot replace BigLake for column- and row-level security on Parquet files.
2. **(Conceptual)** A partner asks for “the raw data, just this once, it is easier.” Write three sentences explaining what breaks and the governed alternative.
3. **(Hands-on)** Reproduce the Thursday lab, then revoke the listing and confirm the linked dataset disappears for the subscriber.
4. **(Hands-on)** Create an Object Table over a prefix of sample PDFs and join it to a structured claims table on document ID.
5. **(Design)** Design the taxonomy for a retailer: three policy tags of increasing sensitivity, which columns get each, and which groups get Fine-Grained Reader.
6. **(Architecture)** Extend the clean-room design with audit-log export, a quarterly access review, and a differencing-attack mitigation note.

8.10 Milestone 2 Capstone: Enterprise Data Lakehouse Build

Capstone Project

Mission: Ingest five years of retail data into Cloud Storage, expose it through BigLake tables with region-based row- and column-level security, accelerate executive KPIs with materialized views and a BI Engine reservation, and publish a governed dataset through Analytics Hub—one governed copy serving internal dashboards and external partners.

Estimated time: 8–10 hours.

What you'll practice:

- Landing five years of Parquet retail data in partitioned Cloud Storage zones.
- Creating BigLake tables with row access policies on `region` and policy tags on customer PII.
- Building materialized views for executive KPIs and sizing a BI Engine reservation.
- Publishing a curated dataset via Analytics Hub and validating subscriber-side read-only access.
- Producing a cost estimate comparing on-demand and Editions for the serving layer.

Scenario

Meridian Retail operates across North America and Europe. Five years of order history exist as Parquet exports. Regional analysts must see only their region's rows; customer email and loyalty identifiers are restricted to a small data-steward group; executives need sub-second KPI dashboards; a logistics partner needs a governed, read-only feed of aggregated fulfillment metrics. One platform must satisfy all four without data copies.

Requirements

- Land the five-year Parquet dataset in a curated Cloud Storage zone, partitioned by date.
- Create BigLake tables over the files through a least-privilege Cloud resource connection.
- Apply a row access policy on `region` and policy tags on customer PII columns; verify each with a negative test.
- Build materialized views for the executive KPIs and add a BI Engine reservation sized to the hot window.
- Publish the curated, secured dataset through an Analytics Hub listing and subscribe from a partner project.
- Produce a monthly cost estimate (on-demand versus Editions) with two documented optimization decisions.

Reference Architecture

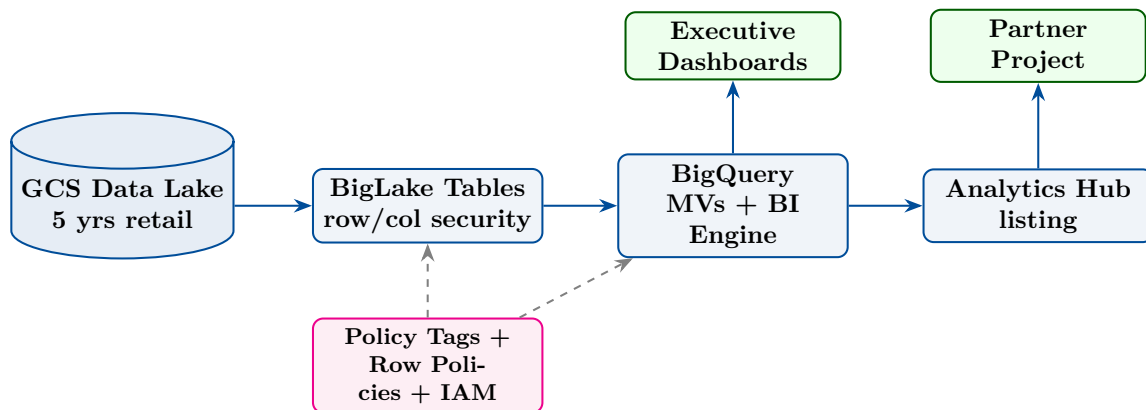


Figure 8.2: Milestone 2 reference architecture: a governed lakehouse from Cloud Storage to shared analytics.

Implementation Steps

1. **Land the data.** Upload five years of Parquet to `gs://$BUCKET/curated/...` with a hive-partitioned layout by sale date; validate row counts against the source export manifest.
2. **Create the connection and BigLake tables.** One connection per environment, `storage.objectViewer` on the curated bucket only.
3. **Secure the tables.** Row access policy on `region` for the analyst groups; policy tags on `customer_email` and `loyalty_id` granted only to the data-steward group. Run negative tests for both.
4. **Accelerate serving.** Materialized views for daily revenue by region and fulfillment SLA by carrier; BI Engine reservation covering the trailing 90 days.
5. **Publish.** Analytics Hub exchange and listing over the curated views; subscribe from the partner project and confirm read-only, policy-enforced access.
6. **Estimate cost.** Four weeks of `INFORMATION_SCHEMA` job telemetry; model on-demand versus an Enterprise Edition reservation; document two optimizations (for example, partition pruning and the MV/BI Engine serving layer).

Deliverables

- A repository that rebuilds the lakehouse from a clean environment: ingest scripts, DDL, security policies, and MV definitions.
- An architecture diagram showing zones, security policies, and serving layers.
- At least three automated data-quality tests (row counts, null checks, freshness) with a failing-case demonstration.
- Monitoring evidence: query-cost and slot-utilization dashboards or exported alert policies.
- The monthly cost estimate and the security review with negative-test evidence.

Acceptance Criteria

- The end-to-end lakehouse rebuilds from a clean environment following the README.
- Row- and column-level policies are verified with negative tests; no hardcoded secrets anywhere.
- The listing serves a live, read-only, policy-enforced dataset to the partner project.
- The cost estimate compares on-demand and Editions with two justified optimizations.

Stretch Goals

- Rebuild the curated zone as Iceberg tables and read them from both BigQuery and Dataproc Spark.
- Add Dataplex AutoDQ scans (Chapter 22) to replace hand-written quality tests.
- Convert the partner listing into a data clean room with aggregation thresholds.
- Add a FinOps dashboard breaking storage, slots, and BI Engine cost down by label and team.

Part III

Batch Processing and ETL Orchestration

Managed Hadoop and Spark with Cloud Dataproc

Learning Objectives

- Explain Dataproc cluster architecture, the role of ephemeral clusters, and pre-emptible/spot secondary workers.
- Plan the migration of on-premises HDFS data to Cloud Storage and decouple compute from storage.
- Compare cluster-based Dataproc with Dataproc Serverless for interactive and batch Spark workloads.
- Tune Spark memory, parallelism, and autoscaling policies for batch ETL jobs.
- Deploy an ephemeral cluster, run a PySpark sales aggregation, and self-delete the cluster.
- Design a migration strategy for a 100-node on-premises Cloudera cluster prioritizing cost reduction.

Cross-Cloud Equivalence

Managed Hadoop/Spark platforms converge on the same modernization story: decouple storage from compute, make clusters ephemeral, and let a serverless layer absorb cluster administration.

Capability	GCP Dataproc	AWS EMR	Azure
Managed Spark/Hadoop	Dataproc clusters	EMR on EC2	HDInsight / Synapse Spark / Fabric Spark
Serverless Spark	Dataproc Serverless	EMR Serverless	Synapse serverless Spark pools
Object storage for HDFS	Cloud Storage (GCS connector)	S3 (EMRFS)	ADLS Gen2 (ABFS)
Cheap workers	Preemptible/spot secondary workers	Spot task nodes	Low-priority/spot nodes
Autoscaling	Autoscaling policies	Managed scaling	Autoscale pools
Cluster lifecycle	Ephemeral per job	Transient clusters	Job-scoped pools

Databricks (available on all three clouds) and **Snowflake**'s Snowpark absorb the same Spark workloads into a managed notebook/runtime model; **MongoDB**'s aggregation framework answers smaller analytical questions without a cluster at all. The migration question is always the same: which workloads need the full Spark ecosystem, and which can be rewritten into SQL or serverless engines entirely.

Key Terms

Dataproc: Google Cloud’s managed service for Hadoop, Spark, Hive, Flink, and related ecosystem clusters. **Ephemeral cluster:** a cluster created for a single job or workflow and deleted immediately after. **Preemptible/spot VM:** deeply discounted compute that Google can reclaim with short notice; used for secondary workers only. **Dataproc Serverless:** a fully managed Spark runtime that executes batches and sessions without a user-managed cluster. **GCS connector:** the Hadoop-compatible filesystem layer that lets Spark read and write Cloud Storage as if it were HDFS. **Autoscaling policy:** declarative rules that grow and shrink worker counts based on YARN memory and pending resources.

9.1 Week 9 Roadmap

Week 9 opens Part III by confronting the workload type most enterprises actually have: years of Hadoop and Spark investment running on-premises. Monday establishes Dataproc architecture and the ephemeral-cluster discipline. Tuesday migrates HDFS data to Cloud Storage. Wednesday covers Dataproc Serverless and Spark tuning. Thursday runs a PySpark aggregation on a self-deleting cluster. Friday designs the migration of a 100-node Cloudera estate.

Definition

Cloud Dataproc is a managed Hadoop ecosystem service. It provisions clusters in about ninety seconds, integrates natively with Cloud Storage through the GCS connector, and supports per-job ephemeral clusters, autoscaling policies, and a serverless Spark runtime [19].

9.2 Monday: Dataproc Architecture, Ephemeral Clusters, and Preemptible VMs

9.2.1 Cluster Anatomy

A Dataproc cluster has a master node (running YARN ResourceManager, HDFS NameNode for scratch, and job drivers) and worker nodes (running NodeManagers and DataNodes). Initialization actions run at creation to install libraries. Component gateways expose web UIs such as the Spark History Server and Jupyter. Critically, HDFS on a Dataproc cluster is *scratch space*: durable data belongs in Cloud Storage, read and written through the GCS connector.

9.2.2 The Ephemeral Discipline

The pivotal mental shift from on-premises Hadoop is ephemerality. A long-lived shared cluster accumulates snowflake configuration, contends between teams, and bills around the clock. An ephemeral cluster exists for exactly one job or workflow template: created with precisely the machine types, image version, and initialization actions the job needs, then deleted. Dataproc’s ninety-second provisioning makes this practical; workflow templates and Composer operators (Chapter 12) make it routine.

Common Pitfall

Never use preemptible or spot VMs as primary workers holding HDFS data or as master nodes. Reclamation kills in-flight tasks; as *secondary* workers they are pure elastic capacity, but as primaries they can fail the entire job and corrupt scratch state.

9.2.3 Cost Anatomy

Dataproc bills the underlying Compute Engine VMs plus a small per-vCPU Dataproc premium. The cost levers, in descending order of impact: ephemeral clusters (pay per job, not per day), preemptible/spot secondary workers (up to roughly 80% cheaper), autoscaling policies (right-size through the job’s life), and custom machine types (match memory-to-core ratios to the workload instead of accepting family defaults).

Tip

Label every cluster (`team`, `job`, `env`) at creation. Dataproc cost is dominated by forgotten long-lived “temporary” clusters; labels make them visible in billing export within a day.

9.3 Tuesday: Migrating from HDFS to Cloud Storage

9.3.1 Why the Move Matters

On-premises Hadoop couples storage and compute in the same nodes: capacity planning means buying disks to get CPUs or CPUs to get disks. Moving data to Cloud Storage decouples the two—storage scales and is billed independently, clusters size to the compute a job actually needs, and multiple clusters (or BigQuery, or Dataproc Serverless) read the same data concurrently. Cloud Storage’s eleven-nines durability also replaces three-way HDFS replication, shrinking both hardware and operational burden [28].

9.3.2 Migration Mechanics

The standard tool is `hadoop distcp` between the on-premises HDFS and `gs://` targets, throttled to protect the WAN link, run in incremental passes. For large estates, sequence it: bulk-copy cold data first, then run repeated incremental syncs of changing datasets, then cut over producers. Preserve Hive metadata separately—the Hive Metastore schema migrates to a Cloud SQL instance or Dataproc Metastore, with table locations rewritten from `hdfs://` to `gs://`.

```
1 hadoop distcp -update -delete \  
2   -bandwidth 200 \  
3   hdfs://nn1.prod.local:8020/data/warehouse/orders \  
4   gs://my-migration-bucket/warehouse/orders
```

Listing 9.1: Representative `distcp` from on-premises HDFS to Cloud Storage.

Note

Validate every dataset with both counts and checksums (`distcp -update` compares size and CRC). A silent partial copy discovered during cutover rehearsal is embarrassing; the same discovery during real cutover is an incident.

9.4 Wednesday: Dataproc Serverless and Spark Tuning

9.4.1 Serverless Spark

Dataproc Serverless runs Spark batches and interactive sessions without any cluster: you submit a batch, the service provisions capacity, executes, and tears down, billing for the compute consumed. It removes image management, node sizing, and idle cost, at the price of less control over machine shapes and startup latency. Use it for scheduled batch ETL and ad hoc analysis; keep managed clusters for workloads needing specific images, GPU attachment, long-lived sessions, or deep ecosystem components.

9.4.2 The Tuning Shortlist

Most Spark performance problems on Dataproc reduce to a handful of properties. `spark.executor.memory` and `spark.executor.cores` set the container shape; oversizing executors wastes YARN scheduling granularity, undersizing them causes spills and out-of-memory kills. `spark.sql.shuffle.partitions` (default 200) is routinely wrong in both directions. `spark.dynamicAllocation` lets the cluster breathe with the job. On the platform side, an autoscaling policy watches YARN's pending and available memory to add or remove workers.

Symptom	Likely cause	First response
Executor OOM, fetch failures	Memory too small for skewed partition	Raise <code>executor.memory</code> ; fix skew (salting)
Hours in a single long stage	Too few shuffle partitions	Raise <code>spark.sql.shuffle.partitions</code>
Thousands of tiny tasks	Too many partitions, small files	Coalesce inputs; lower shuffle partitions
Workers idle, driver saturated	Over-broadcast or collect to driver	Remove <code>collect</code> ; check broadcast hints
Cluster never scales down	<code>dynamicAllocation</code> off; sticky policy	Enable dynamic allocation; review policy cooldown

Table 9.1: Common Spark-on-Dataproc symptoms and first responses [68].

Common Pitfall

A job that reads hundreds of thousands of tiny files spends its life in listing and task-scheduling overhead, not computation. Compact small files upstream (or read with tuned split sizing) before touching executor settings.

9.5 Thursday Hands-On Project: Ephemeral PySpark Sales Aggregation

This lab creates an ephemeral Dataproc cluster, submits a PySpark job that reads sales data from Cloud Storage, aggregates it, writes results back, and deletes the cluster. Run in a sandbox project and replace all placeholders.

9.5.1 Stage Data and Job Code

```

1 $env:PROJECT_ID = "my-gcp-dataproc-lab"
2 $env:BUCKET = "my-gcp-dataproc-lab-data"
3 $env:REGION = "us-central1"
4
5 @'
6 order_id,store_id,amount,order_date
7 1,S01,19.99,2026-07-01
8 2,S02,149.50,2026-07-01
9 3,S01,5.25,2026-07-02
10 '@ | Set-Content -Path .\sales.csv
11
12 gcloud storage cp .\sales.csv gs://$env:BUCKET/raw/sales/sales.csv

```

Listing 9.2: Staging sample sales data and the PySpark job.

```

1 import sys
2

```

```

3 from pyspark.sql import SparkSession
4 from pyspark.sql import functions as F
5
6 input_uri, output_uri = sys.argv[1], sys.argv[2]
7
8 spark = SparkSession.builder.appName("sales-agg").getOrCreate()
9
10 (spark.read.option("header", True).csv(input_uri, inferSchema=True)
11     .groupBy("store_id", "order_date")
12     .agg(F.count("*").alias("orders"),
13         F.round(F.sum("amount"), 2).alias("revenue"))
14     .write.mode("overwrite").parquet(output_uri))
15
16 spark.stop()

```

Listing 9.3: PySpark job aggregating daily sales per store (save as sales_agg.py).

9.5.2 Create, Run, Self-Delete

```

1 $env:CLUSTER = "ephemeral-sales-lab"
2
3 gcloud dataproc clusters create $env:CLUSTER '
4     --region=$env:REGION '
5     --single-node '
6     --image-version=2.2-debian12 '
7     --labels=env=lab,job=sales-agg
8
9 gcloud storage cp .\sales_agg.py gs://$env:BUCKET/code/sales_agg.py
10
11 gcloud dataproc jobs submit pyspark '
12     gs://$env:BUCKET/code/sales_agg.py '
13     --cluster=$env:CLUSTER --region=$env:REGION '
14     -- gs://$env:BUCKET/raw/sales/ gs://$env:BUCKET/out/sales_agg/
15
16 gcloud dataproc clusters delete $env:CLUSTER --region=$env:REGION --quiet
17
18 gcloud storage ls --long gs://$env:BUCKET/out/sales_agg/

```

Listing 9.4: Ephemeral cluster lifecycle around the PySpark job.

Deliverables

- The PySpark job and the command transcript showing cluster creation, job success, and deletion.
- The output Parquet listing and a sample of aggregated rows.
- A cost note: total job wall time and the vCPU-hours consumed.

Acceptance Criteria

- The cluster is created for this job and verifiably deleted afterward.
- The job reads from and writes to Cloud Storage, not cluster HDFS.
- Re-running the job overwrites the deterministic output path instead of appending duplicates.

Stretch Goals

- Re-implement the run as a Dataproc workflow template with automatic cluster teardown on failure.

- Add two preemptible secondary workers and observe the cost and stability effects.
- Port the same job to Dataproc Serverless with `gcloud dataproc batches submit` and compare cold-start latency.

9.6 Friday Use Case and Architecture: Migrating a 100-Node Cloudera Estate

9.6.1 Scenario

An enterprise runs a 100-node Cloudera cluster: 1.5 PB in HDFS, 400 Hive tables, 60 production Spark jobs, 25 Oozie workflows, and a mix of ad hoc analyst notebooks. The directive: reduce cost and operational burden, prioritizing decoupling compute from storage.

9.6.2 Migration Waves

Assess first, move second. Inventory every job, table, and workflow; classify each as *rehost* (runs as-is on Dataproc), *replatform* (Spark job moved to Dataproc Serverless with modest changes), or *retire/rewrite* (Hive queries that belong in BigQuery SQL, Oozie workflows that become Composer DAGs). Typical estates retire 20–40% of jobs outright—duplicates, dead reports, and workloads trivially expressible in BigQuery.

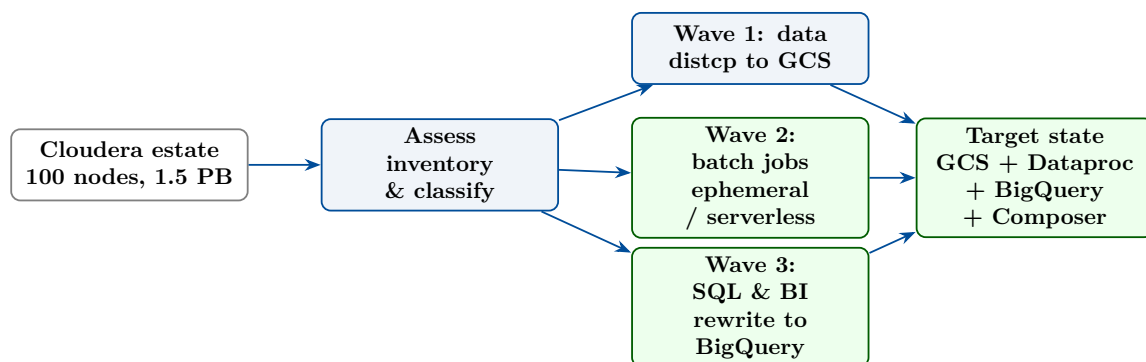


Figure 9.1: Cloudera-to-GCP migration waves: assess, move data, move compute, retire what BigQuery absorbs.

9.6.3 Cost and Risk

The economic case rests on ephemerality: the 100-node cluster runs 24/7 for jobs that occupy it perhaps 15% of the day; ephemeral clusters and serverless Spark bill only for execution. De-risk with a parallel-run period for Tier-1 jobs (both estates produce outputs; reconcile counts and checksums), a rehearsed rollback per wave, and explicit ownership for the Hive Metastore migration, which touches every table reference at once.

Tip

Make idempotency a migration exit criterion, not an afterthought: deterministic output paths and write-then-swap loads mean a preemptible reclamation or retry never double-loads data—the property that makes ephemeral infrastructure safe.

9.7 Interview Q&A

9.7.1 Concepts and Trade-offs

1. **Q: Why should Dataproc clusters be ephemeral for scheduled jobs?**
A: Per-job clusters bill only for execution, carry exactly the configuration the job needs, and avoid the snowflake drift and inter-team contention of long-lived shared clusters.
2. **Q: What is the cost trade-off of preemptible VMs in a Spark cluster?**
A: They cost up to roughly 80% less but can be reclaimed at any time; use them only as secondary workers and make jobs retry- and idempotency-safe.
3. **Q: Why migrate HDFS data to Cloud Storage instead of keeping HDFS?**
A: Decoupling storage from compute lets clusters size to compute need, multiple engines share one copy, and Cloud Storage durability replaces three-way replication.
4. **Q: What Spark property most often explains executor out-of-memory errors?**
A: `spark.executor.memory` sized too small for the largest (often skewed) partition—though the real fix is frequently fixing skew, not adding memory.
5. **Q: How do autoscaling policies decide when to add workers?**
A: They watch YARN metrics—pending versus available memory and containers—and add or remove workers within configured bounds and cooldowns.

9.8 Further Reading

Read the Dataproc documentation for cluster configuration, serverless batches, and autoscaling policies [19], and the Apache Spark documentation for tuning properties and the memory model [68]. Review the Cloud Storage overview for the durability and connector model that replaces HDFS [28]. Chapter 12’s Composer documentation covers orchestrating the ephemeral-cluster pattern at scale [17].

Key Takeaways

- Ephemeral, per-job clusters are the central discipline: pay per job, configure per job, delete after the job.
- Cloud Storage plus the GCS connector replaces HDFS and decouples storage economics from compute economics.
- Preemptible/spot VMs belong in secondary workers only, paired with idempotent job design.
- Dataproc Serverless removes cluster management for batch Spark; managed clusters remain for specialized images and long sessions.
- Spark tuning starts with executor shape, shuffle partitions, and input file sizing—in that order.
- Hadoop migrations succeed in waves: assess, move data, move batch compute, rewrite what SQL engines absorb.

9.9 Exercises

1. **(Conceptual)** Argue both sides: when is a long-lived Dataproc cluster legitimately better than ephemeral clusters?
2. **(Conceptual)** Explain why idempotent writes make preemptible workers safe, with a concrete double-load failure story for the non-idempotent case.
3. **(Hands-on)** Reproduce the Thursday lab, then wrap it in a workflow template that guarantees teardown on job failure.
4. **(Hands-on)** Run the aggregation on Dataproc Serverless and compare submission-to-first-task latency with the managed cluster.
5. **(Analysis)** Given a Spark UI showing one task processing 40% of a stage's rows, diagnose and propose two fixes.
6. **(Design)** Write the wave plan for the 100-node Cloudera migration: assessment criteria, wave membership rules, parallel-run reconciliation, and rollback triggers.

Visual ETL with Cloud Data Fusion

Learning Objectives

- Explain Data Fusion’s CDAP foundations, instance architecture, and private IP deployment.
- Use Wrangler for interactive, visual data cleaning and directive-based transformation.
- Compose pipelines from plugins, parameterize them with macros, and schedule recurring runs.
- Build a no-code pipeline that cleans messy customer data and loads it into BigQuery.
- Evaluate Data Fusion versus Dataflow for analyst-led teams and justify the recommendation.

Cross-Cloud Equivalence

Visual, low-code ETL tools target the same persona everywhere—analysts and integration engineers who need governed pipelines without writing Spark or Beam code.

Capability	GCP Data Fusion	AWS	Azure / Fabric
Visual ETL studio	Data Fusion Studio	Glue Studio	Data Factory pipelines / Fabric Data Factory
Interactive cleaning	Wrangler	Glue DataBrew	Power Query / Dataflow Gen2
Execution engine	Spark on ephemeral Dataproc	Glue Spark runtime	Spark / Data Factory IR
Extensibility	Plugin Hub (CDAP)	Glue connectors / custom transforms	Linked services & connectors
Scheduling	Built-in schedules + triggers	Glue triggers / EventBridge	ADF triggers / Fabric schedules

Snowflake users reach for partner tools (Matillion, Fivetran transformations, dbt) for the same low-code niche; **MongoDB** users lean on Atlas Data Federation and Compass aggregations. The shared trade-off: visual tools democratize pipeline building but can hide cost and semantics behind a friendly canvas.

Key Terms

Cloud Data Fusion: Google Cloud’s managed, visual ETL/ELT service built on the open-source CDAP framework. **Wrangler:** Data Fusion’s interactive data-preparation surface where analysts apply directives to sample data. **Plugin:** a packaged source, transform,

sink, or action available from the Hub or built custom. **Macro:** a runtime parameter placeholder such as a logical date or file path resolved when the pipeline runs. **Pipeline:** a directed graph of plugins deployed to an instance and executed on an ephemeral Dataproc cluster. **Private instance:** a Data Fusion instance reachable only over private IP within a VPC.

10.1 Week 10 Roadmap

Week 10 addresses a different builder: the analyst or integration specialist who owns business logic but not Java or Beam. Monday covers CDAP concepts, instance architecture, and private networking. Tuesday is Wrangler, the interactive cleaning workbench. Wednesday covers plugins, macros, and scheduling. Thursday builds the no-code customer-cleansing pipeline. Friday weighs Data Fusion against Dataflow for analyst teams.

Definition

Cloud Data Fusion is a fully managed, cloud-native data integration service built on CDAP (the Cask Data Application Platform). Pipelines are designed visually as graphs of plugins and executed at scale on ephemeral Dataproc clusters that the service provisions and tears down per run [18].

10.2 Monday: CDAP Concepts, Architecture, and Private IPs

10.2.1 The CDAP Foundation

Data Fusion is managed CDAP. Every pipeline is a CDAP application; every node on the canvas is a plugin with a schema, configuration, and connections to neighbors. Because CDAP is open source, pipeline definitions are portable artifacts (exported JSON) rather than proprietary lock-in—teams version-control them, diff them in review, and promote them across development, test, and production instances.

10.2.2 Instances and Editions

A Data Fusion **instance** is the managed environment hosting the design studio, pipeline store, and execution coordinator. Editions (Basic versus Enterprise) differ in execution guarantees and features; Enterprise adds capabilities such as higher pipeline concurrency and triggers between pipelines. Execution itself does not run in the instance: at run time, Data Fusion provisions an ephemeral Dataproc cluster in your project, runs the pipeline as a Spark job, and deletes the cluster—the ephemeral discipline from Chapter 9, automated.

10.2.3 Private Instances

Data sources in regulated environments rarely expose public endpoints. A **private instance** places Data Fusion on private IP within your VPC, using private services access, so pipelines reach on-premises databases over VPN/Interconnect and internal Cloud SQL instances without public exposure. Pair it with VPC Service Controls (Chapter 21) for a perimeter that contains both the design environment and the data paths.

Common Pitfall

A private instance cannot reach the public internet by default. Plugin downloads from the Hub, and calls to public APIs, fail unless you provide Cloud NAT or an explicit egress path—a common first-day surprise that looks like a product bug.

10.3 Tuesday: Wrangler—Interactive Data Cleaning

10.3.1 The Wrangler Workflow

Wrangler presents a sample of source data as a spreadsheet-like grid. Each cleaning gesture—filter, split, parse, extract, mask, rename—compiles into a **directive**, a declarative transformation step listed beside the grid. The analyst iterates on the sample until the data looks right, then “creates a pipeline” that embeds the Wrangler node with its directives, to be executed at full scale on Spark. This sample-first loop is the core productivity claim: cleaning logic is validated interactively before it ever touches production volume.

10.3.2 Directives in Practice

Directives cover parsing (CSV, JSON, dates), extraction (regex groups, URL and email components), quality rules (null handling, type coercion), and masking. For the Thursday lab, the key directive extracts the domain from an email column: split on the @ and keep the right-hand side, lowercased. Directives are readable enough to serve as reviewable documentation of business cleaning rules.

Tip

Increase the Wrangler sample size when a rule depends on rare values—a 1,000-row default sample can hide the malformed rows that motivated the rule. Also profile columns (null rate, distinct count, min/max) before writing directives, not after.

Note

Wrangler directives execute on Spark at run time; they are not a separate engine. Anything Wrangler cannot express drops down to a custom transform plugin—the escape hatch keeps the visual tool honest without blocking edge cases.

10.4 Wednesday: Plugins, Macros, and Scheduling

10.4.1 The Plugin Ecosystem

The Hub offers hundreds of plugins: sources (Cloud Storage, BigQuery, relational databases via JDBC, SaaS connectors), transforms (joins, aggregations, Wrangler, Python via custom plugins), and sinks (BigQuery, Cloud Storage, Pub/Sub). Choosing a plugin is a governance decision: prefer maintained first-party plugins, pin versions, and review what each connector does with credentials—database plugins want a service account or stored credentials, which belong in secure keys, not in exported pipeline JSON.

10.4.2 Macros and Parameterization

Macros make one pipeline definition serve many contexts: `${logicalStartTime}` resolves to the run’s logical date, `${input.path}` to a runtime argument. The canonical pattern parameterizes

input and output paths by date—`gs://bucket/raw/dt=${logicalStartTime(yyyy-MM-dd)}`—so backfills, reruns, and daily schedules all flow through one pipeline definition with deterministic, idempotent paths.

10.4.3 Scheduling and Triggers

Deployed pipelines run on time-based schedules (cron-like), on triggers from another pipeline's completion, or on-demand via REST. For the daily-reporting pattern: a single time schedule, macros for the date, and a downstream BigQuery load that truncates the target partition keep the whole chain idempotent.

Common Pitfall

Do not let every analyst deploy a personal schedule against production sources. Ungoverned schedules multiply load on source systems and duplicate loads into BigQuery. Centralize scheduling, require partition-overwrite sinks, and alert on unexpected run counts.

10.5 Thursday Hands-On Project: No-Code Customer Cleansing to BigQuery

This lab builds a Data Fusion pipeline that reads messy customer CSVs from Cloud Storage, cleans them in Wrangler—including extracting the domain from each email—and writes curated rows to BigQuery. Work in a sandbox instance; delete the instance after the lab.

10.5.1 Stage the Messy Data

```

1 $env:BUCKET = "my-gcp-datafusion-lab"
2
3 @'
4 customer_id,full_name,email,signup_date,country
5 1,Ada Lovelace,Ada@Example.COM,2026-07-01,UK
6 2,grace hopper,GRACE.HOPPER@example.com,07/02/2026,usa
7 3,Alan Turing,,2026-07-03,UK
8 4,Katherine Johnson,kjohnson@EXAMPLE.org,2026-07-03,USA
9 '@ | Set-Content -Path .\customers-messy.csv
10
11 gcloud storage cp .\customers-messy.csv '
12 gs://${env:BUCKET}/raw/customers/customers-messy.csv

```

Listing 10.1: Staging a deliberately messy customer CSV.

10.5.2 Build the Pipeline

1. In the Data Fusion Studio, add a **Cloud Storage source** plugin pointing at the raw prefix, format CSV, with header.
2. Add a **Wrangler** node. On the sample: parse the CSV, title-case `full_name`, lowercase and trim `email`, extract the email domain into `email_domain`, normalize `country` to ISO codes, and filter rows where `email` is null.
3. Add a **BigQuery sink** plugin writing to `curated.customers` with the table schema explicitly defined, and the write disposition set to truncate the target table (idempotent re-runs).
4. Validate the pipeline (the Studio checks schema propagation), then run once on demand and inspect the BigQuery table.

5. Deploy the pipeline with a daily schedule and a macro-parameterized input path.

Deliverables

- The exported pipeline JSON committed to the repository (credentials externalized as secure keys).
- A screenshot of the Wrangler directives panel alongside the raw sample.
- The resulting `curated.customers` rows showing cleaned names, lowercase emails, extracted domains, and dropped null-email rows.

Acceptance Criteria

- The pipeline runs end-to-end from the Studio with no hand-written code.
- Wrangler directives implement the domain extraction and the null-email filter.
- The BigQuery sink is idempotent: re-running the same date produces no duplicate rows.

Stretch Goals

- Parameterize the input path with the logical-date macro and run a simulated backfill for three dates.
- Add a conditional branch that routes rows with unknown countries to a quarantine prefix.
- Export the pipeline JSON and re-import it into a second instance to prove portability.

10.6 Friday Use Case and Architecture: Data Fusion versus Dataflow for Analyst Teams

10.6.1 Scenario

A retailer's business analysts must build and maintain fifteen daily reporting pipelines: POS extracts, loyalty feeds, inventory snapshots. They are fluent in SQL and spreadsheets but write no Java or Python. Today each pipeline is a support ticket to a central engineering team with a six-week queue.

10.6.2 Decision Framework

Data Fusion fits when builders are analysts, sources and sinks are standard, transformations are row-level cleaning and joins, and the platform team can govern instances, plugins, and schedules centrally. Dataflow (Chapters 14–15) fits when pipelines need custom logic, streaming semantics, windowing, or per-record state—capabilities worth the engineering cost. The honest hybrid, and the recommendation here: Data Fusion for the analyst-owned daily feeds on governed shared instances, Dataflow reserved for the streaming and custom-logic minority, and BigQuery SQL (Chapter 24's Dataform) for in-warehouse transformations.

Criterion	Choose Data Fusion	Choose Dataflow
Builder persona	Analysts, integration engineers	Software engineers
Logic complexity	Row cleaning, joins, lookups	Custom state, windowing, ML calls
Streaming requirements	Batch (or CDAP replication)	First-class, exactly-once streaming
Connector needs	Standard sources/sinks in Hub	Arbitrary custom I/O
Governance model	Central instance, curated plugins	Code review, CI/CD, templates
Cost shape	Instance + ephemeral Dataproc	Per-job worker vCPU/memory

Table 10.1: Data Fusion versus Dataflow decision matrix.

Tip

Whichever tool wins, keep one governance invariant: every pipeline—visual or coded—lands in version control, runs on a central schedule with idempotent sinks, and reports failures to the same on-call channel. Tool pluralism is fine; operational pluralism is not.

Common Pitfall

Data Fusion instance editions are billed continuously while running, even with zero pipelines executing. Development instances left running overnight are the classic budget leak—schedule instance shutdown or use smaller editions for development.

10.7 Interview Q&A

10.7.1 Concepts and Trade-offs

- Q: What underlying framework executes Data Fusion pipelines?**
 A: CDAP applications compiled to Spark jobs running on ephemeral Dataproc clusters provisioned per run.
- Q: What is Wrangler’s role before a pipeline runs at scale?**
 A: It lets an analyst iteratively define and validate cleaning directives on a data sample; the directives then execute at full scale inside the pipeline.
- Q: When should you choose Data Fusion over Dataflow?**
 A: When builders are analysts, sources and sinks are standard, and logic is row-level cleaning and joins rather than custom streaming or stateful processing.
- Q: Why use a private IP Data Fusion instance?**
 A: To reach on-premises and internal data sources over private networking without exposing endpoints publicly—required in most regulated environments.
- Q: What do macros parameterize in a deployed pipeline?**
 A: Runtime values such as the logical date, input/output paths, and environment-specific settings, enabling one definition to serve schedules, backfills, and reruns.

10.8 Further Reading

Read the Cloud Data Fusion documentation for instance setup, Wrangler directives, and plugin development [18]. Review Chapter 9’s Dataproc documentation for the ephemeral clusters that execute pipelines [19], and the BigQuery documentation for sink schemas and write dispositions [9]. For the comparison discussion, the Dataflow documentation previews the streaming capabilities that justify a coded engine [30].

Key Takeaways

- Data Fusion is managed CDAP: visual pipelines compiled to Spark on ephemeral Dataproc clusters.
- Wrangler’s sample-first directive loop validates cleaning logic interactively before production scale.
- Private instances plus VPC integration make visual ETL viable in regulated network topologies.
- Macros and idempotent sinks turn one pipeline definition into safe schedules, backfills, and reruns.
- The tool decision is a persona decision: analysts build in Data Fusion; engineers build streaming and custom logic in Dataflow.
- Instance editions bill while running—govern development instances like any other always-on resource.

10.9 Exercises

1. **(Conceptual)** Explain how the ephemeral-Dataproc execution model aligns Data Fusion with the cost discipline of Chapter 9.
2. **(Conceptual)** A team proposes building a real-time fraud pipeline in Data Fusion. Explain why this strains the tool and what you would propose instead.
3. **(Hands-on)** Reproduce the Thursday lab, then add a Wrangler directive that hashes the email column with SHA-256 for privacy.
4. **(Hands-on)** Export the pipeline JSON, inspect where the email directive and sink schema live, and commit it with a README describing promotion across instances.
5. **(Design)** Draw the platform topology for the Friday retailer: instances per environment, plugin governance, network paths, and the central scheduling pattern.
6. **(Analysis)** A pipeline suddenly doubles its run time after a source change. List the four most likely causes and how you would isolate each in Data Fusion’s run logs.

Change Data Capture with Datastream

Learning Objectives

- Explain change data capture concepts and Datastream's log-based architecture.
- Configure Datastream sources for PostgreSQL, MySQL, and Oracle with correct prerequisites.
- Choose between Cloud Storage and BigQuery destinations and design the load contract around schema enforcement.
- Classify schema changes as breaking or non-breaking and assign data-contract ownership.
- Build a Cloud SQL PostgreSQL to BigQuery CDC stream and test it with additive and renaming schema changes.
- Architect zero-downtime replication from on-premises Oracle to BigQuery for real-time BI.

Cross-Cloud Equivalence

Change data capture is a universal pattern: read the database's transaction log, not its tables, and deliver ordered insert/update/delete events to analytical targets.

Capability	GCP Datastream	AWS DMS	Azure
CDC engine	Log-based, serverless	DMS CDC tasks	Azure Data Factory CDC / SQL CDC
PostgreSQL source	Logical replication (pgoutput / test_decoding)	wal2json / test_decoding	Azure Database for PG logical replication
Oracle source	LogMiner / binary reader	LogMiner / binary reader	(via partner or self-managed)
Managed targets	Cloud Storage, BigQuery	S3, Redshift, Kafka	ADLS, Synapse, Event Hubs
Backfill	Built-in initial snapshot	Full load + CDC	Snapshot + CDC

Open-source **Debezium** on Kafka is the self-managed counterpart everywhere; **Snowflake** consumes CDC through Snowpipe and streams; **MongoDB** has change streams built into the database itself. The architectural constant: the transaction log is the source of truth, and every destination is a eventually-consistent projection of it.

Key Terms

Change data capture (CDC): extracting committed inserts, updates, and deletes from a database's transaction log rather than querying tables. **Datastream:** Google Cloud's serverless CDC and replication service. **Logical replication:** PostgreSQL's mechanism for streaming row-level changes from the write-ahead log (WAL). **Backfill:** the initial snapshot that establishes a baseline before incremental changes stream. **Data contract:** the agreed schema, semantics, and change policy between a data producer and its consumers. **Schema drift:** structural change at the source—added, renamed, or retyped columns—that a replication pipeline must absorb.

11.1 Week 11 Roadmap

Week 11 closes the gap between operational databases and analytical platforms without nightly batch windows. Monday covers CDC concepts and Datastream architecture. Tuesday configures PostgreSQL, MySQL, and Oracle sources. Wednesday covers destinations and the data-contract discipline that keeps replication from silently corrupting consumers. Thursday builds a Cloud SQL PostgreSQL to BigQuery stream and deliberately breaks it. Friday architects zero-downtime Oracle-to-BigQuery replication for real-time BI.

Definition

Change data capture reads a database's transaction log—the PostgreSQL WAL, MySQL binlog, or Oracle redo logs—to produce an ordered stream of committed changes. Unlike polling queries, CDC captures every intermediate state change, adds near-zero load to the source, and sees deletes that queries can never observe [33].

11.2 Monday: CDC Concepts and Datastream Architecture

11.2.1 Why the Log, Not the Table

Polling-based replication (`SELECT * WHERE updated_at > :watermark`) misses deletes, misses intermediate updates between polls, requires perfect watermark discipline, and hammers the source with range scans. Log-based CDC reads the write-ahead log that the database already maintains for its own recovery: every committed insert, update, and delete, in commit order, with almost no additional load on the serving database. This is why CDC has become the default bridge from OLTP to analytics.

11.2.2 Datastream Architecture

Datastream is serverless: you define **connection profiles** (source and destination), a **stream** binding them with an object selection (schemas, tables, columns), and Datastream runs backfill plus continuous replication without servers to size. For BigQuery destinations, Datastream applies changes near-real-time into managed tables; for Cloud Storage, it lands change events as files (Avro or JSON) with a configurable rotation, leaving merge semantics to downstream pipelines—often Dataflow (Chapter 14) or scheduled BigQuery merges.

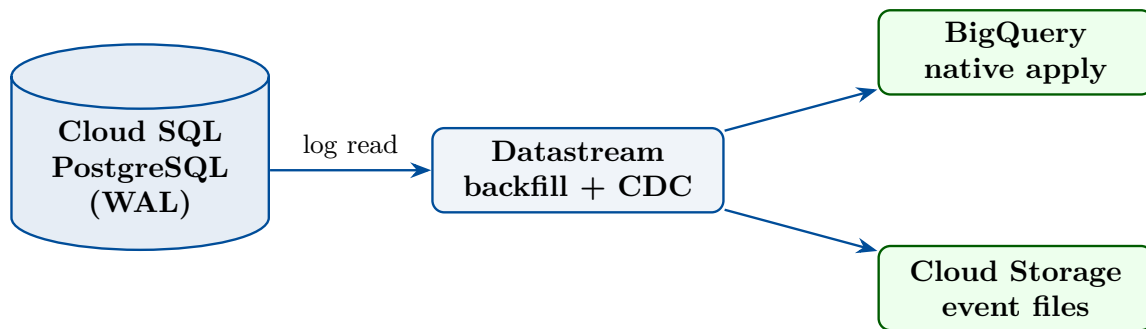


Figure 11.1: Datastream reads the source transaction log once and projects it to BigQuery or Cloud Storage.

Note

Backfill and CDC overlap by design: the snapshot establishes the baseline while the log stream buffers changes; Datastream reconciles the seam. Do not freeze source writes during backfill—that defeats the entire point of CDC-based zero-downtime migration.

11.3 Tuesday: Source Configurations

11.3.1 PostgreSQL

PostgreSQL sources require `wal_level = logical`, a publication over the replicated tables, and a replication slot—the slot is the contract that tells the server to retain WAL segments until Datastream consumes them. Tables without a primary key need `REPLICA IDENTITY FULL` so updates and deletes carry enough row state. On Cloud SQL these map to database flags (`cloudsql.logical_decoding = on`) plus a user with replication privileges.

11.3.2 MySQL and Oracle

MySQL sources stream from the binlog: enable binary logging, row-based format, and a server ID, with binlog retention long enough to survive outages. Oracle sources use LogMiner or binary log reading against redo and archive logs, with supplemental logging enabled on replicated tables—the Oracle path demands the most DBA cooperation and the longest lead time.

Source	Key prerequisites	Classic failure
PostgreSQL	Logical WAL level, publication, replication slot, replica identity	Slot fills disk when consumer stalls
MySQL	Row-based binlog, retention window, server ID	Binlog purged before consumer catches up
Oracle	Archive logging, supplemental logging, LogMiner access	Missing supplemental data breaks row reconstruction

Table 11.1: CDC source prerequisites and classic failures [33].

Common Pitfall

A stalled consumer with a live replication slot retains WAL forever: PostgreSQL will fill its disk and take an outage rather than lose your changes. Monitor slot lag and alert long before the disk fills; this failure mode is the price of guaranteed delivery.

11.4 Wednesday: Destinations and Data Contracts

11.4.1 Cloud Storage versus BigQuery

The **BigQuery destination** applies changes directly into managed tables—append-only with metadata columns, or merged current state—giving near-real-time queryable replicas with zero downstream code. The **Cloud Storage destination** lands versioned change-event files: more control over merge semantics, a durable replayable archive of every change, and a natural hand-off to Dataflow or batch merges—at the cost of building that downstream logic yourself. Choose BigQuery for BI-facing replicas; choose Cloud Storage when you need the immutable event log, multi-target fan-out, or custom merge rules.

11.4.2 Schema Enforcement as Contract

BigQuery load jobs enforce schema by default: rows carrying unknown fields or mismatched types are rejected rather than silently absorbed. This rejection is the *contract speaking*: the producer promised a shape, and the load is the enforcement point. Make the contract explicit—versioned schema, documented semantics, named producer and consumer owners—and treat enforcement failures as contract violations routed to an owner, not as noise to suppress.

11.4.3 Breaking versus Non-Breaking Changes

Non-breaking: adding a nullable column (old consumers ignore it, new consumers tolerate its absence in old rows), widening a type compatibly, adding a table. **Breaking**: renaming or dropping a column, changing a type incompatibly, changing key semantics. The policy follows directly: producers may ship non-breaking changes unilaterally; breaking changes require a versioned rollout—new column alongside old, consumer migration, then removal (the expand-contract pattern from Chapter 24).

Definition

Producer-owned contract: the team writing the source system publishes and versions the schema, announces changes, and honors deprecation windows. **Consumer-owned**: consumers declare required fields and validate them. Healthy platforms do both: producers publish; consumers verify.

Tip

Alert on Datastream’s data-freshness metric and failed events, and write the runbook before the first incident: how to detect a stale log position, how to backfill through a Cloud Storage staging replay when BigQuery apply falls behind, and who owns the contract conversation with the source team.

11.5 Thursday Hands-On Project: PostgreSQL CDC to BigQuery with a Contract Test

This lab configures logical replication on Cloud SQL for PostgreSQL, streams changes to BigQuery through Datastream, then tests the contract: add a nullable column (non-breaking) and rename one (breaking), observing Datastream’s drift handling.

11.5.1 Prepare the Source

```

1 $env:PROJECT_ID = "my-gcp-datastream-lab"
2 $env:INSTANCE = "cdc-source-lab"
3 $env:REGION = "us-central1"
4
5 gcloud sql instances create $env:INSTANCE '
6   --database-version=POSTGRES_16 --region=$env:REGION '
7   --database-flags=cloudsql.logical_decoding=on '
8   --tier=db-custom-2-7680
9
10 gcloud sql users create cdc_user --instance=$env:INSTANCE --password='REPLACEME
11 ,
12 # In psql as a privileged user:
13 #   ALTER USER cdc_user WITH REPLICATION;
14 #   CREATE TABLE shop.orders(order_id int PRIMARY KEY, amount numeric, status
15 #   CREATE PUBLICATION ds_pub FOR TABLE shop.orders;

```

Listing 11.1: Enabling logical decoding and creating a publication on Cloud SQL PostgreSQL.

11.5.2 Create the Stream

1. Create connection profiles: source (Cloud SQL PostgreSQL, private IP or forward-SSH tunnel per network) and destination (BigQuery).
2. Create the stream selecting `shop.orders`, with automatic backfill, targeting dataset `cdc_replica`.
3. Validate the stream; watch it transition through backfill into CDC.
4. Insert, update, and delete rows in the source; confirm the changes appear in BigQuery within the freshness window.

11.5.3 Test the Contract

```

1 -- Non-breaking: add a nullable column. The stream absorbs it.
2 ALTER TABLE shop.orders ADD COLUMN discount_pct numeric;
3
4 -- Breaking: rename a column. Observe how the stream and
5 -- downstream queries respond; document the remediation.
6 ALTER TABLE shop.orders RENAME COLUMN status TO order_status;

```

Listing 11.2: Non-breaking then breaking schema changes against the live stream.

Record what each change does to the destination table and to any consumer queries: the additive change should flow through with nulls in old rows; the rename should surface as a contract violation—old column frozen, new column appearing—requiring a coordinated consumer migration.

Deliverables

- Screenshots or CLI output of the validated stream in CDC state, with the freshness metric.
- A change log demonstrating insert/update/delete propagation within the measured latency.
- A contract-test report: observed behavior for the additive and the renaming change, and the remediation runbook for the breaking case.

Acceptance Criteria

- The source uses logical replication with a publication and replication slot.
- DML on the source appears in BigQuery near-real-time without batch jobs.
- The two schema-change cases are executed and their destination behavior documented accurately.

Stretch Goals

- Add a Cloud Monitoring alert on the stream's freshness metric and a notification channel.
- Re-create the stream to a Cloud Storage destination and compare event-file semantics with the BigQuery apply.
- Write a consumer-side view that tolerates both `status` and `order_status` during a migration window.

11.6 Friday Use Case and Architecture: Zero-Downtime Oracle to BigQuery

11.6.1 Scenario

A manufacturer's on-premises Oracle database runs the ERP of record. The BI team needs near-real-time dashboards in BigQuery; the business forbids downtime beyond a maintenance-minute cutover; the DBAs will permit read-only log access and nothing more.

11.6.2 Architecture

Datastream connects to Oracle through a secure network path (VPN or Interconnect) with LogMiner-based capture, supplemental logging enabled on the replicated schemas. Backfill establishes the baseline while production continues writing; CDC then keeps BigQuery within seconds of the source. A Cloud Monitoring alert on freshness is the operational heartbeat. For BI, expose curated views over the replica tables so dashboard queries never touch raw CDC metadata columns, and schedule a daily reconciliation (row counts and key checksums per table) between Oracle and BigQuery as the trust anchor.

Common Pitfall

Oracle redo/archive retention bounds how long the stream may be down. If retention is 48 hours and the stream stalls over a long weekend, the log position is unrecoverable and the table must be re-backfilled. Size archive storage and alert on stall duration against the retention budget, not against vibes.

Tip

Keep the initial object list narrow: the thirty tables the BI team needs this quarter, not all four thousand. Every replicated table is a schema-drift obligation; start with a contracted subset and expand deliberately.

11.7 Interview Q&A

11.7.1 Concepts and Trade-offs

1. **Q: How does Datastream capture changes without querying the source tables?**
A: It reads the transaction log (WAL, binlog, or redo logs) that the database maintains for recovery, yielding committed changes in order with minimal source load.
2. **Q: Which is a breaking contract change: adding a nullable column or renaming one?**
A: Renaming. Adding a nullable column is backward compatible; renaming breaks every consumer referencing the old name and requires a coordinated rollout.
3. **Q: What does BigQuery schema enforcement on load reject by default?**
A: Rows with unknown fields or type mismatches—the load is the contract enforcement point.
4. **Q: Who should own a data contract—producer, consumer, or both?**
A: Both: producers publish and version the schema with deprecation discipline; consumers declare and validate what they depend on.
5. **Q: What does the data freshness metric tell you about CDC lag?**
A: How far behind real time the destination is; a growing value signals a stalled consumer, a source outage, or an apply bottleneck before users notice stale dashboards.

11.8 Further Reading

Read the Datastream documentation for source prerequisites, stream configuration, and destination behavior [33]. Review Cloud SQL’s logical-decoding flags [26] and BigQuery’s load-time schema enforcement [9]. Cloud Monitoring documentation covers the freshness alerting that makes CDC operationally safe [21]. For the migration context, Database Migration Service documentation describes the related homogeneous-migration path [29].

Key Takeaways

- CDC reads the transaction log: complete, ordered, low-impact change streams that polling can never match.
- Source prerequisites are the project: logical WAL, publications, retention, and replica identity must be right before any stream validates.
- BigQuery destinations give queryable replicas; Cloud Storage destinations give replayable event logs—choose per downstream need.
- Schema enforcement at load is the data contract speaking; route violations to owners instead of suppressing them.
- Breaking changes demand expand-contract rollouts; non-breaking changes flow freely.
- Freshness alerting and a backfill runbook are not optional extras—they are the difference between replication and hope.

11.9 Exercises

1. **(Conceptual)** Explain three classes of changes polling-based replication misses that log-based CDC captures.
2. **(Conceptual)** Why does a stalled consumer threaten the *source* database's availability, and what alerts prevent it?
3. **(Hands-on)** Reproduce the Thursday lab, then measure end-to-end latency from source commit to BigQuery visibility across ten transactions.
4. **(Hands-on)** Force a failed-event condition (for example, a type mismatch) and document the recovery path.
5. **(Design)** Write the data-contract document for `shop.orders`: fields, semantics, versioning rule, change-announcement process, and owners.
6. **(Architecture)** Extend the Friday design with a Cloud Storage event-log destination feeding a replay-based disaster recovery procedure.

Pipeline Orchestration with Cloud Composer

Learning Objectives

- Author Airflow DAGs using operators, sensors, dependencies, XComs, and trigger rules.
- Explain Cloud Composer 2 architecture: GKE-based, autoscaled, and environment-scoped.
- Orchestrate cross-service work—GCS sensors, Dataproc jobs, BigQuery loads—from one DAG.
- Implement idempotent, retry-safe tasks with failure notification and exponential back-off.
- Build the file-triggered Dataproc-to-BigQuery DAG with email-on-failure semantics.
- Design a self-healing orchestration layer for a banking reconciliation across fifteen systems.
- Complete the Milestone 3 capstone: a scheduled, monitored, end-to-end batch ELT pipeline.

Cross-Cloud Equivalence

Managed Airflow is the most portable orchestration story in cloud data engineering—the same DAG Python runs on every platform.

Capability	GCP Composer	AWS MWAA	Azure / Fabric
Managed Airflow	Composer 2 (GKE, autoscaling)	MWAA (ECS-based)	Managed Airflow in ADF / Fabric
Triggering	Cloud Scheduler, sensors, API	EventBridge, sensors	Event Grid, tumbling window triggers
Secret storage	Secret Manager integration	Secrets Manager	Key Vault
Worker identity	GKE Workload Identity	IAM task roles	Managed identity
DAG storage	Cloud Storage bucket synced	S3 bucket synced	ADLS / Git integration

Alternatives map by philosophy: **Snowflake Tasks** and **dbt Cloud** schedules keep orchestration in-warehouse for SQL-only pipelines; **Microsoft Fabric pipelines** target low-code orchestration; **Dagster** and **Prefect** offer asset-centric alternatives. Airflow's task-centric

DAG model remains the lingua franca the others are compared against.

Key Terms

DAG: a directed acyclic graph—an Airflow pipeline definition of tasks and their dependencies. **Operator:** a task template performing an action (run a job, execute SQL); a **sensor** is an operator that waits for a condition. **XCom:** Airflow’s mechanism for passing small values between tasks. **Trigger rule:** the condition under which a task runs relative to its upstream states (e.g., `all_done`). **Catchup:** Airflow’s behavior of running missed historical schedule intervals when a DAG is unpaused. **Cloud Composer:** Google Cloud’s managed Airflow; version 2 runs on autoscaling GKE clusters.

12.1 Week 12 Roadmap

Week 12 assembles Part III’s services into coordinated systems. Monday covers Airflow’s mental model: DAGs, operators, tasks, XComs. Tuesday opens the Composer 2 box. Wednesday orchestrates across Dataproc, Dataflow, and BigQuery. Thursday builds the file-triggered DAG with failure notification. Friday designs the fifteen-system banking reconciliation. The week closes with the Milestone 3 capstone.

Definition

Cloud Composer is Google Cloud’s managed Apache Airflow service. It hosts the scheduler, web server, metadata database, and—in version 2—an autoscaling pool of GKE workers, so teams write DAGs rather than operate Airflow infrastructure [17, 64].

12.2 Monday: Airflow Concepts—DAGs, Operators, Tasks, XComs

12.2.1 The DAG Mental Model

A DAG declares *what* depends on *what*, not *when* code runs beyond the schedule. The scheduler materializes DAG runs per logical interval; each task instance executes an operator with rendered context (the logical date `ds`, run IDs). Dependencies set with `>>` define execution order. The discipline that makes all of it safe: **idempotent tasks**—any task, run twice for the same logical date, produces the same effect as running once. Overwrite the partition; never append blindly.

12.2.2 Sensors and Deferrable Execution

Sensors wait: `GCSObjectExistenceSensor` pokes until a file appears. Classic sensors occupy a worker slot while poking; **deferrable** sensors release the worker and resume from a trigger process—the production-correct choice for long waits. Use sensors for external readiness (a partner’s file), schedules for time-based cadence; mixing them—schedule daily, then sense for the file—covers the real world where “daily” means “when the upstream delivers.”

12.2.3 XComs: Small Messages Only

XComs pass *small* values between tasks—a row count, a validated path, a quality verdict. They are stored in the metadata database and serialized per push. Passing DataFrames, file contents, or large JSON payloads through XComs stalls the scheduler and bloats the metadata database; pass references (a `gs://` path) and let tasks read the payload themselves.

Common Pitfall

`catchup=True` on a newly deployed DAG floods the environment with weeks of backfill runs at once, contending for workers and rate limits. Disable `catchup` for routine DAGs, or cap `max_active_runs` and schedule deliberate backfills instead.

12.3 Tuesday: Cloud Composer 2 Architecture

12.3.1 Under the Hood

A Composer 2 environment is a tenant project running GKE: the Airflow scheduler, web server, triggerer, and workers are workloads on an autoscaling cluster; the metadata database is a managed Cloud SQL instance; the DAGs folder is a Cloud Storage bucket synced continuously. You size the environment by workload config (scheduler and worker CPU/memory), worker count bounds, and environment size tier—then Airflow’s Kubernetes executor scales workers with task demand.

12.3.2 Operational Surface

The operational levers are few but sharp: worker autoscaling bounds, the `[core] parallelism` family of settings, DAG-folder sync (push code by writing to the bucket, ideally via CI), PyPI package installation into the environment image, and environment snapshots for backup and disaster recovery. Monitor scheduler heartbeat, task-queue depth, and the metadata database; most Composer incidents are exhausted workers, a wedged scheduler, or a metadata database under-sized for thousands of task instances.

Tip

Pin PyPI dependencies and Airflow provider versions in the environment configuration. An unpinned provider upgrade changing an operator’s parameters is a deploy failure disguised as a platform upgrade; treat provider upgrades as code changes with review and testing.

Note

Composer environments are regional and have a base cost floor—schedulers and database run continuously even with zero DAGs. Consolidate small teams onto shared environments with clear `dag_prefix` conventions rather than one environment per team.

12.4 Wednesday: Cross-Service Orchestration

12.4.1 Google Cloud Operators

The Google provider package supplies operators and sensors for the whole platform: `GCSObjectExistenceSensor`, `DataprocSubmitJobOperator` (and `DataprocCreateClusterOperator`/`DataprocDeleteClusterOperator` for ephemeral clusters), `DataflowTemplatedJobStartOperator`, `BigQueryInsertJobOperator` for SQL, and `GCSToBigQueryOperator` for loads. A well-formed cross-service DAG sequences the services from earlier chapters: sense the arrival, run the Spark job on an ephemeral cluster, load the partition, verify quality, notify.

12.4.2 Design Rules for Orchestrated Pipelines

Three rules keep multi-service DAGs operable. **Deterministic paths:** every task derives its inputs and outputs from the logical date, so reruns overwrite rather than duplicate. **Verify**

between stages: insert cheap data-quality gates (row-count reconciliation, freshness checks) as tasks between expensive stages, so a bad input fails fast. **Notify on the terminal task:** attach failure callbacks or an `all_done` terminal notifier, so humans hear about every completion state exactly once.

```

1 from datetime import datetime, timedelta
2 from airflow import DAG
3 from airflow.providers.google.cloud.sensors.gcs import GCSObjectExistenceSensor
4 from airflow.providers.google.cloud.operators.dataproc import
  DataprocSubmitJobOperator
5 from airflow.providers.google.cloud.operators.bigquery import
  BigQueryInsertJobOperator
6 from airflow.operators.email import EmailOperator
7
8 default_args = {"retries": 2, "retry_delay": timedelta(minutes=5)}
9
10 with DAG("daily_sales_elt", schedule="@daily", start_date=datetime(2026, 1, 1),
11         default_args=default_args, catchup=False) as dag:
12     wait_for_file = GCSObjectExistenceSensor(
13         task_id="wait_for_file", bucket="raw-sales", object="daily/{{ ds }}/
14         sales.csv")
15     spark_job = DataprocSubmitJobOperator(
16         task_id="spark_transform", project_id="my-project", region="us-central1",
17         job={"placement": {"cluster_name": "ephemeral-etl"},
18             "pyspark_job": {"main_python_file_uri": "gs://code/transform.py"}
19     })
20     load_bq = BigQueryInsertJobOperator(
21         task_id="load_bq",
22         configuration={"query": {"query": "CALL analytics.refresh_daily_sales()",
23                                "useLegacySql": False}})
24     notify = EmailOperator(task_id="notify", to=["data-oncall@example.com"],
25                           subject="daily_sales_elt finished", html_content="
26     Done.",
27                           trigger_rule="all_done")
28     wait_for_file >> spark_job >> load_bq >> notify

```

Listing 12.1: The daily-sales DAG: sense, transform, load, notify.

Common Pitfall

Append-mode loads double-count rows when a task retries. Always write with deterministic partition targets—`WRITE_TRUNCATE` on the logical-date partition or a deterministic object path—so retries and backfills are safe by construction.

12.5 Thursday Hands-On Project: The File-Triggered ELT DAG

This lab deploys the Wednesday DAG to a Composer environment: it waits for a daily file in Cloud Storage, triggers a Dataproc job, loads results to BigQuery, and sends email on completion or failure.

12.5.1 Prepare the Environment

```

1 $env:PROJECT_ID = "my-gcp-composer-lab"
2 $env:REGION = "us-central1"
3 $env:ENV = "daily-elt-lab"
4
5 gcloud composer environments create $env:ENV '
6 --location=$env:REGION --image-version=composer-2-airflow-2

```

```

7
8 # Find the DAGs bucket:
9 gcloud composer environments describe $env:ENV --location=$env:REGION '
10 --format="value(config.dagGcsPrefix)"

```

Listing 12.2: Creating a Composer 2 environment and locating its DAGs bucket.

12.5.2 Deploy and Test

1. Copy the DAG file to the environment's DAGs prefix with `gcloud storage cp`; wait for the DAG to parse (check the Airflow UI for import errors).
2. Create the BigQuery dataset and the `analytics.refresh_daily_sales()` procedure (a simple partition-overwrite insert for the lab).
3. Trigger the DAG unparsed and confirm the sensor waits; upload `daily/2026-07-31/sales.csv` to `raw-sales` and watch the run proceed through all four tasks.
4. Force a failure (point the Dataproc job at a missing script), confirm the email arrives and the retry policy engages.

Deliverables

- The DAG file with a README describing the schedule, sensor target, and notification path.
- Screenshots of the successful and failed runs in the Airflow grid view.
- A one-paragraph idempotency argument: why rerunning any date is safe.

Acceptance Criteria

- The DAG gates on file existence rather than wall-clock assumptions.
- Retries use backoff and the terminal task notifies under `all_done`.
- Rerunning a historical date via the Airflow UI produces no duplicate BigQuery rows.

Stretch Goals

- Convert the sensor to its deferrable variant and compare worker-slot occupancy.
- Wrap the Dataproc stage in create/delete cluster operators for a fully ephemeral run.
- Add a data-quality task between transform and load that fails the DAG when row counts regress by more than 10%.

12.6 Friday Use Case and Architecture: Fifteen-System Banking Reconciliation

12.6.1 Scenario

A bank reconciles fifteen dependent systems nightly: core banking extracts, card network files, general ledger, fraud case queue, and more. Files arrive at different times from different parties; a late file must not silently produce a wrong reconciliation; failures at 03:00 must page the right team with enough context to act.

12.6.2 Design

Model the reconciliation as one orchestrated DAG per business date with explicit system-level tasks. Each of the fifteen inputs gets a sensor with a deadline; a per-system validation task (schema, row counts against control totals, duplicate detection) gates its ingestion; reconciliation tasks consume only validated inputs; a final task writes a signed-off reconciliation report and control totals to an audit table. SLAs are declared on the DAG (`sla` on critical tasks plus Cloud Monitoring alerts on SLA misses), and every task is idempotent so the on-call runbook is “fix the input, clear the task, rerun.”

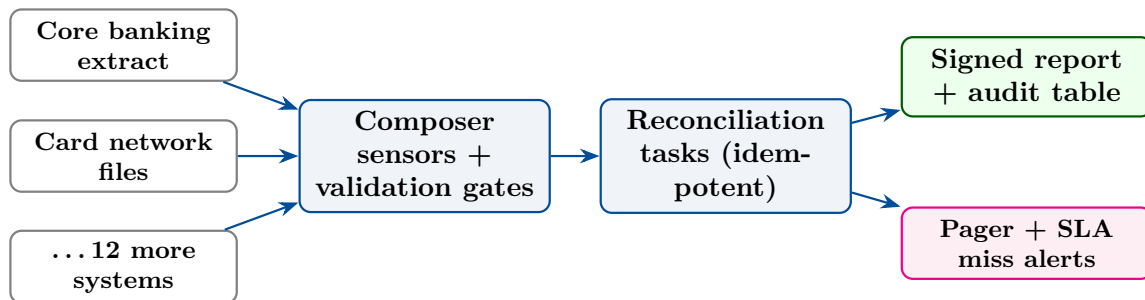


Figure 12.1: Banking reconciliation: sensed, validated inputs; idempotent reconciliation; signed output and alerting.

Tip

“Self-healing” is bounded automation, not magic: retries with exponential backoff for transient failures, automatic rerun-after-fix for data failures, and a page only when the system has exhausted safe recovery. Every page should represent a decision a human must actually make.

12.7 Interview Q&A

12.7.1 Concepts and Trade-offs

1. **Q: Why must Airflow tasks be idempotent for safe retries?**
A: Retries, backfills, and clears rerun tasks freely; only idempotent tasks—deterministic partition overwrites, upserts—make reruns indistinguishable from first runs.
2. **Q: What does the `all_done` trigger rule guarantee here?**
A: The notification task runs after all upstream tasks finish regardless of success or failure, so the team hears exactly once about every run outcome.
3. **Q: What runs underneath Cloud Composer 2?**
A: An autoscaling GKE cluster hosting Airflow scheduler, workers, web server, and triggerer, plus a managed Cloud SQL metadata database and a synced Cloud Storage DAG bucket.
4. **Q: When should you use a GCS sensor instead of a schedule?**
A: When readiness is event-driven—a partner file arrives at variable times—so the pipeline gates on the artifact, not the clock.
5. **Q: What information should flow through XComs—and what should not?**
A: Small control values: paths, counts, verdicts. Never payloads, DataFrames, or large JSON—those belong in storage with a passed reference.

12.8 Further Reading

Read the Apache Airflow documentation for DAG semantics, sensors, trigger rules, and XComs [64], and the Cloud Composer documentation for environment sizing, snapshots, and operations [17]. Review Cloud Scheduler for external cron triggering [22] and Cloud Monitoring for SLA-miss and failure alerting [21]. The Dataproc and Dataflow operator references live in their service documentation [19, 30].

Key Takeaways

- A DAG declares dependencies and cadence; idempotent tasks make retries, backfills, and clears safe.
- Sensors gate on external readiness; prefer deferrable sensors for long waits.
- XComs carry control signals, never payloads.
- Composer 2 is managed GKE-based Airflow: size workers, pin providers, snapshot environments.
- Cross-service DAGs sequence the platform: sense, transform on ephemeral compute, load with partition overwrite, verify, notify.
- Resilient orchestration pages humans only for decisions automation cannot make.

12.9 Exercises

1. **(Conceptual)** Explain why `catchup=True` plus `max_active_runs` defaults can destabilize a new production DAG.
2. **(Conceptual)** Contrast task-centric (Airflow) and asset-centric (Dagster) orchestration for the banking use case.
3. **(Hands-on)** Reproduce the Thursday DAG, then add a SLA to the load task and trigger an SLA miss on purpose.
4. **(Hands-on)** Refactor the DAG to create and delete its own ephemeral Dataproc cluster per run.
5. **(Design)** Specify the fifteen-sensor section of the banking DAG: per-system deadlines, validation tasks, and the dependency structure that isolates one system's lateness.
6. **(Architecture)** Design environment topology (dev/staging/prod), promotion of DAG code through CI, and snapshot-based DR for Composer.

12.10 Milestone 3 Capstone: Automated Batch ELT Pipeline

Capstone Project

Mission: Use Cloud Scheduler to trigger a Composer DAG that verifies a Datastream CDC sync, triggers a Serverless Dataproc Spark job computing ML features, loads them into BigQuery with partition-overwrite idempotency, and refreshes a materialized view—with data-quality tests, failure notification, and a latency alert.

Estimated time: 8–10 hours.

What you'll practice:

- Wiring Cloud Scheduler into Composer via Pub/Sub or the Airflow REST API.
- Verifying CDC freshness and row counts as a pipeline gate.
- Running idempotent Spark feature jobs on serverless compute.
- Loading BigQuery partitions with `WRITE_TRUNCATE` and refreshing materialized views.
- Producing monitoring evidence: failure notification plus a latency alert policy.

Scenario

Northwind Analytics ships ML features nightly: customer order behavior from the Chapter 11 CDC replica, aggregated by Spark, served to downstream models and dashboards. The pipeline must be fully automated, retry-safe, and observable—no manual steps, no silent staleness.

Requirements

- Cloud Scheduler triggers the Composer DAG on a nightly schedule.
- The DAG first verifies CDC: row-count reconciliation between source and BigQuery replica plus a freshness threshold.
- A serverless Dataproc Spark job computes features (`orders_30d`, `spend_30d`) into a deterministic `gs://` path keyed by logical date.
- BigQuery load uses `WRITE_TRUNCATE` on the logical-date partition; a materialized view refreshes afterward.
- At least three automated data-quality tests with a demonstrated failing case.
- Failure email plus a Cloud Monitoring latency alert; cost estimate with two documented optimizations.

Reference Architecture

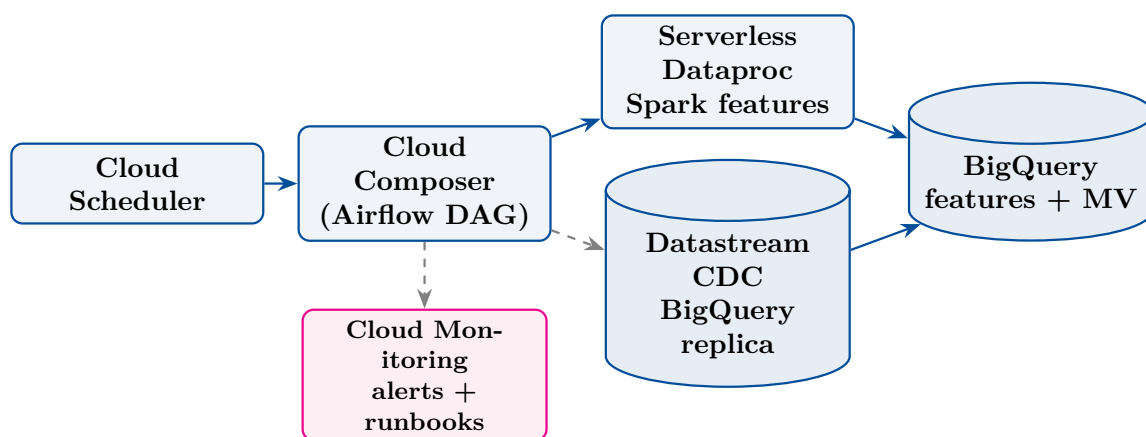


Figure 12.2: Milestone 3 reference architecture: scheduled batch ELT with CDC verification gates.

Implementation Steps

1. **Trigger.** Create the Cloud Scheduler job targeting the DAG (Pub/Sub to a sensor or REST `dagRuns` call).

2. **Gate.** First task: compare source versus replica row counts for the logical date; fail fast on divergence beyond tolerance.
3. **Compute.** Submit the serverless Spark batch with deterministic output path `gs://ml-features/dt={{ds }}`.
4. **Load.** `BigQueryInsertJobOperator` executing a partition-scoped `WRITE_TRUNCATE` insert, then the MV refresh call.
5. **Verify and notify.** Quality tasks (reconciliation, freshness, null-rate), then the `all_done` notifier.
6. **Evidence.** Export the alert policies, the cost estimate, and a screenshot of a deliberately failed quality gate paging the channel.

Deliverables

- A repository that runs end-to-end from a clean environment: DAG, Spark job, SQL, and README.
- The architecture diagram and the data-quality test suite with failing-case demonstration.
- Monitoring evidence and the monthly cost estimate with two optimizations.
- A security review: per-service service accounts, no hardcoded secrets, PII handling noted.

Acceptance Criteria

- Scheduler-triggered runs complete without manual steps; reruns and backfills are duplicate-free.
- The CDC gate demonstrably blocks the pipeline on stale or divergent replicas.
- Quality tests fail the DAG visibly; notifications fire on success and failure exactly once each.

Stretch Goals

- Add an in-DAG backfill command that safely reprocesses a date range with capped parallelism.
- Move feature computation into BigQuery SQL and compare cost and latency with the Spark path.
- Publish feature freshness to Dataplex as a data-product SLO (previewing Chapter 22).

Part IV

Streaming and Real-Time Analytics

Asynchronous Messaging with Pub/Sub

Learning Objectives

- Model Pub/Sub topics, subscriptions (push and pull), and acknowledgements correctly.
- Enforce data contracts at the edge with Avro or Protocol Buffer topic schemas.
- Publish resiliently with retry and exponential backoff, and deduplicate at-least-once delivery downstream.
- Configure dead-letter topics, retry policies, and snapshot/seek replay.
- Compare Pub/Sub Standard and Pub/Sub Lite on cost and operational overhead.
- Architect a global telemetry ingestion point for five million mobile games with exactly-once processing semantics.

Cross-Cloud Equivalence

Managed messaging services share one mental model—producers publish to named channels, consumers subscribe, and the broker owns durability and delivery—with different knobs for ordering, replay, and cost.

Capability	GCP Pub/Sub	AWS	Azure
Managed pub/sub	Pub/Sub (global)	SNS + SQS / EventBridge	Service Bus / Event Grid
Kafka-compatible	Managed Service for Apache Kafka	MSK	Event Hubs (Kafka surface)
Dead-lettering	Dead-letter topics	SQS DLQs / EventBridge DLQ	Service Bus dead-letter queues
Replay	Snapshots + seek	(archive + replay in EventBridge)	(replay via offsets on Event Hubs)
Schema enforcement	Avro / Proto topic schemas	EventBridge schemas	Schema registry (Event Hubs)
Cost-profiled tier	Pub/Sub Lite (zonal, capacity)	Kinesis on-demand vs provisioned	Event Hubs throughput units

Self-managed **Apache Kafka** remains the industry reference point for partition-based, replayable logs [67]. Pub/Sub differs deliberately: global by default, partition-free for the user, and priced per GiB delivered rather than per provisioned broker.

Key Terms

Topic: a named channel to which publishers send messages. **Subscription:** a named cursor over a topic delivering messages to consumers by push or pull. **Acknowledgement (ack):** the consumer's confirmation that a message was processed; unacked messages are redelivered. **Dead-letter topic:** a topic receiving messages that exceeded a subscription's maximum delivery attempts. **Snapshot/seek:** Pub/Sub's replay mechanism, rewinding a subscription to a retained point in time. **Exactly-once delivery:** a subscription mode where Pub/Sub itself suppresses redelivery of already-acked messages within a region.

13.1 Week 13 Roadmap

Week 13 opens Part IV with the front door of every streaming platform. Monday covers topics, subscriptions, acknowledgements, and schema enforcement at the edge. Tuesday covers failure engineering: dead-letter topics, retry policies, snapshots, and seek. Wednesday compares Pub/Sub Lite with Standard. Thursday publishes mock financial transactions with a dead-letter safety net. Friday architects global game telemetry with exactly-once semantics.

Definition

Pub/Sub is Google Cloud's global, serverless messaging service. Publishers send messages to topics; subscriptions deliver them to consumers with at-least-once semantics by default, message retention, flow control, and optional exactly-once delivery and ordering keys [51].

13.2 Monday: Topics, Subscriptions, Acks, and Schema Enforcement

13.2.1 The Delivery Model

A topic is global and durable; messages are retained for a configurable window regardless of consumption. Each subscription is an independent cursor: ten subscriptions on one topic each receive every message—fan-out is free, which is why a single ingestion topic commonly feeds a serving pipeline, an analytics sink, and an audit archive simultaneously. Pull subscriptions have consumers request batches; push subscriptions deliver to an HTTPS endpoint with OIDC authentication—push fits serverless consumers (Cloud Run, Cloud Functions) that should not run polling loops.

13.2.2 Acknowledgement Semantics

Delivery is at-least-once: Pub/Sub redelivers any message not acked within the subscription's acknowledgement deadline. Consumers must therefore be idempotent or deduplicate. Ack deadlines can be extended per message (`modifyAckDeadline`) while processing continues; abandoning extensions is how a slow consumer generates its own duplicate storm.

13.2.3 Schema Validation at the Edge

Attaching an Avro or Protocol Buffer schema to a topic makes the broker reject non-conforming publishes: the data contract enforced at the edge, before a malformed message can reach any of the ten subscriptions. Schema evolution follows compatibility rules (additive fields); versioning lives on the schema, and breaking changes mean a new schema revision with a deployment plan.

Common Pitfall

Schema enforcement protects message *shape*, not message *semantics*. A schema-valid message can still carry a negative price or a timestamp from 1970. Edge validation is the first contract layer, not the only one—validate semantics in the pipeline (Chapters 14–15) and at load (Chapter 11).

Tip

Publish with attributes (`origin`, `schema_version`, `event_type`) even when the payload is self-describing. Attributes are filterable on subscriptions and visible in dead-letter triage without deserializing the body.

13.3 Tuesday: Dead Letters, Retries, Snapshots, and Seek

13.3.1 Dead-Letter Topics

Configure a dead-letter topic *on the subscription* with a maximum delivery attempt count: after N failed deliveries the message forwards to the DLQ with its attempt metadata attached. The DLQ is a first-class topic—subscribe an alerting pipeline to it, sample its messages daily, and own the triage runbook. A DLQ nobody reads is a data-loss mechanism with good branding.

13.3.2 Retry Policies and Backoff

Subscription retry policies add exponential backoff between redeliveries (minimum and maximum delay), smoothing thundering-herd redelivery against a struggling consumer. Publisher-side, the client libraries retry transient failures with their own backoff; both layers compose, so reason about worst-case latency of a message that retries on both sides.

13.3.3 Snapshots and Seek

Snapshots capture a subscription’s ack state; **seek** rewinds a subscription to a snapshot or a timestamp within the topic retention window. Seek is the streaming answer to “the consumer had a bug for six hours”: fix the consumer, seek back to before the bad deploy, replay. It is also the migration tool for consumer changes that must reprocess history.

```

1 $env:PROJECT_ID = "my-gcp-pubsub-lab"
2
3 # DLQ is configured on the subscription, not the publisher:
4 gcloud pubsub subscriptions create card-sub '
5   --topic=card-transactions '
6   --dead-letter-topic=card-dlq '
7   --max-delivery-attempts=5 '
8   --min-retry-delay=10s --max-retry-delay=600s
9
10 # Replay: seek the subscription back two hours after fixing a consumer bug.
11 gcloud pubsub subscriptions seek card-sub --time=$((Get-Date).ToUniversalTime()
    .AddHours(-2).ToString('yyyy-MM-ddTHH:mm:ssZ'))

```

Listing 13.1: DLQ configuration and a seek-based replay from PowerShell.

Note

Seek replays within retention. A topic retaining one day cannot replay last week’s incident. Size retention to your recovery-time expectations, not to the default.

13.4 Wednesday: Pub/Sub Lite versus Pub/Sub Standard

Pub/Sub Lite trades Standard’s global, fully-managed elasticity for zonal, capacity-provisioned, partition-based topics at substantially lower per-GiB cost. Lite requires you to reason about partitions, throughput reservations, and zonal availability; Standard reasons about none of them. The decision rule: Standard by default; Lite when message volume is large and steady, latency tolerance includes zonal failure modes, and the team can operate capacity planning. Many platforms use both: Standard for control-plane and low-volume topics, Lite for the firehose.

Dimension	Pub/Sub Standard	Pub/Sub Lite
Scope	Global, multi-zone durability	Zonal (Lite) or regional
Pricing	Per GiB delivered + retention	Provisioned capacity, lower per-GiB
Partitions	Abstracted away	Explicit; ordering per partition
Capacity	Elastic	Reserved throughput
Best for	Default; variable workloads	High steady volume, cost-sensitive

Table 13.1: Pub/Sub Standard versus Lite [51].

13.5 Thursday Hands-On Project: Financial Transactions with a Dead-Letter Net

This lab creates a topic and pull subscription, publishes one thousand mock card transactions from Python with retry and backoff, and routes poison messages to a dead-letter topic.

13.5.1 Provision

```

1 $env:PROJECT_ID = "my-gcp-pubsub-lab"
2 gcloud config set project $env:PROJECT_ID
3 gcloud services enable pubsub.googleapis.com
4
5 gcloud pubsub topics create card-transactions
6 gcloud pubsub topics create card-dlq
7
8 gcloud pubsub subscriptions create card-sub '
9   --topic=card-transactions '
10  --dead-letter-topic=card-dlq '
11  --max-delivery-attempts=5 '
12  --ack-deadline=60

```

Listing 13.2: Creating topic, DLQ, and subscription with delivery guardrails.

13.5.2 Publish with Backoff

```

1 import json, random, time
2 from google.cloud import pubsub_v1
3
4 publisher = pubsub_v1.PublisherClient(
5     publisher_options=pubsub_v1.types.PublisherOptions(
6         retry=pubsub_v1.types.Retry(initial=0.1, maximum=10.0, multiplier=2.0))
7     )
8 topic_path = publisher.topic_path("my-project", "card-transactions")

```

```

9 def publish(txn):
10     future = publisher.publish(topic_path, json.dumps(txn).encode("utf-8"),
11                               origin="pos-simulator")
12     future.add_done_callback(lambda f: print("acked:", f.result()))
13
14 for i in range(1000):
15     publish({"card_id": f"card-{random.randint(1, 500)}",
16            "amount": round(random.uniform(1, 500), 2),
17            "city": random.choice(["NYC", "LON", "TYO"]), "ts": time.time()})

```

Listing 13.3: Publishing mock card transactions with retry and exponential backoff.

13.5.3 Consume, Fail, and Inspect the DLQ

Write a pull consumer that acks valid messages and deliberately nacks (or lets the ack deadline expire on) messages with `amount > 490` to simulate a consumer bug. After five attempts those messages should land in `card-dlq`; pull from a DLQ subscription to verify, and inspect the `delivery_attempt` attribute.

Deliverables

- Publisher and consumer scripts with a README describing retry and DLQ behavior.
- Evidence of poison messages arriving in `card-dlq` with delivery-attempt metadata.
- A screenshot of subscription metrics: oldest unacked message age and ack rate during the run.

Acceptance Criteria

- The publisher retries transient failures with exponential backoff.
- The DLQ receives exactly the messages that exhaust delivery attempts—no more, no fewer.
- The consumer is idempotent: redelivered messages do not double-apply effects.

Stretch Goals

- Attach an Avro schema to the topic and show a non-conforming publish rejected.
- Enable exactly-once delivery on the subscription and describe what changes.
- Seek the subscription back fifteen minutes and observe redelivery of acked messages.

13.6 Friday Use Case and Architecture: Telemetry for Five Million Mobile Games

13.6.1 Scenario

A games publisher ingests session telemetry from five million concurrent mobile clients world-wide: events per second in the hundreds of thousands at peak, highly variable per region, with fraud, analytics, and live-ops consumers. The requirement reads “exactly-once processing”—which must be decomposed before it can be built.

13.6.2 Decomposing Exactly-Once

No system delivers exactly-once *delivery* across arbitrary failures; what platforms build is exactly-once *effect*. The chain: publishers retry with backoff (duplicates possible); Pub/Sub deduplicates within its window and offers exactly-once delivery mode within a region; the Dataflow consumer (Chapters 14–15) uses Streaming Engine’s exactly-once shuffle; the sink is BigQuery’s Storage Write API in exactly-once mode or an idempotent key-based write. Each hop removes a duplicate class; the composition yields exactly-once effect end to end [1].

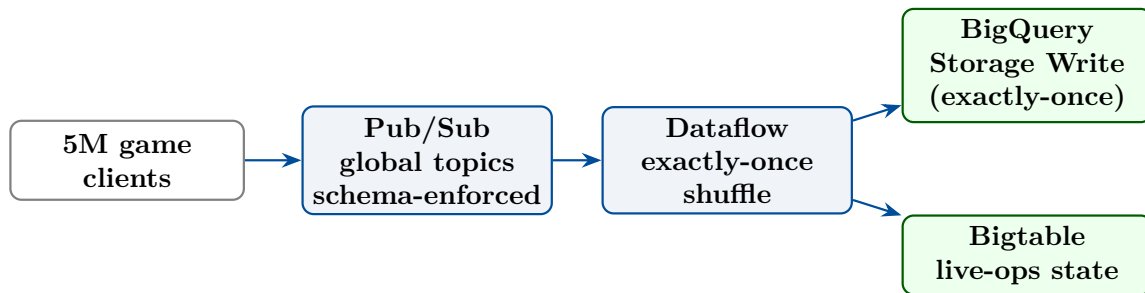


Figure 13.1: Global game telemetry: edge schema enforcement, exactly-once shuffle, idempotent sinks.

13.6.3 Operational Reality

The interesting failures are not message loss—retention and redelivery cover that—but backlog: alert on oldest unacked message age and backlog bytes per subscription, because a consumer that falls behind at global scale catches up slowly. Per-region traffic shaping is free via subscription flow control; per-tenant abuse is handled by publisher-side quotas and schema/attribute filtering rather than by the broker.

Common Pitfall

Exactly-once delivery mode is regional and changes redelivery behavior; combining it casually with ordering keys and high fan-out can surprise throughput. Validate the mode under a load test that includes consumer crashes—the scenario it exists for.

13.7 Interview Q&A

13.7.1 Concepts and Trade-offs

1. **Q: What happens to a message that exceeds max delivery attempts?**
A: It forwards to the subscription’s dead-letter topic with attempt metadata, where a triage process owns it—it is not silently deleted.
2. **Q: How does a topic schema enforce a data contract at publish time?**
A: The broker validates every published message against the Avro/Proto schema and rejects non-conforming publishes before any subscriber sees them.
3. **Q: Why is Pub/Sub publishing at-least-once, and how do you deduplicate?**
A: Acknowledgements can be lost after successful processing, forcing redelivery; deduplicate with message IDs, idempotent consumers, Dataflow exactly-once mode, or exactly-once sinks.
4. **Q: When does Pub/Sub Lite beat Standard on cost?**
A: At high, steady throughput where provisioned zonal capacity is cheaper per GiB than

Standard’s elastic per-GiB pricing and the team can operate partitions.

5. **Q: What does the oldest-unacked-message-age metric warn you about?**

A: Consumer lag or stuck messages: a growing value means the pipeline is falling behind or specific messages are never being acked (poison messages heading for the DLQ).

13.8 Further Reading

Read the Pub/Sub documentation for subscriptions, schemas, exactly-once delivery, and Lite [51]. The Kafka documentation provides the partition-log comparison point [67]. The Dataflow Model paper is the theoretical foundation for the exactly-once chain assembled in the Friday use case [1], and Cloud Monitoring documentation covers the backlog alerting [21].

Key Takeaways

- Fan-out is free: model one ingestion topic with many subscriptions rather than many producer paths.
- Edge schemas reject malformed messages before they reach any consumer—shape first, semantics next.
- At-least-once delivery makes idempotent consumers and deduplication a requirement, not an optimization.
- DLQs, retry policies, and seek are the failure-engineering kit; a DLQ without triage is data loss.
- Pub/Sub Lite trades operational simplicity for cost at high steady volume.
- Exactly-once is a property of the whole chain—publisher, broker, shuffle, sink—assembled deliberately.

13.9 Exercises

1. **(Conceptual)** Walk a single message through publish, delivery, ack-loss, redelivery, and deduplication, naming the duplicate-suppression mechanism at each hop.
2. **(Conceptual)** A product manager demands “no duplicate analytics events, ever.” Explain what is and is not achievable, and the design that satisfies the intent.
3. **(Hands-on)** Reproduce the Thursday lab, then raise `max-delivery-attempts` and observe the latency before DLQ arrival.
4. **(Hands-on)** Attach a Proto schema, publish a non-conforming message, and capture the rejection error.
5. **(Design)** Choose Lite versus Standard for three workloads: 200 msg/s control events, 2 GB/s steady telemetry, and 50 GB/s Black-Friday bursts.
6. **(Architecture)** Add a replay drill to the telemetry design: consumer bug deployed for six hours, detected, fixed, and replayed via seek—write the runbook.

Dataflow and Apache Beam Fundamentals

Learning Objectives

- Apply the Beam programming model: pipelines, PCollections, PTransforms, ParDo, and runners.
- Explain Dataflow’s managed architecture: worker autoscaling, Shuffle and Streaming Engine.
- Unit-test pipelines with TestPipeline, PAssert, and isolated DoFn tests before deploying.
- Package pipelines as Classic or Flex Templates for repeatable, parameterized launches.
- Build the WordCount pipeline and run it on the Dataflow runner with autoscaling.
- Design a real-time pipeline that anonymizes healthcare HL7 messages before warehousing.

Cross-Cloud Equivalence

Managed runners for portable pipelines exist on every cloud, but Beam’s model—one API, many runners—is the most explicit separation of pipeline logic from execution engine.

Capability	GCP Dataflow	AWS	Azure
Managed runner	Dataflow (Beam native)	Kinesis Data Analytics (Flink), EMR Flink/Spark	HDInsight/Synapse Spark, Fabric
Programming model	Beam (Java/Python/Go)	Flink API, Spark API, Kinesis SQL	Spark API, KQL
Portability	Beam runners (Flink, Spark, Samza)	Moderate	Moderate
Autoscaling	Native, per-stage	Application scaling	Pool-level scaling
Batch + streaming	One model, both modes	Separate services traditionally	Separate engines traditionally

The Beam model’s contribution is conceptual as much as practical: its unified treatment of bounded and unbounded data, event time, and windowing [1] is now the vocabulary in which streaming is discussed on every platform, including **Snowflake**’s streaming features and **MongoDB**’s change-stream processors.

Key Terms

PCollection: an immutable, distributed dataset—bounded or unbounded—flowing through a pipeline. **PTransform**: a data-processing operation consuming and producing PCollections. **ParDo**: the general parallel “do” transform executing a user DoFn per element. **Runner**: the engine executing a Beam pipeline—Direct (local), Dataflow, Flink, Spark. **Streaming Engine**: Dataflow’s mode that moves streaming shuffle and state off worker VMs into the service backend. **Flex Template**: a containerized pipeline image plus a template spec, launched with parameters without rebuilding code.

14.1 Week 14 Roadmap

Week 14 teaches the model and the managed runner together. Monday is the Beam model: PCollections, PTransforms, ParDo, and the runner abstraction. Tuesday is Dataflow’s service architecture: autoscaling, Shuffle, Streaming Engine. Wednesday covers templates. Thursday runs WordCount on the Dataflow runner. Friday anonymizes healthcare HL7 streams before they reach BigQuery.

Definition

Apache Beam is a unified programming model for batch and streaming data processing. A Beam pipeline is a graph of PTransforms over PCollections, compiled to a **runner**; **Dataflow** is Google Cloud’s fully managed Beam runner with autoscaling, exactly-once shuffle, and streaming state management [65, 30].

14.2 Monday: The Beam Model—PCollections, PTransforms, ParDo, Runners

14.2.1 Pipelines as Graphs

A Beam program builds a directed graph: sources produce PCollections; transforms map, filter, key, window, group, and join them; sinks write results. The same graph runs bounded (a batch of files) or unbounded (a Pub/Sub subscription)—the model unifies both, which is the thesis of the Dataflow paper [1]: ask *what* results, *where* in event time, *when* in processing time to emit, and *how* refinements accumulate, and the model handles the rest.

14.2.2 ParDo and DoFn

ParDo is the workhorse: your `DoFn.process` runs per element, emitting zero or more outputs, with access to side inputs, state, and timers (Chapter 15). Higher-level transforms—`Map`, `Filter`, `GroupByKey`, `CombinePerKey`—are convenience wrappers over the same machinery. Fusions and combiner lifting are why idiomatic Beam (combine before group, filter early) runs dramatically cheaper than naive graphs.

```

1 import re
2 import apache_beam as beam
3
4 with beam.Pipeline() as p:
5     (p
6      | "Read" >> beam.io.ReadFromText("gs://dataflow-samples/shakespeare/
        kinglear.txt")
7      | "Words" >> beam.FlatMap(lambda line: re.findall(r"[A-Za-z ']+", line))
8      | "Pair" >> beam.Map(lambda w: (w.lower(), 1))
9      | "Count" >> beam.CombinePerKey(sum))

```

```

10 | "Format" >> beam.Map(lambda kv: f"{kv[0]}: {kv[1]}")
11 | "Write" >> beam.io.WriteToText("gs://my-bucket/out/wordcount")

```

Listing 14.1: The canonical WordCount expressed in Beam.

14.2.3 Runners and Portability

Develop on the Direct runner locally, test with TestPipeline, deploy to Dataflow. Because the model is portable, the same pipeline can run on Flink or Spark—a hedge some organizations value—but portability is at the API level: runner-specific features (Streaming Engine, exactly-once modes) are where managed value concentrates.

Note

Beam pipelines are lazily constructed: transforms build the graph, and `pipeline.run()` (or context exit) submits it. Code before submission runs on the launching machine; DoFn bodies run on workers. Confusing the two is the most common beginner bug—a database connection opened at graph-construction time does not exist on workers.

14.3 Tuesday: Dataflow Architecture—Autoscaling, Shuffle, Streaming Engine

14.3.1 The Managed Service

You submit a graph; Dataflow provisions worker VMs, distributes fused stages, and scales workers between configured bounds as backlog and utilization demand. The service monitors per-stage progress, drains stragglers with dynamic work rebalancing (stealing unprocessed ranges from slow workers), and reports per-stage metrics—system lag, watermark, throughput—that become your alerting surface in Chapter 15.

14.3.2 Shuffle and Streaming Engine

Batch jobs use the service-based **Shuffle**: intermediate data moves through a managed shuffle tier instead of worker disks, so workers stay stateless and autoscale freely. **Streaming Engine** does the same for streaming pipelines—plus moving per-key state into the backend—reducing worker count, improving autoscaling latency, and enabling features like exactly-once shuffle. Unless you have a specific reason (custom network placement, specialized disks), both should be on.

Signal	Meaning	Response
Workers at max, backlog growing	Under-provisioned or hot stage	Raise <code>max_num_workers</code> ; inspect stage
Workers idle, low throughput	Input-bound or over-provisioned	Check source parallelism; lower bounds
High system lag, stable watermark	Processing capacity short	Scale up; enable Streaming Engine
Watermark stuck, lag stable	Late/stranded data upstream	Inspect upstream; check event timestamps

Table 14.1: Reading Dataflow job metrics during tuning.

Tip

Autoscaling reacts to backlog; it does not predict it. For known daily peaks, either accept the scale-up lag (minutes) or pin a higher `num_workers` floor during the peak window via scheduled job updates.

14.4 Wednesday: Templates—Classic versus Flex

Templates separate pipeline *definition* from pipeline *launch*: operators launch parameterized jobs without a development environment. **Classic templates** stage the serialized graph at build time—parameters must be `ValueProviders` resolved at launch, and graph shape cannot depend on them. **Flex Templates** package the pipeline as a container image; at launch the container runs, builds the graph with full access to parameters, and submits it—conditional graph construction, custom dependencies, and non-`ValueProvider` parameters all work. Google ships both as Google-provided templates (GCS to BigQuery, Pub/Sub to BigQuery, and dozens more); prefer them over writing your own when they fit.

Common Pitfall

A Classic template whose graph depends on a launch parameter silently bakes the build-time value. If a parameter must change the DAG shape—choose a source, toggle a transform—you need a Flex Template. This constraint has bitten every team that ignored it exactly once.

14.5 Thursday Hands-On Project: WordCount on the Dataflow Runner

This lab runs WordCount locally, unit-tests a transform, then submits to Dataflow with autoscaling—the full develop-test-deploy loop in miniature.

14.5.1 Local Run and Unit Test

```

1 import apache_beam as beam
2 from apache_beam.testing.test_pipeline import TestPipeline
3 from apache_beam.testing.util import assert_that, equal_to
4
5
6 def count_words(lines):
7     return (lines
8             | "Words" >> beam.FlatMap(str.split)
9             | "Pair" >> beam.Map(lambda w: (w, 1))
10            | "Count" >> beam.CombinePerKey(sum))
11
12
13 def test_count_words():
14     with TestPipeline() as p:
15         out = p | beam.Create(["a b a", "b"])
16         assert_that(count_words(out), equal_to(["a", 2], ("b", 2)))

```

Listing 14.2: Unit-testing a Beam transform with TestPipeline and PAssert.

14.5.2 Submit to Dataflow

```

1 $env:PROJECT_ID = "my-gcp-dataflow-lab"
2 $env:REGION = "us-central1"

```

```

3 $env:BUCKET = "my-gcp-dataflow-lab-bucket"
4
5 python .\wordcount.py '
6   --runner=DataflowRunner '
7   --project=$env:PROJECT_ID '
8   --region=$env:REGION '
9   --temp_location=gs://$env:BUCKET/tmp '
10  --staging_location=gs://$env:BUCKET/staging '
11  --output=gs://$env:BUCKET/out/wordcount '
12  --autoscaling_algorithm=THROUGHPUT_BASED '
13  --max_num_workers=4

```

Listing 14.3: Running WordCount on Dataflow with autoscaling from PowerShell.

Open the Dataflow job graph in the console: identify the fused stages, worker count over time, and per-stage throughput. Then rerun with `-runner=DirectRunner` on a small input and compare—the graph is identical; only the runner changed.

Deliverables

- The pipeline, the passing unit test, and the submission command transcript.
- A screenshot of the Dataflow job graph with stages and autoscaling behavior annotated.
- A paragraph comparing the local and managed runs: what the runner changed and what it did not.

Acceptance Criteria

- The transform logic is covered by a PAssert unit test that fails when logic changes.
- The Dataflow job succeeds with autoscaling enabled and bounded workers.
- Outputs are written to a deterministic Cloud Storage path, overwritable on rerun.

Stretch Goals

- Package the pipeline as a Flex Template with a Docker image and launch it with `gcloud dataflow flex-template run`.
- Add a Google-provided template run (Text Files on Cloud Storage to BigQuery) and compare with hand-rolled code.
- Benchmark the job with and without service-based shuffle on a large input.

14.6 Friday Use Case and Architecture: Real-Time HL7 Anonymization

14.6.1 Scenario

A hospital network streams HL7 admission and lab messages from facilities to a central analytics platform. Before any message lands in BigQuery, direct identifiers (patient names, MRNs, addresses) must be pseudonymized; the mapping from real to pseudonymous identity lives in a restricted system; analysts query only the safe projection.

14.6.2 Architecture

Facilities publish to a schema-validated Pub/Sub topic (Chapter 13). A streaming Dataflow pipeline parses HL7 segments, calls the tokenization service (or Sensitive Data Protection’s de-identification, Chapter 23) through a bounded-batch `ParDo` with backoff, and writes two outputs: the pseudonymized stream to BigQuery for analytics, and the raw stream to a VPC-SC-protected, CMEK-encrypted Cloud Storage bucket reachable only by the compliance service account. Side inputs carry the day’s code mappings; dead-lettered parse failures go to a triage topic with alerts.

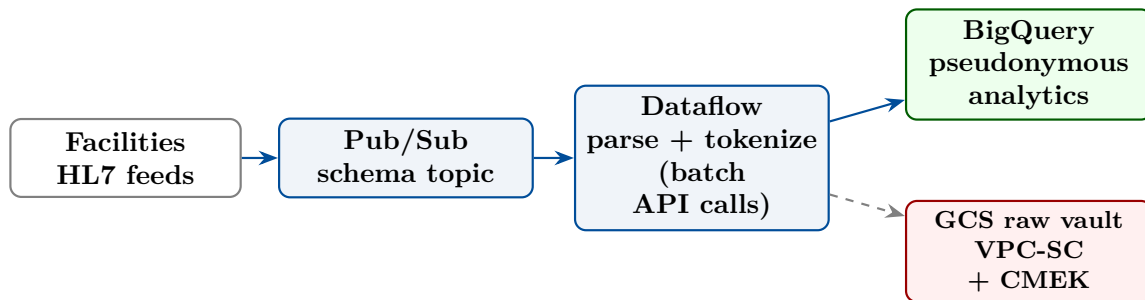


Figure 14.1: HL7 anonymization: parse and tokenize in flight; analysts see only the pseudonymous projection.

Tip

Batch external API calls inside the `DoFn` (accumulate a bundle, call once) and bound concurrency with backoff—per-message calls to a tokenization endpoint are the classic way to turn a cheap pipeline into an expensive, rate-limited one. Chapter 20 develops this pattern fully.

Common Pitfall

“Anonymized” pipelines leak through timestamps, free-text notes, and rare-diagnosis combinations. Pseudonymization of direct identifiers is step one; quasi-identifier risk analysis (Chapter 23) decides whether the analytics projection is actually safe to expose.

14.7 Interview Q&A

14.7.1 Concepts and Trade-offs

1. **Q: What is the difference between a `PTransform` and a `ParDo`?**
 A: A `PTransform` is any graph node transforming `PCollections`; `ParDo` is the specific general-purpose transform applying your `DoFn` element-wise.
2. **Q: Why unit-test Beam with `TestPipeline` before deploying to Dataflow?**
 A: Logic errors cost minutes locally and money plus data-correction effort in production; `PAssert` proves transform semantics, and `TestStream` (Chapter 15) proves windowing behavior, all in CI.
3. **Q: What does Streaming Engine move off the worker VMs?**
 A: Streaming shuffle and per-key state into the service backend, leaving workers stateless, smaller, and faster to autoscale.
4. **Q: When should you package a pipeline as a Flex Template?**

A: When operators must launch parameterized jobs without a dev environment, or when graph shape depends on launch parameters—which Classic templates cannot express.

5. **Q: How does Dataflow autoscaling react to a growing backlog?**

A: It adds workers up to `max_num_workers` based on backlog and utilization, then removes them as the backlog drains—reactive, with minutes of ramp time.

14.8 Further Reading

Read the Beam programming guide for the model, transforms, and testing utilities [65], and the Dataflow documentation for runner options, Streaming Engine, and templates [30]. The Dataflow Model paper grounds the unified batch/streaming semantics [1]. Chapter 13’s Pub/Sub documentation covers the ingestion contracts feeding these pipelines [51].

Key Takeaways

- Beam unifies batch and streaming: one graph, bounded or unbounded inputs, many runners.
- ParDo/DoFn is where user logic lives; prefer combinable transforms and early filters for cheap graphs.
- Streaming Engine and service shuffle make workers stateless—enable both by default.
- Autoscaling reacts to backlog within worker bounds; plan peaks deliberately.
- Test with `TestPipeline` and `PAssert` before any managed run; templates industrialize launches.
- Regulated pipelines separate raw vaults from pseudonymous analytics at the architecture level, not by convention.

14.9 Exercises

1. **(Conceptual)** Explain graph-construction time versus worker execution time, with one bug that arises from confusing them.
2. **(Conceptual)** Why does moving state into Streaming Engine improve autoscaling for streaming pipelines?
3. **(Hands-on)** Reproduce the Thursday lab; then modify the tokenizer to strip stopwords, update the `PAssert`, and redeploy.
4. **(Hands-on)** Package `WordCount` as a Flex Template and launch two runs with different parameters.
5. **(Analysis)** A streaming job shows rising system lag with a stable watermark. Diagnose using Table 14.1 and propose the fix.
6. **(Design)** Extend the HL7 design with a Bigtable latest-state store per patient pseudonym, including row-key sketch and TTL policy.

Advanced Dataflow: Windowing, State, and Exactly-Once Semantics

Learning Objectives

- Distinguish event time from processing time and explain what a watermark estimates.
- Choose fixed, sliding, and session windows for concrete workloads.
- Configure triggers, accumulation modes, and allowed lateness for late-arriving data.
- Manage per-key state with the Beam State and Timers API, bounding state with TTL via watermark garbage collection.
- Join streams over windows with CoGroupByKey and reason about join state retention.
- Compare end-to-end exactly-once with at-least-once plus downstream dedup, and troubleshoot a late-data dashboard inaccuracy.

Cross-Cloud Equivalence

Stream-processing semantics are platform-neutral: every serious engine answers the Dataflow Model’s four questions, with different defaults and vocabulary.

Concept	Beam/Dataflow	Apache Flink	Spark Structured Streaming
Event-time windows	Fixed/sliding/session	Tumbling/hopping/session	Tumbling/sliding/session
Completeness estimate	Watermark	Watermark	Watermark (threshold)
Late data	Allowed lateness + triggers	Allowed lateness + side outputs	Watermark delay
Per-key state	State & Timers API	Keyed state + timers	mapGroupsWithState
Exactly-once	Exactly-once shuffle + sinks	Checkpointing + two-phase sinks	Checkpoint + idempotent sinks

On the warehousing side, **Snowflake** streams and **Microsoft Fabric** eventstreams land micro-batches with simpler semantics; **MongoDB** change streams feed processors that must implement these semantics themselves. Beam’s vocabulary—watermark, trigger, allowed lateness—is the common reference even when the runtime differs [1].

Key Terms

Event time: the timestamp at which an event occurred at its source; **processing time:** when the pipeline observes it. **Watermark:** the engine’s estimate of event-time completeness—“we have probably seen all data before time t .” **Trigger:** the rule deciding when a window emits a pane (early, on-time, late). **Allowed lateness:** how long a window stays open for late events before being garbage-collected. **State and Timers API:** Beam’s per-key state bags/values and event-time/processing-time timers for custom stateful logic. **CoGroupByKey:** the windowed join primitive grouping two keyed PCollections by key and window.

15.1 Week 15 Roadmap

Week 15 is the conceptual summit of Part IV. Monday separates event time from processing time and defines watermarks. Tuesday covers the three window strategies. Wednesday handles late data, triggers, and the State and Timers API. Thursday builds the session-window clickstream pipeline with two minutes of allowed lateness. Friday joins streams with CoGroupByKey, assembles end-to-end exactly-once, and debugs a dashboard corrupted by offline mobile uploads.

Definition

A **watermark** is a monotonic estimate of event-time progress: the engine’s declaration that it expects no more data older than a given event time. Data arriving behind the watermark is **late**; windows stay open for it only as long as **allowed lateness** permits [1, 65].

15.2 Monday: Event Time, Processing Time, and Watermarks

15.2.1 Two Clocks

Every event lives in two time domains. Event time is when the user clicked, the sensor read, the trade executed. Processing time is when your worker saw it. Under perfect conditions they coincide; under mobile offline periods, cross-region queueing, or source outages they diverge arbitrarily. Computing “clicks per 5-minute window” against processing time counts events by when the network felt like delivering them; event time counts them by when they happened. For anything humans interpret as behavior, event time is the only defensible axis.

15.2.2 Watermark Mechanics

The watermark advances as the pipeline observes event timestamps—typically tracking near the maximum seen event time minus a heuristic skew estimate. Windowing, timers, and state garbage collection all key off it. A watermark stuck behind reality stalls window output (dashboards freeze); a watermark racing ahead discards legitimately late data (dashboards lie). Both failure modes are visible in Dataflow’s per-stage watermark and system-lag metrics, which is why the monitoring story in this chapter’s production notes starts there [30].

Common Pitfall

A single pathological upstream—one mobile device that buffered a week of events, one partition of a source lagging—holds the watermark hostage for every downstream window. Cap the damage: bound event-time skew at the source, drop or side-channel events older than a sanity horizon, and monitor oldest-event-age alongside watermark lag.

15.3 Tuesday: Windowing Strategies—Fixed, Sliding, Session

15.3.1 Fixed and Sliding

Fixed (tumbling) windows partition time into disjoint buckets—hourly revenue, daily aggregates. **Sliding (hopping) windows** are defined by size and period: a ten-minute window emitted every minute is a rolling detector—the fraud rule “same card in two cities within ten minutes” in the Milestone 4 capstone is exactly a sliding window over per-card events. Note the cost asymmetry: each event belongs to *size/period* sliding windows, multiplying downstream state and computation.

15.3.2 Session Windows

Session windows group events by activity bursts: a window opens at a key’s first event, extends while gaps stay under a threshold, and closes after the gap elapses. User analytics (visits, app sessions, support interactions) are the canonical fit. Sessions are per-key and dynamically merged, which makes them more expensive than fixed windows and makes their state retention directly proportional to allowed lateness.

Window	Definition	Canonical use
Fixed	Disjoint buckets of size s	Periodic aggregates: hourly revenue, daily KPIs
Sliding	Size s , period p (overlapping)	Rolling detectors: fraud rules, moving averages
Session	Gap-based, per key, dynamically merged	User activity sessions, device connectivity bursts

Table 15.1: Beam windowing strategies and their canonical workloads [65].

Tip

Choose the window from the *question*, not the data shape. “Daily totals” is fixed; “any 10-minute span with two cities” is sliding; “how long do users engage per visit” is session. The wrong choice is not a style error—it changes the answer.

15.4 Wednesday: Late Data, Triggers, and the State and Timers API

15.4.1 Triggers and Accumulation

A window fires panes according to its trigger: the default fires once when the watermark passes the window end. `AfterWatermark(early=..., late=...)` adds speculative early panes (freshness) and per-late-event refinement (correctness). The **accumulation mode** decides whether a new pane *accumulates* with prior panes or *discards* them—discarding fits counters and per-pane detectors; accumulating fits running totals. **Allowed lateness** keeps window state alive past the watermark for late refinement, at direct state-storage cost; when it expires, the window and its state are garbage-collected and stragglers are dropped (or side-output if you coded for it).

15.4.2 Per-Key State and Timers

Windowing is declarative state; the State and Timers API is the imperative layer underneath: `BagStateSpec` to accumulate, `ValueStateSpec` to remember, `TimerSpec` to fire callbacks in event or processing time. The canonical use is custom deduplication and pattern detection with explicit expiry.

```

1 import apache_beam as beam
2 from apache_beam.coders import coders
3 from apache_beam.transforms.userstate import BagStateSpec, TimerSpec, on_timer
4
5 class DedupWithTimer(beam.DoFn):
6     SEEN = BagStateSpec("seen", coders.StrUtf8Coder())
7     EXPIRY = TimerSpec("expiry")
8
9     def process(self, element, seen=beam.DoFn.StateParam(SEEN),
10                expiry=beam.DoFn.TimerParam(EXPIRY)):
11         key, event = element
12         expiry.set(event["event_ts"] + 600) # state TTL: 10 min
13         if event["event_id"] in set(seen.read()):
14             return # duplicate: discard
15         seen.add(event["event_id"])
16         yield event
17
18     @on_timer(EXPIRY)
19     def on_expiry(self, seen=beam.DoFn.StateParam(SEEN)):
20         seen.clear() # free per-key state at timer fire

```

Listing 15.1: Stateful deduplication with a 10-minute event-time TTL.

Common Pitfall

Unbounded per-key state kills streaming jobs—slowly, then at 03:00 on a Saturday. Every `BagStateSpec` needs a removal story: watermark garbage collection via allowed lateness, an explicit expiry timer, or a bounded data structure. If you cannot state when state is freed, the job owns a memory leak with a billing address.

15.5 Thursday Hands-On Project: Sessionized Clicks with Late Data

This lab builds a streaming pipeline reading clicks from Pub/Sub, grouping them into session windows (with a fixed-window variant for 5-minute counts), allowing two minutes of late data, and writing results to BigQuery—then proves the windowing logic with a `TestStream` unit test.

15.5.1 The Streaming Pipeline

```

1 import json
2 import apache_beam as beam
3 from apache_beam.options.pipeline_options import PipelineOptions,
4     StandardOptions
5 from apache_beam.transforms.window import TimestampedValue, FixedWindows
6 import apache_beam.transforms.trigger as trigger
7
8 options = PipelineOptions(project="my-project", region="us-central1",
9                           temp_location="gs://my-bucket/tmp")
10 options.view_as(StandardOptions).streaming = True
11
12 with beam.Pipeline(options=options) as p:
13     (p | "Read" >> beam.io.ReadFromPubSub(

```

```

13         subscription="projects/my-project/subscriptions/clicks-sub")
14     | "Parse" >> beam.Map(lambda b: json.loads(b.decode("utf-8")))
15     | "Stamp" >> beam.Map(lambda e: TimestampedValue(e, e["event_ts"]))
16     | "Window" >> beam.WindowInto(
17         FixedWindows(300), # 5-minute windows
18         trigger=trigger.AfterWatermark(late=trigger.AfterCount(1)),
19         allowed_lateness=120, # 2 minutes of late data
20         accumulation_mode=trigger.AccumulationMode.DISCARDING)
21     | "KeyByUser" >> beam.Map(lambda e: (e["user_id"], 1))
22     | "Count" >> beam.CombinePerKey(sum)
23     | "Write" >> beam.io.WriteToBigQuery(
24         "my-project:analytics.clicks_5min",
25         create_disposition=beam.io.BigQueryDisposition.CREATE_IF_NEEDED))

```

Listing 15.2: Pub/Sub clicks windowed with late-data tolerance into BigQuery.

15.5.2 Session Variant and Unit Test

Replace the Window step with `beam.WindowInto(Sessions(gap_size=300), allowed_lateness=120)` for true sessionization, keying by `user_id` first. Then prove behavior locally:

```

1 from apache_beam.testing.test_stream import TestStream
2 from apache_beam.testing.test_pipeline import TestPipeline
3 from apache_beam.testing.util import assert_that, equal_to_windowed
4 from apache_beam.transforms.window import TimestampedValue
5
6 def test_late_click_counted_once():
7     with TestPipeline() as p:
8         stream = (TestStream()
9             .add_elements([TimestampedValue({"user_id": "u1", "event_ts":
10             0}, 0)])
11             .advance_watermark_to(310) # first window closes
12             .add_elements([TimestampedValue({"user_id": "u1", "event_ts":
13             100}, 310)]) # late
14             .advance_watermark_to_infinity())
15         # apply Parse/Stamp/Window/KeyByUser/Count, then:
16         # assert two panes: on-time pane and one late refinement
17         _ = p | stream
18         # (assertions omitted for brevity; use assert_that with pane matchers)

```

Listing 15.3: TestStream proving on-time and late panes.

Deliverables

- The fixed-window and session-window pipelines, with a note on when each is correct.
- The TestStream test output proving late events refine the window exactly once.
- Console screenshots of per-stage watermark lag while replaying a late-data scenario.

Acceptance Criteria

- Events are stamped from their source event time, not arrival time.
- The windowing declares trigger, accumulation mode, and allowed lateness explicitly.
- The unit test demonstrates an on-time pane and a late refinement without double counting.

Stretch Goals

- Emit dropped-because-too-late events to a side output and a monitoring metric.

- Replace `CombinePerKey` with the stateful `DedupWithTimer` DoFn keyed on `event_id`.
- Run the pipeline against a simulated Pub/Sub replay and chart watermark behavior.

15.6 Friday: Stream-Stream Joins, Exactly-Once, and the Late-Mobile Dashboard

15.6.1 Joining Streams over Windows

Stream-stream joins in Beam are windowed `CoGroupByKey`s: key both inputs identically, apply a shared windowing, and the join emits when matching keys co-occur in a window. Two disciplines follow. First, **watermark alignment**: the joined output’s completeness is bounded by the slower input’s watermark—a lagging dimension stream stalls the fact stream’s joins. Second, **state retention**: join state lives until window closing plus allowed lateness; wide windows times high cardinality times long lateness is a state-size multiplication you must compute, not discover.

15.6.2 End-to-End Exactly-Once

The assembly from Chapter 13 completed: Streaming Engine provides exactly-once shuffle between stages; the BigQuery Storage Write API provides exactly-once sink semantics within its stream commit model; deterministic keys make reruns idempotent. The alternative—at-least-once delivery plus downstream dedup by event ID (a `MERGE` or a dedup query over a raw table)—is cheaper in pipeline complexity and often the right choice when a deduplication key is natural and consumers tolerate a dedup step. Choose deliberately: exactly-once infrastructure where duplicates are dangerous (billing, fraud), at-least-once plus dedup where they are merely untidy.

15.6.3 Use Case: The Dashboard That Lies

A streaming dashboard shows daily aggregations that do not match finance’s numbers. Investigation finds mobile devices that go offline for hours, then upload event batches—arriving after windows closed, silently dropped by default watermarks. The fix is layered: extend `allowed_lateness` to the business’s true tolerance (hours, not minutes); add a late trigger so refinements republish; make the BigQuery sink upsert by window key so refined panes replace rather than duplicate; and add a Cloud Monitoring alert on watermark lag and on a “very late” side-output metric so the next incident pages instead of being discovered in a board meeting.

Tip

Alert on **per-stage data watermark lag** and system lag, and page on sustained growth. Watermark lag growing while input rate is normal is the streaming equivalent of a disk filling: everything still works, right up until it very much does not [21].

Common Pitfall

Changing stateful logic—state specs, timers, windowing—usually breaks in-place job updates because existing state cannot map to the new schema. Prefer drain-and-replace for stateful changes: drain the old job, start the new one from a snapshot or fresh subscription, and plan the cutover to avoid reprocessing or gaps.

15.7 Interview Q&A

15.7.1 Concepts and Trade-offs

1. **Q: What does a watermark estimate, and what lies behind it?**
 A: Event-time completeness: the engine believes it has seen all events up to time t . Late data—offline devices, delayed sources—arrives behind it.
2. **Q: Why does unbounded per-key state kill a streaming job, and how do timers plus watermark garbage collection bound it?**
 A: State accumulates in worker or backend storage until memory/disk exhausts; expiry timers free state explicitly and watermark progress garbage-collects closed windows, bounding lifetime.
3. **Q: In the dedup DoFn, what do the SEEN bag state and EXPIRY timer each do?**
 A: SEEN remembers event IDs already emitted for the key; EXPIRY fires after ten minutes of event time to clear them, bounding the dedup horizon.
4. **Q: What must align for a CoGroupByKey stream-stream join to emit, and what state does it retain?**
 A: Matching keys within the same window on both inputs; it buffers each side's elements per key per window until closing plus allowed lateness.
5. **Q: How do Streaming Engine shuffle and the Storage Write API deliver end-to-end exactly-once, and when is at-least-once plus dedup cheaper?**
 A: Exactly-once shuffle removes intra-pipeline duplicates and the sink commits records idempotently; dedup-based designs skip that infrastructure and are cheaper when a natural dedup key exists and consumers tolerate a merge step.

15.8 Further Reading

Read the Beam programming guide's windowing, triggers, and state-and-timers chapters [65], and the Dataflow Model paper for the foundational semantics [1]. Dataflow documentation covers watermark and system-lag metrics, exactly-once modes, and job update constraints [30]; Cloud Monitoring documentation covers the alert policies [21]. The BigQuery documentation details the Storage Write API's exactly-once behavior [9].

Key Takeaways

- Event time is the defensible axis for behavior analytics; processing time is an operational detail.
- Watermarks trade completeness for latency; monitor them like disk usage.
- Fixed, sliding, and session windows answer different questions—choose from the question.
- Triggers plus allowed lateness define the freshness-correctness contract of every window.
- All state needs an eviction story: expiry timers, watermark GC, or bounded structures.
- Exactly-once end-to-end is assembled hop by hop; at-least-once plus dedup is a legitimate, often cheaper alternative.

15.9 Exercises

1. **(Conceptual)** Explain why a processing-time daily report disagrees with finance after a mobile network outage, even with zero bugs.
2. **(Conceptual)** A fraud rule uses 10-minute sliding windows with 1-minute period. Compute the per-event window multiplicity and its effect on state.
3. **(Hands-on)** Reproduce the Thursday lab; then set `allowed_lateness=0`, replay late events, and document where they go.
4. **(Hands-on)** Add the dedup DoFn upstream of the window and verify via `TestStream` that a duplicated event ID emits once.
5. **(Analysis)** Given watermark lag growing at one stage only, list three causes and the metric or log that distinguishes each.
6. **(Design)** Specify the drain-and-replace cutover plan for a stateful pipeline whose windowing must change from 5-minute fixed to session windows.

Business Intelligence, Looker, and Real-Time Serving

Learning Objectives

- Explain Looker’s architecture and the role of the LookML semantic layer in governed BI.
- Model dimensions, measures, views, and explores that encode metric definitions once.
- Contrast Looker Studio with Looker (Core) for self-serve versus governed analytics.
- Implement reverse ETL: push curated BigQuery segments to operational APIs with identity resolution.
- Build a real-time streaming-sales dashboard with drill-down filters and a reverse-ETL sync.
- Design a governed BI access layer for five hundred analysts with non-conflicting metrics.
- Complete the Milestone 4 capstone: an end-to-end real-time fraud-detection pipeline.

Cross-Cloud Equivalence

Every platform eventually confronts the same BI governance question: are metrics defined once in a semantic layer, or independently in every dashboard?

Capability	GCP Looker	AWS	Azure / Fabric
Governed BI + semantic layer	Looker (LookML)	QuickSight (datasets)	Power BI (semantic models)
Self-serve dashboards	Looker Studio	QuickSight	Power BI / Fabric
Semantic model as code	LookML in Git	QuickSight APIs	TMDL / Fabric git integration
In-memory acceleration	BI Engine (Ch. 6)	QuickSight SPICE	Import mode / DirectLake
Reverse ETL	Scheduled queries + Functions (partner tools)	Partner tools (Census, Hightouch)	Partner tools

Snowflake users pair with the same BI tools over its warehouses; **MongoDB** charts serve operational analytics directly on Atlas. The portable insight: a semantic layer is a *code artifact*—versioned, reviewed, tested—and the organizations that treat it as such are the ones where “Gross Revenue” means one thing.

Key Terms

LookML: Looker’s modeling language defining dimensions, measures, joins, and explores as versioned code. **View:** a LookML file mapping a table or derived table into dimensions and measures. **Explore:** a curated join of views exposed to analysts as a self-serve query surface. **Measure:** a LookML aggregate definition (count, sum, average) with declared type and filters. **Reverse ETL:** syncing curated warehouse data back into operational systems (CRM, marketing platforms). **Identity resolution:** reconciling entity keys across systems so a segment in BigQuery matches the same humans in the CRM.

16.1 Week 16 Roadmap

Week 16 closes Part IV where streaming data meets human decision-making. Monday covers Looker’s architecture and LookML. Tuesday models dimensions, measures, and explores. Wednesday contrasts self-serve and governed BI and introduces reverse ETL. Thursday builds the streaming-sales dashboard and a reverse-ETL sync to a marketing webhook. Friday designs the governed access layer for five hundred analysts. The week ends with the Milestone 4 real-time fraud capstone.

Definition

Looker is Google Cloud’s governed BI platform. Its defining feature is the **LookML semantic layer**: metric and join definitions live as versioned code between the database and every dashboard, so all consumers compute the same numbers [48].

16.2 Monday: Looker Architecture and the Semantic Layer

16.2.1 How Looker Differs

Most BI tools extract data into extracts or connect dashboards directly to tables; Looker connects to the database (BigQuery, via a service account) and generates SQL per user interaction. Nothing is extracted by default; freshness is the warehouse’s freshness. The price of this elegance is that warehouse performance (Chapters 5–6) *is* dashboard performance—partitioning, clustering, and BI Engine decisions surface directly in tile load times.

16.2.2 Why a Semantic Layer

Without a shared layer, every analyst re-implements “Gross Revenue” in a SQL snippet, a sheet formula, or a dashboard calculated field, and the definitions diverge silently. LookML defines it once—`measure: gross_revenue { type: sum sql: ${order_amount} ;; filters: [is_refunded: "no"] }`—and every explore, dashboard, and API consumer inherits the same definition, reviewable in Git like any other code.

Note

The semantic layer is also a security layer: LookML access grants and user attributes restrict which explores, fields, and rows a user can see, composing with (not replacing) the warehouse’s row access policies from Chapter 8.

16.3 Tuesday: Dimensions, Measures, Explores, and Views

16.3.1 Modeling Vocabulary

A **view** maps one table (or derived table) to LookML: **dimensions** for attributes and **measures** for aggregates, each with types, SQL expressions, and formatting. An **explore** joins views with declared relationships and join keys, producing the analyst-facing query surface. Discipline points: always declare **relationship** (*one_to_many*, *many_to_one*) so Looker can apply symmetric aggregates and avoid fan-out inflation; expose only fields analysts should see; name measures as the business names them.

```

1 # file: views/order_items.view.lkml
2 view: order_items {
3   sql_table_name: analytics.fact_order_items ;;
4   dimension: order_id { type: string sql: ${TABLE}.order_id ;; }
5   dimension: order_date { type: date sql: ${TABLE}.order_date ;; }
6   dimension: is_refunded { type: yesno sql: ${TABLE}.refund_flag = 'Y' ;; }
7   measure: gross_revenue {
8     type: sum
9     sql: ${TABLE}.order_amount ;;
10    filters: [is_refunded: "no"]
11    value_format_name: usd
12  }
13 }
14
15 # file: models/retail.model.lkml
16 explore: orders {
17   join: customers {
18     sql_on: ${orders.customer_id} = ${customers.customer_id} ;;
19     relationship: many_to_one
20   }
21 }

```

Listing 16.1: A minimal LookML view and explore (LookML syntax in SQL listing).

Common Pitfall

Fan-out is the silent killer of joined explores: joining one-to-many without declared **relationship** inflates sums when Looker cannot apply symmetric aggregates. Declare every join's cardinality; spot-check a known total after every model change.

16.4 Wednesday: Looker Studio versus Looker, and Reverse ETL

16.4.1 Self-Serve versus Governed

Looker Studio is the free, self-serve dashboard tool: connect to BigQuery (or Sheets, or CSV), drag fields, publish. It is the right answer for departmental reporting and the Thursday lab's fast path. **Looker (Core)** is the governed platform: LookML, Git workflow, access grants, API, embeds. The enterprise pattern uses both—Studio for velocity at the edges, Looker for the certified metrics at the core—with BigQuery's governed tables as the common floor.

16.4.2 Reverse ETL

Dashboards inform; reverse ETL *acts*: pushing the warehouse's computed segments (high-value customers, churn risks, fraud watchlists) back into the operational systems where work

happens—CRM, marketing automation, ad platforms. The reference implementation is a scheduled BigQuery query materializing the segment, plus a Cloud Function (or Dataform operation) posting deltas to the operational API on a cadence. Two disciplines make it safe: **identity resolution**—the segment keys must map to the operational system’s identifiers, or you sync a segment of strangers—and **sync-frequency trade-offs**: freshness versus API rate limits and cost, which usually lands on hourly deltas, not per-event pushes.

Tip

Sync deltas, not full snapshots: track a `last_synced_at` watermark per entity and push only changed segment membership. Full syncs multiply API calls by segment size daily; delta syncs keep both cost and rate-limit pressure flat.

16.5 Thursday Hands-On Project: Streaming Dashboard and Reverse-ETL Sync

This lab connects Looker Studio to a BigQuery streaming-sales dataset with drill-down filters, then reverse-ETLs a high-value customer segment to a mock marketing webhook via a scheduled query and Cloud Function.

16.5.1 The Live Dashboard

1. Create (or reuse) an `analytics.sales_live` table receiving streaming inserts—the fraud capstone’s alert table or a simple streaming sink works.
2. In Looker Studio, add a BigQuery data source with a custom query selecting the trailing hour: `WHERE sale_ts >= TIMESTAMP_SUB(CURRENT_TIMESTAMP(), INTERVAL 1 HOUR)`.
3. Build tiles: revenue time series, sales-by-region map, top products table; add drill-down controls for region and product category; set data freshness to one minute.

16.5.2 The Reverse-ETL Sync

```
1 CREATE OR REPLACE TABLE crm_sync.high_value_segment AS
2 SELECT customer_id, email, lifetime_value,
3        CURRENT_TIMESTAMP() AS computed_at
4 FROM analytics.dim_customer
5 WHERE lifetime_value > 10000 AND is_current = TRUE;
```

Listing 16.2: Scheduled query materializing the high-value segment with sync watermarks.

```
1 import os
2 import requests
3 from google.cloud import bigquery
4
5 WEBHOOK = os.environ["MARKETING_WEBHOOK_URL"]
6
7
8 def sync_segment(request):
9     bq = bigquery.Client()
10    rows = bq.query("""
11        SELECT customer_id, email FROM crm_sync.high_value_segment
12        WHERE computed_at > (
13            SELECT COALESCE(MAX(synced_at), '1970-01-01')
14            FROM crm_sync.sync_log)
15    """).result()
16    payload = {"members": [dict(r) for r in rows]}
17    resp = requests.post(WEBHOOK, json=payload, timeout=30)
```

```

18     resp.raise_for_status()
19     bq.query("INSERT INTO crm_sync.sync_log (synced_at, member_count) "
20             f"VALUES (CURRENT_TIMESTAMP(), {len(payload['members'])})").result
21     ()
22     return {"synced": len(payload["members"])}

```

Listing 16.3: Cloud Function posting segment deltas to a mock marketing webhook.

Deliverables

- The dashboard URL or screenshots showing live tiles and drill-down filters.
- The scheduled query, Cloud Function, and sync-log table DDL.
- Evidence of a sync run: webhook received payload and the sync-log row.

Acceptance Criteria

- The dashboard reflects new streaming rows within its freshness interval.
- The reverse-ETL sync pushes deltas only and records each run in the sync log.
- The segment definition lives in exactly one place (the scheduled query), not in the Function.

Stretch Goals

- Rebuild one tile in Looker Core with a LookML measure and compare governance properties.
- Add identity resolution: map `customer_id` to the CRM’s contact IDs through a mapping table.
- Add alerting on sync failures and on segment-size anomalies (a segment that doubles overnight).

16.6 Friday Use Case and Architecture: Governed BI for Five Hundred Analysts

16.6.1 Scenario

An enterprise with five hundred analysts across finance, marketing, and operations suffers metric chaos: three departments report three different “Gross Revenue” numbers to the same executive meeting. The mandate: one governed access layer over the central warehouse, with self-service that cannot fork definitions.

16.6.2 Design

Certify a core LookML model owned by the analytics-engineering team: conformed explores for the shared fact tables, with measures defined once, access grants per department, and content validators in CI (LookML lint plus known-total tests). Analysts build dashboards freely *on the explores* but cannot redefine measures—new metric requests enter as pull requests. Looker Studio remains available for departmental scratch work against curated datasets, clearly badged as uncertified. Adoption is enforced socially as much as technically: the data-literacy program teaches “explores are the source of truth,” and executive reviews cite only certified dashboards.

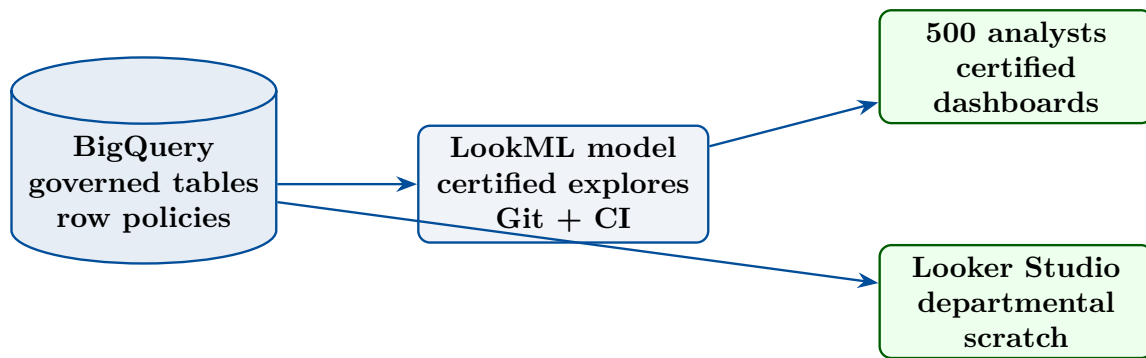


Figure 16.1: Governed BI: certified explores for shared truth, self-serve Studio at the edges.

Common Pitfall

A semantic layer nobody adopts is worse than none—it adds a third competing definition. Budget the socialization: training, office hours, and executive insistence on certified numbers are part of the deliverable, not an afterthought.

16.7 Interview Q&A

16.7.1 Concepts and Trade-offs

1. **Q: Why does a LookML semantic layer prevent conflicting metric definitions?**
A: Measures are defined once in versioned code and inherited by every explore, dashboard, and API call—analysts consume the definition instead of re-implementing it.
2. **Q: What problem does reverse ETL solve that dashboards cannot?**
A: Dashboards inform humans who must then act elsewhere; reverse ETL delivers the warehouse’s computed decisions into the operational systems where action happens.
3. **Q: What breaks in a sync without identity resolution?**
A: Warehouse keys do not match CRM identifiers, so segments sync to wrong or non-existent contacts—the pipeline runs green while marketing emails strangers.
4. **Q: What trade-off governs reverse-ETL sync frequency?**
A: Freshness versus API rate limits and cost; deltas on an hourly cadence are the usual optimum.
5. **Q: When is Looker Core justified over Looker Studio?**
A: When metrics must be governed—defined once, access-controlled, auditable—across many teams; Studio wins for velocity and departmental self-service.

16.8 Further Reading

Read the Looker documentation for LookML, explores, and content governance [48], and Looker Studio documentation for self-serve reporting [49]. The BigQuery documentation covers scheduled queries and the Storage Write API behind live tiles [9], and Chapter 6’s BI Engine reference covers dashboard acceleration [8].

Key Takeaways

- A semantic layer is code: versioned, reviewed, CI-tested—and the only durable cure for metric chaos.
- Declare join cardinality; fan-out inflation is silent and plausibly wrong.
- Live dashboards inherit warehouse performance: partitioning and BI Engine are BI decisions.
- Reverse ETL closes the loop from insight to action; identity resolution and delta syncs keep it safe.
- Studio for velocity, Looker for governance, curated tables as the common floor.
- Adoption is a deliverable: training and executive habits matter as much as the model.

16.9 Exercises

1. **(Conceptual)** Explain symmetric aggregates to an analyst whose revenue doubled after adding a join.
2. **(Conceptual)** Argue when *not* to reverse-ETL: give two cases where a dashboard or a native integration is the better answer.
3. **(Hands-on)** Reproduce the Thursday lab, then add a second tile showing per-city sales with a cross-filter.
4. **(Hands-on)** Add a content-validation test to the LookML model that fails CI when `gross_revenue` changes without a version bump in a comment convention you define.
5. **(Design)** Write the metric-governance policy for the 500-analyst enterprise: certification tiers, PR process, SLAs for new-metric requests.
6. **(Architecture)** Extend the reverse-ETL design with consent enforcement: only customers with marketing consent flags sync to the ad platform.

16.10 Milestone 4 Capstone: Real-Time Fraud Detection

Capstone Project

Mission: Simulate credit-card swipes into Pub/Sub, detect with a sliding-window Dataflow rule any card used in two different cities within ten minutes, stream alerts to BigQuery, and visualize them live in Looker—serverless end to end, with DLQ, tests, and watermark-lag alerting.

Estimated time: 8–10 hours.

What you'll practice:

- Publishing simulation traffic with retry/backoff and a dead-letter topic (Chapter 13).
- Sliding-window stateful detection with allowed lateness (Chapter 15).
- Exactly-once sinks and live BI on streaming results (Chapters 14, 16).
- Unit-testing the fraud rule with PAssert including a late-data case.

Scenario

Meridian Card Services wants a real-time fraud signal: a card physically presented in two distant cities within ten minutes is near-certainly compromised. Swipes stream from terminals; alerts

must reach the fraud team’s dashboard in under a minute; false negatives from late-arriving offline terminals are acceptable only within a documented tolerance.

Requirements

- Python simulator publishing plausible swipes (with seeded cross-city anomalies) to a schema-validated topic with a DLQ.
- Dataflow pipeline: key by card, 10-minute sliding windows with 1-minute period, two minutes allowed lateness, flag windows containing two or more distinct cities.
- Alerts stream to BigQuery via exactly-once write; Looker (or Studio) dashboard shows live alerts with drill-down.
- PAssert unit tests for the detection rule, including a late-data refinement case.
- Alert policies on Dataflow watermark lag and Pub/Sub backlog; documented monthly cost estimate.

Reference Architecture and Core Rule

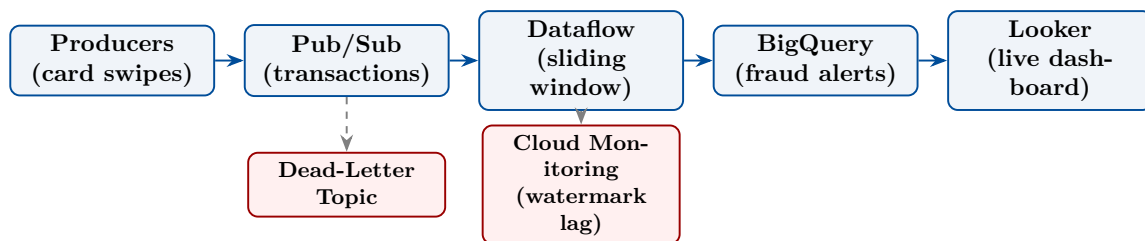


Figure 16.2: Milestone 4 reference architecture: real-time fraud detection, serverless end to end.

```

1 alerts = (
2     events
3     | "KeyByCard" >> beam.Map(lambda e: (e["card_id"], e))
4     | "Window" >> beam.WindowInto(SlidingWindows(600, 60),
5                                   allowed_lateness=120,
6                                   accumulation_mode=trigger.AccumulationMode.DISCARDING)
7     | "Group" >> beam.GroupByKey()
8     | "Flag" >> beam.FlatMap(lambda kv: [(kv[0], w)]
9                               for w in [1] if len({e["city"] for e in kv[1]}) > 1))

```

Listing 16.4: The two-cities-in-ten-minutes detection rule.

Deliverables

- A repository that runs end-to-end from a clean environment: simulator, pipeline, tests, dashboard link.
- The architecture diagram including DLQ, windowing semantics, and serving layer.
- At least three automated tests with a failing-case demonstration.
- Monitoring evidence and the monthly cost estimate with two optimizations documented.
- A security review: least-privilege runner service account, card data pseudonymized, no hardcoded secrets.

Acceptance Criteria

- A seeded cross-city anomaly surfaces as a BigQuery alert row within a minute of ingestion.
- Late events (within tolerance) refine rather than duplicate alerts; out-of-tolerance events are measurably accounted.
- The pipeline survives a worker kill test without losing or duplicating alerts.
- The dashboard reflects the alert stream live with drill-down by city and card.

Stretch Goals

- Key the rule on event time from the terminal (not arrival) and quantify the difference on replayed traffic.
- Add a second rule—velocity (more than N swipes per minute per card)—using the State and Timers API.
- Reverse-ETL confirmed-fraud card IDs back to a mock authorization API to auto-decline.
- Load-test to ten thousand events per second and document the autoscaling curve and cost.

Part V

Machine Learning and MLOps

BigQuery ML: Machine Learning in SQL

Learning Objectives

- Train regression, classification, and clustering models in pure SQL where the data already lives.
- Forecast time series with `ARIMA_PLUS` and detect anomalies from residuals.
- Evaluate models with `ML.EVALUATE`, read ROC and precision/recall, and detect overfitting.
- Tune hyperparameters and understand BQML's automatic preprocessing and encoding.
- Train a logistic-regression churn model on Google Analytics sample data and evaluate its ROC curve.
- Architect a pure-SQL forecasting system for daily inventory needs across one thousand stores.

Cross-Cloud Equivalence

In-warehouse machine learning eliminates the extract-train-reload loop; every major warehouse now ships a variant.

Capability		GCP ML	BigQuery	Snowflake	Azure / Fabric
SQL-native training		CREATE MODEL		Snowpark ML / ML functions	Synapse/Fabric ML (Spark MLlib)
Forecasting		ARIMA_PLUS		Forecasting ML function	AutoML forecasting
In-warehouse inference		ML.PREDICT		Snowpark ML inference	Synapse predictions
Python escape hatch		Remote functions / Vertex AI		Snowpark Python	Spark notebooks
Best fit		SQL-fluent tabular data	teams,	SQL/Python teams	Fabric-native teams

MongoDB Atlas users export to analytical stores for ML; **Redshift ML** wraps SageMaker behind SQL similarly to BQML's managed training. The pattern's shared virtue: governance, lineage, and access control of the warehouse extend automatically to training data and models.

Key Terms

BQML: BigQuery ML—model training, evaluation, and prediction expressed in GoogleSQL against warehouse tables. **CREATE MODEL:** the DDL statement that trains a model over a query result. **ARIMA_PLUS:** BQML’s managed time-series forecaster handling seasonality, holidays, and anomalies automatically. **ML.EVALUATE:** the function returning model quality metrics on held-out or supplied data. **ROC AUC:** area under the receiver operating characteristic curve; a threshold-independent classifier quality measure. **Feature preprocessing:** BQML’s automatic one-hot encoding of categoricals and standardization of numerics.

17.1 Week 17 Roadmap

Week 17 opens Part V with the fastest path from data to model: no exports, no notebooks, no infrastructure. Monday surveys BQML’s model catalog. Tuesday forecasts with **ARIMA_PLUS**. Wednesday evaluates and tunes. Thursday trains a churn classifier on the Google Analytics sample dataset. Friday architects thousand-store inventory forecasting in pure SQL.

Definition

BigQuery ML executes machine learning inside the warehouse: **CREATE MODEL** trains over a SQL query, **ML.EVALUATE** scores it, **ML.PREDICT** serves it—with preprocessing, train/test splitting, and hyperparameter defaults managed by the service [41].

17.2 Monday: The BQML Catalog—Regression, Classification, Clustering

17.2.1 Model Types

The practical catalog: **linear and logistic regression** for numeric and binary targets; **boosted trees** (XGBoost-style) and **deep neural networks** for nonlinear tabular problems; **k-means** for segmentation; **matrix factorization** for recommendations; **ARIMA_PLUS** for time series; and **AutoML Tables** integration when you want BQML syntax over Vertex’s AutoML engine. Choosing is mostly standard ML judgment: start with the interpretable baseline (logistic regression), graduate to boosted trees when interactions matter, reach for DNNs last in SQL.

```

1 CREATE OR REPLACE MODEL analytics.churn_model
2 OPTIONS(model_type = 'LOGISTIC_REG',
3         input_label_cols = ['will_churn'],
4         auto_class_weights = TRUE)
5 AS
6 SELECT
7     totals.visits,
8     totals.pageviews,
9     totals.timeonsite,
10    geoNetwork.country,
11    device.deviceCategory,
12    IF(totals.transactions IS NULL, 0, 1) AS will_churn
13 FROM `bigquery-public-data.google_analytics_sample.ga_sessions_*`
14 WHERE _TABLE_SUFFIX BETWEEN '20160801' AND '20170630';

```

Listing 17.1: Training a churn classifier in pure SQL.

17.2.2 Why Train Where Data Lives

Moving data to a training silo costs pipeline code, freshness, and governance: the extract drifts from the warehouse, and the warehouse’s access controls no longer apply. In-warehouse training keeps one copy, one permission model, one lineage graph—and makes retraining a scheduled SQL query (Chapter 24 makes it a Dataform model). The limits are equally real: exotic architectures, GPU-heavy training, and non-tabular data belong to Vertex AI (Chapters 18–19).

Common Pitfall

CREATE MODEL reads the whole training query every run. Point it at a four-year unfiltered scan and you pay for it repeatedly. Materialize a curated training table (partitioned, clustered) first, train from that, and version it so models are reproducible.

17.3 Tuesday: Time-Series Forecasting with ARIMA_PLUS

17.3.1 Managed Forecasting

ARIMA_PLUS automates the classical ARIMA workflow—differencing, seasonal decomposition, holiday effects, hyperparameter search—behind OPTIONS(model_type='ARIMA_PLUS', time_series_timestamp_col=..., time_series_data_col=...). With time_series_id_col it trains one model per series (per store, per SKU) in a single statement: the thousand-store forecast is one query, not one thousand jobs.

```

1 CREATE OR REPLACE MODEL analytics.store_demand
2 OPTIONS(model_type = 'ARIMA_PLUS',
3         time_series_timestamp_col = 'sale_date',
4         time_series_data_col = 'units_sold',
5         time_series_id_col = 'store_id',
6         holiday_region = 'US')
7 AS SELECT sale_date, store_id, units_sold
8 FROM analytics.daily_store_sales;
9
10 SELECT store_id, forecast_timestamp, forecast_value,
11         prediction_interval_lower_bound, prediction_interval_upper_bound
12 FROM ML.FORECAST(MODEL analytics.store_demand,
13                 STRUCT(30 AS horizon, 0.9 AS confidence_level))
14 ORDER BY store_id, forecast_timestamp;

```

Listing 17.2: Per-store daily demand forecasting and forecast inspection.

17.3.2 Anomaly Detection from Residuals

ML.DETECT_ANOMALIES flags points whose observed values fall outside the model’s prediction interval—for ARIMA_PLUS, anomalies in the *history* (a demand spike the seasonality cannot explain) and, applied to new data, anomalies in *operations* (today’s sales diverging from forecast). The same primitive powers data-quality monitoring: forecast the row-count time series of a critical table and alert when reality leaves the interval.

Tip

Feed ARIMA_PLUS clean, gap-filled daily series: missing dates become implicit zeros or gaps depending on options, and both distort seasonality. A GENERATE_DATE_ARRAY spine left-joined to actuals is the standard preprocessing idiom.

17.4 Wednesday: Evaluation and Hyperparameter Tuning

17.4.1 Reading ML.EVALUATE

For classifiers, `ML.EVALUATE` returns precision, recall, accuracy, F1, log loss, and ROC AUC. `ML.ROC_CURVE` returns per-threshold true/false-positive rates for threshold selection. For imbalanced targets—churn, fraud—accuracy is a liar (predicting “no churn” everywhere scores high); precision/recall at the operating threshold and ROC AUC are the honest metrics. Evaluate on a time-split holdout for temporal problems: random splits leak future behavior into training and inflate every metric.

Metric	Question answered	Churn-model caution
ROC AUC	Ranking quality across all thresholds	Threshold-free; hides operating-point pain
Precision	Of predicted churners, how many churn?	Low precision floods retention campaigns
Recall	Of true churners, how many caught?	Low recall misses saveable customers
Accuracy	Overall correctness	Misleading under class imbalance
Log loss	Probability calibration quality	Penalizes confident wrongness; tuning target

Table 17.1: Classifier evaluation metrics and their churn-prediction reading.

17.4.2 Tuning and Overfitting

BQML supports hyperparameter tuning (`num_trials`, `hparam_tuning_objectives`) over learning rate, regularization, and architecture knobs, reporting trials in `ML.TRIAL_INFO`. Detect overfitting by comparing `ML.EVALUATE` on training versus held-out data: a wide gap (training AUC 0.99, holdout 0.78) says the model memorized; regularize, simplify features, or gather data. With `auto_class_weights=TRUE`, verify the implied threshold on the ROC curve rather than trusting the default 0.5 cutoff.

Common Pitfall

A model evaluated on the same period it trained on is a horoscope, not a forecast. For temporal problems, always hold out the most recent period and evaluate there—and repeat the evaluation after every data-pipeline change upstream, because feature drift silently invalidates yesterday’s metrics.

17.5 Thursday Hands-On Project: Churn Prediction on Google Analytics Data

This lab trains a logistic-regression churn model on the public Google Analytics sample dataset and evaluates its ROC curve, entirely in SQL.

17.5.1 Train

Run Wednesday’s `CREATE MODEL` against `bigquery-public-data.google_analytics_sample.ga_sessions_*` using the first eleven months for training. The label flags visitors with transactions; treat the

exercise as churn-propensity scoring (high-transaction propensity as the positive class for a clean binary).

17.5.2 Evaluate and Predict

```

1 SELECT *
2 FROM ML.EVALUATE(MODEL analytics.churn_model, (
3   SELECT totals.visits, totals.pageviews, totals.timeonsite,
4          geoNetwork.country, device.deviceCategory,
5          IF(totals.transactions IS NULL, 0, 1) AS will_churn
6   FROM `bigquery-public-data.google_analytics_sample.ga_sessions_*`
7   WHERE _TABLE_SUFFIX BETWEEN '20170701' AND '20170801'));
8
9 SELECT threshold, recall, true_positives, false_positives, true_negatives
10 FROM ML.ROC_CURVE(MODEL analytics.churn_model, (
11   SELECT totals.visits, totals.pageviews, totals.timeonsite,
12          geoNetwork.country, device.deviceCategory,
13          IF(totals.transactions IS NULL, 0, 1) AS will_churn
14   FROM `bigquery-public-data.google_analytics_sample.ga_sessions_*`
15   WHERE _TABLE_SUFFIX BETWEEN '20170701' AND '20170801'))
16 ORDER BY threshold;
17
18 SELECT fullVisitorId, predicted_will_churn_probs
19 FROM ML.PREDICT(MODEL analytics.churn_model, (
20   SELECT fullVisitorId, totals.visits, totals.pageviews, totals.timeonsite,
21          geoNetwork.country, device.deviceCategory
22   FROM `bigquery-public-data.google_analytics_sample.ga_sessions_20170801`)
23 ORDER BY (SELECT prob FROM UNNEST(predicted_will_churn_probs) WHERE label = 1)
24          DESC
25 LIMIT 100;

```

Listing 17.3: Evaluating the churn model and reading its ROC curve.

Deliverables

- The training, evaluation, and prediction SQL committed with a README on the time split.
- The ML.EVALUATE output and the chosen operating threshold with justification.
- A short paragraph comparing boosted-tree and logistic-regression results on the same data.

Acceptance Criteria

- Training and evaluation use disjoint time periods.
- Metrics reported include ROC AUC and precision/recall at a stated threshold, not accuracy alone.
- The prediction query produces calibrated probability output, not just labels.

Stretch Goals

- Retrain with BOOSTED_TREE_CLASSIFIER and ML.EXPLAIN_PREDICT for feature attributions.
- Run a ten-trial hyperparameter search and report the trial table.
- Schedule weekly retraining via a scheduled query and log metrics per run.

17.6 Friday Use Case and Architecture: SQL-Only Forecasting for 1,000 Stores

17.6.1 Scenario

A grocery chain must forecast daily unit demand per store per category—one thousand stores, fifty categories—for replenishment planning. The analytics team is SQL-fluent; there is no ML platform team; the forecast must refresh nightly and feed the ordering system by 05:00.

17.6.2 Architecture

The whole system is scheduled SQL: a Dataform or scheduled-query pipeline (1) rebuilds the gap-filled daily series per store-category from the sales replica, (2) retrains `ARIMA_PLUS` weekly, (3) nightly runs `ML.FORECAST` with a seven-day horizon into a forecast table partitioned by forecast date, (4) applies business rules (min/max stock, case-pack rounding) in SQL, and (5) exports the order file to the ERP landing bucket. Monitoring is itself a forecast: `ML.DETECT_ANOMALIES` over realized-versus-forecast error, paged when sustained.

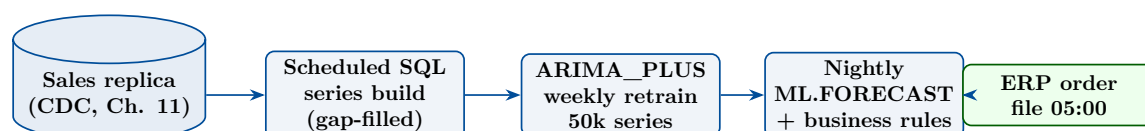


Figure 17.1: SQL-only inventory forecasting: CDC replica to nightly per-series forecasts to the ERP.

Tip

Fifty thousand series in one `CREATE MODEL` is normal for `ARIMA_PLUS`, but watch the long tail: stores with sparse histories produce wide intervals. Segment by data sufficiency and route thin series to a simpler fallback (moving average) rather than trusting a heroic model.

Common Pitfall

A forecast table without a forecast-generation timestamp is unusable for accountability: you can never reconstruct “what did we predict on Tuesday for Friday.” Always persist `forecast_for_date`, `generated_at`, and model version with every batch.

17.7 Interview Q&A

17.7.1 Concepts and Trade-offs

1. **Q: Which BQML model type fits customer churn prediction?**

A: Logistic regression as the interpretable baseline; boosted trees when nonlinear feature interactions justify the opacity.

2. **Q: What does `ARIMA_PLUS` automate compared to classic `ARIMA`?**

A: Differencing, seasonality, holiday effects, hyperparameter search, and per-series parallelism via `time_series_id_col`.

3. **Q: Which `ML.EVALUATE` metrics matter most for imbalanced churn?**

A: ROC AUC for ranking quality plus precision and recall at the chosen operating threshold; accuracy misleads under imbalance.

4. **Q: What is the advantage of training where the data already lives?**

A: No extract pipelines, one permission model, full lineage, and retraining as a scheduled SQL query.

5. **Q: How do you detect overfitting with BQML evaluation output?**

A: Compare ML.EVALUATE on training versus time-held-out data; a large quality gap indicates memorization.

17.8 Further Reading

Read the BigQuery ML introduction and model reference for model types, options, and evaluation functions [41]. The BigQuery documentation covers scheduled queries that productionize retraining [9]. Vertex AI documentation previews the custom-training path for when BQML's catalog is insufficient [57], and Chapter 24's Dataform reference covers SQL pipeline discipline [31].

Key Takeaways

- BQML moves ML to the data: governance, freshness, and lineage come free.
- ARIMA_PLUS turns thousand-series forecasting into one query—given clean, gap-filled inputs.
- Evaluate on time-held-out data; report threshold-aware metrics for imbalanced problems.
- Materialize curated training tables; never train over raw multi-year scans.
- Persist forecast provenance (generated-at, version) with every batch.
- Know the boundary: exotic architectures and non-tabular data belong to Vertex AI.

17.9 Exercises

1. **(Conceptual)** When does BQML's in-warehouse training lose to a Vertex AI custom job? Give three concrete cases.
2. **(Conceptual)** Explain why random train/test splits inflate metrics for temporal targets.
3. **(Hands-on)** Reproduce the Thursday lab, then build the ROC-based threshold analysis for a retention-budget constraint (campaign capacity of the top 5%).
4. **(Hands-on)** Forecast one public time series with ARIMA_PLUS and verify interval coverage on a held-out month.
5. **(Design)** Specify the anomaly-detection layer for the store forecasts: error metrics, alert thresholds, and the human triage loop.
6. **(Architecture)** Add a feedback table capturing realized orders versus forecasts, and design the weekly accuracy review query.

Vertex AI Foundations: Training, Registry, and Serving

Learning Objectives

- Map the Vertex AI ecosystem and choose between AutoML and custom training.
- Train an AutoML Tabular model on a BigQuery dataset and version it in the Model Registry.
- Deploy models to managed endpoints and choose between online and batch prediction.
- Interpret feature attributions from Explainable AI for model consumers.
- Request an online prediction with a cURL call against a deployed endpoint.
- Design a secure GPU training workflow for data scientists working from governed BigQuery data.

Cross-Cloud Equivalence

Managed ML platforms converge on the same lifecycle—train, register, deploy, explain—behind different consoles.

Capability	GCP Vertex AI	AWS SageMaker	Azure ML / Fabric
AutoML (tabular)	AutoML Tabular	Autopilot	Automated ML
Custom training	Training jobs (any framework)	Training jobs	Compute clusters/jobs
Model registry	Vertex Model Registry	Model Registry	MLflow model registry
Online serving	Endpoints	Endpoints	Online endpoints
Batch scoring	Batch prediction	Batch transform	Batch endpoints
Explainability	Explainable AI attributions	Clarify	Responsible AI dashboards

Snowflake embeds Snowpark ML and Cortex for in-warehouse inference; **MongoDB** users typically serve models adjacent to Atlas via app-tier integrations; **Databricks ML** unifies lakehouse training with MLflow. For the data engineer, the constant is the interface contract: features in, predictions out, lineage and governance around both.

Key Terms

Vertex AI: Google Cloud’s unified ML platform for training, registry, deployment, and monitoring. **AutoML:** managed model search and training over a dataset with minimal

code. **Model Registry**: the versioned store of trained models with lineage and aliases. **Endpoint**: a managed serving resource hosting one or more deployed models with traffic splitting. **Batch prediction**: offline scoring of a dataset to files or BigQuery, without a live endpoint. **Feature attribution**: per-feature contribution scores (e.g., sampled Shapley) explaining a prediction.

18.1 Week 18 Roadmap

Week 18 moves from SQL-native ML to the full platform. Monday maps the Vertex AI ecosystem and the AutoML-versus-custom decision. Tuesday covers the Model Registry and endpoint deployment. Wednesday covers Explainable AI. Thursday trains and deploys an AutoML Tabular model from BigQuery and calls it with cURL. Friday designs the secure GPU workflow for data scientists.

Definition

Vertex AI is Google Cloud’s managed machine-learning platform: datasets from BigQuery or Cloud Storage, AutoML and custom training on managed infrastructure, a versioned Model Registry, online endpoints and batch prediction, Explainable AI, pipelines, and model monitoring under one API and permission model [57].

18.2 Monday: The Ecosystem and AutoML versus Custom Training

18.2.1 The Platform Map

Vertex AI’s components compose: **Datasets** reference BigQuery tables or Cloud Storage files; **training** runs AutoML or custom jobs (your container, any framework, managed GPUs); the **Model Registry** versions artifacts with lineage back to dataset and code; **endpoints** serve online predictions; **batch prediction** scores datasets offline; **Feature Store**, **Pipelines**, and **Model Monitoring** (Chapter 19) close the loop. For the data engineer, the key property is that data never leaves governed storage: training reads BigQuery in place, and predictions can write back to it.

18.2.2 AutoML or Custom

AutoML Tabular searches architectures and hyperparameters over your table—strong baselines with zero modeling code, ideal when the problem is standard tabular classification/regression and the team’s constraint is time or ML depth. Custom training wins when you need a specific architecture, custom loss, non-tabular data, fine control over the training loop, or a framework the team already knows (TensorFlow, PyTorch, XGBoost). The professional pattern: AutoML establishes the baseline in days; custom training must beat it measurably to justify its operational cost.

Common Pitfall

AutoML’s convenience hides a data contract: schema, label semantics, and time-splits are inputs you still own. An AutoML model trained on leaky features (columns unknowable at prediction time) performs beautifully in evaluation and collapses in production. Leakage review is a human responsibility no platform automates.

Tip

Always declare the time column for temporal problems so AutoML’s split respects chronology. Random splits on temporal data leak the future into training—the same trap as Chapter 17, one platform layer up.

18.3 Tuesday: Model Registry and Endpoint Deployment

18.3.1 Registry as the System of Record

Every model version in the Registry carries its artifact, framework, metrics, dataset lineage, and labels; aliases (**production**, **challenger**) decouple consumers from version numbers. The registry is where governance lives: approvals are alias changes, rollbacks are alias moves, audits are lineage queries. Data engineers feed it (training pipelines) and consume it (batch-scoring jobs that resolve **production** rather than hardcoding a version).

18.3.2 Online versus Batch Serving

Online endpoints serve single predictions with millisecond latency for interactive flows—checkout fraud checks, next-best-action—and bill for provisioned machine time with configurable min/max replicas per deployed model, plus traffic splitting for canary rollouts. **Batch prediction** scores a BigQuery table or file set into BigQuery or Cloud Storage with no endpoint to run—the right answer for nightly scoring of millions of rows. The decision is latency and traffic shape, not model quality: the same registered model serves both.

Dimension	Online endpoint	Batch prediction
Latency	Milliseconds, per request	Minutes to hours, per dataset
Cost shape	Always-on replicas (min nodes)	Per job, scales to zero
Traffic management	Split across model versions	One model per job
Best for	Interactive flows (checkout, API)	Nightly scores, list generation

Table 18.1: Online versus batch serving on Vertex AI [57].

Common Pitfall

`min_replica_count=0` eliminates idle cost but reintroduces cold-start latency on the first request after idle—tens of seconds of spinner on a checkout flow. If an endpoint serves humans, keep at least one warm replica or accept and measure the cold-start tail.

18.4 Wednesday: Explainable AI and Feature Attributions

Explainable AI returns **feature attributions** with each prediction: per-feature contribution scores (sampled Shapley or integrated gradients, method depending on model type) answering “why this score for this input.” For data engineers, attributions serve two operational purposes beyond model trust. First, **debugging the data pipeline**: when a feature’s attribution suddenly dominates or vanishes, the feature computation upstream likely broke—attributions are a distribution-shift canary. Second, **consumer communication**: a credit-decision or churn-retention system owes its users reasons, and attribution payloads are the raw material of those

explanations. Persist attributions alongside predictions in BigQuery so both uses are queryable later.

Note

Attributions explain the model, not the world: they describe what the model used, which is exactly what you need for debugging and disclosure, and not evidence that the model's reasoning is causally correct.

18.5 Thursday Hands-On Project: AutoML Tabular from BigQuery to a Live Endpoint

This lab trains an AutoML Tabular model on a cleaned BigQuery dataset, registers it, deploys it to an endpoint, and scores it with cURL.

18.5.1 Train via the Console or API

1. Create a Vertex AI tabular dataset from `analytics.churn_features` (the Chapter 17 feature table, cleaned and labeled).
2. Launch an AutoML Tabular classification training job: label `will_churn`, a declared time column if present, a one-hour training budget, and explainability enabled.
3. When training completes, review evaluation metrics in the Registry—the same reading as Chapter 17: AUC, precision/recall at threshold.

18.5.2 Deploy and Predict

```

1 $env:PROJECT_ID = "my-gcp-vertex-lab"
2 $env:REGION = "us-central1"
3 $env:MODEL = "churn-automl-lab"
4 $env:ENDPOINT = "churn-endpoint-lab"
5
6 gcloud ai endpoints create --region=$env:REGION --display-name=$env:ENDPOINT
7
8 # Deploy with one warm replica (note the endpoint/model IDs from output):
9 gcloud ai endpoints deploy-model <ENDPOINT_ID> '
10 --region=$env:REGION --model=<MODEL_ID> '
11 --display-name=churn-v1 '
12 --machine-type=n1-standard-4 --min-replica-count=1 --max-replica-count=2
13
14 @'
15 {"instances": [{"visits": 12, "pageviews": 45, "timeonsite": 620,
16                  "country": "United States", "deviceCategory": "desktop"}]}
17 '@ | Set-Content -Path .\instance.json
18
19 $token = (gcloud auth print-access-token)
20 Invoke-RestMethod -Method Post '
21 -Uri "https://$env:REGION-aiplatform.googleapis.com/v1/projects/$env:
22     PROJECT_ID/locations/$env:REGION/endpoints/<ENDPOINT_ID>:predict" '
23 -Headers @{ Authorization = "Bearer $token" } '
24 -ContentType "application/json" '
25 -InFile .\instance.json

```

Listing 18.1: Deploying the model to an endpoint and requesting an online prediction.

Deliverables

- Training-job configuration and the evaluation report screenshot from the Registry.
- The endpoint ID, deployed-model details, and a successful prediction response with attributions.
- A cost note: training hours, endpoint replica-hours, and the batch alternative for this workload.

Acceptance Criteria

- The model trains from a governed BigQuery table with a declared time-based split where applicable.
- The endpoint serves a prediction with feature attributions via a scripted call.
- The Registry shows the model version with lineage to the dataset.

Stretch Goals

- Score the same dataset via batch prediction to BigQuery and compare per-row cost with the endpoint.
- Run a challenger AutoML config behind a 90/10 traffic split and compare live metrics.
- Wire prediction logging into BigQuery and chart attribution drift over a week of synthetic traffic.

18.6 Friday Use Case and Architecture: Secure GPU Training over Governed Data

18.6.1 Scenario

A fintech's data scientists train custom TensorFlow models on GPUs. Training data lives in BigQuery under row access policies and policy tags (Chapter 8); regulatory policy forbids exporting row-level PII to local laptops; the platform team must provide self-service GPU training without weakening the governance perimeter.

18.6.2 Architecture

Scientists work in Vertex AI Workbench instances inside the project VPC—no public IPs, access via Identity-Aware Proxy, Workload Identity instead of downloaded keys (Chapter 21). Training jobs read BigQuery through the BigQuery Storage API with the scientist's or pipeline's identity, so policy tags and row policies filter what training can even see; derived feature tables carry their own policy tags. Custom training jobs run as managed Vertex jobs on GPU machine types with the training service account scoped to read the feature dataset and write only to a designated model-artifacts bucket under CMEK. Registered models flow to staging endpoints automatically; production alias moves require an approver group.

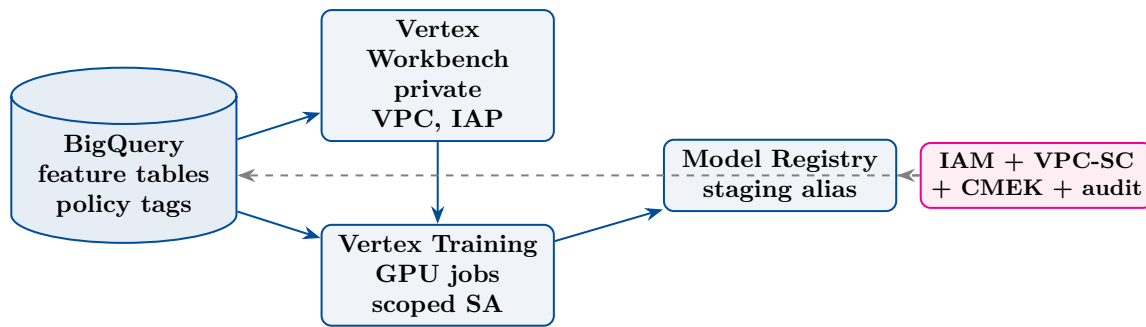


Figure 18.1: Secure GPU workflow: governed reads, scoped training identities, gated production promotion.

Tip

Give scientists their speed through *pre-approved paths*: a golden container image with their frameworks, a feature-store dataset contract, and a one-command job submission. Governance that makes the secure path the easy path gets followed; governance that blocks work gets routed around.

Common Pitfall

Notebook persistence is a data-exfiltration channel: a Workbench instance with a mounted home directory and egress to the internet can carry training data anywhere. Disable public IPs, route egress through controlled NAT with logging, and include notebook disks in the CMEK and VPC-SC posture.

18.7 Interview Q&A

18.7.1 Concepts and Trade-offs

1. **Q: When is AutoML Tabular sufficient versus custom training?**
A: When the problem is standard tabular classification/regression and time or ML depth is the constraint; custom training earns its cost for specific architectures, custom losses, or non-tabular data—and must beat the AutoML baseline measurably.
2. **Q: What does the Model Registry version control?**
A: Model artifacts with framework, metrics, dataset and code lineage, labels, and aliases—making promotion an alias move and rollback trivial.
3. **Q: Online versus batch prediction: which serves a checkout flow?**
A: Online endpoints—millisecond, per-request latency with warm replicas. Batch prediction serves nightly offline scoring where latency is irrelevant.
4. **Q: What do feature attributions tell a model consumer?**
A: Which features contributed most to this specific prediction—enough for reason codes and debugging, not causal proof.
5. **Q: Why deploy through an endpoint rather than calling the model directly?**
A: Endpoints add the operational layer—scaling, traffic splitting, monitoring, IAM—that raw model artifacts lack; consumers integrate once with the endpoint, not with every model version.

18.8 Further Reading

Read the Vertex AI documentation for datasets, AutoML, custom training, endpoints, and Explainable AI [57]. The BigQuery Storage API and policy-tag enforcement behind secure training reads are covered in the BigQuery documentation [9]. Chapter 19’s pipeline and monitoring documentation completes the lifecycle [59], and Chapter 21’s VPC Service Controls documentation covers the perimeter referenced here [60].

Key Takeaways

- AutoML is the baseline generator; custom training must beat it to justify its operational weight.
- The Registry is the system of record: lineage in, aliases out, approvals as alias moves.
- Online endpoints serve interactive flows with warm-replica cost; batch prediction scales to zero for offline scoring.
- Feature attributions double as pipeline debugging canaries and consumer-facing reason codes.
- Secure training means governed reads in place, scoped service accounts, private workbenches, and gated promotion.
- Leakage and time-split discipline are human responsibilities at every level of platform abstraction.

18.9 Exercises

1. **(Conceptual)** A team wants custom PyTorch training for a standard tabular churn model before any AutoML baseline exists. Construct the argument you would have with them.
2. **(Conceptual)** Explain how traffic splitting plus the **production** alias gives you canary deployment with instant rollback.
3. **(Hands-on)** Reproduce the Thursday lab, then batch-score the same dataset to BigQuery and reconcile scores against endpoint predictions.
4. **(Hands-on)** Persist ten attribution payloads to BigQuery and write the query finding the feature with the largest week-over-week attribution shift.
5. **(Design)** Write the IAM and network specification for the Friday fintech: roles per persona, service accounts, egress rules, and the promotion approval flow.
6. **(Architecture)** Add a cost model: endpoint replicas versus batch prediction for three traffic shapes (interactive API, nightly scores, monthly deep analysis).

MLOps and Vertex AI Pipelines

Learning Objectives

- Serve features consistently through Vertex AI Feature Store’s online and offline paths.
- Compose Kubeflow pipelines (Extract, Train, Deploy) as versioned, reproducible components.
- Gate model registration and deployment on evaluation metrics, not on hope.
- Wire continuous-training triggers to model-monitoring drift and skew alerts.
- Distinguish training-serving skew from production data drift and respond to each.
- Architect the self-retraining MLOps lifecycle for a production recommendation engine.

Cross-Cloud Equivalence

MLOps platforms converge on the same loop—features, pipelines, registry, monitoring, retraining—with different orchestration defaults.

Capability	GCP Vertex AI	AWS SageMaker	Azure ML / Databricks
Pipelines	Vertex Pipelines (KFP/TFX)	SageMaker Pipelines	Azure ML pipelines / MLflow + Workflows
Feature store	Vertex AI Feature Store	SageMaker Feature Store	Feature Store (Databricks)
Monitoring	Vertex Model Monitoring	Model Monitor	ML monitoring / Lakehouse monitoring
Continuous training	Pipeline triggers + Functions	EventBridge + Pipelines	Event Grid triggers
Metadata	ML Metadata lineage	SageMaker lineage	MLflow tracking

Snowflake users run ML workflows in Snowpark with tasks and streams as triggers; **MongoDB** users build feature pipelines over change streams. The portable principle: a model is a pipeline artifact with a lineage graph, and retraining is an *event-driven reaction to evidence*, not a calendar entry.

Key Terms

Feature Store: a managed store serving the same feature definitions offline (training datasets) and online (low-latency lookups), preventing training-serving skew by construction. **Kubeflow pipeline:** a DAG of containerized components with typed inputs/outputs, compiled to YAML and executed on Vertex AI Pipelines. **Component:** one reusable

pipeline step—a container image plus an interface—with tracked artifacts in and out. **Continuous training (CT)**: event-triggered retraining, typically from drift or freshness signals. **Training-serving skew**: divergence between features as computed in training versus as served in production. **Data drift**: production input distributions diverging from the training distribution over time.

19.1 Week 19 Roadmap

Week 19 industrializes Part V. Monday covers Feature Store’s dual serving paths. Tuesday composes Kubeflow pipelines. Wednesday wires continuous training to model monitoring. Thursday builds and runs the Extract-Train-Deploy pipeline. Friday architects drift-triggered retraining for a recommendation engine.

Definition

MLOps is the engineering discipline that makes model lifecycle—data, features, training, evaluation, deployment, monitoring, retraining—reproducible, automated, and observable. On GCP it assembles from Feature Store, Vertex AI Pipelines, Model Registry, endpoints, and Model Monitoring [59, 58].

19.2 Monday: Vertex AI Feature Store—Online and Offline Serving

19.2.1 One Definition, Two Paths

Features computed twice—once in the training SQL, once in the serving application—will eventually diverge: a null-handling difference, a timezone, a library version. That divergence is **training-serving skew**, and it is the most common cause of “the model was great in evaluation.” Feature Store’s answer is to make divergence structurally impossible: feature values live in the store; **offline serving** exports training datasets from historical values; **online serving** exposes the latest values through low-latency lookups to the prediction path. Training and serving read the same feature definitions from the same system.

19.2.2 Point-in-Time Correctness

Offline serving joins entity IDs and timestamps against feature history *as of* each example’s time—preventing the subtlest leakage, where a training row sees a feature value that did not exist yet (yesterday’s training example joining today’s aggregate). When you build training sets by hand, the point-in-time join is your responsibility; the store makes it the default.

Common Pitfall

The Feature Store stores *values*, not *computation*: feature engineering still lives upstream in BigQuery/Dataflow jobs you own (Chapters 14–17). Skew survives if those upstream jobs compute training features one way and the streaming featurization another—the store synchronizes reads, not logic. Keep one featurization codebase feeding both paths.

Tip

Register feature freshness as a first-class signal: each feature group should carry an expected update cadence, and staleness beyond it should alarm before it silently degrades predictions.

Stale features are drift you can prevent.

19.3 Tuesday: Kubeflow Pipelines and Vertex AI Pipelines

19.3.1 Components and Composition

A Kubeflow pipeline is a Python-authored DAG of components; each component is a container image with a typed interface (inputs, outputs, artifacts). Vertex AI Pipelines executes the DAG on managed GKE, tracking every artifact—dataset snapshots, metrics, models—into ML Metadata, so any produced model traces back through code, data, and parameters. Components are shareable: an “extract features from BigQuery” component written once becomes organizational vocabulary.

```

1 from kfp import dsl
2 from kfp.dsl import component
3
4 @component(base_image="python:3.10", packages_to_install=["google-cloud-
    bigquery"])
5 def extract_features(project: str, dataset: dsl.Output[dsl.Dataset]) -> None:
6     from google.cloud import bigquery
7     rows = bigquery.Client(project=project).query(
8         "SELECT * FROM analytics.churn_features").to_dataframe()
9     rows.to_csv(dataset.path, index=False)
10
11 @component(base_image="python:3.10", packages_to_install=["scikit-learn", "
    pandas", "joblib"])
12 def train_model(dataset: dsl.Input[dsl.Dataset],
13                model: dsl.Output[dsl.Model], metrics: dsl.Output[dsl.Metrics])
14     -> None:
15     import joblib, pandas as pd
16     from sklearn.linear_model import LogisticRegression
17     df = pd.read_csv(dataset.path)
18     X, y = df.drop(columns=["will_churn"]), df["will_churn"]
19     clf = LogisticRegression(max_iter=1000).fit(X, y)
20     metrics.log_metric("train_accuracy", float(clf.score(X, y)))
21     joblib.dump(clf, model.path)
22
23 @dsl.pipeline(name="extract-train-deploy")
24 def pipeline(project: str) -> None:
25     d = extract_features(project=project)
26     t = train_model(dataset=d.outputs["dataset"])
27     _ = t # deploy component follows, gated on metrics

```

Listing 19.1: A minimal extract-train-deploy Kubeflow pipeline.

Note

Every pipeline run is immutable evidence: input dataset snapshot, code image digests, parameters, metrics, produced model. When someone asks “what changed between the model from March and the model from May,” the lineage graph answers from facts, not memory.

19.4 Wednesday: Continuous Training and Model Monitoring

19.4.1 Gates, Not Just Triggers

Automated retraining without a gate automates the shipping of bad models. The production pattern: monitoring raises a signal; a trigger launches the pipeline; the pipeline retrains *and*

evaluates; a gate component compares candidate metrics against the incumbent’s (and against absolute thresholds); only a passing candidate registers and deploys, with notification either way.

```

1 @component(base_image="python:3.10")
2 def register_gate(evaluation: dsl.Input[dsl.Metrics]) -> bool:
3     auc = evaluation.metadata["roc_auc"]
4     if auc < 0.80:
5         raise ValueError(f"ROC AUC {auc:.3f} below 0.80 gate")
6     return True # downstream register/deploy tasks only run if this passes

```

Listing 19.2: An evaluation gate that blocks bad models from registration.

19.4.2 Skew versus Drift

Training-serving skew: serving features differ from training features *right now*—a bug or divergence, detectable by comparing request features against the training distribution at deploy time. **Prediction/data drift:** production inputs diverge from training *over time*—the world changed, detectable by monitoring input distributions against the training baseline. Vertex Model Monitoring computes both (categorical and numeric distribution distances) per feature and raises alerts on threshold breach. Skew says “fix the pipeline”; drift says “retrain the model”; confusing them pages the wrong responder.

Tip

Wire monitoring alerts to a notification channel *and a runbook*: triage the drifting feature, validate upstream schema drift, then trigger retraining only after data-quality checks pass. The runbook—not the alert—is the automation’s real content.

19.5 Thursday Hands-On Project: Build and Run an Extract-Train-Deploy Pipeline

This lab composes the extract and train components from Tuesday into a compiled pipeline, adds a register gate, and runs it on Vertex AI Pipelines.

19.5.1 Compile and Submit

```

1 from google.cloud import aiplatform
2 from kfp import compiler
3
4 compiler.Compiler().compile(
5     pipeline_func=pipeline, package_path="extract_train_deploy.yaml")
6
7 aiplatform.init(project="my-gcp-mlops-lab", location="us-central1",
8     staging_bucket="gs://my-gcp-mlops-lab-pipelines")
9
10 job = aiplatform.PipelineJob(
11     display_name="churn-extract-train-deploy",
12     template_path="extract_train_deploy.yaml",
13     parameter_values={"project": "my-gcp-mlops-lab"})
14 job.run()

```

Listing 19.3: Compiling the pipeline and launching a run on Vertex AI.

19.5.2 Inspect the Run

In the Vertex AI Pipelines console, open the run: the DAG shows each component’s status; expanding the train component shows the logged accuracy metric; the lineage view links dataset

artifact to model artifact. Deliberately degrade the gate (set the threshold above the achieved AUC), rerun, and confirm registration is skipped with a visible, explained failure.

Deliverables

- The component definitions, pipeline DSL, compiled YAML, and submission script.
- Console evidence of a successful run and of the gated run that correctly refused to register.
- A lineage screenshot from dataset artifact to model artifact.

Acceptance Criteria

- The pipeline runs end-to-end from the compiled YAML with no console clicks.
- Metrics are logged as artifacts, and registration is conditional on the gate.
- Rerunning with the same parameters reproduces the run (data snapshot permitting).

Stretch Goals

- Add a deploy component with traffic splitting between incumbent and challenger.
- Parameterize the BigQuery source query and feature snapshot date for backfills.
- Trigger the pipeline from a Cloud Function on a Pub/Sub monitoring alert (rehearsing Friday).

19.6 Friday Use Case and Architecture: Self-Retraining Recommendation Engine

19.6.1 Scenario

A streaming service's recommendation model degrades as catalogs and tastes shift. The requirement: detect degradation automatically, retrain with human-auditable gates, and never ship a regression. Business stakeholders want a written account of when the system may act without a human.

19.6.2 Architecture

Feature engineering runs nightly in Dataflow/BigQuery into Feature Store (online and offline). Vertex Model Monitoring watches the production endpoint's request features and prediction distribution against the training baseline; drift beyond threshold publishes to a Pub/Sub topic; a Cloud Function validates preconditions (feature freshness, data-quality checks green, no active incident freeze) and launches the retraining pipeline. The pipeline extracts a fresh point-in-time training set, trains, evaluates on a recent holdout, gates against the incumbent's metrics, registers the challenger, deploys to 10% traffic, and reports to a metrics dashboard. Weekly, a human reviews auto-promotions; a promotion to 100% requires the review's sign-off in the early maturity phase and can relax to automatic-with-alerts as trust is earned.

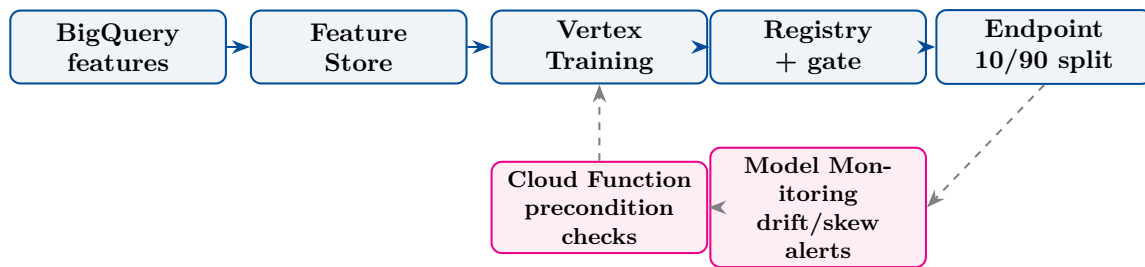


Figure 19.1: Closed-loop MLOps: monitoring detects drift, a preconditioned trigger retrains, gates protect production.

Common Pitfall

Auto-deploying without an evaluation gate ships regressions silently—and the first person to notice is a customer. The gate is not optional ceremony; it is the difference between automation and negligence. Page a human on gate failures: they mean either the world changed faster than the model could follow or the training data is broken.

Tip

Document the autonomy envelope in writing: which signals trigger retraining, which gates must pass, which promotions need human review, and the freeze conditions (incidents, holiday code freezes) that suspend automation. Regulators and incident reviewers will ask for exactly this document.

19.7 Interview Q&A

19.7.1 Concepts and Trade-offs

1. **Q: What problem does a Feature Store solve between training and serving?**
A: Training-serving skew and leakage: one feature definition serves offline point-in-time training sets and online lookups, so both paths read identical semantics.
2. **Q: Online versus offline serving: which backs a real-time recommender?**
A: Online serving—low-latency latest-value lookups per entity at request time; offline serving builds training datasets.
3. **Q: What does a Kubeflow pipeline component encapsulate?**
A: A containerized step with typed inputs, outputs, and artifacts—reusable, versioned, and lineage-tracked across runs.
4. **Q: Which signal should gate an automated retraining trigger?**
A: Preconditioned drift alerts (feature freshness and data-quality green first), and the candidate must beat the incumbent on evaluation metrics before registration.
5. **Q: How does prediction skew differ from training-data drift?**
A: Skew is a present mismatch between served and trained features (a pipeline bug); drift is the world's distribution moving away from training data over time (a model-freshness problem).

19.8 Further Reading

Read the Vertex AI Pipelines documentation for components, compilation, and lineage [59], and the Feature Store documentation for online/offline serving and point-in-time lookups [58]. Model Monitoring’s skew and drift detection is covered in the Vertex AI documentation [57]. OpenLineage is worth reading for cross-platform lineage thinking [32], and Chapter 12’s Composer patterns apply when orchestrating the surrounding data work [17].

Key Takeaways

- Feature Store prevents skew by construction—but only if one featurization codebase feeds both paths.
- Pipelines are immutable evidence: artifacts, parameters, and lineage per run.
- Gates, not triggers, protect production: evaluation thresholds before registration, always.
- Skew means fix the pipeline; drift means retrain the model—monitor distinguishes them.
- The drift-to-retrain loop includes precondition checks and human-auditable promotion rules.
- Write down the autonomy envelope; undocumented automation is unauditable automation.

19.9 Exercises

1. **(Conceptual)** Trace a skew incident: training reads a feature computed daily, serving reads it computed hourly. Describe the symptom and the permanent fix.
2. **(Conceptual)** Why is point-in-time correctness impossible to retrofit onto a naive join of features to labels?
3. **(Hands-on)** Reproduce the Thursday pipeline, then add a comparison component that logs challenger-versus-incumbent AUC as a metric.
4. **(Hands-on)** Configure Model Monitoring on a deployed endpoint and generate synthetic drift (shift one feature’s distribution) to observe the alert.
5. **(Design)** Write the autonomy-envelope document for the Friday recommendation system, including freeze conditions and the promotion-review ritual.
6. **(Architecture)** Extend the loop with a champion-challenger analysis dashboard and an automatic rollback trigger on live-metric regression.

Unstructured Data and AI APIs

Learning Objectives

- Select among Vision, Video Intelligence, Natural Language, and Translation APIs for unstructured workloads.
- Integrate per-call AI APIs inside Dataflow pipelines with batching, backoff, and rate-limit discipline.
- Make OCR output searchable and joinable against structured data in BigQuery.
- Forecast per-call API costs and choose between API calls, BQML, and custom models.
- Build the streaming sentiment pipeline: Pub/Sub reviews, Natural Language API, BigQuery sink.
- Architect terabyte-scale image ingestion with OCR, object tagging, and searchable metadata.
- Complete the Milestone 5 capstone: an end-to-end MLOps pipeline with monitoring.

Cross-Cloud Equivalence

Pretrained AI APIs are the fastest path from unstructured data to structured signal on every cloud; the pricing and rate-limit shapes differ, the integration patterns do not.

Capability	GCP	AWS	Azure
Image analysis/OCR	Vision API	Rekognition / Text-tract	Azure AI Vision / Document Intelligence
Video analysis	Video Intelligence	Rekognition Video	Azure Video Indexer
Text NLP	Natural Language API	Comprehend	Azure AI Language
Translation	Translation API	Translate	Azure Translator
Speech	Speech-to-Text	Transcribe	Azure Speech
Generative/LLM	Gemini (Vertex AI)	Bedrock	Azure OpenAI

MongoDB Atlas Vector Search and **Snowflake Cortex** bring embeddings and LLM functions into the data layer; the GCP analog is Vertex AI’s Gemini integration with BigQuery. The engineering constants: per-call pricing demands cost forecasting, and external calls inside distributed pipelines demand batching and backoff.

Key Terms

Vision API: pretrained image understanding—labels, faces, landmarks, logos, OCR—per image call. **Natural Language API:** entity extraction, sentiment, syntax, and classifica-

tion per text call. **InfoType-independent enrichment**: using pretrained APIs to derive structured fields (labels, sentiment, entities) from unstructured payloads. **OCR**: optical character recognition—text extraction from images and scanned documents. **Batching**: aggregating pipeline elements into a single API call to reduce overhead and rate-limit pressure. **Backoff**: retrying failed or throttled calls with increasing delays (exponential with jitter).

20.1 Week 20 Roadmap

Week 20 closes Part V by bringing unstructured data into the platform. Monday covers Vision and Video Intelligence. Tuesday covers Natural Language and Translation. Wednesday is the integration day: AI APIs inside Dataflow. Thursday builds the streaming sentiment pipeline. Friday architects terabyte-scale image ingestion. The week ends with the Milestone 5 MLOps capstone.

Definition

Google Cloud’s **pretrained AI APIs** expose production-grade models—vision, language, translation, speech—behind simple per-call interfaces [34]. They trade model control for operational simplicity: no training, no endpoints to size, per-call pricing, and immediate capability.

20.2 Monday: Vision and Video Intelligence

20.2.1 Vision API

Vision’s feature menu per image: `LABEL_DETECTION` (objects and scenes), `TEXT_DETECTION` (OCR; `DOCUMENT_TEXT_DETECTION` for dense pages), `SAFE_SEARCH_DETECTION` (moderation signals), `FACE_DETECTION` (presence and attributes, not identity), `LOGO_DETECTION`, and `OBJECT_LOCALIZATION`. For data engineering, the design questions are throughput (images per second times cost per image), feature selection (each requested feature is billed), and storage of results (structured rows keyed by object URI, joining back to the Object Table from Chapter 8).

20.2.2 Video Intelligence

Video Intelligence annotates at shot, segment, and frame level: label detection over time, shot changes, explicit content, speech transcription, object tracking. The economics differ from images: one call per video with per-minute pricing, asynchronous operation, and results as annotation JSON. The platform pattern is event-driven: a Cloud Storage finalize event (or Object Tables plus a scheduled diff) triggers annotation, results land in BigQuery, and downstream queries answer “videos containing logos after minute ten.”

Common Pitfall

Requesting every Vision feature on every image multiplies the bill by the feature count. Profile first: run a sample through all features, keep the two or three your downstream queries actually use, and make feature selection a reviewed pipeline parameter.

20.3 Tuesday: Natural Language and Translation APIs

20.3.1 Natural Language API

The workhorse methods: `analyzeSentiment` (document and sentence scores, -1 to 1 , with magnitude), `analyzeEntities` (typed entities with salience), `classifyText` (content categories), and `analyzeEntitySentiment` (sentiment *per entity*—the difference between “the battery died but support was great”). Sentiment score needs magnitude to be interpretable: a score of 0.1 with magnitude 8 is a long, mildly positive document, not a neutral one.

20.3.2 Translation

The Translation API (Basic and Advanced tiers) translates between a hundred-plus languages, with glossaries and batch document translation on Advanced. In data platforms it appears in two places: normalizing multilingual text before NLP (translate to one analysis language, acknowledging translation-loss caveats) and as a data product itself (localized catalog content pipelines).

Tip

Translate-then-analyze versus analyze-in-language is a measurable decision, not a philosophy: sample both on your domain text and compare downstream metrics. Modern multilingual models have narrowed the gap, but domain vocabulary still travels badly through translation.

20.4 Wednesday: AI APIs Inside Dataflow Pipelines

20.4.1 The Integration Pattern

Calling a per-call API from a `ParDo` is correct and dangerous at scale: a streaming pipeline at ten thousand elements per second becomes ten thousand API calls per second—rate limits, tail latency, and cost explode together. The disciplined pattern: **batch** elements into bundles inside the `DoFn` (or use the API’s batch method), call once per bundle with a bounded thread pool, apply exponential backoff with jitter on 429/5xx, and route permanently failing elements to a dead-letter output with the error attached. Beam’s stateful timers (Chapter 15) implement token-bucket throttling when you need a hard calls-per-second ceiling.

```

1 import time
2 import apache_beam as beam
3 from google.cloud import language_v1
4
5
6 class SentimentFn(beam.DoFn):
7     def setup(self):
8         self.client = language_v1.LanguageServiceClient()
9         self.batch = []
10
11     def process(self, element):
12         self.batch.append(element)
13         if len(self.batch) >= 25:
14             yield from self.flush()
15
16     def finish_bundle(self):
17         for row in self.flush():
18             yield row
19
20     def flush(self):

```



```

21     results, batch = [], self.batch
22     self.batch = []
23     for element in batch:
24         results.append(self.call_with_backoff(element))
25     return results
26
27     def call_with_backoff(self, element):
28         doc = {"content": element["text"],
29              "type_": language_v1.Document.Type.PLAIN_TEXT}
30         for attempt in range(5):
31             try:
32                 resp = self.client.analyze_sentiment(document=doc)
33                 return {**element, "score": resp.document_sentiment.score,
34                        "magnitude": resp.document_sentiment.magnitude}
35             except Exception:
36                 if attempt == 4:
37                     raise
38                 time.sleep(min(2 ** attempt, 30))

```

Listing 20.1: Bounded-batch Natural Language calls with retry inside a DoFn.

Common Pitfall

Retries without idempotency analysis double-bill: most AI API calls are safe to retry (pure functions of the payload), but billing is per call—a retry loop around a flaky network path is a billing multiplier. Alert on retry rate as a cost signal, not just a reliability one.

20.4.2 Choosing API versus BQML versus Custom

Call the API when the task is generic (OCR, sentiment, translation) and volume is moderate. Use BQML (Chapter 17) when the task is tabular and data already lives in BigQuery. Train custom (Chapter 18) when the domain is specific (medical imaging, proprietary taxonomies) or volume makes per-call pricing worse than owned inference. Write the per-1,000-units price of all three options on one slide before choosing; the answer is usually arithmetic.

20.5 Thursday Hands-On Project: Streaming Review Sentiment to BigQuery

This lab streams support-review text from Pub/Sub through the Natural Language API into BigQuery, using the batching DoFn from Wednesday.

20.5.1 Provision and Publish

```

1 $env:PROJECT_ID = "my-gcp-aiapi-lab"
2 gcloud config set project $env:PROJECT_ID
3 gcloud services enable language.googleapis.com dataflow.googleapis.com pubsub.
   googleapis.com
4
5 gcloud pubsub topics create support-reviews
6 gcloud pubsub subscriptions create support-reviews-sub --topic=support-reviews
7
8 bq mk --dataset analytics
9 bq query --nouse_legacy_sql '
10     "CREATE TABLE IF NOT EXISTS analytics.review_sentiment
11      (review_id STRING, text STRING, score FLOAT64, magnitude FLOAT64,
12       event_ts TIMESTAMP)"
13
14 gcloud pubsub topics publish support-reviews '

```

```

15  --message='{ "review_id": "r1", "text": "The agent resolved my issue in minutes,
    wonderful!", "event_ts": "2026-08-31T12:00:00Z"}'

```

Listing 20.2: Creating the reviews topic, BigQuery sink, and a test publish.

20.5.2 Run the Pipeline

Extend Wednesday’s DoFn with Pub/Sub read, JSON parse, and `WriteToBigQuery` to `analytics.review_sent` (Chapter 15’s pattern), then launch on the Dataflow runner. Publish twenty reviews—including a mixed-sentiment one and one in Spanish—and query the sink.

```

1  SELECT review_id, LEFT(text, 40) AS snippet, score, magnitude
2  FROM analytics.review_sentiment
3  ORDER BY event_ts DESC
4  LIMIT 20;
5
6  SELECT CASE WHEN score >= 0.25 THEN 'positive'
7             WHEN score <= -0.25 THEN 'negative'
8             ELSE 'mixed/neutral' END AS bucket,
9             COUNT(*) AS reviews, AVG(magnitude) AS avg_magnitude
10 FROM analytics.review_sentiment
11 GROUP BY bucket;

```

Listing 20.3: Inspecting scored reviews and sentiment distribution.

Deliverables

- The pipeline code with the batching/backoff DoFn and the dead-letter output path.
- Query results showing scored reviews across the sentiment buckets.
- A cost calculation: reviews per month times per-call price, versus the BQML-alternative estimate.

Acceptance Criteria

- API calls are batched and retried with backoff; poison messages reach a dead-letter output.
- Every review lands in BigQuery with score, magnitude, and original identifiers.
- The cost note compares per-call API pricing with at least one alternative.

Stretch Goals

- Add `analyzeEntities` and store top entities per review as a repeated field.
- Add Translation for non-English reviews before sentiment, and document the caveat.
- Add a throughput ceiling with a stateful token bucket and load-test at one thousand messages per second.

20.6 Friday Use Case and Architecture: Terabytes of User Images, Searchable

20.6.1 Scenario

A marketplace ingests terabytes of user-uploaded product images monthly. Requirements: extract embedded text (labels, serial numbers), tag objects and logos, flag policy-violating content, and make all of it searchable—sub-second—for trust-and-safety analysts and catalog search.

20.6.2 Architecture

Uploads land in a Cloud Storage raw bucket (Object Table indexes them for SQL enumeration, Chapter 8). A Dataflow batch job—triggered on schedule over new Object Table rows—calls Vision in batches for OCR, labels, logos, and safe-search scores, writing results to BigQuery tables keyed by object URI. A search index (Chapter 7) over the OCR text column makes extracted strings instantly searchable; entity rows join to catalog tables for “images whose OCR mentions brand X without a listing relationship.” Safe-search scores feed a moderation queue with human review for the borderline band. Cost is governed by feature selection, batch size, and a sampled preview pass on 1% of volume to validate configuration before full runs.

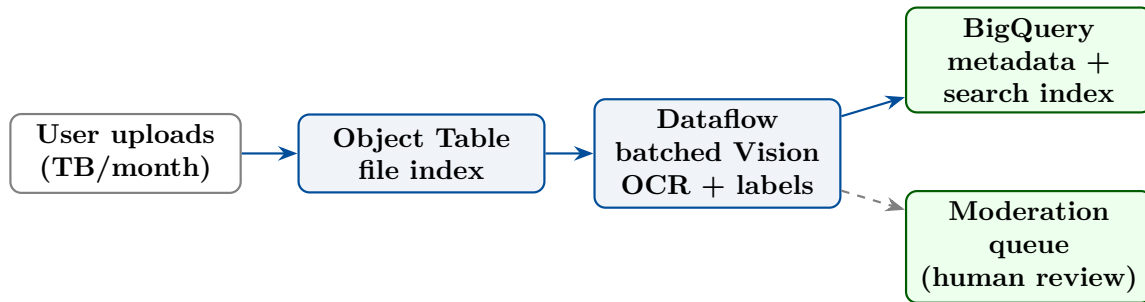


Figure 20.1: Image enrichment at scale: Object Tables enumerate, Dataflow batches API calls, BigQuery serves search.

Tip

Per-call APIs turn volume into a financial forecast: $\text{images/month} \times \text{features/image} \times \text{unit price}$, with a growth scenario. Present that number at design review—it routinely converts “call the API for everything” into “call it for what the queries need.”

Common Pitfall

OCR confidence is not uniform: stylized logos, low resolution, and handwritten text degrade silently. Sample-audit extraction quality weekly against human labels and track an accuracy metric; a search index over quietly-wrong text is worse than none because it looks authoritative.

20.7 Interview Q&A

20.7.1 Concepts and Trade-offs

1. **Q: When should a Dataflow pipeline call an AI API versus BQML?**
A: APIs for generic unstructured tasks (OCR, sentiment, translation) at moderate volume; BQML for tabular problems where the data already lives in BigQuery.
2. **Q: What rate-limit handling belongs around external API calls?**
A: Batching, bounded concurrency, exponential backoff with jitter, dead-lettering of permanent failures, and a throttle when a hard calls-per-second ceiling exists.
3. **Q: Which API extracts entities and sentiment from support text?**
A: The Natural Language API: `analyzeEntities`, `analyzeSentiment`, and `analyzeEntitySentiment` for per-entity sentiment.
4. **Q: How do you make OCR output searchable in BigQuery?**

A: Land extracted text in a structured column keyed by object URI and build a BigQuery search index over it for `SEARCH` queries.

5. **Q: What cost risk do per-call AI APIs add to streaming jobs?**

A: Cost scales linearly with throughput and feature count, invisible until the bill—plus retry multipliers; forecast and alert on both.

20.8 Further Reading

Read the Google Cloud AI API documentation for Vision, Video Intelligence, Natural Language, and Translation capabilities and quotas [34]. The Beam documentation covers bundle lifecycle methods (`setup`, `finish_bundle`) used by the batching DoFn [65]. Chapter 7’s search-index reference and Chapter 8’s Object Tables complete the architecture [9, 40]. Dataflow documentation covers the runner-side tuning [30].

Key Takeaways

- Pretrained APIs trade model control for immediate capability and per-call pricing.
- Select billed features deliberately; sample before scaling.
- Inside distributed pipelines: batch, bound concurrency, back off, dead-letter.
- OCR becomes searchable through structured landing tables plus a search index.
- API versus BQML versus custom is an arithmetic decision: price all three.
- Enrichment quality drifts silently—sample-audit extracted text against human labels.

20.9 Exercises

1. **(Conceptual)** Justify when Video Intelligence beats per-frame Vision calls, and vice versa.
2. **(Conceptual)** Explain score-versus-magnitude to a product manager who wants “just a thumbs up/down.”
3. **(Hands-on)** Reproduce the Thursday lab, then add a per-language breakdown using detected language from a Translation call.
4. **(Hands-on)** Load one hundred sample images, run OCR via a batch pipeline, and measure accuracy against hand-transcribed text for a ten-image audit sample.
5. **(Design)** Produce the three-option cost slide (Vision API, custom Vision model on endpoints, hybrid) for the Friday scenario at three volume tiers.
6. **(Architecture)** Add abuse protections to the image pipeline: size limits, content pre-filters, and a cost circuit breaker.

20.10 Milestone 5 Capstone: End-to-End MLOps Pipeline

Capstone Project

Mission: Orchestrate with Composer a pipeline that extracts features from BigQuery, syncs them to Vertex AI Feature Store, trains a custom model on Vertex Training, gates registration on an evaluation threshold, deploys to an endpoint, and enables Vertex Model Monitoring for skew and drift—closing the loop the MLOps chapters built piece by piece.

Estimated time: 10–12 hours.

What you'll practice:

- Feature extraction and Feature Store synchronization with freshness guarantees.
- Custom training with evaluation-metric artifacts and a registration gate.
- Endpoint deployment with traffic splitting and monitored skew/drift.
- Composer orchestration of the whole ML lifecycle (Chapter 12 applied to Part V).
- Cost governance across training, endpoints, and monitoring.

Scenario

The churn model from Chapters 17–18 must become a production system: features refreshed nightly, models retrained weekly and on drift, deployments gated and canaried, and every run leaving lineage evidence. The platform team owns it after you hand it over—the README is part of the deliverable.

Requirements

- A Composer DAG orchestrating: feature extraction (BigQuery), Feature Store sync, Vertex Training, evaluation gate, registration, deployment, monitoring configuration.
- The evaluation gate blocks registration below an AUC threshold and blocks deployment on a failed challenger comparison.
- Vertex Model Monitoring configured for training-serving skew and prediction drift with a notification channel.
- At least three automated tests: feature-quality checks, an evaluation-threshold gate test, a pipeline unit test.
- A monthly cost estimate with two optimization decisions (machine types, endpoint min/max nodes, batch versus online).
- Security review: least-privilege pipeline service account, no hardcoded secrets, training-data PII handling noted.

Reference Architecture

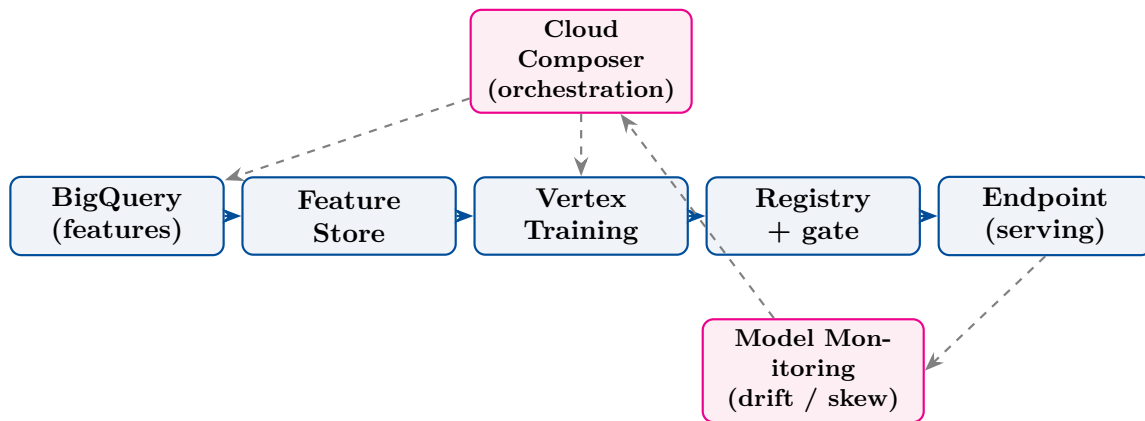


Figure 20.2: Milestone 5 reference architecture: the full MLOps lifecycle orchestrated by Composer.

Deliverables

- A repository that runs the full lifecycle from a clean environment, with README and architecture diagram.
- Evidence of the gate blocking a deliberately weakened candidate model.
- Monitoring configuration export (or screenshots) showing skew and drift alerts.
- The cost estimate, test suite with failing-case demonstration, and security review.

Acceptance Criteria

- One DAG run produces a registered, deployed, monitored model from raw features with no manual steps.
- The gate demonstrably blocks a sub-threshold candidate; the incumbent keeps serving.
- Drift injected into a shadow traffic sample triggers the monitoring alert and the documented runbook path.

Stretch Goals

- Add a challenger-champion traffic split with automatic rollback on live-metric regression.
- Trigger retraining from the monitoring alert via Pub/Sub and a preconditioned Cloud Function (Chapter 19's Friday design).
- Publish model and feature freshness to Dataplex as data-product SLOs (previewing Chapter 22).

Part VI

Security, Governance, and Exam Prep

Security, IAM, and Networking for Data Platforms

Learning Objectives

- Configure VPC Service Controls perimeters and access levels that block exfiltration even with stolen credentials.
- Apply least-privilege custom roles, service accounts per workload, and Workload Identity over service-account keys.
- Choose between Google-managed encryption, CMEK, and CSEK for BigQuery and Cloud Storage.
- Validate a perimeter with a negative test and wire audit-log alerting on violations.
- Connect privately to Google APIs with Private Google Access and Private Service Connect.
- Design a healthcare data perimeter that contains compromise blast radius.

Cross-Cloud Equivalence

Perimeter-based exfiltration defense and key management have direct counterparts on every cloud; the GCP names are the ones to memorize for this book’s exam.

Capability	GCP	AWS	Azure
Service perimeter	VPC Service Controls	SCPs + endpoint policies	(Private Link + policy; Purview for data)
Private API access	Private Google Access / PSC	VPC endpoints (PrivateLink)	Private Link / private endpoints
Key management	Cloud KMS (CMEK/CSEK)	KMS (CMK/BYOK)	Key Vault (CMK/BYOK)
Identity federation	Workload Identity Federation	IAM Roles Anywhere / IRSA	Workload identity federation
Audit trail	Cloud Audit Logs	CloudTrail	Azure Monitor / Activity Log

Snowflake implements perimeter control with network policies and Tri-Secret Secure (BYOK); **MongoDB Atlas** with private endpoints, customer key management, and field-level encryption. The shared mental model: IAM answers *who may*, perimeters answer *from where and to where*, and keys answer *who can read it at all*.

Key Terms

VPC Service Controls (VPC SC): organization-level perimeters around GCP services that restrict API access by network origin and identity, independent of IAM grants. **Access level:** a named set of request attributes (IP range, device, identity) permitted to cross a perimeter. **Workload Identity Federation:** keyless authentication letting external or CI workloads impersonate service accounts via short-lived tokens. **CMEK:** customer-managed encryption keys in Cloud KMS protecting service data, with rotation and revocation under your control. **CSEK:** customer-supplied encryption keys, where key material never persists in Google systems. **Private Service Connect:** private, IP-based connectivity to Google APIs or producer services from a VPC.

21.1 Week 21 Roadmap

Week 21 opens Part VI by hardening everything the book has built. Monday covers VPC Service Controls and private connectivity. Tuesday is the IAM deep dive: custom roles, service accounts, Workload Identity. Wednesday covers Cloud KMS and the CMEK/CSEK decision. Thursday builds and negative-tests a BigQuery perimeter. Friday designs the healthcare anti-exfiltration architecture.

Definition

VPC Service Controls create a service perimeter: API calls to protected services (BigQuery, Cloud Storage, and dozens more) succeed only from authorized networks and identities, regardless of how permissive IAM is [60]. IAM controls *who*; the perimeter controls *where from*, *where to*—and it is the control that survives credential theft.

21.2 Monday: VPC Service Controls and Private Connectivity

21.2.1 Perimeter Semantics

A perimeter lists projects and protected services. Inside-to-inside calls flow freely; crossing the boundary—reading a BigQuery dataset from outside, copying a bucket’s objects to an outside project—is denied unless an **access level** (IP, device, identity conditions) or an explicit **ingress/egress rule** permits it. The security property that matters: an attacker holding a perfectly valid service-account key, operating from an unauthorized network, gets `PERMISSION_DENIED` from the perimeter. IAM alone cannot say that.

21.2.2 Perimeters and Data Engineering

Data pipelines cross boundaries constantly: Composer calls BigQuery; Dataflow writes Cloud Storage; Cloud Build deploys Dataform. Each crossing needs to be inside the perimeter or explicitly ruled in—the classic production incident is deploying a perimeter and breaking every CI/CD job that runs outside it. Design in layers: one perimeter for the data platform’s projects, ingress rules for CI identities and specific methods, and dry-run mode (log-only) before enforcement.

21.2.3 Private Connectivity

Private Google Access lets VPC resources reach Google APIs over internal IPs without public addresses; **Private Service Connect** publishes Google APIs (or producer services)

as private endpoints with DNS control. Both remove the public-internet path for data-plane traffic—complementary to VPC SC, which governs the API plane regardless of network path.

Common Pitfall

Dry-run first, always. An enforced perimeter with a forgotten service—Composer’s GKE autoscaling APIs, a Cloud Build trigger, a Dataproc job’s staging bucket—takes down production data movement while looking, from IAM’s perspective, perfectly configured. Run dry-run for two weeks and triage every violation log before enforcing.

21.3 Tuesday: IAM Deep Dive—Custom Roles, Service Accounts, Workload Identity

21.3.1 Roles and the Least-Privilege Discipline

GCP IAM binds principals to roles on resources. Predefined roles (e.g., `roles/bigquery.dataEditor`) are the everyday tool; **custom roles** package exactly the permissions a workload needs when predefined roles are too broad—a Dataflow runner that must write one dataset and read one topic, nothing else. Basic roles (Owner, Editor, Viewer) grant sweeping access across services and have no place on production resources; Chapter 1’s storage lab and every chapter since have assumed this discipline.

21.3.2 Service Accounts as Workload Identity

Every pipeline identity is a service account: one per workload (not per project), named for its function, granted at the narrowest resource scope. The cardinal sin is the **downloaded service-account key**: a bearer credential that never expires, exfiltrates with a single copy, and audits poorly. **Workload Identity** replaces keys with short-lived tokens—GKE workloads via Workload Identity for GKE, external CI (GitHub Actions, on-premises runners) via Workload Identity Federation—so there is no key file to steal, rotate, or accidentally commit.

Pattern	Risk	Preferred alternative
Downloaded SA key in CI secret store	Never expires; exfiltrates silently	Workload Identity Federation
One shared SA for all jobs	Blast radius of every workload	One SA per workload
Basic roles on data resources	Cross-service sweep	Predefined/custom roles at resource scope
Keys committed to git	Permanent, public compromise	Secret Manager + keyless auth; rotate and investigate if it happens

Table 21.1: Identity anti-patterns and their least-privilege alternatives [38].

Tip

Use IAM Policy Analyzer and the `lastUsed` insights to hunt over-granted permissions quarterly: a role whose unused permissions exceed its used ones is a custom-role candidate. Least privilege is a maintenance activity, not a launch-day configuration.

21.4 Wednesday: Cloud KMS—CMEK versus CSEK in BigQuery and Cloud Storage

21.4.1 The Encryption Ladder

All GCP data is encrypted at rest by default with Google-managed keys. **CMEK** upgrades control: keys live in your Cloud KMS key rings, services access them through a per-service-agent grant, rotation and revocation are yours, and every use is audit-logged. **CSEK** (customer-supplied keys) goes further: you present key material per request and Google never persists it—maximal control, maximal operational burden (lose the key, lose the data, permanently). Regulatory drivers usually land on CMEK; CSEK serves narrow, high-sensitivity enclaves.

21.4.2 CMEK in Practice

Location binds: a CMEK key's location must be compatible with the protected resource (a US multi-region bucket wants a `us` key ring). Key **rotation** is non-disruptive (new versions encrypt new writes; old versions decrypt old data). Key **disablement** is the kill switch: disable the key and protected data becomes unreadable within the propagation window—powerful for crypto-shredding (Chapter 23) and terrifying as an accident. Guard keys with separation of duties: the team administering data should not administer the keys protecting it.

Common Pitfall

Disabling a CMEK key is a data outage, not a permission change: BigQuery tables, buckets, and disks protected by it fail reads broadly. Test the disable/restore procedure in a sandbox before you ever need it under incident pressure, and restrict key-admin roles to a small, monitored group.

Note

Cloud KMS destruction has a mandatory 24-hour scheduled-destruction delay—a deliberate safety net against accidental or coerced deletion. Chapter 23 uses this property as the foundation of GDPR crypto-shredding.

21.5 Thursday Hands-On Project: Perimeter Around a BigQuery Dataset

This lab creates a VPC Service Controls perimeter protecting BigQuery in a lab project, proves the block with a negative test from an unauthorized context, then adds an access level permitting a specific IP.

21.5.1 Create the Perimeter

```

1 $env:ORG_ID = "123456789012"
2 $env:PROJECT_ID = "my-gcp-vpcsc-lab"
3 $env:PROJECT_NUMBER = "999999999999"
4
5 gcloud access-context-manager policies create '
6   --organization=$env:ORG_ID --title="data-platform-policy"
7
8 gcloud access-context-manager levels create lab_analyst_ip '
9   --policy=<POLICY_ID> --title="Lab analyst IP" '
10  --ip-based-access-levels=<YOUR_IP>/32
11
12 gcloud access-context-manager perimeters create data_perimeter '

```

```

13 --policy=<POLICY_ID> --title="Data platform perimeter" '
14 --resources=projects/$env:PROJECT_NUMBER '
15 --restricted-services=bigquery.googleapis.com,storage.googleapis.com '
16 --access-levels=lab_analyst_ip '
17 --enable-dry-run

```

Listing 21.1: Creating an access policy, access level, and dry-run perimeter.

21.5.2 Negative Test and Enforcement

1. Query a BigQuery dataset in the project from an IP *outside* the access level (a Cloud Shell session or a different network). In dry-run, the violation appears in audit logs with `dryRun: true`.
2. Query again from the permitted IP: success. The contrast is the evidence.
3. Review dry-run violation logs for a period, triage each (CI identities? Composer? scheduled queries?), add justified ingress rules.
4. Remove `-enable-dry-run` (update the perimeter to enforced), re-run the negative test, and capture the `PERMISSION_DENIED` response.

Deliverables

- The perimeter and access-level configuration commands or Terraform.
- Negative-test evidence from both dry-run (logged violation) and enforced (denied query) phases.
- A triage log of dry-run violations with the ingress rules they justified.

Acceptance Criteria

- The perimeter restricts BigQuery and Cloud Storage for the lab project.
- An external query is verifiably blocked while the access-level IP succeeds.
- No legitimate workload broke during enforcement—the triage log explains why.

Stretch Goals

- Add an ingress rule scoped to a CI service account and specific BigQuery methods only.
- Create a Cloud Monitoring alert on perimeter-violation audit logs.
- Extend the perimeter to protect a Vertex AI training path (Chapter 18’s Friday design).

21.6 Friday Use Case and Architecture: The Healthcare Anti-Exfiltration Perimeter

21.6.1 Scenario

A healthcare analytics platform holds PHI in BigQuery and Cloud Storage. The threat model includes a stolen service-account key and a compromised analyst credential. Requirement: even with valid credentials, no data may leave the governed environment.

21.6.2 Layered Design

Perimeter: one VPC SC perimeter encloses the data projects and the CI/orchestration project; restricted services include BigQuery, Storage, Pub/Sub, Dataflow, and Vertex AI. **Access levels:** analyst access only from corporate device IP ranges with verified device context; service accounts only from inside the perimeter. **Identity:** Workload Identity everywhere—no keys exist to steal; analyst access uses short-lived federated sessions. **Encryption:** CMEK on all stores with key administration separated from data administration. **Observation:** audit logs stream to a locked logging project (its own perimeter), with alerts on perimeter violations, mass exports, and policy changes. **Negative testing:** a monthly game day runs the stolen-credential scenario from an external network and files the denial evidence.

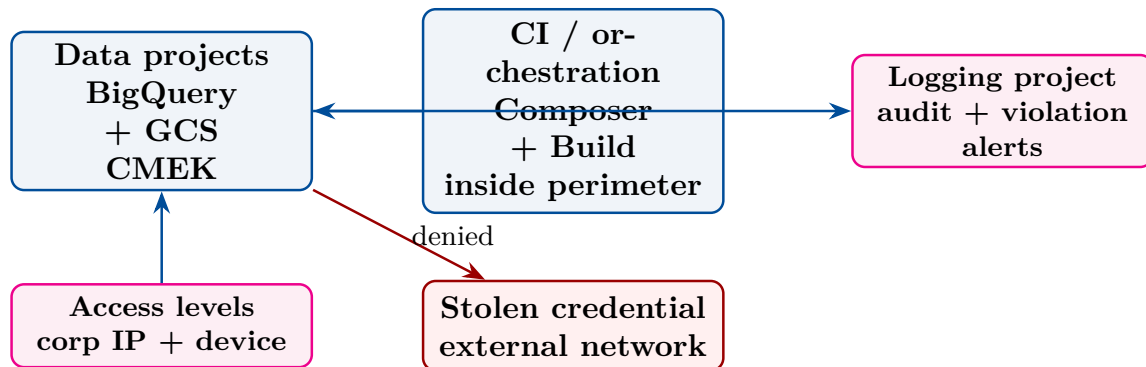


Figure 21.1: Healthcare perimeter: valid credentials from an unauthorized network are denied by VPC SC.

Tip

The design review question that matters: “enumerate every path bytes can leave the perimeter”—exports, public shares, signed URLs, CI artifacts, support bundles. Each path needs a rule or a documented acceptance. Exfiltration defense fails through the path nobody enumerated.

21.7 Interview Q&A

21.7.1 Concepts and Trade-offs

1. **Q: What does a VPC SC perimeter block that IAM cannot?**
A: Requests from unauthorized networks or to unauthorized destinations—even with perfectly valid, fully-authorized IAM credentials. IAM authenticates; the perimeter constrains context.
2. **Q: Why prefer Workload Identity over service-account keys?**
A: Keys are long-lived bearer files that exfiltrate and audit poorly; Workload Identity issues short-lived tokens with no key material to steal, rotate, or commit.
3. **Q: CMEK versus CSEK: who holds the key material in each?**
A: CMEK keys live in your Cloud KMS, used by services under grant and audit; CSEK material is supplied per request and never persisted by Google—with total-loss risk if you lose it.
4. **Q: How do access levels modify perimeter behavior?**
A: They carve conditional entry (IP ranges, device state, identity) into the perimeter

boundary for requests that would otherwise be denied.

5. **Q: Why test a perimeter with a negative (denied) query?**

A: Security controls fail silently toward openness; only a denied-request test proves the control exists and works, and only repeated tests prove it keeps working.

21.8 Further Reading

Read the VPC Service Controls documentation for perimeter design, dry-run, and ingress/egress rules [60], the IAM documentation for roles, service accounts, and federation [38], and Cloud KMS for key lifecycle and CMEK integration [20]. The Architecture Framework’s security pillar ties these controls into reviewable posture [35], and Chapter 23 builds the DLP and erasure layer on top [54].

Key Takeaways

- IAM controls who; VPC SC controls where-from and where-to; KMS controls readability. Defense requires all three.
- Perimeters are deployed dry-run-first, with every violation triaged before enforcement.
- No downloaded keys, ever: Workload Identity Federation covers CI and external workloads.
- Custom roles and per-workload service accounts keep blast radius proportional to function.
- CMEK key disablement is a kill switch and an outage mechanism—test it deliberately, guard it with separation of duties.
- Security controls are proven by negative tests and audit evidence, not by configuration screenshots.

21.9 Exercises

1. **(Conceptual)** Explain to a skeptical engineer why “we already have least-privilege IAM” does not cover the stolen-key scenario.
2. **(Conceptual)** Compare CMEK rotation, disablement, and destruction: effect, reversibility, and appropriate use of each.
3. **(Hands-on)** Reproduce the Thursday lab, then add an ingress rule for a specific service account and one BigQuery method; verify other methods still deny.
4. **(Hands-on)** Create a custom role for the Chapter 14 Dataflow runner (read one subscription, write one dataset) and diff it against the predefined roles it replaces.
5. **(Design)** Enumerate the exfiltration paths for the healthcare platform and assign each a control or documented acceptance.
6. **(Architecture)** Design the monthly stolen-credential game day end to end: scenario, execution, evidence collection, and review ritual.

Data Governance and Data Mesh with Dataplex

Learning Objectives

- Organize Dataplex lakes, zones (raw and curated), and assets over Cloud Storage and BigQuery.
- Apply tag templates and policy tags for classification and column-level enforcement.
- Run automatic data-quality (AutoDQ) scans and data profiling against governed assets.
- Trace lineage from a broken dashboard back to the responsible source system.
- Build a governed lake with assets, a tag template, and a protected Salary column.
- Architect a five-domain data mesh with centralized governance and decentralized ownership.

Cross-Cloud Equivalence

Governance fabrics attach metadata, quality, and policy to data wherever it lives; each platform’s answer differs in scope and tightness of integration.

Capability	GCP Dataplex	AWS	Azure / Fabric
Governance fabric	Dataplex (lakes/zones)	Lake Formation	Microsoft Purview
Catalog & search	Data Catalog (in Dataplex)	Glue Data Catalog	Purview / Fabric OneLake catalog
Lineage	Dataplex lineage	(partner/Glue)	Purview lineage
Auto data quality	AutoDQ scans	Glue Data Quality	Purview / Fabric quality rules
Data mesh units	Domains as lakes per team	Accounts per domain	Domains in Fabric/Purview

Snowflake governs through Horizon (tags, masking policies, lineage) inside its boundary; **MongoDB** relies on schema validation and Atlas governance. The mesh principle is platform-agnostic: domains own data products; the platform provides federated guardrails—discovery, quality, and policy enforcement—without centralizing every decision.

Key Terms

Lake: a Dataplex top-level governance container for a domain or environment. **Zone:** a logical grouping within a lake (raw, curated) carrying its own policy posture. **Asset:** a resource mapped into a zone—a bucket or BigQuery dataset—whose metadata Dataplex discovers and governs. **Tag template:** a schema of business metadata fields (owner, classification, retention) attachable to catalog entries. **AutoDQ:** Dataplex’s rule-based data-quality scanning with suggested and custom rules and score publication. **Lineage:** the recorded graph of how data moves and transforms across systems, queryable for impact analysis.

22.1 Week 22 Roadmap

Week 22 turns the platform into a governed system of record. Monday structures lakes, zones, and assets. Tuesday covers the catalog: tag templates, search, and lineage. Wednesday automates quality with AutoDQ and profiling. Thursday builds the governed lake with a protected Salary column. Friday architects the five-domain data mesh.

Definition

Dataplex is Google Cloud’s governance fabric: it organizes storage and warehouse resources into lakes and zones, discovers their metadata into a searchable catalog, enforces policy through tags and security bindings, profiles and quality-scans data automatically, and records lineage across pipelines [32].

22.2 Monday: Lakes, Zones, and Assets

22.2.1 The Organizational Vocabulary

A **lake** scopes governance to a domain or environment (“sales,” “finance,” “nonprod”). **Zones** subdivide it by processing stage and trust: a *raw* zone holding source-fidelity data with strict write access, a *curated* zone holding modeled products with broader read access. **Assets** map physical resources—buckets, datasets—into zones; once mapped, Dataplex discovers tables and filesets, and zone-level IAM bindings govern access to everything the zone contains. The structure mirrors the zone discipline from Chapter 1, now with a governance control plane over it.

22.2.2 Design Consequences

Zone placement is a policy decision: attaching a bucket to a curated zone grants the zone’s readers access to everything in the bucket—prefix-level exceptions belong in bucket design, not in workarounds. Lakes are the federation unit for the mesh: one lake per domain keeps ownership boundaries clean, while Dataplex’s cross-lake catalog keeps discovery global.

Common Pitfall

Mapping one giant bucket into a curated zone because “it is mostly curated” leaks the raw prefixes it also contains to every curated-zone reader. Zone boundaries follow trust boundaries: split buckets first, map zones second.

Tip

Standardize zone names and their policy meaning across domains—**raw**, **staging**, **curated**—so a data engineer moving between teams reads any lake’s structure instantly. Conventions are governance that costs nothing to enforce.

22.3 Tuesday: Data Catalog, Tag Templates, and Lineage

22.3.1 Searchable, Tagged Metadata

Every discovered asset becomes a catalog entry; **tag templates** attach business metadata—**owner**, **data_classification**, **retention_class**, **cost_center**—as structured, searchable fields. The discipline that separates a useful catalog from a metadata swamp: a small, governed set of templates with controlled vocabularies, populated by pipeline automation (a Dataform or Composer step tags what it publishes), not by analyst goodwill.

22.3.2 Policy Tags versus Descriptive Tags

Chapter 8 used **policy tags** for enforcement: taxonomy-based labels on BigQuery columns that gate access at query time. Tag templates are descriptive metadata for discovery and stewardship. The two compose: a descriptive tag says **classification=PII-direct** for discovery and review; the policy tag on the same column enforces access. Healthy platforms generate one from the other in automation so discovery and enforcement never disagree.

22.3.3 Lineage

Dataplex records lineage from BigQuery jobs, Dataflow pipelines, and Composer runs: which tables a job read, which it wrote. The operational payoff is impact analysis—a broken executive dashboard traces upstream through views and models to the staging table whose CDC stream stalled (Chapter 11’s freshness alert), turning a “the number is wrong” escalation into a named owner and a root cause in minutes.

Note

Lineage is only as complete as the paths that report it. Custom Python reading and writing tables outside known systems leaves gaps; document manual lineage for known gaps rather than pretending the graph is complete.

22.4 Wednesday: Auto Data Quality and Profiling

22.4.1 Profiling

Data profiling scans tables for statistical fingerprints: null rates, distinct-value counts, min/max, string-length distributions, top values. Profiling is the cheapest data-quality investment: it finds the column that suddenly went 40% null, the ID that stopped being unique, the country field that grew a new unexpected value—before the consumers do.

22.4.2 AutoDQ

AutoDQ evaluates declarative rules—completeness (**email IS NOT NULL**), validity (regex or range), uniqueness, freshness, consistency across tables—on a schedule, publishing a per-table quality score to the catalog and alerting on rule failures. Rules can be suggested from profiling output or authored explicitly. The platform pattern: every curated data product ships with its

AutoDQ scan spec in version control, reviewed with the model code, with scores surfaced next to the product in the catalog [32].

Rule type	Example	Typical failure meaning
Completeness	Null rate < 0.1% on <code>order_id</code>	Upstream extraction dropped a field
Uniqueness	<code>customer_key</code> distinct in dim	Duplicate-join regression in the model
Validity	<code>country</code> in ISO list	New unmapped source value
Freshness	Max <code>event_ts</code> < 2h old	CDC or stream stall
Consistency	Order totals match across fact tables	Partial load or currency bug

Table 22.1: AutoDQ rule types and what their failures usually mean.

Common Pitfall

Quality rules without an incident path decay into ignored red tiles. Every rule needs a severity, an owner, and a response expectation; a page-worthy rule pages, a warning rule feeds a weekly review, and anything nobody would act on gets deleted.

22.5 Thursday Hands-On Project: A Governed Lake with a Protected Salary Column

This lab creates a Dataplex lake with raw and curated zones, maps a bucket and a BigQuery dataset as assets, defines a tag template, and protects an HR dataset’s `salary` column with a policy tag verified by a negative test.

22.5.1 Create Lake, Zones, and Assets

```

1 $env:PROJECT_ID = "my-gcp-dataplex-lab"
2 $env:REGION = "us-central1"
3 gcloud config set project $env:PROJECT_ID
4 gcloud services enable dataplex.googleapis.com
5
6 gcloud dataplex lakes create hr-lake --location=$env:REGION '
7   --display-name="HR Lake"
8
9 gcloud dataplex zones create hr-raw --location=$env:REGION '
10  --lake=hr-lake --type=RAW --resource-location-type=SINGLE_REGION '
11  --discovery-enabled
12
13 gcloud dataplex zones create hr-curated --location=$env:REGION '
14  --lake=hr-lake --type=CURATED --resource-location-type=SINGLE_REGION '
15  --discovery-enabled
16
17 gcloud dataplex assets create hr-dataset --location=$env:REGION '
18  --lake=hr-lake --zone=hr-curated '
19  --resource-type=BIGQUERY_DATASET '
20  --resource-name=projects/$env:PROJECT_ID/datasets/hr_data '
21  --discovery-enabled

```

Listing 22.1: Creating the lake, zones, and attaching assets.

22.5.2 Tag Template and Policy Tag

Create a tag template `data_product_meta` with fields `owner` (string), `classification` (enum: public/internal/confidential), and `review_date` (date); attach it to the discovered `employees` table entry. In Data Catalog, create a taxonomy HR PII with a policy tag `compensation`; apply it to `employees.salary`; grant Fine-Grained Reader on the tag only to the HR-analyst group. Run the negative test: a non-member queries `SELECT salary FROM hr_data.employees` and is denied, while `SELECT name, department` succeeds.

Deliverables

- The lake/zone/asset configuration with discovery enabled and findings visible in the catalog.
- The tag template definition and the tagged table entry.
- Negative- and positive-test query outputs proving the policy tag enforcement.

Acceptance Criteria

- Raw and curated zones carry distinct access bindings, verified by inspection.
- The `salary` column is inaccessible without the Fine-Grained Reader grant on the policy tag.
- The catalog entry is discoverable by searching the tag's classification value.

Stretch Goals

- Add an AutoDQ scan over `hr_data.employees` with completeness and uniqueness rules and a failing-case demonstration.
- Trace lineage from a Looker dashboard field back through the curated table to the raw zone.
- Automate tag application from the pipeline that publishes the curated table.

22.6 Friday Use Case and Architecture: A Five-Domain Data Mesh

22.6.1 Scenario

A global conglomerate—retail, logistics, finance, manufacturing, and digital domains—has outgrown its central data team: every dataset change queues behind a team that owns neither the context nor the consequences. The target operating model: domains own and serve data products; a platform team provides the governed substrate; central standards keep products interoperable.

22.6.2 Mesh on GCP

Each domain runs its own project(s) and its own Dataplex lake with the standard raw/curated zones. Domains publish data products—BigQuery datasets or BigLake tables—with mandatory tag-template metadata (owner, classification, SLA), AutoDQ scores, and policy tags from the shared taxonomy. Discovery is global via the catalog; access is granted by domain owners through zone bindings and Analytics Hub listings (Chapter 8) for cross-domain consumption. The platform team owns the taxonomy, the tag templates, the AutoDQ rule minimums, and

the audit dashboards—federated governance—while domains own schemas, quality, and documentation: decentralized accountability.

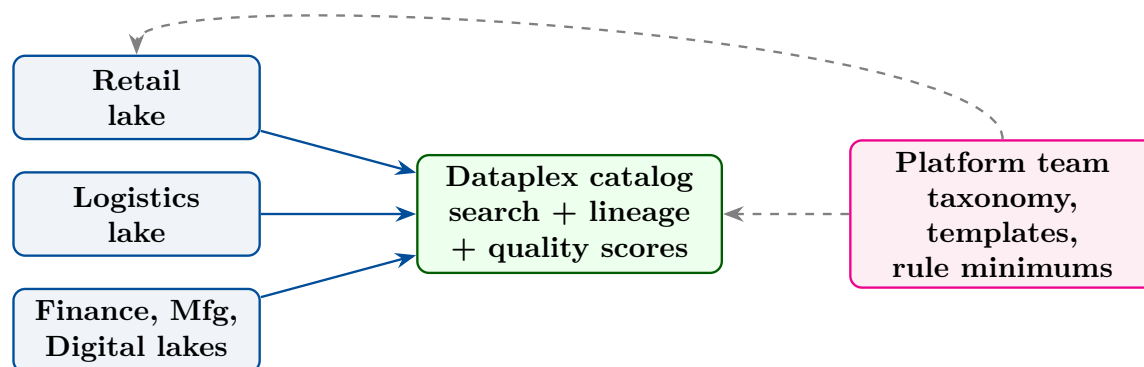


Figure 22.1: Data mesh on Dataplex: domain lakes publish governed products into one catalog under shared standards.

Tip

Fund the platform team as a product team with adoption metrics (products published, catalog search usage, mean time to discover), not as a ticket queue. Meshes fail when the platform’s incentives reward gatekeeping instead of self-service.

Common Pitfall

Decentralized ownership without enforced minimum standards produces five incompatible definitions of “customer” wearing a mesh costume. The non-negotiables—tag template, taxonomy, quality floor, lineage reporting—must be platform-enforced, or the mesh decentralizes into silos with better branding.

22.7 Interview Q&A

22.7.1 Concepts and Trade-offs

1. **Q: What distinguishes a raw zone from a curated zone in Dataplex?**
A: Trust and processing stage: raw holds source-fidelity data with narrow write access; curated holds modeled products with broader read access—each carrying its own policy posture over attached assets.
2. **Q: What does a tag template attach to catalog entries?**
A: Structured business metadata—owner, classification, retention—as searchable, governed fields; distinct from policy tags, which enforce access.
3. **Q: What does AutoDQ check without you writing rules?**
A: Profiling surfaces statistics automatically; AutoDQ can suggest rules from them—but declared rules (completeness, uniqueness, freshness) are yours to author and own.
4. **Q: In a data mesh, who owns the data product—central team or domain?**
A: The domain: schema, quality, documentation, and SLAs. The central platform owns the federated guardrails—taxonomy, templates, minimum quality rules.
5. **Q: How does lineage help when a downstream dashboard breaks?**
A: Impact analysis runs upstream from the broken field through transformations to the

responsible source and its owner, collapsing triage from archaeology to a lookup.

22.8 Further Reading

Read the Dataplex documentation for lakes, zones, assets, AutoDQ, profiling, and lineage [32]. The Analytics Hub reference covers cross-domain sharing that the mesh relies on [39], and Chapter 8’s policy-tag discussion covers the enforcement half of tagging [9]. DataHub is worth reading as the open-source catalog comparison point [35].

Key Takeaways

- Lakes and zones make trust boundaries explicit and enforceable; map assets along those boundaries only.
- Tag templates carry stewardship metadata; policy tags carry enforcement; automation keeps them in agreement.
- Profiling is the cheapest anomaly detector you own; AutoDQ turns rules into published scores.
- Lineage converts “the number is wrong” into an owner and a root cause.
- A mesh is domain ownership plus platform-enforced standards—neither half works alone.
- Catalogs succeed through automation and adoption metrics, not through documentation mandates.

22.9 Exercises

1. **(Conceptual)** A domain asks to be exempt from the shared taxonomy “because our data is special.” Write the negotiation position.
2. **(Conceptual)** Explain the difference between a tag template and a policy tag to a new analyst, with one failure story of confusing them.
3. **(Hands-on)** Reproduce the Thursday lab, then add a second zone with stricter bindings and verify the boundary with two test identities.
4. **(Hands-on)** Author an AutoDQ scan with one suggested rule and one custom rule; capture the published quality score.
5. **(Design)** Define the minimum publishable standard for a data product in the five-domain mesh: required tags, quality floor, SLA fields, documentation.
6. **(Architecture)** Design the cross-domain consumption path (Analytics Hub listing versus direct grant) with the access-review cadence for each.

Compliance and Data Loss Prevention

Learning Objectives

- Detect PII with InfoTypes, inspection jobs, and scheduled discovery scans over BigQuery and Cloud Storage.
- Apply masking, tokenization, and crypto-based transformations with KMS-wrapped keys.
- Propagate GDPR erasure across lakes, warehouses, streams, and backups via crypto-shredding.
- Reconcile retention policies with legal holds and erasure obligations.
- Build a DLP inspection and de-identification pipeline over sensitive files.
- Architect third-party data ingestion that tokenizes PII with an end-to-end erasure map.

Cross-Cloud Equivalence

Sensitive-data discovery and de-identification exist on every platform, though GCP’s DLP (Sensitive Data Protection) is unusually deep as a standalone service.

Capability	GCP	AWS	Azure / Fabric
PII discovery	Sensitive Data Protection (DLP)	Macie	Purview Information Protection
De-identification	DLP transforms (mask, tokenize, FPE)	(partner/Lambda transforms)	Purview / ADF masking
Warehouse masking	BigQuery policy tags + routines	(via Lake Formation + UDFs)	Dynamic data masking
Format-preserving	DLP FFX/FPE	(partner)	(partner)
Risk analysis	DLP k-anonymity / l-diversity	(partner)	(partner)

Snowflake provides masking policies and tag-based protection; **MongoDB** provides field-level encryption. The portable compliance lesson: deletion is not erasure—time travel, backups, and replicas retain copies, and only cryptographic erasure or exhaustive replay satisfies a strict right-to-erasure obligation.

Key Terms

InfoType: a category of sensitive data (email, credit card, national ID) that DLP detectors recognize—built-in, custom, or dictionary-based. **De-identification:** transforming data to remove or obscure identifying content: masking, redaction, tokenization, generalization, bucketing. **Format-preserving encryption (FPE):** deterministic encryption producing ciphertext in the input’s format (a token still looks like an email), enabling joins on protected values. **Crypto-shredding:** rendering data unrecoverable by destroying the key that encrypts it—erasing all copies, including backups, at once. **k-anonymity:** a risk metric—each combination of quasi-identifiers appears at least k times, limiting re-identification. **Legal hold:** a suspension of deletion/erasure obligations for data under litigation or investigation.

23.1 Week 23 Roadmap

Week 23 carries the platform through the compliance layer. Monday covers DLP concepts: InfoTypes, inspection, de-identification. Tuesday covers the transformation menu: masking, tokenization, cryptographic hashing. Wednesday covers automated scanning and the GDPR erasure problem done properly. Thursday builds the inspection-and-de-identify pipeline. Friday architects third-party PII ingestion with a complete erasure-propagation map.

Definition

Sensitive Data Protection (Cloud DLP) is Google Cloud’s discovery and de-identification service: it inspects storage and text for InfoTypes, profiles tables and buckets for sensitive content, transforms data with masking and tokenization primitives, and quantifies re-identification risk [54].

23.2 Monday: DLP Concepts—InfoTypes, Inspection, De-identification

23.2.1 Detection

An InfoType names a sensitive category: EMAIL_ADDRESS, CREDIT_CARD_NUMBER, US_SOCIAL_SECURITY_NUMBER plus hundreds more, extensible with custom regex detectors, dictionaries, and hotword rules. Inspection scans return *findings*—InfoType, location, likelihood, quote—with likelihood tuned by context (a number near the word “card” scores higher). The tuning knob that matters in pipelines: minimum likelihood and detector selection, because scanning a data lake with every detector at LOW drowns you in false positives.

23.2.2 Discovery and Inspection Modes

DLP operates in three modes with different economics: **content inspection** (API calls over text/images you send—used inside pipelines), **storage inspection jobs** (scheduled scans of BigQuery tables, buckets, and databases—used for estate-wide discovery), and **data profiles** (continuous per-table sensitivity scoring feeding the catalog). A governed platform runs discovery continuously and content inspection at ingestion boundaries.

Common Pitfall

Inspection results are themselves sensitive: a finding’s quote field contains the PII it found. Restrict who can read findings, prefer quote-free configurations where possible, and route findings into a monitored, access-controlled project—not a shared bucket.

23.3 Tuesday: Masking, Tokenization, and Crypto-Based Hashing

23.3.1 The Transformation Menu

DLP's de-identification primitives serve different consumers. **Redaction** removes content outright. **Masking** replaces characters (jo***@example.com)—readable to humans, useless for joins. **Crypto-based tokenization** (deterministic AES or FFX format-preserving encryption keyed by a KMS-wrapped key) produces stable tokens: the same input always yields the same token, so analytics can still join on `user_token` without ever seeing the identifier. **Generalization and bucketing** (age to ranges, timestamps to dates) reduce precision for quasi-identifiers.

23.3.2 Who Can Reverse It

Masking is irreversible by anyone; tokenization is reversible by whoever holds KMS decrypt permission—which is precisely the control: detokenization becomes an auditable, least-privilege API call rather than a property of the data. Deterministic tokenization's weakness is dictionary attackability on low-entropy values (emails follow patterns); where the threat model includes offline guessing, use keyed hashing with salt management or non-deterministic surrogate replacement plus a protected mapping table.

Definition

Deterministic tokenization maps each plaintext to one token under a key: join-compatible and reversible only with the key. **Non-deterministic replacement** assigns random surrogates with an explicit mapping table: stronger against guessing, but the mapping table becomes a crown-jewel asset requiring its own perimeter.

Tip

Wrap the tokenization key with Cloud KMS (`kms_wrapped` crypto keys) so the DLP call presents an encrypted key that KMS unwraps under IAM and audit logging. Keys in configuration files—even in a secret manager without an audit trail per use—undermine the entire reversibility control.

23.4 Wednesday: Automated Scanning and GDPR Erasure Propagation

23.4.1 Scheduled Discovery

Storage inspection jobs scan BigQuery datasets and bucket prefixes on a schedule with configured InfoTypes and sampling, writing findings to BigQuery for dashboarding: sensitive data discovered in unexpected tables, new columns introducing PII into previously clean datasets, scan coverage versus the asset inventory. Sampling controls cost—a 10% random sample finds systemic PII presence at a fraction of full-scan price, while regulated tables get 100% coverage.

23.4.2 Why DELETE Is Not Erasure

A naive `DELETE FROM customers WHERE id = :subject` leaves the subject recoverable almost everywhere: BigQuery time travel serves the row via `FOR SYSTEM_TIME AS OF` during its window; fail-safe retains it seven more days; backups persist it until they expire; Pub/Sub snapshots

replay it; Cloud Storage versioning retains pre-delete generations; derived tables copied it onward. Each layer’s retention was designed for durability—and durability is exactly what an erasure request fights.

23.4.3 Crypto-Shredding as the Architecture

Encrypt each subject’s data under a per-subject (or per-subject-group) key in Cloud KMS at ingestion. Erasure then schedules destruction of that key: after the mandatory 24-hour destruction delay, every copy—raw lake objects, warehouse rows, backups, snapshots—becomes permanently unrecoverable ciphertext, in one auditable KMS operation [20]. The alternative—row-level deletion with replay across every layer—must enumerate and verify each copy path, and fails the audit when a new layer forgets to join the runbook. Legal holds suspend erasure for identified subjects and must be recorded as documented exceptions with owners and review dates.

Layer	Why DELETE survives	Erasure strategy
GCS raw zone	Object versions, retention policies	Per-subject keys; crypto-shred
BigQuery curated	Time travel, fail-safe, backups	Crypto-shred; or delete + window expiry + backup aging
Pub/Sub	Message retention, snapshots	Short retention; crypto-shredded payloads
Derived tables	Copies copied onward	Lineage-driven replay (Ch. 22) or crypto-shred
Backups	Immutable by design	Crypto-shred; or wait for rotation

Table 23.1: Erasure propagation across platform layers.

Common Pitfall

Crypto-shredding requires key-per-subject granularity decided *at ingestion*. Retrofitting per-subject encryption onto a petabyte lake means re-writing the lake. If erasure obligations apply, the ingestion pipeline’s key management is the compliance architecture—decide it on day one.

23.5 Thursday Hands-On Project: Inspect and De-identify Sensitive Files

This lab inspects Cloud Storage files for PII with DLP, then de-identifies the content—masking emails and tokenizing customer IDs with a KMS-wrapped key—writing the safe output to a BigQuery table.

23.5.1 Stage Sensitive Samples and Create a Key

```

1 $env:PROJECT_ID = "my-gcp-dlp-lab"
2 $env:BUCKET = "my-gcp-dlp-lab-data"
3 gcloud config set project $env:PROJECT_ID
4 gcloud services enable dlp.googleapis.com cloudkms.googleapis.com
5
6 @'
7 customer_id,email,note
8 C001,ada@example.com,Called about invoice 4412; card on file

```

```

9 C002,grace@example.com,Email follow-up scheduled for Tuesday
10 '@ | Set-Content -Path .\support-notes.csv
11
12 gcloud storage cp .\support-notes.csv gs://$env:BUCKET/pii/support-notes.csv
13
14 gcloud kms keyrings create dlp-ring --location=global
15 gcloud kms keys create dlp-token-key --location=global '
16 --keyring=dlp-ring --purpose=encryption

```

Listing 23.1: Staging sample PII and creating the KMS-wrapped tokenization key.

23.5.2 Inspect and De-identify

```

1 from google.cloud import dlp_v2
2
3 dlp = dlp_v2.DlpServiceClient()
4 crypto = {"crypto_key": {"kms_wrapped": {
5     "wrapped_key": wrapped_key_bytes,
6     "crypto_key_name": kms_key_resource}},
7     "surrogate_info_type": {"name": "USER_TOKEN"}}
8 # deterministic FFX: same input -> same token, joins still work,
9 # but detokenization requires an auditable Cloud KMS permission

```

Listing 23.2: Deterministic, KMS-wrapped tokenization with DLP.

1. Create an inspection job over `gs://$BUCKET/pii/` with InfoTypes `EMAIL_ADDRESS`, `CREDIT_CARD_NUMBER`, minimum likelihood `POSSIBLE`, findings to BigQuery.
2. De-identify the CSV content via the DLP API: mask `EMAIL_ADDRESS` findings, tokenize `customer_id` with the FFX configuration above.
3. Load the de-identified rows to `safe.support_notes` and compare against the source: emails masked, IDs tokenized, notes preserved.
4. Verify reversibility control: call `ReidentifyContent` once as the key-holding identity (success, audit-logged), then as an identity without KMS decrypt (denied).

Deliverables

- The inspection-job configuration and its findings table excerpt.
- The de-identification configuration (mask + FFX token) and before/after row samples.
- Audit-log evidence of the permitted detokenization and the denied one.

Acceptance Criteria

- Inspection finds the planted PII with likelihood at or above the configured minimum.
- Tokenization is deterministic: the same customer ID tokenizes identically across runs.
- Detokenization succeeds only for the KMS-authorized identity, with both attempts in audit logs.

Stretch Goals

- Add k-anonymity risk analysis over the “safe” table using ZIP and birth-date quasi-identifiers.
- Wrap the flow in a Dataflow pipeline using DLP’s streaming-friendly transforms.

- Simulate an erasure: destroy a per-subject key version (24-hour delay) and document the recovery window.

23.6 Friday Use Case and Architecture: Third-Party PII Ingestion with an Erasure Map

23.6.1 Scenario

A marketing platform ingests customer files from third-party partners nightly. Files contain emails, names, and phone numbers. Requirements: PII never persists in the clear beyond a quarantined landing zone; analytics runs on tokens; GDPR erasure requests complete provably within thirty days; legal holds suspend erasure for flagged subjects.

23.6.2 Architecture

Partner files land in a quarantine bucket inside the VPC-SC perimeter (Chapter 21), CMEK-encrypted, access-limited to the ingestion service account. A Dataflow job validates the file contract, then per record: encrypts the raw payload under the subject’s KMS key (created on first sight), tokenizes identifiers with FFX, and writes tokenized rows to the analytics dataset and encrypted raw records to the vault bucket. The erasure service—triggered by request tickets—schedules destruction of the subject’s key and records the request, the key destruction ID, and the 24-hour completion in an audit table. Legal holds are a flag on the subject that blocks key destruction and is itself permission-gated and review-dated. The erasure-propagation map—which layer, which mechanism, which SLA—is a maintained document reviewed quarterly and tested in an annual game day.

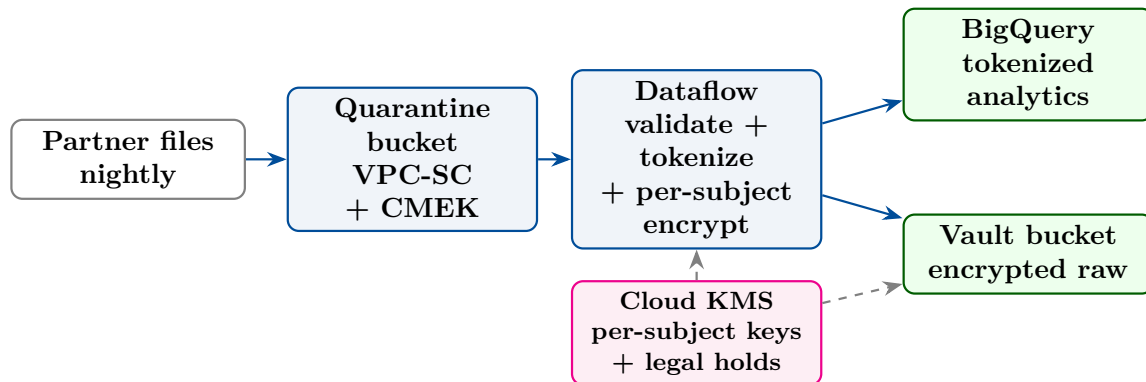


Figure 23.1: Third-party PII ingestion: quarantine, tokenize for analytics, encrypt per subject for provable erasure.

Tip

Prove erasure with a canary subject: a synthetic person whose data flows through every layer. The annual game day requests erasure of the canary and verifies unrecoverability per layer—an audit artifact more persuasive than any architecture diagram.

Common Pitfall

Tokenized PII can be re-identified through quasi-identifiers (ZIP plus birthdate plus gender uniquely identifies most populations). Run DLP risk analysis (k-anonymity, l-diversity) on the “safe” tables before declaring them shareable—the token removes the direct identifier,

not the person-shaped pattern.

23.7 Interview Q&A

23.7.1 Concepts and Trade-offs

1. **Q: What is an InfoType, and name two built-in examples.**

A: A category of sensitive data DLP recognizes—for example EMAIL_ADDRESS and CREDIT_CARD_NUMBER—with built-in, custom-regex, and dictionary detectors.

2. **Q: Masking versus tokenization: which is reversible, and by whom?**

A: Tokenization is reversible—only by identities holding KMS decrypt permission, under audit. Masking is irreversible by design.

3. **Q: Why use a KMS-wrapped key for tokenization?**

A: The key material is only ever handled encrypted; KMS unwraps it under IAM control with per-use audit logging, making reversibility a governed API call.

4. **Q: What does scheduled DLP scanning of BigQuery tables detect?**

A: PII presence, new sensitive columns, and coverage gaps across the estate—findings land in BigQuery for dashboards and alerting.

5. **Q: How do you verify de-identified data is actually unreadable?**

A: Negative tests (detokenize as an unauthorized identity), risk analysis on quasi-identifiers, and—for erasure—canary-subject game days proving per-layer unrecoverability.

23.8 Further Reading

Read the Sensitive Data Protection documentation for InfoTypes, inspection, de-identification templates, and risk analysis [54]. Cloud KMS documentation covers key destruction semantics and the 24-hour delay that crypto-shredding relies on [20]. BigQuery documentation details time travel and fail-safe behavior that erasure designs must account for [9], and Chapter 21's perimeter controls bound the whole design [60].

Key Takeaways

- DLP discovers (scheduled scans, profiles) and transforms (mask, tokenize, generalize)—different modes, different economics.
- Tokenization moves reversibility into KMS permissions and audit logs; masking removes it entirely.
- Deletion is not erasure: time travel, fail-safe, backups, snapshots, and versions all retain copies.
- Crypto-shredding erases every layer at once—but only if per-subject keys were designed in at ingestion.
- Legal holds suspend erasure as documented, permission-gated, review-dated exceptions.
- Prove compliance with canary subjects and game days, not with policy documents alone.

23.9 Exercises

1. **(Conceptual)** A PM proposes “just delete the rows within thirty days.” Write the technical reply enumerating the layers where the data survives.
2. **(Conceptual)** Compare deterministic FFX tokenization with a salted keyed hash for email joining, including the dictionary-attack consideration.
3. **(Hands-on)** Reproduce the Thursday lab, then add a custom dictionary detector for partner-specific loyalty IDs.
4. **(Hands-on)** Run k-anonymity analysis on a public dataset’s quasi-identifier columns and interpret the result.
5. **(Design)** Write the erasure-propagation map for the Friday architecture: per layer, mechanism, owner, SLA, and verification method.
6. **(Architecture)** Add the legal-hold workflow: flag storage, permissioning, review cadence, and the audit evidence it produces.

Production DataOps and the Professional Data Engineer Exam

Learning Objectives

- Provision data infrastructure—buckets, datasets, Composer environments—with Terraform and remote state.
- Ship Dataform SQLX models with assertions as data-quality tests through scheduled runs.
- Build CI/CD for data pipelines: plan/apply on merge, Dataform compilation and assertions on every pull request.
- Execute zero-downtime schema evolution with the expand-migrate-contract pattern.
- Map the Professional Data Engineer exam domains to this book’s weeks and identify weak areas.
- Complete the Milestone 6 capstone: a secure, governed, three-project data mesh.

Cross-Cloud Equivalence

Infrastructure-as-code and SQL transformation pipelines are platform-portable disciplines; only the resource names change.

Capability	GCP	AWS	Azure / Fabric
IaC	Terraform google provider / Deployment Manager	Terraform aws / CloudFormation	Terraform azurerm / Bicep
SQL pipelines	Dataform (SQLX, assertions)	dbt on Redshift / Athena	dbt on Synapse/Fabric
CI/CD	Cloud Build + Workload Identity	CodePipeline / GitHub Actions	Azure DevOps / GitHub Actions
State backend	GCS backend with locking	S3 + DynamoDB locking	Azure Storage backend
Exam analog	Professional Data Engineer	Data Engineer Associate	DP-203 / DP-600

Snowflake practitioners reach for dbt or Terraform’s Snowflake provider; **MongoDB** for the Atlas provider and schema-versioned migrations. The DataOps constant: infrastructure and transformations are code—reviewed, planned, applied, and rolled back through the same pipeline machinery as application software.

Key Terms

Terraform: declarative infrastructure-as-code; the google provider maps resources (`google_bigquery_dataset`) to GCP APIs. **State:** Terraform's recorded mapping of managed resources; remote backends (GCS) with locking make it team-safe. **SQLX:** Dataform's SQL extension adding model configs, dependencies (`ref`), and assertions to plain SQL. **Assertion:** a declarative data-quality query that fails the run when it returns rows (i.e., when violations exist). **Expand-contract:** zero-downtime schema evolution: add new shape, dual-write and backfill, migrate consumers, remove old shape. **Workload Identity Federation:** keyless CI authentication to GCP (Chapter 21)—the only acceptable credential story in pipelines.

24.1 Week 24 Roadmap

Week 24 completes the program by industrializing delivery and certifying mastery. Monday provisions with Terraform. Tuesday models with Dataform. Wednesday wires CI/CD and the expand-contract discipline. Thursday builds the full IaC-plus-Dataform-plus-CI project with an expand-contract drill. Friday runs the compressed exam preparation. The book closes with the Milestone 6 capstone.

Definition

DataOps applies software-delivery discipline to data platforms: infrastructure and transformations as versioned code, continuous integration with automated tests, staged deployment, and observable production operation. On GCP the reference stack is Terraform for infrastructure, Dataform for warehouse transformations, and Cloud Build for the pipeline itself [61, 31, 16].

24.2 Monday: Infrastructure as Code with Terraform

24.2.1 The google Provider

Terraform declares desired state; the google provider maps resources to GCP APIs. The data engineer's resource vocabulary: `google_storage_bucket`, `google_bigquery_dataset`, `google_bigquery_table` (with schema JSON and partitioning blocks), `google_composer_environment`, `google_pubsub_topic`, plus the IAM binding resources that wire them. Every resource from Chapters 1–23 is provisionable this way—the `gcloud` commands throughout this book were the learning surface; Terraform is the production surface.

```

1 terraform {
2   required_providers {
3     google = { source = "hashicorp/google", version = "~> 5.0" }
4   }
5 }
6
7 provider "google" {
8   project = var.project_id
9   region  = "us-central1"
10 }
11
12 resource "google_bigquery_dataset" "analytics" {
13   dataset_id    = "analytics_curated"
14   location      = "US"
15   description   = "Curated data products, managed by Terraform"
16   default_table_expiration_ms = 7776000000 # 90 days

```

```

17 labels      = { env = "prod", team = "data-engineering" }
18 }

```

Listing 24.1: Terraform provisioning a governed BigQuery dataset.

24.2.2 State, Backends, and Workspaces

Terraform’s state file is the source of truth for what it manages—and a sensitive artifact (it can contain attribute values). Teams store state in a **GCS remote backend** with state locking, never in git and never on laptops. **Workspaces** or—better for data platforms—**separate state per environment and per project** bound blast radius: the dev apply cannot touch prod because it cannot see prod’s state. Per-project state also prevents the cross-project apply races that corrupt multi-project mesh deployments.

Common Pitfall

terraform apply from a laptop with broad credentials is the shadow-IaC incident generator: unreviewed changes, stale state, and no audit trail. Gate apply behind CI with PR approval (Wednesday) and give humans plan-only permissions in production.

Tip

Manage IAM with authoritative resources per surface: google_project_iam_binding for project roles you fully own, ..._member for additive grants. Mixing authoritative and additive resources on the same role produces plan flapping that erodes trust in the pipeline.

24.3 Tuesday: Dataform—SQL Pipelines as Code

24.3.1 SQLX Models

Dataform is BigQuery’s native SQL pipeline tool. A SQLX file is a query plus a config block declaring its type (table, incremental, view), schema, and dependencies via \${ref('model')}. From refs, Dataform builds the dependency DAG and executes models in topological order—no scheduler configuration, no orchestration code; the DAG is derived from the SQL. **Incremental models** process only new data per run with when(incremental()) predicates, which is partition-overwrite idempotency (Chapter 12) expressed natively.

```

1  -- definitions/curated/daily_sales.sqlx
2  config {
3    type: "incremental",
4    schema: "curated",
5    bigquery: { partitionBy: "sale_date", clusterBy: ["store_id"] }
6  }
7
8  SELECT sale_date, store_id, SUM(amount) AS revenue
9  FROM ${ref('stg_sales')}
10 ${when(incremental(), "WHERE sale_date = CURRENT_DATE()")}
11 GROUP BY sale_date, store_id
12
13 -- definitions/curated/daily_sales_asserts.sqlx
14 config { type: "assertion", schema: "curated" }
15 SELECT sale_date FROM ${ref('daily_sales')}
16 GROUP BY sale_date HAVING COUNT(*) = 0 OR SUM(revenue) < 0

```

Listing 24.2: A SQLX model with an assertion file.

24.3.2 Assertions as Quality Gates

Assertions run as part of the pipeline: a failing assertion (the query returns rows—violations) fails the run and blocks downstream dependents. This is the AutoDQ idea (Chapter 22) embedded in the transformation layer: row counts, null checks, uniqueness, and freshness travel with the model code through review and CI. Note dbt as the cross-platform alternative with the same philosophy; Dataform’s advantage on GCP is native integration and zero external infrastructure.

Note

dbt users will find Dataform immediately familiar—models, refs, tests, and a DAG. The exam and this book use Dataform; production portability arguments sometimes favor dbt. Know both, choose per organization’s multi-warehouse posture.

24.4 Wednesday: CI/CD and Zero-Downtime Schema Evolution

24.4.1 The Pipeline

The reference CI/CD flow: on every pull request, Cloud Build (or GitHub Actions, authenticated via Workload Identity Federation—never committed keys) runs `terraform plan`, compiles the Dataform project, and executes assertions against a CI dataset; on merge to main, `terraform apply` and a Dataform release deploy. Budget alerts on the CI project guard cost; branch protection makes the pipeline the only path to production.

24.4.2 Expand-Contract Migrations

Zero-downtime schema change is a four-stage sequence, each stage a separate ordered deployment through the same pipeline. **Expand**: add the nullable new column. **Migrate**: dual-write and backfill it. **Switch**: move consumers to the new column behind a backward-compatible view shim. **Contract**: drop the old column—only after every consumer has moved. The deploy-order rule: schema expands first, consumers migrate to the shim, contraction ships last—never bundled.

Common Pitfall

The failure story that teaches this pattern: a team renamed `email` to `email_address` in the same release as the Dataform model change; the Looker dashboard deployed an hour later, and every tile errored on the missing column, paging on-call. Expand-contract would have kept both columns alive until consumers switched. Bundling all three stages in one release is how a routine rename becomes an incident review.

24.5 Thursday Hands-On Project: Terraform + Dataform + CI with an Expand-Contract Drill

This lab builds the full DataOps loop: a GitHub repository with Terraform provisioning a dataset and bucket, a Dataform project with three SQLX models and five assertions, a Cloud Build trigger compiling and testing on every pull request—then a four-stage column rename across four PRs, keeping a dashboard green throughout.

24.5.1 Repository Skeleton

1. **infra/**: Terraform for the GCS state backend (bootstrap, hand-run once), the **analytics** dataset, a raw bucket, and the CI service account with Workload Identity Federation binding.
2. **dataform/**: **dataform.json** plus **definitions/**: **stg_sales.sqlx** (view over raw), **daily_sales.sqlx** (incremental, partitioned), **sales_by_region.sqlx** (table), and assertion files covering row counts, nulls, uniqueness, freshness, and non-negative revenue.
3. **cloudbuild.yaml**: steps to run **terraform plan**, **dataform compile**, and **dataform test** against the CI dataset on every PR.

```

1 cd infra
2 terraform init
3 terraform plan -out=plan.tfplan
4
5 cd ../dataform
6 dataform compile
7 dataform test

```

Listing 24.3: Local verification before pushing the first PR.

24.5.2 The Expand-Contract Drill

Rename **amount** to **amount_usd** in **stg_sales** across four PRs: (1) add **amount_usd** alongside **amount**; (2) backfill and dual-write both; (3) switch **daily_sales** and the dashboard’s view shim to **amount_usd**; (4) drop **amount**. After each PR’s merge, confirm the dashboard tile querying the model still renders. Record the four green checks as the drill’s evidence.

Deliverables

- The repository: Terraform, Dataform project, Cloud Build configuration, and README with bootstrap steps.
- PR-check evidence: compile-and-test results on a pull request.
- The four-PR expand-contract sequence with dashboard-green evidence at each stage.

Acceptance Criteria

- **terraform apply** runs only from CI after PR approval; local applies are impossible by IAM design.
- All five assertions pass on the CI dataset, and a deliberately seeded bad row fails one visibly.
- The rename completes in four stages with zero consumer-visible downtime or errors.

Stretch Goals

- Add a scheduled Dataform release (nightly) with a notification channel on run failure.
- Extend CI with a bytes-scanned budget check that fails PRs whose SQLX models exceed a dry-run threshold.
- Reproduce the pipeline with GitHub Actions instead of Cloud Build and compare the Workload Identity setup.

24.6 Friday: Professional Data Engineer Exam Preparation

24.6.1 Exam Domains Mapped to This Book

The Professional Data Engineer exam assesses design and operational judgment, not syntax recall [50]. Map its domains to your study weeks: **designing data processing systems** (Chapters 1–4: storage, database selection, migration); **ingesting and processing data** (Chapters 9–15: Dataproc, Data Fusion, Datastream, Composer, Pub/Sub, Dataflow); **storing data and preparing/using data for analysis** (Chapters 5–8, 16–17: BigQuery depth, lakehouse, BI, BQML); **maintaining and automating workloads** (Chapters 12, 18–20, 24: orchestration, MLOps, DataOps); and the security/compliance thread woven through Chapters 21–23.

24.6.2 The Case Studies

The exam’s case studies present fixed business scenarios—**EHR Healthcare** (compliance-driven migration: the Chapter 11 CDC, Chapter 21 perimeter, and Chapter 23 DLP patterns) and **Helium Sports** (real-time analytics at scale: Chapters 13–16 streaming patterns). Questions reference these scenarios; read both case-study documents before the exam and annotate each requirement with the GCP service and this book’s chapter that answers it. Most case-study questions reduce to a selection decision you have already made hands-on: which storage class, which ingestion path, which security control.

24.6.3 Readiness Method

Take the first full-length mock exam closed-book and untimed-but-tracked; score by domain, not in aggregate—a 60% in one domain matters more than an 82% average. Weak domains map back to weeks for a targeted one-week review loop: rebuild that week’s lab, re-read the Friday use case, re-answer the self-checks. Take the second mock timed after the review loop; a score above 85% with no domain below 70% is the schedule-the-real-exam threshold. Exam questions reward operational judgment: prefer managed services over self-managed, least-privilege over convenient, and the answer that includes monitoring over the one that merely works.

Exam domain	Book chapters	Typical trap
Designing data processing systems	1–4, 21	Self-managed over managed answers
Ingesting and processing	9–15	Batch answers to streaming requirements
Storing data	1–8	Ignoring lifecycle/partition cost levers
Preparing and using data	5–8, 16–17	Skipping governance and semantics
Maintaining and automating	12, 18–20, 24	Solutions without monitoring or CI/CD
Security and compliance	21–23	IAM-only answers ignoring perimeters

Table 24.1: Exam domains mapped to book chapters, with the trap each domain sets.

Tip

In scenario questions, the correct answer usually optimizes the stated constraint—cost, latency, compliance, operational simplicity—not technical elegance. Identify the constraint in the stem first, then eliminate the two answers that ignore it; the remaining two differ on a trade-off you practiced in a Friday use case.

24.7 Interview Q&A

24.7.1 Concepts and Trade-offs

1. **Q: Why must Terraform state live in a remote backend for team work?**
A: Local state diverges between engineers and laptops, causing conflicting applies and untracked resources; a locked GCS backend serializes changes and makes state an auditable shared artifact.
2. **Q: What does a Dataform assertion guarantee about a model's output?**
A: That no row violates the assertion query at run time—failures block the run and its dependents, making quality a pipeline gate rather than a report.
3. **Q: What should run on every pull request: plan, apply, or both?**
A: Plan (plus Dataform compile and assertions). Apply belongs post-merge behind approval—PRs propose; merges deploy.
4. **Q: Why is Workload Identity Federation safer than committed keys?**
A: No long-lived secret exists to leak from the repository or CI variables; access is short-lived tokens bound to the workload's identity, fully audit-logged.
5. **Q: How do SQLX dependencies determine Dataform's execution order?**
A: `ref()` calls build the dependency DAG; Dataform executes models in topological order—orchestration derived from the SQL itself.

24.8 Further Reading

Read the Terraform google provider reference for the full resource vocabulary [61], the Dataform documentation for SQLX, assertions, and releases [31], and Cloud Build for triggers and Workload Identity Federation [16]. The Professional Data Engineer certification page hosts the current exam guide and case studies [50], and the Architecture Framework supplies the review questions the exam's design domains reward [35].

Key Takeaways

- Infrastructure and transformations are code: versioned, reviewed, planned, applied, and observable.
- Remote locked state and CI-gated applies are non-negotiable for team Terraform.
- Dataform derives orchestration from `refs` and gates quality with assertions—the DAG lives in the SQL.
- Expand-contract is four deployments, never one release; dashboards stay green at every stage.
- CI authenticates with Workload Identity Federation; committed keys are disqualifying.

- Exam readiness is measured per domain against the guide, with hands-on rebuilds as the review loop.

24.9 Exercises

1. **(Conceptual)** Explain why `terraform apply` belongs in CI even for a solo engineer.
2. **(Conceptual)** A colleague proposes skipping the view shim “since we deploy everything at once anyway.” Write the incident story that answers them.
3. **(Hands-on)** Reproduce the Thursday lab, then break one assertion deliberately and capture the failed PR check.
4. **(Hands-on)** Add a sixth assertion (freshness) and a CI step failing on dry-run bytes above a threshold.
5. **(Design)** Write your personal exam-readiness plan: domain self-scores, the two weakest domains, and the one-week rebuild loop for each.
6. **(Architecture)** Design the CI/CD topology for the three-project mesh capstone below: state layout, apply ordering, and cross-project dependency handling.

24.10 Milestone 6 Capstone: Secure, Governed Data Mesh

Capstone Project

Mission: Build a multi-project secure data mesh: Project A ingests streaming PII and tokenizes it with DLP; Project B hosts BigLake analytics over the tokenized data; Project C runs Dataplex governance with row-level security—all three inside one VPC Service Controls perimeter, everything provisioned by Terraform, so the blast radius of any single compromise is one project.

Estimated time: 12–16 hours.

What you’ll practice:

- Terraform across three projects with per-project state and explicit ordering.
- DLP deterministic tokenization with KMS-wrapped keys (Chapter 23).
- BigLake exposure with row- and column-level security (Chapter 8).
- Dataplex policy enforcement and AutoDQ scans (Chapter 22).
- VPC Service Controls perimeter with negative testing (Chapter 21).

Scenario

The conglomerate from Chapter 22 must now demonstrate its mesh under regulatory scrutiny: PII exists only tokenized in analytics, governance is centralized, and the perimeter is enforced and tested. This capstone integrates the entire book—ingestion, warehousing, streaming, ML-readiness, security, governance, and DataOps—into one defensible platform.

Requirements

- Terraform provisions all three projects, the perimeter, and every resource; applies run from CI with PR approval.
- Project A streams synthetic PII through Pub/Sub and Dataflow, tokenizing with DLP FFX under a KMS-wrapped key.

- Project B exposes only tokenized tables as BigLake assets with row access policies.
- Project C owns the Dataplex lake, taxonomy, policy tags, and AutoDQ scans; row-level security is verified with negative tests.
- The VPC SC perimeter encloses all three projects; the negative perimeter test passes and an audit-log alert on violations is exported.
- Cost estimate with two documented optimizations (DLP scan sampling, Dataplex tier, per-project budgets).

Reference Architecture

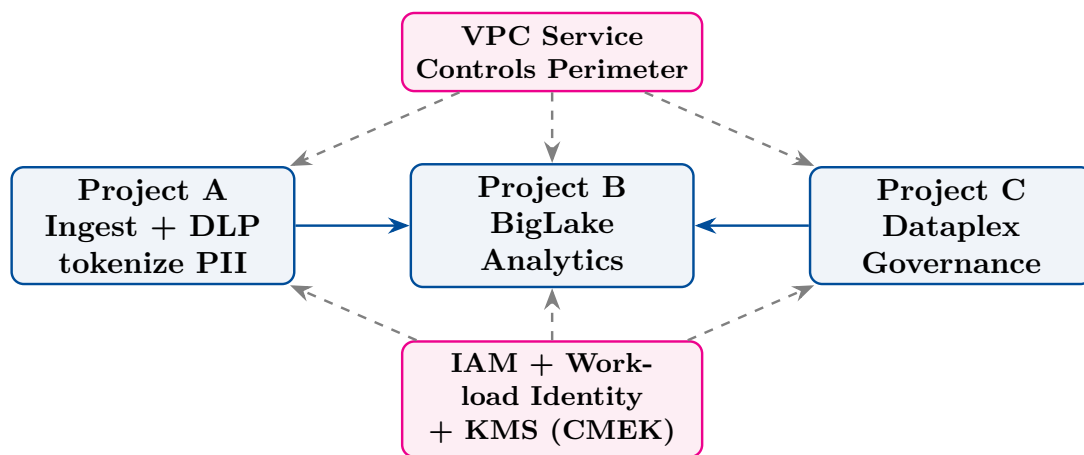


Figure 24.1: Milestone 6 reference architecture: three projects, one perimeter, centralized governance.

Implementation Notes

Order the Terraform applies per project with explicit `depends_on` or separate state—cross-project races are the classic failure. Remember that the perimeter blocks CI and orchestration API calls from outside: add the CI’s federated identity and Composer’s project to the perimeter or to ingress rules before enforcement. Finally, run DLP risk analysis on the “safe” tokenized tables: quasi-identifiers re-identify where tokens cannot (Chapter 23).

Deliverables

- A repository deploying the mesh from a clean organization: Terraform for all three projects plus documented gcloud fallbacks.
- The architecture diagram showing projects, perimeter, and governance flows.
- At least three automated data-quality tests (Dataplex AutoDQ or Dataform assertions) with failing-case evidence.
- Monitoring evidence: audit-log alerts on perimeter violations and DLP findings.
- Cost estimate and the security review: least-privilege per project, tokenized PII verified unreadable in the clear, negative perimeter test passed.

Acceptance Criteria

- The mesh deploys end-to-end from the README without manual console steps.

- Tokenized analytics in Project B contain no cleartext PII, verified by DLP inspection of the “safe” tables.
- The perimeter denies an external API call with valid credentials; the denial is alerted on.
- Row-level security in Project C passes its negative test, and policy tags deny by default with a verified positive path.

Stretch Goals

- Add a fourth domain project onboarded entirely through the Terraform module, proving the mesh scales.
- Extend the perimeter game day with a stolen-credential drill and a legal-hold suspension drill (Chapter 23).
- Publish the mesh’s data products through Analytics Hub clean-room listings for partner access (Chapter 8).
- Add the book-closing retrospective: map every capstone requirement back to the exam domains and score your readiness per domain.

Appendix A

Cross-Cloud Service Equivalence

This appendix is a consolidated reference that maps conceptually equivalent services and ideas across the seven platforms studied in this book series: Amazon Web Services (AWS), Microsoft Azure, Microsoft Fabric, Google Cloud Platform (GCP), MongoDB, Snowflake, and Databricks. The names differ, but the underlying data-engineering concepts—durable object storage, tiered lifecycle economics, immutability, managed relational engines, elastic compute, and disciplined data organization—recur everywhere.

Note

This appendix grew with each chapter. Every chapter contributed the equivalence tables introduced in its cross-cloud callout, so the book now holds a single consolidated Rosetta Stone for moving fluently between clouds. Where a clean one-to-one mapping does not exist, the prose notes the closest analog and the caveat. Databricks, which runs on all three hyperscalers, receives both per-domain treatment and a consolidated at-a-glance table at the end.

A.1 Object and Data-Lake Storage

The foundational layer covered in Chapter 1. Every platform offers durable, HTTP-addressable object storage with tiered pricing, lifecycle transitions, versioning, and write-once-read-many (WORM) controls.

Capability	AWS	Azure	GCP
Object storage	Amazon S3	ADLS Gen2 / Blob	Cloud Storage
Hot tier	S3 Standard	Hot	Standard
Infrequent access	S3 Standard-IA	Cool	Nearline
Archive (instant)	Glacier Instant Re-trieval	Cold	Coldline
Deep archive	Glacier Deep Archive	Archive	Archive
Lifecycle rules	Lifecycle policies	Lifecycle management	Object Lifecycle Mgmt
Versioning	S3 Versioning	Blob versioning	Object Versioning
Immutability (WORM)	Object Lock	Immutable blob storage	Bucket Lock
Cross-region copy	CRR / SRR	Object replication	Dual/multi-region + Transfer
Encryption at rest	SSE-S3 / SSE-KMS	SSE + CMK (Key Vault)	Google-managed / CMEK

The three remaining platforms approach storage differently. **Microsoft Fabric** exposes a single logical lake, *OneLake*, and uses *shortcuts* to virtualize S3, ADLS Gen2, and Google Cloud Storage in place without copying. **Snowflake** abstracts storage entirely: table data lives in

cloud object storage as immutable, columnar *micro-partitions* that the service manages on your behalf. **MongoDB** persists documents through the *WiredTiger* storage engine, balancing an internal cache against the operating-system page cache rather than exposing storage tiers.

A.2 Managed Relational Databases

The focus of Chapter 2 for the three hyperscalers. Each provides a fully managed relational engine, a cloud-native high-performance variant, high availability, read replicas, and a migration service.

Capability	AWS	Azure	GCP
Managed relational	Amazon RDS	Azure SQL Database	Cloud SQL
Cloud-native engine	Amazon Aurora	SQL Hyperscale	AlloyDB
Managed PostgreSQL	RDS / Aurora PostgresSQL	Azure DB for PostgreSQL	Cloud SQL / AlloyDB
High availability	Multi-AZ	Zone-redundant HA	Regional HA
Read scaling	Read Replicas	Read replicas / geo-replicas	Read Replicas
Global / geo	Aurora Database Global	Active geo-replication	Cross-region replicas
Serverless compute	Aurora Serverless v2	Azure SQL Serverless	AlloyDB (elastic)
Migration service	DMS + SCT	Azure Database Migration Svc	Database Migration Svc

Snowflake, **Fabric**, and **MongoDB** are not managed relational engines, but they solve overlapping problems: Snowflake and Fabric provide SQL analytics over warehouse/lakehouse storage, while MongoDB provides a distributed document store whose schema design (Chapter 2) replaces rigid relational normalization with intentional, access-driven modeling.

A.3 Elastic Compute and Analytical Warehouses

Snowflake’s Chapter 2 topic—separating compute from storage and scaling it elastically—has a counterpart in every analytics platform.

Concept	Snowflake	AWS	Azure / GCP
Compute unit	Virtual Warehouse	Redshift RA3 / Serverless	Synapse pool / Big-Query slots
Elastic concurrency	Multi-cluster warehouse	Concurrency scaling	Autoscale / on-demand
Pause when idle	Auto-suspend / resume	Pause / resume	Auto-pause / serverless
Billing unit	Credits	RPU / node-hours	DWU / slot-hours
Result reuse	Result cache	Result cache	Result / BI cache

Microsoft Fabric meters all workloads against a shared pool of Capacity Units (CU) on an F-SKU, an approach closer to a single elastic budget than to per-engine warehouses. **MongoDB Atlas** scales through cluster tiers with optional auto-scaling of compute and storage.

A.4 Data Organization and the Medallion Pattern

Fabric’s Chapter 2 topic—the Bronze/Silver/Gold medallion—is a portable design pattern rather than a product. Every platform implements the same progressive-refinement idea with its own primitives.

Layer	Fabric	Data (AWS/Azure/GCP)	lake	Snowflake
Bronze (raw)	Bronze Lakehouse	raw/ (S3/ADLS/GCS)	prefix	RAW schema / stage
Silver (validated)	Silver Lakehouse	validated/ prefix		STAGING schema
Gold (curated)	Gold Lakehouse	curated/ prefix		ANALYTICS schema
Storage format	Delta-Parquet	Parquet / Delta / Ice- berg		Micro-partitions

MongoDB realizes the same separation with distinct collections or databases (for example **raw_events**, **validated**, **curated**) governed by schema-design patterns rather than a lake layout.

A.5 Document Data Modeling

MongoDB’s Chapter 2 schema-design patterns generalize to every document and wide-column store.

Concept	MongoDB	Amazon nomoDB	Dy-	Azure Cosmos DB
Primary access unit	Document / collection	Item / table		Item / container
Partitioning	Shard key	Partition key		Partition key
One-table modeling	Embedding + patterns	Single-table design		Embedding + parti- tioning
Time-series grouping	Bucket Pattern	Composite sort keys		Bucketing by key
Optional attributes	Attribute Pattern	Sparse attributes		Sparse properties

Google Cloud’s closest analogs are **Firestore** and **Bigtable**. In **Snowflake** and **Fabric**, semi-structured documents are stored in **VARIANT** (Snowflake) or JSON columns and queried with path expressions rather than modeled as collections.

A.6 Globally Distributed and NoSQL Databases

Chapter 3 branches by platform into high-scale NoSQL (AWS DynamoDB, Azure Cosmos DB), globally consistent NewSQL (GCP Cloud Spanner), and advanced document modeling (MongoDB). They all answer the same question: how to scale a database horizontally across partitions and regions.

Concept	DynamoDB	Cosmos DB	Cloud Spanner	MongoDB
Model	Key-value / doc	Multi-model	Distributed SQL	Document
Partitioning	Partition key	Partition key	Split by key range	Shard key
Throughput unit	RCU / WCU	Request Units (RU)	Nodes / processing units	Cluster tier
Secondary indexes	GSI / LSI	Automatic indexing	Secondary indexes	Indexes
Change stream	DynamoDB Streams	Change Feed	Change streams	Change streams
Expiry	TTL	TTL	(manual)	TTL index
Multi-region	Global Tables	Multi-region writes	Multi-region (syn- chronous)	Zones / global clusters
Consistency	Eventual / strong	5 tunable levels	External (True- Time)	Tunable read/write concern

Snowflake and **Fabric** are analytical platforms rather than OLTP key-value stores; they ingest from these operational databases and store semi-structured payloads in **VARIANT**/JSON columns. Google’s dedicated NoSQL analogs are **Firestore** (document) and **Bigtable** (wide-column).

A.7 Managed Apache Spark

Microsoft Fabric’s Chapter 3 topic—managed Spark pools and notebooks—has a first-class equivalent on every cloud.

Capability	Fabric	AWS	Azure / GCP
Managed Spark	Starter / Custom pools	Glue / EMR / EMR Serverless	Synapse Spark / Databricks; Dataproc
Notebook UX	Fabric Notebooks	Glue Studio / EMR Studio	Synapse / Databricks; Vertex AI
Serverless option	Autoscaling pools	Glue / EMR Serverless	Synapse serverless; Dataproc Serverless
Lake format	Delta on OneLake	Delta / Iceberg / Hudi	Delta; Delta / Iceberg

Snowflake offers **Snowpark** for DataFrame-style processing, and **MongoDB** integrates through the Spark Connector.

A.8 Query Acceleration and Materialized Views

Snowflake’s Chapter 3 acceleration tools—materialized views, the Search Optimization Service, and transient/temporary tables—map to comparable features in every warehouse.

Feature	Snowflake	AWS Redshift	Azure Synapse / GCP BigQuery
Materialized views	Materialized Views	Materialized views	Materialized views
Point-lookup accel.	Search Optimization Svc	Sort keys / zone maps	(indexing) / search indexes
Result reuse	Result cache	Result cache	Result-set cache / BI Engine
Ephemeral tables	Transient / temporary	Temp tables	Temp tables

Fabric accelerates reads through Direct Lake and its Warehouse engine, while **MongoDB** relies on secondary and compound indexes plus the in-memory working set.

A.9 In-Memory Caching and Hybrid Ingestion

The AWS and Azure Chapter 4 topic—low-latency caching in front of a durable store, fed by hybrid (edge-to-cloud) ingestion—has a managed Redis-compatible equivalent on every cloud.

Capability	AWS	Azure	GCP / others
Managed cache	ElastiCache (Redis/Memcached)	Cache for Redis	Memorystore (Redis/Memcached)
Real-time API layer	AppSync / API Gateway	API Management / SignalR	API Gateway / Apigee
Hybrid ingestion	DataSync / Snow family / IoT Core	Data Box / IoT Hub / Arc	Transfer Appliance / IoT Core
Streaming intake	Kinesis / MSK	Event Hubs	Pub/Sub

Snowflake and **Fabric** rely on result and Direct Lake caches rather than a standalone Redis tier, while **MongoDB** serves hot data from its in-memory working set and Atlas can pair with an external cache.

A.10 Wide-Column and Key-Value NoSQL

GCP’s Chapter 4 topic—Cloud Bigtable—is a wide-column, high-throughput, low-latency NoSQL store. Its peers span the managed and open-source ecosystems.

Characteristic	GCP Bigtable	AWS	Azure / open source
Data model	Wide-column	DynamoDB (key-value/doc)	Cosmos DB (multi-model)
HBase-compatible	HBase API	Keyspaces (Cassandra)	Managed Instance for Cassandra
Scaling unit	Nodes / clusters	On-demand / provisioned	RU/s (Cosmos)
Best fit	Time-series, IoT, ad-tech	Serverless key-value	Global multi-model

Snowflake and **Fabric** address these workloads analytically rather than as operational key-value stores; **MongoDB** covers the document end of the same NoSQL spectrum.

A.11 Data Protection: Cloning, Time Travel, and Migration

Snowflake’s Chapter 4 topic—zero-copy cloning, Time Travel, and Fail-safe—plus MongoDB’s relational-to-document migration map to data-protection and migration tooling across the clouds.

Capability	Snowflake	AWS / Azure	GCP / Fabric
Instant clone	Zero-copy clone	Aurora clone / SQL DB copy	BigQuery table clone; OneLake shortcut
Point-in-time recovery	Time Travel	PITR (RDS/SQL DB backups)	PITR; Delta time travel
Retention safety net	Fail-safe (7 days)	Automated backups	Backups; Delta VACUUM window
Schema migration	—	DMS / SCT	Database Migration Service

MongoDB migrations use **Relational Migrator** plus the embed-vs-reference patterns from Chapter 4, and **Delta Lake** exposes historical versions through `VERSION AS OF` time-travel queries.

A.12 Serverless Analytical Warehouses

Chapter 5’s topic—BigQuery’s serverless, columnar warehouse—is the analytical heart of every platform. Pricing mechanics and tuning surfaces differ; decoupled storage and compute do not.

Capability	GCP BigQuery	AWS Redshift	Azure / Databricks
Warehouse engine	Dremel over Colossus	RA3 managed storage	Synapse / SQL warehouse (Photon)
Unit of compute	Slots	RPUs / node hours	DWU / DBU
On-demand pricing	Per TiB scanned	Serverless RPUs	Serverless SQL per TB / serverless DBU
Capacity pricing	Editions + reservations	Provisioned clusters	Provisioned pools / pro & classic warehouses
Result acceleration	BI Engine	Materialized views, AQUA	Result-set cache / DB-SQL cache
Nested data	ARRAY and STRUCT	SUPER (semi-structured)	OPENJSON / Spark nested types

Snowflake is the closest philosophical cousin (virtual warehouses billed per credit); **Fabric**

meters everything against capacity units. Databricks SQL warehouses query the *same* Delta tables pipelines write—no copy into a proprietary store.

A.13 Query Optimization and Physical Layout

Chapter 6’s two levers—eliminate data before it is read, precompute what is read often—exist on every analytical platform under different names.

Capability	GCP BigQuery	Snowflake	Databricks (Delta)
Coarse pruning	Partitioning	Micro-partition pruning	Partitioning / file stats
Fine-grained ordering	Clustering (4 columns)	Clustering keys	ZORDER BY / liquid clustering
Compaction	Managed automatically	Auto recluster	OPTIMIZE
Pre-aggregation	Materialized views	Materialized views	Materialized views
In-memory tier	BI Engine	Result / warehouse cache	Disk cache / Photon
Auto-maintenance	Autoclustering	Automatic	Predictive optimization

AWS Redshift exposes sort and dist keys plus zone maps; **Fabric** adds V-Order; **MongoDB** answers the same question with indexes and covered queries. The physics is shared: touch less data per query.

A.14 Federated and Cross-Cloud Analytics

Chapter 7’s questions—“can I query data I have not loaded?” and “can I query data in another cloud?”—have answers everywhere, with different mechanics and egress bills.

Capability	GCP BigQuery	AWS	Azure / Databricks
Query files in place	External tables / BigLake	Redshift Spectrum / Athena	Synapse serverless / external tables, Delta over S3/ADLS/GCS
Cross-cloud analytics	BigQuery Omni	Athena federated queries	Fabric shortcuts / multi-cloud workspaces
Geospatial	GEOGRAPHY (S2)	GEOMETRY / GEOGRAPHY	KQL geo / H3, spatial SQL
Full-text search	SEARCH indexes	OpenSearch integration	AI Search / Lakehouse search

Snowflake queries external and Iceberg tables in customer storage; **MongoDB Atlas Data Federation** reaches into S3 and Atlas clusters. The universal lesson: federation moves compute to data only when the engine supports it—otherwise bytes travel and the egress bill follows.

A.15 Lakehouse Governance, Sharing, and Clean Rooms

Chapter 8’s lakehouse pattern—open files in object storage with warehouse-grade security on top—exists on every platform, with different names for the unification layer.

Capability	GCP	AWS	Databricks / Fabric
Lakehouse tables	BigLake (+ Iceberg)	Lake Formation governed tables	Delta + Unity Catalog / OneLake lakehouse
Unstructured indexing	Object Tables	Athena + external metadata	Volumes + AI/BI / OneLake files
Row-level security	Row access policies	Lake Formation row filters	UC row filters / SQL RLS
Column-level security	Policy tags	Lake Formation column grants	UC column masks / Purview
Data sharing	Analytics Hub	Data Exchange / Redshift sharing	Delta Sharing / external sharing
Data clean rooms	Analytics Hub clean rooms	AWS Clean Rooms	UC clean rooms / confidential

Snowflake delivers the same story through external tables, masking policies, and Marketplace listings; **MongoDB**'s analog is field-level encryption and role-based views. Governance belongs on the data asset itself, not on each downstream copy.

A.16 Managed Spark and Hadoop Modernization

Chapter 9's modernization story—decouple storage from compute, make clusters ephemeral, let a serverless layer absorb administration—converges across clouds.

Capability	GCP Dataproc	AWS EMR	Databricks / Azure
Managed Spark/Hadoop	Dataproc clusters	EMR on EC2	Databricks Runtime (Photon) / HDInsight, Synapse
Serverless Spark	Dataproc Serverless	EMR Serverless	Serverless compute / Synapse serverless
Object storage for HDFS	Cloud Storage connector	S3 (EMRFS)	Unity Catalog volumes / ADLS (ABFS)
Cheap workers	Preemptible secondaries	Spot task nodes	Spot instances / low-priority nodes
Vectorized engine	(vanilla Spark)	EMR runtime (partial)	Photon / Fabric native engine
Cluster lifecycle	Ephemeral per job	Transient clusters	Job clusters / job-scoped pools

Snowflake's Snowpark absorbs DataFrame workloads without a cluster; **MongoDB**'s aggregation framework answers smaller questions cluster-free. The migration question is always: which workloads need the full Spark ecosystem, and which rewrite into SQL or serverless engines entirely.

A.17 Visual, Low-Code ETL

Chapter 10's topic—visual ETL for analysts and integration engineers—targets the same persona on every cloud.

Capability	GCP Data Fusion	AWS	Azure / Fabric
Visual ETL studio	Data Fusion Studio	Glue Studio	Data Factory / Fabric Data Factory
Interactive cleaning	Wrangler	Glue DataBrew	Power Query / Dataflow Gen2
Execution engine	Spark on ephemeral Dataproc	Glue Spark runtime	Spark / Data Factory IR
Extensibility	Plugin Hub (CDAP)	Glue connectors	Linked services & connectors
Scheduling	Built-in schedules + triggers	Glue triggers / EventBridge	ADF triggers / Fabric schedules

Databricks covers this niche with Lakeflow Declarative Pipelines and partner-built connectors; **Snowflake** users reach for Matillion, Fivetran, or dbt; **MongoDB** users lean on Atlas Data Federation and Compass. The shared trade-off: visual tools democratize pipeline building but can hide cost and semantics behind a friendly canvas.

A.18 Change Data Capture

Chapter 11’s pattern is universal: read the database’s transaction log, not its tables, and deliver ordered insert/update/delete events to analytical targets.

Capability	GCP Datastream	AWS DMS	Databricks / Azure
CDC engine	Log-based, serverless	DMS CDC tasks	Lakeflow Connect / ADF CDC, SQL CDC
PostgreSQL source	Logical replication (pgoutput)	wal2json / test_decoding	Partner tools / PG logical replication
Oracle source	LogMiner / binary reader	LogMiner / binary reader	Partner or self-managed
Managed targets	Cloud Storage, BigQuery	S3, Redshift, Kafka	Delta tables / ADLS, Synapse
CDC apply	BigQuery MERGE (staging)	Glue/EMR MERGE	AUTO CDC / T-SQL MERGE
Backfill	Built-in initial snapshot	Full load + CDC	Snapshot + CDC

Open-source **Debezium** on Kafka is the self-managed counterpart everywhere; **Snowflake** consumes CDC through Snowpipe and streams; **MongoDB** has change streams built into the database. The architectural constant: the transaction log is the source of truth, and every destination is an eventually-consistent projection of it.

A.19 Workflow Orchestration

Chapter 12’s managed Airflow is the most portable orchestration story in cloud data engineering—the same DAG Python runs on every platform.

Capability	GCP Composer	AWS MWAA	Databricks / Azure
Orchestrator	Composer 2 (GKE)	MWAA (ECS-based)	Workflows (jobs + tasks) / ADF managed Airflow
Parameters	Airflow variables, XComs	SFN input/output	Job params + task values / pipeline params
Conditional flow	Branch operators	Choice / Map states	if/else, for-each tasks / If Condition
Partial recovery	Retries, clears	Redrive / retries	Repair run / rerun from activity
Secret storage	Secret Manager	Secrets Manager	Secret scopes / Key Vault
DAG storage	Cloud Storage bucket	S3 bucket synced	Workspace files, Asset Bundles / Git

Snowflake Tasks and **dbt Cloud** schedules keep orchestration in-warehouse for SQL-only pipelines; **Fabric pipelines** target low-code orchestration; **Dagster** and **Prefect** offer asset-centric alternatives. Airflow’s task-centric DAG model remains the lingua franca the others are compared against.

A.20 Messaging and Event Ingestion

Chapter 13’s managed messaging shares one mental model—producers publish to named channels, consumers subscribe, the broker owns durability—with different knobs for ordering, replay, and cost.

Capability	GCP Pub/Sub	AWS	Azure / Kafka
Managed pub/sub	Pub/Sub (global)	SNS + SQS / EventBridge	Service Bus / Event Grid
Kafka-compatible	Managed Service for Kafka	MSK	Event Hubs Kafka surface / MSK, Confluent
Dead-lettering	Dead-letter topics	SQS DLQs	Service Bus dead-letter queues
Replay	Snapshots + seek	EventBridge archive+replay	Event Hubs offsets / Kafka retention
Schema enforcement	Avro / Proto topic schemas	EventBridge schemas	Schema Registry / Azure Schema Registry
Cost-profiled tier	Pub/Sub Lite (zonal)	Kinesis on-demand vs provisioned	Event Hubs throughput units / partitions

Databricks and **Snowflake** sit downstream of these buses (Spark Kafka source; Snowpipe Streaming, Kafka Connector); **MongoDB** consumes via Atlas Stream Processing. Pub/Sub differs deliberately from Kafka: global by default, partition-free for the user, priced per GiB delivered rather than per provisioned broker.

A.21 Stream Processing Engines and Semantics

Chapters 14 and 15 teach Beam’s unified model; every serious engine answers the Dataflow Model’s four questions with different defaults and vocabulary.

Concept	Beam / Dataflow	Apache Flink	Spark tured (Databricks)	Struc- tured Streaming
Event-time windows	Fixed / sliding / session	Tumbling / hopping / session	<code>window()</code> , session windows	session windows
Completeness	Watermark	Watermark	<code>withWatermark</code>	
Late data	Allowed lateness + triggers	Allowed lateness + side outputs	Watermark delay	
Per-key state	State & Timers API	Keyed state + timers	<code>flatMapGroupsWithState</code>	
Dedup	Stateful DoFn	Keyed state + TTL	<code>dropDuplicates</code> + watermark	
Exactly-once	Shuffle + Storage Write API	Checkpointing + 2PC sinks	Checkpoint + idempotent sink	
Batch unification	One model, both modes	Flink batch mode	<code>availableNow</code> trigger	

Managed runners exist on every cloud: Dataflow (GCP), Managed Service for Apache Flink and Kinesis Data Analytics (AWS), Stream Analytics and Fabric eventstreams (Azure). **Snowflake** lands micro-batches with simpler semantics; **MongoDB** change streams feed processors that implement these semantics themselves. Beam’s vocabulary—watermark, trigger, allowed lateness—is the common reference even when the runtime differs [1].

A.22 Governed BI and Semantic Layers

Chapter 16’s governance question—are metrics defined once in a semantic layer, or independently in every dashboard?—confronts every platform.

Capability	GCP Looker	AWS	Azure / Databricks
Governed BI + semantics	Looker (LookML)	QuickSight datasets	Power BI semantic models / AI/BI + metrics
Self-serve dashboards	Looker Studio	QuickSight	Power BI / AI/BI dashboards
Model as code	LookML in Git	QuickSight APIs	TMDL / Genie instructions
Natural-language BI	Gemini in Looker	Amazon Q in QuickSight	Copilot / Genie spaces
In-memory acceleration	BI Engine	SPICE	Import / DirectLake / SQL warehouse cache
Reverse ETL	Scheduled queries + partners	Census, Hightouch	Partner tools

Snowflake pairs with the same BI tools over its warehouses; **MongoDB Charts** serves operational analytics on Atlas. The portable insight: a semantic layer is a *code artifact*—versioned, reviewed, tested—and organizations that treat it as such are the ones where “Gross Revenue” means one thing.

A.23 In-Warehouse Machine Learning and Feature Stores

Chapter 17’s in-warehouse ML eliminates the extract-train-reload loop; every major warehouse ships a variant, and every serious ML platform implements a feature store as the contract between training and serving.

Capability	GCP	Snowflake	Databricks / AWS
SQL-native training	BigQuery ML <code>CREATE MODEL</code>	Snowpark ML / ML functions	SQL + MLlib / Redshift ML
Forecasting	ARIMA_PLUS	Forecasting ML function	AutoML / SageMaker forecasting
Feature registry	Vertex AI Feature Store	Snowflake Feature Store	Feature Engineering in UC / SageMaker FS
Offline store	BigQuery tables	Snowflake tables	Delta tables (UC) / S3 + Glue catalog
Online serving	Feature Store online	(hybrid tables)	Feature serving endpoints / ElastiCache
Point-in-time join	Training-set builders	ASOF joins	<code>create_training_set</code> / event-time joins

Azure / Fabric fields Synapse and Fabric ML with Spark MLlib; **MongoDB Atlas** exports to analytical stores for ML. The shared virtue: governance, lineage, and access control of the data platform extend automatically to training data, features, and models.

A.24 ML Platforms, Pipelines, and Serving

Chapters 18 and 19’s managed ML platforms converge on the same lifecycle—train, register, deploy, explain, monitor, retrain—behind different consoles.

Capability	GCP Vertex AI	AWS SageMaker	Databricks / Azure ML
AutoML (tabular)	AutoML Tabular	Autopilot	AutoML / Automated ML
Experiment tracking	Vertex Experiments	SageMaker Experiments	Managed MLflow / Azure ML runs
Model registry	Model Registry	Model Registry	Unity Catalog registry / MLflow registry
Pipelines	Vertex Pipelines (KFP/TFX)	SageMaker Pipelines	Workflows + MLflow / Azure ML pipelines
Online serving	Endpoints	Endpoints	Model Serving (serverless) / online endpoints
Batch scoring	Batch prediction	Batch transform	Spark UDF from registry / batch endpoints
Monitoring	Model Monitoring	Model Monitor	Lakehouse Monitoring / Azure ML monitoring
Request logging	Endpoint logs	Data capture	Inference tables (Delta) / App Insights

Snowflake embeds Snowpark ML and Cortex for in-warehouse inference; **MongoDB** users serve models adjacent to Atlas. For the data engineer, the constant is the interface contract: features in, predictions out, lineage and governance around both—and retraining as an *event-driven reaction to evidence*, not a calendar entry.

A.25 Pretrained AI APIs and Generative AI

Chapter 20’s pretrained APIs and the retrieval-augmented generation (RAG) pattern are the fastest paths from unstructured data to structured signal on every cloud.

Capability	GCP	AWS	Azure / Databricks
Image analysis / OCR	Vision API	Rekognition / Text-tract	AI Vision / Document Intelligence
Text NLP	Natural Language API	Comprehend	Azure AI Language
Translation	Translation API	Translate	Azure Translator
Speech	Speech-to-Text	Transcribe	Azure Speech
Foundation models	Gemini (Vertex AI)	Bedrock	Azure OpenAI / Foundation Model APIs
Vector index	Vertex Vector Search	OpenSearch / Kendra	AI Search / Mosaic AI Vector Search
RAG orchestration	Vertex RAG Engine	Bedrock Knowledge Bases	Azure AI RAG / LangChain + MLflow

MongoDB Atlas Vector Search and **Snowflake Cortex** bring embeddings and LLM functions into the data layer. The engineering constants: per-call pricing demands cost forecasting, and external calls inside distributed pipelines demand batching and backoff.

A.26 Security: Perimeters, Keys, and Identity

Chapter 21’s perimeter-based exfiltration defense and key management have direct counterparts on every cloud.

Capability	GCP	AWS	Azure / Databricks
Service perimeter	VPC Service Controls	SCPs + endpoint policies	Private Link + policy / IP access lists
Private API access	Private Google Access / PSC	VPC endpoints (PrivateLink)	Private Link / private connectivity
Key management	Cloud KMS (CMEK/CSEK)	KMS (CMK/BYOK)	Key Vault (CMK/BYOK) / CMK
Identity federation	Workload Identity Federation	IAM Roles Anywhere / IRSA	Workload identity federation
Machine identity	Service accounts	IAM roles	Managed identity / service principal + OAuth
Audit trail	Cloud Audit Logs	CloudTrail	Activity Log / <code>system.access.audit</code>

Snowflake implements perimeter control with network policies and Tri-Secret Secure; **MongoDB Atlas** with private endpoints and field-level encryption. The shared mental model: IAM answers *who may*, perimeters answer *from where and to where*, and keys answer *who can read it at all*.

A.27 Governance Fabrics, Lineage, and Data Quality

Chapter 22’s governance fabric—metadata, quality, and policy attached to data wherever it lives—differs per platform in scope and tightness of integration.

Capability	GCP Dataplex	AWS	Azure / Databricks
Governance fabric	Dataplex (lakes/zones)	Lake Formation	Purview / Unity Catalog
Catalog & search	Data Catalog (in Dataplex)	Glue Data Catalog	Purview / UC metastore
Lineage	Dataplex lineage	Glue (partial) / SageMaker	Purview lineage / UC auto-lineage
Auto data quality	AutoDQ scans	Glue Data Quality (Deequ)	Purview / Lakehouse Monitoring, DLT expectations
Data mesh units	Lakes per domain	Accounts per domain	Fabric domains / catalogs per domain

Snowflake governs through Horizon (tags, masking, lineage) inside its boundary; **MongoDB** relies on schema validation and Atlas roles. The mesh principle is platform-agnostic: domains own data products; the platform provides federated guardrails—discovery, quality, and policy enforcement—without centralizing every decision.

A.28 Sensitive-Data Discovery and De-identification

Chapter 23’s sensitive-data discovery and de-identification exist on every platform, though GCP’s Sensitive Data Protection is unusually deep as a standalone service.

Capability	GCP	AWS	Azure / Databricks
PII discovery	Sensitive Data Protection	Macie	Purview Information Protection / UC discovery
De-identification	DLP transforms (mask, tokenize, FPE)	Partner / Lambda transforms	Purview / ADF masking / UC masks
Warehouse masking	Policy tags + routines	Lake Formation + UDFs	Dynamic data masking / column masks
Format-preserving Risk analysis	DLP FFX/FPE k-anonymity / l-diversity	(partner) (partner)	(partner) (partner)

Snowflake provides masking policies and tag-based protection; **MongoDB** provides field-level encryption. The portable compliance lesson: deletion is not erasure—time travel, backups, and replicas retain copies, and only cryptographic erasure or exhaustive replay satisfies a strict right-to-erasure obligation.

A.29 Infrastructure as Code, SQL Pipelines, and CI/CD

Chapter 24’s disciplines—infrastructure and transformations as code—are platform-portable; only the resource names change.

Capability	GCP	AWS	Azure / Databricks
IaC	Terraform google provider / Deployment Manager	Terraform aws / CloudFormation	Terraform azurearm / Bicep / Asset Bundles
SQL pipelines	Dataform (SQLX, assertions)	dbt on Redshift / Athena	dbt on Synapse / Databricks SQL, DLT
CI/CD	Cloud Build + Workload Identity	CodePipeline / GitHub Actions	Azure DevOps / GitHub Actions
State backend	GCS backend with locking	S3 + DynamoDB locking	Azure Storage / Terraform remote state
Validate before deploy	terraform plan	cfn-lint / CDK synth	what-if / bundle validate
Cost telemetry	Billing export to BigQuery	CUR 2.0	Cost Management / system.billing.usage
Exam analog	Professional Data Engineer	Data Engineer Associate	DP-203/DP-600 / Databricks Data Engineer

Snowflake practitioners reach for dbt and Terraform’s Snowflake provider; **MongoDB** for the Atlas provider and schema-versioned migrations. The DataOps constant: the merge is the deploy, and the pipeline is the only writer to production.

A.30 Databricks Equivalents at a Glance

Because Databricks runs on all three hyperscaler clouds, it deserves a single consolidated mapping against this book’s GCP services, harvested from the companion Databricks book’s cross-cloud boxes.

GCP service / concept	Databricks equivalent	Book chapter
GCP project + console	Workspace + account console	Ch. 0 / DB Ch. 1
Cloud Storage (lake layout)	Volumes + external locations (Delta on GCS/S3/ADLS)	Ch. 1 / DB Ch. 2
BigQuery table	Delta table in Unity Catalog	Ch. 5 / DB Ch. 2
Partitioning + clustering	Partitioning, <code>ZORDER BY</code> , liquid clustering	Ch. 6 / DB Ch. 5
BigQuery reservations / slots	SQL warehouses (serverless / pro / classic)	Ch. 6 / DB Ch. 6
BigLake governance	Unity Catalog (grants, row filters, masks)	Ch. 8 / DB Ch. 8
Looker / Looker Studio	AI/BI dashboards + Genie	Ch. 16 / DB Ch. 7
Dataproc (managed Spark)	Databricks Runtime clusters (Photon)	Ch. 9 / DB Ch. 4
Data Fusion (visual ETL)	Lakeflow Declarative Pipelines (DLT)	Ch. 10 / DB Ch. 10
Datastream (CDC)	Lakeflow Connect + <code>AUTO CDC</code>	Ch. 11 / DB Ch. 11
Composer (Airflow)	Workflows (jobs, tasks, repair runs)	Ch. 12 / DB Ch. 12
Pub/Sub + Dataflow (streaming)	Kafka sources + Structured Streaming	Ch. 13–15 / DB Ch. 13–16
Storage Write API upserts	<code>foreachBatch</code> + <code>MERGE</code>	Ch. 15 / DB Ch. 16
Vertex Experiments / Registry	Managed MLflow + UC model registry	Ch. 18 / DB Ch. 17
Vertex Feature Store	Feature Engineering in UC + serving	Ch. 17, 19 / DB Ch. 18
Vertex endpoints / monitoring	Model Serving + Lakehouse Monitoring	Ch. 18, 20 / DB Ch. 20
Vertex Vector Search / RAG	Mosaic AI Vector Search + Foundation Models	Ch. 20 / DB Ch. 19
VPC SC, KMS, Workload Identity	Private connectivity, CMK, service principals	Ch. 21 / DB Ch. 21
Dataplex (lineage, audit)	UC lineage + <code>system.access.audit</code>	Ch. 22 / DB Ch. 22
Analytics Hub (sharing)	Delta Sharing (open protocol)	Ch. 8 / DB Ch. 22
Terraform + Cloud Build	Asset Bundles + Terraform + CI/CD	Ch. 24 / DB Ch. 23

The recurring contrast: GCP splits the platform into purpose-built managed services with separate consoles, while Databricks consolidates the same capabilities around one runtime, one table format (Delta), and one governance plane (Unity Catalog). Neither is simpler—they distribute complexity differently.

Tip

Use this appendix as a study aid for the book's lockstep, cross-cloud approach: when you learn a concept on one platform, immediately locate its equivalent here and predict how the other five would express the same idea. That habit turns six separate vocabularies into one mental model.

Appendix B

Environment Setup Reference

This appendix consolidates, in one place, everything a workstation needs before any chapter’s labs will run: the Google Cloud project and billing foundation, the `gcloud` CLI, language client libraries, Terraform, and the cost guardrails that keep a self-study budget intact. Each section is a checklist distilled from Chapter 0 and the setup steps scattered through the book; treat it as the page you revisit whenever a lab fails with an authentication, quota, or “API not enabled” error.

B.1 Project and Billing Foundation

Every lab in this book assumes one dedicated learner project (Chapter 0 recommends a name such as `gcp-de-study-<your-initials>`) linked to a billing account protected by the free trial’s \$300 / 90-day credit.

- ☐ Google Cloud account created; free trial activated with a card (no charge until the trial is manually upgraded).
- ☐ One learner project created; nothing runs in an organization default project.
- ☐ Billing account linked to the project (Billing console or `gcloud billing projects link`).
- ☐ Personal account holds *Editor* (or narrower per-chapter roles) on the learner project only—never *Owner* as a daily-driver habit (Chapter 0).
- ☐ 2-Step Verification enabled on the Google account before any resource exists.
- ☐ Regions and zones chosen deliberately and recorded (the book standardizes on `us-central1` / `us` multi-region unless a chapter says otherwise).

APIs are disabled by default; each chapter’s first command fails until its services are enabled. Enable them per lab rather than all at once:

```
1 gcloud services enable storage.googleapis.com bigquery.googleapis.com ‘
2 dataflow.googleapis.com dataproc.googleapis.com composer.googleapis.com ‘
3 pubsub.googleapis.com datastream.googleapis.com datafusion.googleapis.com ‘
4 aiplatform.googleapis.com dataplex.googleapis.com dlp.googleapis.com
```

Listing B.1: Enabling the most common service APIs used across the book.

B.2 The `gcloud` CLI

Install the Google Cloud CLI on Windows via the interactive installer or `winget install Google.CloudSDK`, then initialize authentication and defaults once (Chapter 0):

```

1 gcloud init                                # account, project, default
   region/zone
2 gcloud auth application-default login      # credentials for client
   libraries
3 gcloud config list                         # verify
4 gcloud auth list                          # verify active account

```

Listing B.2: One-time CLI initialization from PowerShell.

- ☐ `gcloud init` completed; `gcloud config list` shows the learner project and chosen region.
- ☐ Application Default Credentials created—client libraries (Python, Java) and Terraform read these when no explicit key is provided.
- ☐ Cloud Shell verified as the zero-install fallback (`shell.cloud.google.com`); it ships `gcloud`, `bq`, `gsutil`, Terraform, and an editor pre-authenticated.
- ☐ Components updated periodically: `gcloud components update`.

Tip

When a command mysteriously acts on the wrong project, the active configuration is almost always the cause. `gcloud config configurations create/learn/activate` keeps separate named configurations per project—cheap insurance against writing to the wrong place.

B.3 Client Libraries and Local Development

The book’s code listings assume Python 3.10+ in a virtual environment, with per-chapter extras:

```

1 python -m venv .venv
2 .\.venv\Scripts\Activate.ps1
3 pip install google-cloud-storage google-cloud-bigquery google-cloud-pubsub '
4 google-cloud-aiplatform apache-beam[gcp] dbt-bigquery

```

Listing B.3: Python environment with the libraries used across the book.

Key rules the chapters rely on:

- Client libraries authenticate through Application Default Credentials; never download a service-account JSON key for local work (Chapter 21 explains why keys are disqualifying in production and a bad habit in study).
- Apache Beam pipelines run locally with the DirectRunner before any Dataflow submission (Chapter 14); the `apache-beam[gcp]` extra is required for GCP I/O connectors.
- Keep one virtual environment per machine, not per chapter; dependency conflicts between chapters are rare and a shared environment keeps disk use down.

B.4 Terraform

Chapters 12 and 24 (and the Milestone 6 capstone) provision with Terraform. Setup is three steps:

1. Install Terraform on Windows (`winget install Hashicorp.Terraform`) or use the copy preinstalled in Cloud Shell.

2. Point Terraform at the same Application Default Credentials as the client libraries; the `google` provider picks them up automatically.
3. For any team or CI scenario, move state to a locked GCS backend before the first shared apply (Chapter 24):

```

1 terraform {
2   backend "gcs" {
3     bucket = "gcp-de-study-tfstate"
4     prefix = "book/state"
5   }
6 }

```

Listing B.4: Remote state backend block used from Chapter 24 onward.

Common Pitfall

Do not run Chapter 24's multi-project applies with local state: a laptop crash between `plan` and `apply` leaves orphaned resources the next run cannot see. The GCS backend takes five minutes and is the single highest-value setup step in this appendix.

B.5 Cost Guardrails

Chapter 0 sets these before the first lab; they are repeated here because every expensive accident in self-study traces to a skipped guardrail.

- ☐ A budget exists on the billing account with alerts at 50%, 90%, and 100% of a small monthly amount (for example \$25), emailed to you (`gcloud billing budgets create` or the console).
- ☐ Billing export to BigQuery is enabled once spend becomes non-trivial (Chapter 8 and Chapter 23 analyze this export).
- ☐ BigQuery custom query cost quotas are set per user and per project (Chapter 5's Friday use case; dry-run bytes checks in CI per Chapter 24).
- ☐ Every cluster, Composer environment, Data Fusion instance, and Vertex endpoint created in a lab is deleted or stopped the same day—these are the services that bill while idle.
- ☐ A weekly five-minute review: Billing console reports grouped by service, compared against the budget forecast.

Idle-cost offender	Bills while idle?	Shutdown habit
Dataproc cluster	Yes (VMs)	Ephemeral per job; delete after
Composer environment	Yes (GKE + Cloud SQL)	Delete between weeks 11–12 labs
Data Fusion instance	Yes (instance hours)	Stop/delete after each lab
Vertex AI endpoint	Yes (replica hours)	Undeploy model, delete endpoint
Bigtable cluster	Yes (node hours)	Delete node/cluster after lab
Pub/Sub, Cloud Storage	Effectively no (usage-priced)	Lifecycle rules; delete test buckets
BigQuery	No (storage only)	Delete datasets; watch bytes scanned

B.6 Per-Chapter Tool Requirements

The full matrix of what each chapter needs beyond the base setup (project, billing, gcloud, Python). Enable APIs and install extras the day you start the chapter, not before.

Ch.	Services to enable	Local tools / notes
0	Cloud Resource Manager, Billing, Storage	gcloud CLI, Cloud Shell fallback
1	Cloud Storage, Storage Transfer	gcloud storage / gsutil; Python storage client
2	Cloud SQL, AlloyDB, DMS	psql client; private IP networking
3	Cloud Spanner	Spanner client libs; Key Visualizer in console
4	Cloud Bigtable	cbt CLI; HBase shell optional
5	BigQuery	bq CLI; public datasets (free tier)
6	BigQuery (BI Engine)	Execution-plan reading; INFORMATION_SCHEMA
7	BigQuery (external tables, Omni)	External CSVs; geography functions
8	BigLake, Analytics Hub, BI Engine	BigLake connection; Looker Studio (free)
9	Dataproc, Dataflow (Beam)	PySpark local optional; apache-beam[gcp]
10	Data Fusion (+ ephemeral Dataproc)	Wrangler UI; private IP instance for labs
11	Datastream, Composer	Source PostgreSQL with logical replication
12	Composer, Dataform, Dataplex	Airflow local optional; Terraform intro
13	Pub/Sub (+ Lite)	Python publisher/subscriber samples
14	Dataflow, Pub/Sub	Beam DirectRunner locally first
15	Dataflow (streaming), BigQuery	Storage Write API; windowing test harness
16	Looker / Looker Studio	LookML Git repo; reverse-ETL partner optional
17	BigQuery ML, Vertex AI	google-cloud-aiplatform
18	Vertex AI (AutoML, endpoints)	Endpoint teardown discipline (cost)
19	Vertex Pipelines, Feature Store	KFP SDK; pipeline compiler
20	Vision / NL / Translation APIs, Vertex	API keys not needed—use ADC
21	VPC SC, Cloud KMS, Audit Logs	Second test identity for negative tests
22	Dataplex, Data Catalog, AutoDQ	Tag templates; lake/zone planning worksheet
23	Sensitive Data Protection (DLP), KMS	Synthetic PII generator; risk-analysis jobs
24	Cloud Build, Dataform, Terraform	GCS state backend; GitHub repo for CI

Note

If a lab fails at its first API call, debug in this order: (1) active configuration and account (**gcloud config list**, **gcloud auth list**); (2) API enabled on *this* project; (3) billing linked; (4) quota; (5) IAM role. In self-study the cause is (2) or (3) nine times out of ten.

CLI and SQL Quick Reference

This appendix collects the commands the book's labs actually use, grouped by service, each with a one-line purpose. It is a lookup table, not a tutorial—each row is taught in context in the chapter shown in parentheses. All commands assume an authenticated gcloud CLI with the learner project set (Appendix on environment setup).

C.1 Core gcloud

Command	Purpose
<code>gcloud init</code>	Interactive login, project, region/zone defaults (Ch. 0)
<code>gcloud auth application-default login</code>	Create ADC for client libraries and Terraform (Ch. 0)
<code>gcloud config set project P</code>	Switch active project
<code>gcloud config configurations activate NAME</code>	Swap between saved project/identity sets
<code>gcloud services enable S.googleapis.com</code>	Enable an API before first use (Ch. 0)
<code>gcloud projects list</code>	List visible projects
<code>gcloud billing budgets create ...</code>	Create budget with alert thresholds (Ch. 0)
<code>gcloud iam service-accounts create NAME</code>	Create a service account (Ch. 0, 21)
<code>gcloud projects add-iam-policy-binding P \</code>	Grant a role to a member (Ch. 0)
<code>gcloud logging read "FILTER" --limit=N</code>	Query Cloud Logging from the shell (Ch. 15, 21)

C.2 Cloud Storage (gcloud storage / gsutil)

Command	Purpose
<code>gcloud storage buckets create gs://B --location=US</code>	Create a bucket (Ch. 0, 1)
<code>gcloud storage cp LOCAL gs://B/</code>	Upload a file or object (Ch. 1)
<code>gcloud storage cp -r gs://B/PREFIX .</code>	Download a prefix recursively
<code>gcloud storage ls -r gs://B/</code>	List objects recursively
<code>gcloud storage rm -r gs://B</code>	Delete objects and bucket (careful)
<code>gcloud storage buckets update gs://B \</code>	Set lifecycle, class, or labels on a bucket
<code>gsutil versioning set on gs://B</code>	Enable object versioning (Ch. 1)
<code>gsutil retention set 30d gs://B</code>	Apply a retention policy (Ch. 1)
<code>gsutil -m cp -r SRC gs://B/</code>	Parallel multi-threaded copy of large trees
<code>gcloud transfer jobs create ...</code>	Storage Transfer Service scheduled transfer (Ch. 1)

gcloud storage is the modern, parallel-by-default CLI; gsutil remains for versioning, retention, and a few legacy flags—the book uses both deliberately (Chapter 0).

C.3 BigQuery (bq and SQL)

Command	Purpose
bq mk DATASET	Create a dataset (Ch. 5)
bq show PROJECT:DATASET.TABLE	Inspect schema and metadata (Ch. 5)
bq query --nouse_legacy_sql "SQL"	Run a query non-interactively (Ch. 5)
bq query --dry_run "SQL"	Estimate bytes billed before running (Ch. 5, 6)
bq load --autodetect DS.TBL gs://B/f.csv	Load CSV from Cloud Storage (Ch. 7)
bq extract DS.TBL gs://B/out-*.csv	Export a table to Cloud Storage
bq ls --jobs -a	List recent jobs for forensics (Ch. 6)
bq mk --table DS.TBL schema.json	Create a table with an explicit schema
bq update --time_partitioning_field=D ...	Convert/define partitioning (Ch. 6)
bq mkdef --source_format=CSV ...	External table definition over GCS (Ch. 7)
SQL pattern	Purpose
FROM t, UNNEST(t.arr) AS x	Flatten repeated fields (Ch. 5)
AVG(x) OVER (PARTITION BY k ORDER BY d \ ROWS BETWEEN 6 PRECEDING AND CURRENT ROW)	Seven-row rolling window (Ch. 5)
MERGE t USING s ON key AND is_current ...	SCD Type 2 upsert (Ch. 5)
CREATE TABLE ... PARTITION BY DATE(d) \ CLUSTER BY customer_id	Partition + cluster in one DDL (Ch. 6)
CREATE MATERIALIZED VIEW ...	Precompute a hot aggregation (Ch. 6)
ST_WITHIN(point, polygon)	Geospatial containment test (Ch. 7)
CREATE MODEL ... OPTIONS(model_type=...)	BigQuery ML training (Ch. 17)
ML.EVALUATE(MODEL m, ...)	Model metrics on held-out data (Ch. 17)
ML.PREDICT(MODEL m, ...)	In-warehouse inference (Ch. 17)
SELECT * FROM DS.INFORMATION_SCHEMA.JOBS BY PROJECT	Estimate bytes forensics (Ch. 6, 8)
CREATE ROW ACCESS POLICY ...	Row-level security (Ch. 8)

C.4 Dataflow and Apache Beam

Command	Purpose
python pipeline.py	Local test before cloud submission (Ch. 14)
--runner=DirectRunner	
python pipeline.py	Submit to Dataflow (Ch. 14)
--runner=DataflowRunner \ --project=P --region=R	
--temp_location=gs://B/tmp	
--streaming	Run in streaming mode (Ch. 14, 15)
--max_num_workers=N	Cap autoscaling ramp (Ch. 14)
gcloud dataflow jobs list --region=R	List jobs and states
gcloud dataflow jobs cancel JOB_ID	Stop a streaming job
gcloud dataflow jobs drain JOB_ID	Stop gracefully, flushing in-flight data (Ch. 15)
gcloud dataflow flex-template run NAME \ --template-file-gcs-location=gs://B/t.json	Launch a packaged Flex Template (Ch. 14)

C.5 Dataproc

Command	Purpose
<code>gcloud dataproc clusters create C \</code> <code>--region=R --num-workers=2</code>	Create a cluster (Ch. 9)
<code>--secondary-worker-type=preemptible</code> <code>gcloud dataproc jobs submit pyspark</code> <code>f.py \</code> <code>--cluster=C --region=R</code>	Add cheap preemptible workers (Ch. 9) Submit a Spark job (Ch. 9)
<code>gcloud dataproc clusters delete C</code> <code>--max-idle=30m</code>	Tear down the same day (cost) Auto-delete an idle cluster
<code>gcloud dataproc batches submit pyspark</code> <code>f.py</code> <code>--properties spark.executor.memory=8g</code>	Serverless Spark, no cluster at all (Ch. 9) Override Spark configuration (Ch. 9)

C.6 Cloud Composer (Airflow)

Command	Purpose
<code>gcloud composer environments create E \</code> <code>--location=L --image-version=...</code>	Create a Composer 2 environment (Ch. 12)
<code>gcloud composer environments storage</code> <code>dags \</code> <code>import --environment=E --location=L</code> <code>--source=dag.py</code>	Upload a DAG to the synced bucket
<code>gcloud composer environments run E \</code> <code>--location=L dags trigger -- DAG_ID</code>	Trigger a DAG manually
<code>gcloud composer environments run E \</code> <code>--location=L dags list-runs -- DAG_ID</code>	List task states for triage
<code>gcloud composer environments delete E</code>	Delete the environment (idle cost)

C.7 Pub/Sub

Command	Purpose
<code>gcloud pubsub topics create T</code>	Create a topic (Ch. 13)
<code>gcloud pubsub topics create T \</code> <code>--schema=SCHEMA</code> <code>--message-encoding=BINARY</code>	Enforce an Avro/Proto schema (Ch. 13)
<code>gcloud pubsub subscriptions create S</code> <code>--topic=T</code>	Create a pull subscription
<code>--dead-letter-topic=DLT \</code> <code>--max-delivery-attempts=5</code>	Attach a dead-letter topic (Ch. 13)
<code>gcloud pubsub topics publish T</code> <code>--message="..."</code>	Publish a test message
<code>gcloud pubsub subscriptions pull S</code> <code>--auto-ack</code>	Pull and acknowledge test messages
<code>gcloud pubsub snapshots create SNAP \</code> <code>--subscription=S</code>	Snapshot for later replay/seek (Ch. 13)
<code>gcloud pubsub lite-topics create T</code> <code>--zone=Z</code>	Provisioned zonal Lite topic (Ch. 13)

C.8 Datastream

Command	Purpose
<code>gcloud datastream connection-profiles create CP \</code> <code>--location=L --type=postgresql ...</code>	Define a source or destination (Ch. 11)
<code>gcloud datastream streams create S \</code> <code>--location=L --source=CP1</code> <code>--destination=CP2</code>	Start CDC from source to target (Ch. 11)
<code>gcloud datastream streams list</code> <code>--location=L</code>	Check stream state and lag
<code>gcloud datastream streams update S</code> <code>--state=PAUSED</code>	Pause a stream for maintenance
<code>gcloud datastream streams backfill S \</code> <code>--objects=...</code>	Re-run initial load for objects (Ch. 11)

C.9 Cloud Data Fusion

Command	Purpose
<code>gcloud data-fusion instances create I \</code> <code>--location=L --edition=developer</code> <code>--enable-private-ip</code>	Create an instance (Ch. 10) Reach on-prem sources over private networking (Ch. 10)
<code>gcloud data-fusion instances describe I</code> <code>--location=L</code>	Get the Studio URL and state
<code>gcloud data-fusion instances stop I</code> <code>--location=L</code>	Stop instance billing between labs

Pipelines themselves are deployed and scheduled inside the Studio UI or via the REST/CDAP API; Wrangler directives are authored on samples and executed at scale by ephemeral Dataproc (Chapter 10).

C.10 Vertex AI

Command	Purpose
<code>gcloud ai models list --region=R</code>	List registered models (Ch. 18)
<code>gcloud ai endpoints create --region=R \</code> <code>--display-name=N</code>	Create a serving endpoint (Ch. 18)
<code>gcloud ai endpoints deploy-model E \</code> <code>--model=M --region=R ...</code>	Attach a model version with replicas (Ch. 18)
<code>gcloud ai endpoints predict E</code> <code>--region=R \</code> <code>--json-request=in.json</code>	Smoke-test online prediction
<code>gcloud ai custom-jobs create --region=R</code> <code>...</code>	Submit a custom training job (Ch. 18)
<code>gcloud ai endpoints undeploy-model E \</code> <code>--deployed-model-id=D --region=R</code>	Stop replica billing (cost habit)
<code>gcloud ai index-endpoints list</code> <code>--region=R</code>	Vector Search endpoints (Ch. 20)

C.11 Dataplex

Command	Purpose
<code>gcloud dataplex lakes create LAKE --location=L</code>	Create a lake (Ch. 22)
<code>gcloud dataplex zones create ZONE \ --lake=LAKE --location=L --type=RAW</code>	Add a raw or curated zone (Ch. 22)
<code>gcloud dataplex assets create A \ --lake=LAKE --zone=ZONE --location=L</code>	Attach a bucket or dataset to a zone
<code>... gcloud dataplex datascans create data-quality \ SCAN --location=L --data=...</code>	Declare and schedule quality rules (Ch. 22)
<code>gcloud dataplex datascans run SCAN --location=L</code>	Run a quality scan on demand

C.12 Sensitive Data Protection (DLP)

Command	Purpose
<code>gcloud dlp inspect-templates create ...</code>	Reusable inspection configuration (Ch. 23)
<code>gcloud dlp deidentify-templates create ...</code>	Reusable de-identification config (Ch. 23)
<code>... gcloud dlp jobs triggers create \ --storage-type=bigquery ...</code>	Scheduled scan of BigQuery / GCS (Ch. 23)
<code>gcloud dlp jobs list</code>	Inspect scan results and status
<code>curl dlp.googleapis.com/v2/...:deidentify</code>	Inline tokenization from pipelines (Ch. 23)

Tip

Every `gcloud` command accepts `--format=json` and `--filter=...`; scripting the CLI with JSON output plus `jq` (or PowerShell's `ConvertFrom-Json`) turns every lab step into an automatable check—which is exactly how Chapter 24's CI pipelines verify deployments.

Production-Readiness and Governance Checklists

This appendix distills the production notes, pitfalls, and acceptance criteria scattered through the chapters into domain checklists. Use them two ways: as a pre-launch gate (every box ticked or consciously deferred with a reason) and as exam preparation—Professional Data Engineer scenario questions are, in large part, these checklists wearing a story. Items marked (*neg*) require a negative test: proving the control denies what it must deny.

D.1 Storage and Data Lake (Chapters 1, 8)

- ☐ Bucket location matches compute locality; dual/multi-region only where an availability requirement justifies the price.
- ☐ Lifecycle rules transition and expire objects; no bucket grows unbounded (Archive minimum-duration cost considered before archiving active data).
- ☐ Versioning, retention policy, or soft delete chosen deliberately per bucket; the three mechanisms are not conflated.
- ☐ Uniform bucket-level access enabled; no legacy ACLs leaking object-level exceptions.
- ☐ Encryption posture recorded: Google-managed, CMEK, or CSEK, with key rotation and (*neg*) access revocation tested.
- ☐ Transfer jobs (Storage Transfer Service) validated on bytes, object counts, sampled checksums, and incremental sync—not just “it ran.”
- ☐ Lake zones mapped to a medallion layout (*raw/*, *validated/*, *curated/*) with IAM boundaries per zone.
- ☐ BigLake connections enforce policy tags and row access policies on lake files; (*neg*) a direct GCS read by an unauthorized identity fails.

D.2 Warehousing and Analytics (Chapters 5–8)

- ☐ Fact tables declare a grain; every measure and key respects it.
- ☐ Large tables partitioned (usually by date) and clustered on the dominant high-cardinality filter columns; bytes-billed before/after measured.
- ☐ Materialized views cover the top recurring aggregations; refresh cost is budgeted.

- ☐ Pricing model chosen from measured slot usage (`INFORMATION_SCHEMA.JOBS_BY_PROJECT`), not guessed: on-demand for spiky, Editions reservation for steady workloads.
- ☐ Custom query cost quotas per user/project; dry-run byte checks in CI.
- ☐ SCD Type 2 dimensions match on key *and* `is_current`; history reconstruction tested on a known entity.
- ☐ External/BigLake tables accepted with their limits (no clustering, no managed statistics) or promoted to native tables where performance matters.
- ☐ Semantic layer (LookML or equivalent) owns metric definitions; no dashboard re-implements a governed measure.
- ☐ BI Engine capacity sized for hot dashboard tables only, not batch ELT.
- ☐ Analytics Hub listings grant read-only linked datasets; revocation tested.

D.3 Batch ETL and Orchestration (Chapters 9–12, 24)

- ☐ Dataproc clusters are ephemeral per job (or serverless batches), with preemptible capacity confined to secondary workers and jobs made retry-safe.
- ☐ Spark jobs sized from evidence: skew inspected in the Spark UI before memory is raised; autoscaling policy bounds and cooldowns set.
- ☐ Data Fusion instances use private IP where sources are internal; macros parameterize dates and paths so one definition serves schedules, backfills, and reruns.
- ☐ Datastream streams monitored for freshness lag; backfill strategy documented per source table.
- ☐ Every Airflow task is idempotent (deterministic partition overwrites or upserts); retries and clears are safe by construction.
- ☐ XComs carry small control values only—paths, counts, verdicts—never payloads.
- ☐ Event-driven readiness uses sensors (file arrival) rather than clock guesses; schedules have catchup behavior chosen deliberately.
- ☐ Dataform (or dbt) assertions gate the pipeline; a failing test blocks dependents.
- ☐ Terraform state is remote and locked; applies run from CI after review; Workload Identity Federation replaces all committed keys.
- ☐ Schema evolution follows expand–contract: additive change, dual-run shim, consumer migration, contract cleanup—never a one-release breaking change.

D.4 Streaming and Real-Time (Chapters 13–16)

- ☐ Topics carry Avro/Proto schemas so the broker rejects contract violations at publish time.
- ☐ Every subscription has a dead-letter topic and a triage owner; poison messages never silently vanish.
- ☐ Deduplication is designed in: message IDs, idempotent sinks, or Dataflow exactly-once mode—at-least-once delivery is assumed.
- ☐ Pub/Sub Lite chosen only after the steady-throughput cost comparison against Standard.

- ☐ `oldest_unacked_message_age` alerts exist per subscription.
- ☐ Windowing, watermark, and allowed-lateness choices are written down per pipeline; late-data handling (side output or update) is explicit.
- ☐ Per-key state is bounded by timers or watermark garbage collection; unbounded state growth is treated as a bug.
- ☐ Sinks are exactly-once (Storage Write API) or at-least-once with a merge/dedup step—and the choice is justified by cost.
- ☐ Streaming jobs drain (not cancel) for deploys; autoscaling floor/ceiling set for known traffic peaks.
- ☐ Reverse-ETL syncs resolve identity before writing to operational systems and respect API rate limits on a freshness-versus-cost cadence.

D.5 Machine Learning and MLOps (Chapters 17–20)

- ☐ The simplest sufficient model is the baseline (BQML logistic regression or AutoML) before any custom training spend.
- ☐ Training/serving skew prevented by construction: one feature definition (feature store or shared SQL) feeds both paths.
- ☐ Evaluation uses the metric that matches the decision—precision/recall at a chosen threshold for imbalanced problems, never bare accuracy.
- ☐ Models live in a registry with lineage to dataset and code; promotion is an alias move; rollback is tested.
- ☐ Online endpoints serve only latency-bound paths; batch prediction covers bulk scoring; endpoints are undeployed when idle.
- ☐ Retraining triggers fire on drift evidence (with data-quality preconditions), not on a calendar; candidates must beat the incumbent before registration.
- ☐ External AI API calls inside pipelines are batched, bounded in concurrency, retried with backoff and jitter, and dead-lettered on permanent failure; per-call cost is forecast and alerted.
- ☐ GenAI/RAG pipelines govern source chunks and prompts like data assets: versioned, access-controlled, evaluated.

D.6 Security, Governance, and Compliance (Chapters 21–23)

- ☐ Least privilege everywhere: predefined or custom roles, no primitive Owner/ Editor grants in shared projects; service accounts per workload, not shared.
- ☐ No service-account key files exist; Workload Identity (Federation) issues short-lived tokens instead.
- ☐ VPC Service Controls perimeters enclose sensitive projects; access levels carve only documented exceptions; (*neg*) a valid-credential request from outside the perimeter is denied and alerted.
- ☐ CMEK on regulated datasets and buckets with rotation; key destruction procedure rehearsed (it is the erasure backstop).

- ☐ Cloud Audit Logs (including Data Access where required) route to a log bucket or BigQuery with alerting on policy denials and IAM changes.
- ☐ Dataplex lakes/zones map to data-mesh domains; domains own product quality and SLAs; the platform owns taxonomy and minimum rules.
- ☐ Policy tags (not per-role views) enforce column-level security; row access policies have (*neg*) tests.
- ☐ AutoDQ scans plus declared quality rules cover completeness, uniqueness, validity, and freshness on curated assets; findings route to owners.
- ☐ DLP discovery scans the estate on a schedule; sensitive columns are masked (irreversible) or tokenized (KMS-wrapped, reversible under audit)—never left cleartext “temporarily.”
- ☐ Right-to-erasure design accounts for time travel, backups, and replicas; quasi-identifier risk analysis runs on “safe” de-identified tables.
- ☐ Lineage is queryable from dashboard tile back to source system; impact analysis is a lookup, not archaeology.

D.7 Reliability and FinOps (Chapter 23 and throughout)

- ☐ RPO/RTO stated per pipeline; restore and failover are drilled, not assumed.
- ☐ Budgets alert at 50/90/100%; billing export to BigQuery feeds a weekly cost-by-service review.
- ☐ Idle-billing resources (clusters, Composer, endpoints, instances) have same-day teardown or auto-stop.
- ☐ Every architecture review answers: what breaks when this component fails, who is paged, and what does it cost at 10× volume.

Tip

During the exam, each checklist above compresses into one elimination rule: prefer the managed service, the least-privilege grant, the answer that includes monitoring, and the design that satisfies the stem’s stated constraint—then verify none of these checklists is violated.

Lab Solutions and Solution Sketches

This appendix sketches a reference solution for every chapter's Thursday hands-on project plus selected exercises (Chapters 0–24). Each sketch gives the **approach**, the **key commands or code**, and the **common failure modes** that consume the most debugging time. These are sketches, not the only correct solutions—if your version satisfies the chapter's acceptance criteria differently, it is equally valid.

E.1 Chapter 0: Environment Verification

Approach. Prove the toolchain end-to-end before any service work: authenticate, set defaults, create/list/delete a temporary bucket.

Key commands. `gcloud init; gcloud auth list; gcloud config list; gcloud storage buckets create gs://$env:BUCKET --location=us-central1; gcloud storage ls; gcloud storage rm -r gs://$env:BUCKET.`

Common failure modes. API not enabled on the learner project (enable `storage.googleapis.com`); billing not linked (buckets cannot be created); wrong active configuration after switching projects; PowerShell quoting of `gs://` URIs (use single quotes or variables).

E.2 Chapter 1: Bucket, Lifecycle, Transfer, and Client Code

Approach. Create a regional bucket next to the compute; upload objects with `gcloud storage cp`; attach a lifecycle JSON transitioning to Nearline at 30 days and deleting at 365; run a Storage Transfer Service job from a second bucket; finish with the Python `google-cloud-storage` client listing and downloading via ADC.

Key commands. `gcloud storage buckets update gs://B --lifecycle-file=life.json; gsutil versioning set on gs://B; gcloud transfer jobs create SRC gs://B.`

Common failure modes. Lifecycle rules that never fire because the condition field name is wrong (`age` is days since creation, not modification); transfers validating nothing (bytes/counts/checksums are the acceptance evidence); client code failing on missing `application-default` login.

E.3 Chapter 2: HA Cloud SQL PostgreSQL, Read Replica, and Failover

Approach. Create a regional (HA) Cloud SQL PostgreSQL instance with private IP; add a cross-zone read replica; connect with `psql` over the private address; trigger failover from the

console or CLI and time the promotion; verify the replica continues serving reads during primary failover.

Key commands. `gcloud sql instances create pg --region=us-central1 --availability-type=REGIONAL --network=default --no-assign-ip; gcloud sql instances create pg-replica --master-instance-name=pg gcloud sql instances failover pg.`

Common failure modes. Private IP without allocated ranges/peering (allocate with `compute networks vpc-peerings`); confusing HA standby with read replica—the standby is not readable; testing read-after-write on the replica and hitting replication lag.

E.4 Chapter 3: Multi-Region Cart Lab

Approach. Create a multi-region Spanner instance, a `Carts` table keyed by a high-cardinality customer ID, and an interleaved `CartItem`s child; write through the client library; inspect Key Visualizer for hotspotting after a load burst.

Key commands. `gcloud spanner instances create cart --config=nam3 --nodes=1; DDL via gcloud spanner databases ddl update with INTERLEAVE IN PARENT.`

Common failure modes. Monotonically increasing keys (timestamps as leading key) creating one hot split; interleaving a child that is scanned independently; expecting Cloud-SQL-style replica semantics—Spanner’s replicas are synchronous quorum members, not read copies.

E.5 Chapter 4: IoT Temperature Sensor Ingestion

Approach. Design a row key `device_id#reversed_timestamp`; create the table with one column family; batch-write synthetic readings with the Python client or `cbt`; read recent history with a small prefix range scan; confirm even load in Key Visualizer.

Key commands. `gcloud bigtable instances create iot --cluster=iot-c1 --cluster-zone=us-central1 --cluster-num-nodes=1; cbt createtable readings; cbt set readings rowkey cf:temp=22.5.`

Common failure modes. Lexicographic key mistakes (unpadded numbers sort wrong: zero-pad); forgotten reversed timestamp making “latest” reads scan the whole entity; salting applied without planning the read fan-out; leaving the one-node cluster running after the lab (it bills).

E.6 Chapter 5: GitHub Public Dataset Analytics

Approach. Query `bigquery-public-data.github_repos.sample_commits`: UNNEST the repeated `repo_name`, aggregate daily commits per repository, and compute a seven-day rolling average with an explicit `ROWS BETWEEN 6 PRECEDING AND CURRENT ROW` frame. Verify one window by hand.

Key commands. `bq show bigquery-public-data:github_repos.sample_commits; bq query --nouse_legacy_sql "SELECT ..."; dry-run first with --dry_run.`

Common failure modes. Forgetting UNNEST and counting array rows instead of elements; window without `PARTITION BY` (one giant global average); legacy SQL syntax errors (the backtick identifier style); skipping the bytes-processed estimate and scanning the full table.

E.7 Chapter 6: Measuring Scan Strategies

Approach. Take one unpartitioned table and create three variants—partitioned, partitioned+clustered, and a materialized view over the hot aggregation. Run the same selective query against each, recording bytes billed from dry runs and execution plans; identify the skewed stage by per-slot output.

Key commands. `bq query --dry_run` per variant; DDL with `PARTITION BY DATE(d) CLUSTER BY customer_id`; `CREATE MATERIALIZED VIEW mv AS SELECT d, customer_id, SUM(amount)`
....

Common failure modes. Comparing cached results (disable result cache for timing); partitioning on a high-cardinality column (partition explosion); expecting the materialized view to accelerate queries whose SQL it cannot match; clustering column chosen with low filter selectivity.

E.8 Chapter 7: External CSVs Joined with Native GIS Data

Approach. Define an external table over CSVs in Cloud Storage, join against a native table with `GEOGRAPHY` columns, and answer a containment question with `ST_WITHIN`. Compare cost and latency against loading the CSVs natively.

Key commands. `bq mkdef --source_format=CSV --autodetect gs://B/sales/*.csv > def.json`; `bq mk --external_table_definition=def.json DS.sales_ext`; SQL with `ST_GEOGRAPHYFROMTEXT` and `ST_WITHIN`.

Common failure modes. Header row parsed as data (`skip_leading_rows`); schema drift between CSV files (`autodetect` masks it until the join fails); assuming external tables prune—every query scans every file; point-in-polygon argument order swapped.

E.9 Chapter 8: BigLake Security and an Analytics Hub Listing

Approach. Create a BigLake connection and an external table over the lake bucket; attach policy tags to the sensitive column and a row access policy; publish the curated dataset as an Analytics Hub listing and subscribe from a second project or identity.

Key commands. `bq mk --connection --location=US --connection_type=CLOUD_RESOURCE lake_conn`; `CREATE ROW ACCESS POLICY`; console/CLI Analytics Hub create listing and subscribe.

Common failure modes. The connection's service account lacking `storage.objectViewer` on the bucket; testing security only with the owner identity (negative tests need a second identity); expecting subscribers to see updates to revoked listings.

E.10 Chapter 9: Ephemeral PySpark Sales Aggregation

Approach. Create a Dataproc cluster sized for one job (or use a serverless batch), submit a PySpark aggregation reading CSVs from Cloud Storage and writing Parquet back, then delete the cluster in the same session. Compare wall time and cost against Dataproc Serverless.

Key commands. `gcloud dataproc clusters create spark9 --max-idle=30m`; `gcloud dataproc jobs submit pyspark agg.py --cluster=spark9 --region=us-central1`; `gcloud dataproc clusters delete spark9`.

Common failure modes. Reading via `file://` instead of `gs://` (workers see nothing); executor OOM on a skewed key (inspect Spark UI, fix skew before raising memory); forgetting teardown; version mismatch between local PySpark and the cluster image.

E.11 Chapter 10: No-Code Customer Cleansing to BigQuery

Approach. In a developer-edition Data Fusion instance, use Wrangler to clean a customer CSV sample (trim, case-normalize, parse dates, dedupe), promote the directives into a pipeline reading Cloud Storage and writing BigQuery, then schedule it and parameterize dates with macros.

Key commands. Studio UI for Wrangler and pipeline deploy; `gcloud data-fusion instances stop I --location=L` after the lab.

Common failure modes. Public-IP instance unable to reach internal sources (use private IP); directives validated on a 100-row sample failing on full-scale edge cases; BigQuery sink schema mismatch rejecting rows silently into errors; leaving the instance running (bills hourly).

E.12 Chapter 11: PostgreSQL CDC to BigQuery with a Contract Test

Approach. Enable logical replication on a source Cloud SQL PostgreSQL (`cloudsql.logical_decoding=on`), create Datastream connection profiles for source and BigQuery, start a stream with backfill, and add a contract test: apply a nullable-column addition (should pass) and a rename (should fail loudly).

Key commands. `gcloud datastream connection-profiles create ... --type=postgresql;`
`gcloud datastream streams create S --source=CP1 --destination=CP2;`
`gcloud datastream streams list` for freshness lag.

Common failure modes. Logical decoding flag unset or replication role missing; publication not covering the migrated tables; BigQuery apply ordering issues when the merge key is ambiguous; renames treated as drop+add breaking downstream views.

E.13 Chapter 12: The File-Triggered ELT DAG

Approach. Build a Composer DAG that senses a partner file in Cloud Storage, loads it to a BigQuery staging table, merges into curated with idempotent partition overwrite, runs a Dataform assertion, and notifies with `trigger_rule=all_done`. Rerun a past date to prove idempotency.

Key commands. `gcloud composer environments storage dags import --source=elt.py;`
`gcloud composer environments run E --location=L dags trigger -- file_elt;`
`dataform compile` locally before upload.

Common failure modes. Sensor timeout shorter than real arrival variance; non-idempotent INSERT duplicating rows on retry; XComs carrying row payloads (task failures on size); environment still building when the first DAG is uploaded (Composer creation takes 20+ minutes).

E.14 Chapter 13: Financial Transactions with a Dead-Letter Net

Approach. Create a transactions topic with an Avro schema, a subscription with a dead-letter topic at five delivery attempts, and a publisher sending both valid and malformed messages. Watch the malformed ones route to the DLQ with attempt metadata, then triage and republish.

Key commands. `gcloud pubsub topics create txn --schema=txn-schema; gcloud pubsub subscriptions create txn-sub --topic=txn --dead-letter-topic=txn-dlq --max-delivery-attempts=5; gcloud pubsub subscriptions pull txn-dlq-sub.`

Common failure modes. Schema encoding mismatch (BINARY vs JSON) rejecting every publish; DLQ without its own subscription (messages accumulate unobserved); assuming ordering without ordering keys; alerting absent on `oldest_unacked_message_age`.

E.15 Chapter 14: WordCount on the Dataflow Runner

Approach. Port the local DirectRunner WordCount to Dataflow: same Beam code, new runner flags. Then unit-test a custom ParDo with `TestPipeline` and `PAssert` in CI before any cloud run; package the result as a Flex Template.

Key commands. `python wordcount.py --runner=DataflowRunner --project=P --region=R --temp_location=gs://B/tmp --staging_location=gs://B/stg; gcloud dataflow flex-template build/run.`

Common failure modes. Missing `temp_location/staging_location`; worker SDK version mismatch with the local Beam version; unserializable DoFn closures (module-level imports inside the DoFn); expecting Classic templates to accept runtime graph parameters (use Flex).

E.16 Chapter 15: Sessionized Clicks with Late Data

Approach. Extend streaming WordCount thinking to sessions: key clicks by user, apply session windows with a 30-minute gap, configure allowed lateness, and emit late arrivals to a side output. Land results in BigQuery via the Storage Write API; test windowing locally with `TestStream`.

Key commands. `Beam Window.into(Sessions(gap_duration=30*60)).with_allowed_lateness(...).accumulating_fired_panels(); --streaming --enable_streaming_engine.`

Common failure modes. Unbounded state from missing watermark GC; session gap too small (over-fragmented sessions); at-least-once sink without dedup double-counting after retries; cancelling instead of draining during redeloys (lost in-flight windows).

E.17 Chapter 16: Streaming Dashboard and Reverse-ETL Sync

Approach. Point Looker Studio (or Looker with a LookML model) at the streaming BigQuery table from Chapter 15 for a live dashboard; then build a reverse-ETL sync pushing a computed segment (for example high-value users) into a CRM-shaped table/file, with identity resolution validated first.

Key commands. LookML `explore/measure` definitions in Git; scheduled query for the segment; `bq query` delta extraction keyed on `updated_at`.

Common failure modes. Dashboard re-implementing a metric that belongs in the semantic layer; syncing on warehouse IDs that do not match CRM identifiers (green pipeline, wrong people); sync frequency set tighter than the API rate limit and budget allow.

E.18 Chapter 17: Churn Prediction on Google Analytics Data

Approach. On the Google Analytics sample dataset in BigQuery, engineer per-user features in SQL, train `BOOSTED_TREE_CLASSIFIER` (after a logistic-regression baseline), evaluate with `ML.EVALUATE` on a time-held-out split, and serve predictions with `ML.PREDICT`.

Key commands. `CREATE OR REPLACE MODEL ds.churn OPTIONS(model_type='BOOSTED_TREE_CLASSIFIER', input_label_cols=['churned']) AS SELECT ...; SELECT * FROM ML.EVALUATE(MODEL ds.churn, ...).`

Common failure modes. Label leakage (features computed from events after the prediction date); reporting accuracy on a 5% churn base rate; random split instead of time split inflating metrics; exporting the model to Vertex before the BQML baseline is even measured.

E.19 Chapter 18: AutoML Tabular from BigQuery to a Live Endpoint

Approach. Register the churn feature table as a Vertex AI dataset, launch an AutoML Tabular training job, review feature attributions, deploy the model to an endpoint, smoke-test with one prediction, then undeploy.

Key commands. `gcloud ai endpoints create --display-name=churn; gcloud ai endpoints deploy-model E --model=M --region=R --min-replica-count=1; gcloud ai endpoints predict E --json-request=req.json; gcloud ai endpoints undeploy-model` immediately after.

Common failure modes. Endpoint left deployed (replica-hours bill); traffic split summing to less than 100%; attributions misread as causal; requesting online latency for a nightly batch problem.

E.20 Chapter 19: Extract–Train–Deploy Pipeline

Approach. Compose a KubeFlow (Vertex Pipelines) DAG: a BigQuery extract component, a train component, an evaluation gate, and a conditional deploy component that registers the model only when metrics beat the incumbent. Compile with the KFP SDK and run on Vertex Pipelines.

Key commands. `from google_cloud_aiplatform import pipeline_jobs; compiler.Compiler().compile('pipeline.json'); pipeline_jobs.PipelineJob(...).run().`

Common failure modes. Components passing large data through parameters instead of artifact URIs; evaluation gate comparing against a stale incumbent metric; missing region match between pipeline job and staging bucket; triggering retraining on a schedule instead of on drift evidence.

E.21 Chapter 20: Streaming Review Sentiment to BigQuery

Approach. Stream synthetic product reviews through Pub/Sub into a Dataflow job that calls the Natural Language API (`analyzeSentiment`) with batching and bounded concurrency, writes annotated rows to BigQuery, and dead-letters permanent API failures.

Key commands. Python client `language_v1.LanguageServiceClient().analyze_sentiment;` pipeline options with `--streaming;` DLQ subscription on the input topic.

Common failure modes. One API call per element at full parallelism (quota errors and a linear bill—batch and throttle); retries without backoff hammering a 429; cost forecast skipped until the invoice; sentiment treated as per-entity when `analyzeEntitySentiment` was needed.

E.22 Chapter 21: Perimeter Around a BigQuery Dataset

Approach. Create a VPC Service Controls perimeter enclosing the analytics project, scoped to the BigQuery and Storage APIs; add an access level for the administrator’s IP; then run the negative test from an outside identity holding valid credentials and confirm denial plus an audit-log alert.

Key commands. `gcloud access-context-manager perimeters create P --resources=projects/N --restricted-services=bigquery.googleapis.com,storage.googleapis.com; gcloud access-context-manager levels create;` negative test via a second gcloud configuration.

Common failure modes. Testing only with the owner (control proves nothing); forgetting CI/orchestration identities in ingress rules (pipelines break at enforcement); dry-run perimeter left in dry-run forever; assuming IAM denial and perimeter denial are the same signal in logs.

E.23 Chapter 22: A Governed Lake with a Protected Salary Column

Approach. Create a Dataplex lake with raw and curated zones, attach the HR bucket and BigQuery dataset as assets, build a tag template (`owner`, `classification`, `retention`), apply a policy tag to the salary column, schedule an AutoDQ scan on the curated table, and verify lineage from the dashboard table back to the raw file.

Key commands. `gcloud dataplex lakes create hr --location=L; gcloud dataplex zones create curated --lake=hr --type=CURATED; gcloud dataplex datascans create data-quality hr-dq --data=//bigquery.../table.`

Common failure modes. Zones created without an access posture decision (raw world-readable); tag templates conflated with policy tags (metadata vs enforcement); AutoDQ suggested rules accepted without declaring owned completeness/freshness rules; lineage gaps from jobs run under personal credentials.

E.24 Chapter 23: Inspect and De-identify Sensitive Files

Approach. Run a DLP inspection job over a bucket of synthetic customer files, land findings in BigQuery, then de-identify with masking for analytics and KMS-wrapped format-preserving tokenization where re-identification is a governed operation; finish with k-anonymity risk analysis on the “safe” output.

Key commands. `gcloud dlp jobs triggers create --storage-type=gcs ...;` DLP deidentify API with `CryptoReplaceFfxFpeConfig` referencing a KMS-wrapped key; risk-analysis job via the DLP API.

Common failure modes. InfoTypes too broad (false-positive floods) or too narrow (missed formats); treating masking as reversible; token key stored unwrapped beside the data; assuming tokenization alone anonymizes—quasi-identifiers re-identify; inspecting only the lake while backups and replicas retain cleartext.

E.25 Chapter 24: Terraform + Dataform + CI with an Expand-Contract Drill

Approach. Provision the lab project (dataset, tables, schedules) with Terraform from a GCS-backend state; define the mart in Dataform SQLX with assertions; wire Cloud Build so every PR runs `terraform plan + dataform compile` and merges run `apply`; then perform one expand-contract column rename across four deployments without a red dashboard.

Key commands. `terraform init/plan/apply`; `dataform compile`; `gcloud builds triggers create github ...`; Workload Identity Federation pool and provider for keyless CI.

Common failure modes. Local state drifting between runs; `apply` running on PR instead of post-merge; rename done in one release (views break at deploy time); CI authenticating with a committed service-account key (disqualifying); assertion written after the fact instead of gating the run.

E.26 Selected Exercises

Chapter 0 (conceptual). Why avoid daily use of Owner? Because every mistake and every compromise inherits Owner’s blast radius; least privilege makes errors survivable.

Chapter 3 (design). Choosing between Spanner and sharded Cloud SQL: pick Spanner when global strong consistency and horizontal scale are requirements; pick Cloud SQL when the workload is regional and relational features outweigh scale—the interview answer names the requirement first, the product second.

Chapter 5 (hands-on). SCD2 MERGE correctness: match on business key *and* `is_current = TRUE`; close the current row (`valid_to, is_current=false`) and insert the new version in one MERGE.

Chapter 13 (conceptual). At-least-once is not a bug to fix but a guarantee to design around: idempotent consumers plus a dedup horizon sized to the replay window.

Chapter 17 (design). Overfitting evidence: a large gap between ML.EVALUATE on training versus time-held-out data; remediation is simpler features and regularization, not more data of the same leaky kind.

Chapter 21 (hands-on). The negative perimeter test is the deliverable, not an afterthought: without a denied request, the perimeter is a hope.

Chapter 24 (architecture). Three-project mesh CI topology: one state prefix per project, applies ordered A (ingest) → C (governance) → B (analytics exposure), with cross-project outputs passed through remote state data sources—never hardcoded project numbers.

Professional Data Engineer Exam Mapping

This appendix maps the Google *Professional Data Engineer* certification onto the book's chapters, so the final study weeks can be run as a targeted loop rather than a reread. It expands the exam-preparation Friday of Chapter 24 [50].

F.1 Exam Domains Mapped to Chapters

The exam assesses design and operational judgment, not syntax recall. The official guide groups objectives into domains; the table below assigns each domain its chapters, the specific objectives the book covers, and the trap the domain sets for unprepared candidates.

Exam domain	Chapters	Objectives covered in this book
Designing data processing systems	1–4, 21–23	Storage class and location selection; relational vs. Spanner vs. Bigtable vs. BigQuery selection; migration and HA design; security and compliance baked into design
Ingesting and processing data	9–15	Batch (Dataproc, Data Fusion, Dataflow batch), CDC (Datastream), orchestration (Composer), messaging (Pub/Sub), streaming (Dataflow windowing, state, exactly-once)
Storing the data	1–8	Object storage lifecycle; warehouse partitioning, clustering, materialized views; external/federated storage; lakehouse (BigLake, Iceberg)
Preparing and using data for analysis	5–8, 16–17	SQL depth, modeling (grain, SCD2), BigLake governance, sharing (Analytics Hub), BI and semantic layers (Looker), in-warehouse ML (BQML)
Maintaining and automating data workloads	12, 18–20, 24	Idempotent orchestration, quality gates (assertions, AutoDQ), MLOps (pipelines, registry, monitoring), DataOps (Terraform, Dataform, CI/CD)
Security, compliance, governance	21–23	IAM least privilege, VPC Service Controls, CMEK/CSEK, Dataplex catalog/lineage/quality, DLP discovery and de-identification, erasure design

Typical trap per domain (from Chapter 24): self-managed answers where a managed service exists; batch answers to streaming requirements; ignoring lifecycle and partition cost levers;

skipping governance and semantics; solutions with no monitoring or CI/CD; IAM-only answers that ignore perimeters.

F.2 The Case Studies

The exam's case studies present fixed business scenarios. **EHR Healthcare** is a compliance-driven migration: the Chapter 11 CDC patterns, Chapter 21 perimeter, and Chapter 23 DLP designs answer most of its questions. **Helium Sports** is real-time analytics at scale: Chapters 13–16 streaming patterns. Read both case-study documents before the exam and annotate each requirement with the GCP service and this book's chapter that answers it; most case-study questions reduce to a selection decision already made hands-on.

F.3 Study Strategy

1. **Baseline mock (week -3).** One full-length practice exam, closed book, untimed but tracked. Score *by domain*, not in aggregate—a 60% in one domain matters more than an 82% average.
2. **Targeted rebuild loop (weeks -3 to -1).** For each weak domain, rebuild that week's Thursday lab from the solution sketches in the lab-solutions appendix, re-read the Friday use case, and re-answer the chapter's self-checks. Hands-on rebuilds beat rereading.
3. **Timed mock (week -1).** Second full mock under time pressure. The schedule-the-real-exam threshold: above 85% overall with no domain below 70%.
4. **Exam day heuristics.** Identify the stem's stated constraint (cost, latency, compliance, operational simplicity) first; eliminate the two answers that ignore it; prefer managed over self-managed, least-privilege over convenient, and the answer that includes monitoring over the one that merely works.

F.4 Self-Assessment Checklist

Tick a box only when you could perform the task hands-on, unprompted, in the console or CLI.

Design (Chapters 1–4, 21)

- ☐ Choose storage class, location, and lifecycle for a stated access pattern and budget.
- ☐ Choose among Cloud SQL, AlloyDB, Spanner, Bigtable, and BigQuery for a stated workload, naming the deciding requirement.
- ☐ Design migration (DMS, Datastream, Transfer Service) with rollback and validation.
- ☐ Sketch the security envelope (IAM, perimeter, CMEK) before the data design.

Ingestion and Processing (Chapters 9–15)

- ☐ Build and tear down an ephemeral Dataproc job; explain preemptible trade-offs.
- ☐ Author a Data Fusion pipeline with Wrangler directives and macros.
- ☐ Stand up a Datastream CDC pipeline with backfill and a contract test.
- ☐ Write an idempotent, sensor-triggered Composer DAG with an `all_done` notifier.
- ☐ Configure Pub/Sub schemas, dead-lettering, and replay.

- ☐ Implement windows, watermarks, triggers, and bounded state in Beam on Dataflow.

Storage and Analysis (Chapters 5–8, 16)

- ☐ Partition and cluster a table and prove the bytes-billed reduction.
- ☐ Decide on-demand versus Editions from measured slot usage.
- ☐ Model a star schema with declared grain and an SCD2 dimension.
- ☐ Govern lake files with BigLake policy tags and row access policies; share via Analytics Hub.
- ☐ Define one metric in a semantic layer and consume it in a dashboard.

ML and Operations (Chapters 17–20, 24)

- ☐ Train, evaluate, and serve a BQML model with leakage-free features.
- ☐ Deploy a Vertex model behind an endpoint with registry lineage and a rollback path.
- ☐ Gate automated retraining on drift evidence with quality preconditions.
- ☐ Wrap external AI API calls in batching, backoff, and dead-lettering, with cost forecast.
- ☐ Provision with Terraform from locked remote state; gate deploys through CI with Workload Identity Federation.

Security and Governance (Chapters 21–23)

- ☐ Enclose a project in a VPC SC perimeter and prove it with a negative test.
- ☐ Enforce column-level security with policy tags and row access policies, both negative-tested.
- ☐ Run DLP discovery and de-identification, including FPE tokenization with a KMS-wrapped key.
- ☐ Explain erasure design across time travel, backups, and replicas.
- ☐ Trace lineage from a dashboard tile to its source system in Dataplex.

Tip

If a box cannot be ticked honestly, that line *is* the study plan: find the chapter, rebuild the lab, and re-attempt the mock for that domain. The exam is passable on judgment built from these labs; it is not passable on vocabulary alone.

Interview Question Bank

This appendix consolidates the Interview Q&A sections of all twenty-five chapters into one categorized bank. Questions are grouped by domain and tagged by difficulty—[F] foundational (definitions, direct trade-offs), [I] intermediate (applied judgment), [A] advanced (design under constraints). Answers are deliberately abbreviated to recall cues; the chapter reference holds the full discussion. Practice aloud: the exam and the interview both reward candidates who name the requirement before the product.

G.1 Storage and Databases (Chapters 1–4)

1. [I] **Regional versus multi-region bucket?** Regional when compute and residency are regional; wider geography only for a concrete availability/access requirement. (Ch. 1)
2. [I] **Why can Archive cost more than Standard?** Retrieval fees and minimum duration dominate when data is frequently restored or reprocessed. (Ch. 1)
3. [F] **Versioning versus retention versus soft delete?** Versioning preserves generations; retention prevents premature deletion; soft delete gives a recovery window. Related but distinct failure modes. (Ch. 1)
4. [A] **Validate a petabyte-scale transfer?** Bytes, object counts, sampled checksums, metadata, prefix coverage, logs, downstream readability—plus incremental sync for changed sources. (Ch. 1)
5. [I] **Cloud SQL regional HA versus read replica?** HA standby fails over and serves no reads; replicas serve reads asynchronously but may lag and are not HA. (Ch. 2)
6. [I] **Why private IP for migrations?** Keeps traffic off public addresses, shrinks attack surface, fits compliance—still requires IAM, firewall, and role controls. (Ch. 2)
7. [F] **Main risk of reading replicas?** Lag; read-after-write workflows belong on the primary or need explicit consistency checks. (Ch. 2)
8. [I] **AlloyDB over Cloud SQL for PostgreSQL?** When compatibility plus higher performance, read pools, or columnar acceleration for operational analytics is needed. (Ch. 2)
9. [F] **What problem does TrueTime solve?** Bounded time uncertainty enabling commit wait and external consistency for global transactions. (Ch. 3)
10. [I] **Why do monotonically increasing keys hurt Spanner?** Writes concentrate at one key-range end—a hot split; use higher-cardinality or hashed leading keys. (Ch. 3)

11. **[I] When interleave a child table?** When children are read with the parent and share its lifecycle; avoid for independent scans or giant parents. (Ch. 3)
12. **[F] Spanner versus Cloud SQL replicas?** Synchronous quorum replication with strong consistency versus asynchronous read copies of a primary. (Ch. 3)
13. **[A] Evidence before adding a Spanner index?** Query text, plan, rows scanned vs returned, latency, CPU, lock wait, existing index usage; add the smallest index that fixes the measured path. (Ch. 3)
14. **[F] Why is row-key design the key Bigtable decision?** It fixes sort order, tablet ranges, scan locality, and load distribution. (Ch. 4)
15. **[F] What is a tablet?** A contiguous row-key range Bigtable splits, moves, and serves independently. (Ch. 4)
16. **[I] Why reversed timestamps in time-series keys?** Newest events first, so recent-history reads stop after a small range scan. (Ch. 4)
17. **[I] Downside of salting row keys?** Writes spread, but entity reads fan out across buckets and must merge. (Ch. 4)
18. **[I] What do app profiles provide?** Traffic routing to specific clusters or multi-cluster policies—workload isolation and locality. (Ch. 4)
19. **[I] When is Bigtable a poor fit?** Ad hoc joins, relational constraints, small OLTP, unpredictable access paths. (Ch. 4)

G.2 Warehousing and Analytics (Chapters 5–8, 16)

1. **[F] What makes BigQuery storage and compute independently scalable?** Colossus stores columnar data; stateless Dremel workers compute; Jupiter bandwidth removes co-location needs. (Ch. 5)
2. **[I] When do Editions beat on-demand?** When steady slot usage makes slot-hours cheaper than per-TiB scanning—continuous ELT and BI with measurable volume. (Ch. 5)
3. **[F] What does declaring a fact table’s grain commit you to?** Every row means one thing at one detail level; all measures and keys must respect it or results double-count. (Ch. 5)
4. **[I] SCD Type 2 versus Type 1?** Type 2 when history must be reconstructed—regulatory reporting, point-in-time joins, “what we knew when.” (Ch. 5)
5. **[A] Why must an SCD2 MERGE match on key and is_current?** Key alone updates every historical version, destroying the timeline. (Ch. 5)
6. **[I] Partitioning versus clustering for bytes billed?** Partitioning skips whole segments; clustering skips blocks inside partitions via sorted-column statistics, even on high-cardinality columns. (Ch. 6)
7. **[I] When does clustering beat partitioning?** High-cardinality or non-temporal filter columns, or a second selective axis on an already date-partitioned table. (Ch. 6)
8. **[F] What does a materialized view cost?** Stored bytes plus background refresh slots, and SQL restrictions—in exchange for incremental precomputation. (Ch. 6)
9. **[A] Where do you spot a skewed stage?** Per-slot output rows or slot time dominated by few workers on a join/group-by over a hot key. (Ch. 6)

10. **[I] When is BI Engine wasted?** On batch ELT and one-off forensics; it accelerates repeated interactive dashboard queries. (Ch. 6)
11. **[F] External versus native BigQuery table?** External reads source files at query time—no clustering or managed statistics; native offers pruning, caching, predictable performance. (Ch. 7)
12. **[F] Which function tests point-in-polygon?** `ST_WITHIN(point, polygon)` (or `ST_CONTAINS(polygon, point)`). (Ch. 7)
13. **[I] What cost does BigQuery Omni add?** Source-cloud egress and cross-cloud latency on moved bytes. (Ch. 7)
14. **[I] Why no clustering on external tables?** BigQuery does not manage the file layout, so no block-level statistics exist to prune. (Ch. 7)
15. **[I] Federated query versus ingestion?** Federate exploratory, low-frequency questions over data you do not own; ingest when query repetition justifies the pipeline. (Ch. 7)
16. **[A] How does BigLake secure lake files?** Access flows through a governed connection and BigQuery's engine, so policy tags and row access policies enforce at query time on files. (Ch. 8)
17. **[F] What can an Object Table index?** Unstructured objects—PDFs, images, audio—as rows with metadata and signed URLs, no shared tabular schema required. (Ch. 8)
18. **[I] How does Analytics Hub share without copying?** Subscribers get read-only linked datasets referencing live publisher tables; revoking the listing revokes access. (Ch. 8)
19. **[I] What does a clean room prevent?** Each party seeing the other's row-level records; only rule-satisfying analyses (aggregation thresholds, join-only columns) run. (Ch. 8)
20. **[I] Policy tags versus per-role views?** Tags attach enforcement to the column across every query path; views multiply, drift, and are bypassed by direct table access. (Ch. 8)
21. **[I] Why does a LookML layer prevent conflicting metrics?** Measures are defined once in versioned code and inherited by every explore, dashboard, and API call. (Ch. 16)
22. **[I] What does reverse ETL solve that dashboards cannot?** It delivers computed decisions into operational systems where action happens; dashboards only inform. (Ch. 16)
23. **[A] What breaks in a sync without identity resolution?** Warehouse keys mismatch CRM identifiers—green pipeline, wrong people. (Ch. 16)
24. **[I] Reverse-ETL frequency trade-off?** Freshness versus API rate limits and cost; hourly deltas are the usual optimum. (Ch. 16)
25. **[I] Looker Core versus Looker Studio?** Core when metrics must be governed across teams; Studio for velocity and departmental self-service. (Ch. 16)

G.3 Batch Processing and Orchestration (Chapters 9–12)

1. **[I] Why ephemeral Dataproc clusters?** Bill only for execution, exact per-job configuration, no snowflake drift or shared-cluster contention. (Ch. 9)
2. **[I] Preemptible VM trade-off?** Up to roughly 80% cheaper but reclaimable at any time; secondary workers only, with retry-safe jobs. (Ch. 9)

3. **[I] Why migrate HDFS to Cloud Storage?** Decoupled storage and compute, one shared copy for many engines, durability without three-way replication. (Ch. 9)
4. **[A] Property behind executor OOM?** `spark.executor.memory` too small for the largest (often skewed) partition—but the real fix is usually skew, not memory. (Ch. 9)
5. **[F] How do Dataproc autoscaling policies decide?** YARN metrics—pending versus available memory/containers—within configured bounds and cooldowns. (Ch. 9)
6. **[F] What executes Data Fusion pipelines?** CDAP applications compiled to Spark on ephemeral Dataproc clusters provisioned per run. (Ch. 10)
7. **[F] Wrangler’s role?** Iteratively define and validate cleaning directives on a sample; they execute at full scale inside the pipeline. (Ch. 10)
8. **[I] Data Fusion versus Dataflow?** Fusion when builders are analysts with standard sources and row-level logic; Dataflow for custom streaming or stateful processing. (Ch. 10)
9. **[I] Why a private-IP Data Fusion instance?** To reach on-prem/internal sources over private networking—required in regulated environments. (Ch. 10)
10. **[F] What do macros parameterize?** Runtime values—logical date, paths, environment settings—so one definition serves schedules, backfills, reruns. (Ch. 10)
11. **[F] How does Datastream capture changes without querying tables?** It reads the transaction log (WAL/binlog/redo) maintained for recovery. (Ch. 11)
12. **[I] Nullable column add versus rename—which breaks contracts?** Renaming; a nullable addition is backward compatible. (Ch. 11)
13. **[F] What does BigQuery schema enforcement reject on load?** Rows with unknown fields or type mismatches—the load is the contract enforcement point. (Ch. 11)
14. **[I] Who owns a data contract?** Both sides: producers version schemas with deprecation discipline; consumers declare and validate dependencies. (Ch. 11)
15. **[I] What does a freshness metric reveal about CDC?** How far behind real time the destination is; growth signals a stalled consumer or source outage before dashboards go stale. (Ch. 11)
16. **[I] Why must Airflow tasks be idempotent?** Retries, backfills, and clears rerun freely; only deterministic overwrites/upserts make reruns indistinguishable. (Ch. 12)
17. **[F] What does `all_done` guarantee?** The notifier runs after all upstreams regardless of outcome—exactly-once notification of every run. (Ch. 12)
18. **[F] What runs underneath Composer 2?** An autoscaling GKE cluster (scheduler, workers, web server, triggerer), a managed Cloud SQL metadata DB, and a synced GCS DAG bucket. (Ch. 12)
19. **[I] GCS sensor versus schedule?** Sensor when readiness is event-driven—gate on the artifact, not the clock. (Ch. 12)
20. **[I] What belongs in XComs?** Small control values (paths, counts, verdicts); never payloads or DataFrames—those live in storage with a passed reference. (Ch. 12)

G.4 Streaming and Real-Time (Chapters 13–15)

1. **[F] Message exceeding max delivery attempts?** Forwarded to the dead-letter topic with attempt metadata; a triage process owns it—never silently deleted. (Ch. 13)
2. **[I] How does a topic schema enforce contracts?** The broker validates each published message against the Avro/Proto schema and rejects violations before subscribers see them. (Ch. 13)
3. **[I] Why at-least-once, and how to dedupe?** Acks can be lost after processing; dedupe via message IDs, idempotent consumers, exactly-once sinks, or Dataflow exactly-once mode. (Ch. 13)
4. **[I] When does Pub/Sub Lite win on cost?** High steady throughput where provisioned zonal capacity beats per-GiB elastic pricing and the team can operate partitions. (Ch. 13)
5. **[F] What does oldest-unacked-message-age warn about?** Consumer lag or poison messages never acked and heading for the DLQ. (Ch. 13)
6. **[F] PTransform versus ParDo?** PTransform is any graph node over PCollections; ParDo is the element-wise transform applying your DoFn. (Ch. 14)
7. **[I] Why unit-test Beam locally first?** PAssert proves transform semantics and TestStream proves windowing in minutes; production failures cost money and corrections. (Ch. 14)
8. **[F] What does Streaming Engine move off workers?** Shuffle and per-key state into the service backend—workers become stateless and autoscale faster. (Ch. 14)
9. **[I] When package a Flex Template?** When operators launch parameterized jobs without a dev environment, or graph shape depends on launch parameters. (Ch. 14)
10. **[I] How does Dataflow autoscaling react to backlog?** Adds workers up to `max_num_workers` on backlog/utilization, removes them as the backlog drains—reactive, minutes of ramp. (Ch. 14)
11. **[F] What does a watermark estimate?** Event-time completeness: all events up to time t believed seen; late data arrives behind it. (Ch. 15)
12. **[A] Why does unbounded state kill a streaming job?** State accumulates until storage exhausts; expiry timers and watermark GC bound its lifetime. (Ch. 15)
13. **[A] In a dedup DoFn, what do SEEN state and EXPIRY timer do?** SEEN remembers emitted event IDs per key; EXPIRY clears them after ten minutes of event time. (Ch. 15)
14. **[A] What must align for a stream-stream join to emit?** Matching keys in the same window on both inputs; each side buffers per key per window until closing plus allowed lateness. (Ch. 15)
15. **[A] Exactly-once versus at-least-once plus dedup?** Exactly-once shuffle plus idempotent sinks is simplest but costlier; dedup designs win when a natural key exists and consumers tolerate a merge. (Ch. 15)

G.5 Machine Learning and MLOps (Chapters 17–20)

1. **[F] BQML model for churn?** Logistic regression as interpretable baseline; boosted trees when nonlinear interactions justify opacity. (Ch. 17)

2. [I] **What does ARIMA_PLUS automate?** Differencing, seasonality, holidays, hyperparameter search, and per-series parallelism via `time_series_id_col`. (Ch. 17)
3. [I] **Metrics for imbalanced churn?** ROC AUC plus precision/recall at the chosen threshold; accuracy misleads. (Ch. 17)
4. [I] **Advantage of training where data lives?** No extract pipelines, one permission model, full lineage, retraining as scheduled SQL. (Ch. 17)
5. [A] **Detecting overfitting in BQML?** Compare `ML.EVALUATE` on training versus time-held-out data; a large gap is memorization. (Ch. 17)
6. [I] **AutoML versus custom training?** AutoML for standard tabular problems under time/skill constraints; custom must measurably beat the AutoML baseline. (Ch. 18)
7. [F] **What does the Model Registry version?** Artifacts with framework, metrics, dataset/code lineage, labels, aliases—promotion is an alias move, rollback trivial. (Ch. 18)
8. [F] **Online versus batch prediction for checkout?** Online endpoints (millisecond per-request); batch prediction for nightly offline scoring. (Ch. 18)
9. [I] **What do feature attributions tell consumers?** Which features drove this prediction—reason codes and debugging, not causal proof. (Ch. 18)
10. [I] **Why deploy through an endpoint?** Endpoints add scaling, traffic splitting, monitoring, IAM; consumers integrate once, not per model version. (Ch. 18)
11. [I] **What problem does a Feature Store solve?** Training-serving skew and leakage: one definition feeds point-in-time training sets and online lookups. (Ch. 19)
12. [F] **Online versus offline serving for a recommender?** Online: low-latency latest-value lookups at request time; offline builds training datasets. (Ch. 19)
13. [F] **What does a Kubeflow component encapsulate?** A containerized step with typed inputs, outputs, artifacts—reusable, versioned, lineage-tracked. (Ch. 19)
14. [A] **What should gate automated retraining?** Preconditioned drift alerts (freshness/quality green first), and the candidate must beat the incumbent on evaluation. (Ch. 19)
15. [A] **Prediction skew versus drift?** Skew is a present served/trained feature mismatch (pipeline bug); drift is the world moving away from training data (freshness problem). (Ch. 19)
16. [I] **Dataflow calling an AI API versus BQML?** APIs for generic unstructured tasks at moderate volume; BQML for tabular problems on data already in BigQuery. (Ch. 20)
17. [I] **Rate-limit handling around external APIs?** Batching, bounded concurrency, exponential backoff with jitter, dead-lettering, throttling at hard ceilings. (Ch. 20)
18. [F] **Which API extracts entities and sentiment?** Natural Language API: `analyzeEntities`, `analyzeSentiment`, `analyzeEntitySentiment`. (Ch. 20)
19. [I] **Making OCR output searchable?** Land text keyed by object URI in BigQuery and build a search index for `SEARCH` queries. (Ch. 20)
20. [A] **Cost risk of per-call AI APIs in streaming?** Cost scales with throughput and retries, invisibly until the bill—forecast and alert on both. (Ch. 20)

G.6 Security, Governance, and Compliance (Chapters 21–23)

1. **[I] What does a VPC SC perimeter block that IAM cannot?** Requests from unauthorized networks or to unauthorized destinations—even with valid credentials. IAM authenticates; the perimeter constrains context. (Ch. 21)
2. **[I] Why Workload Identity over service-account keys?** Keys are long-lived bearer files that exfiltrate and audit poorly; Workload Identity issues short-lived tokens with no key material. (Ch. 21)
3. **[F] CMEK versus CSEK?** CMEK keys live in your Cloud KMS under grant and audit; CSEK material is supplied per request, never persisted—lose it, lose the data. (Ch. 21)
4. **[I] How do access levels modify perimeters?** They carve conditional entry (IP, device state, identity) into the boundary for otherwise-denied requests. (Ch. 21)
5. **[I] Why test perimeters with a denied query?** Security controls fail silently toward openness; only a denial proves the control exists. (Ch. 21)
6. **[F] Raw versus curated zone in Dataplex?** Trust and stage: raw holds source-fidelity data with narrow write access; curated holds modeled products with broader reads—each with its own policy posture. (Ch. 22)
7. **[F] What does a tag template attach?** Structured business metadata (owner, classification, retention) as searchable fields—distinct from policy tags, which enforce access. (Ch. 22)
8. **[I] What does AutoDQ check without authored rules?** Profiling statistics and suggested rules; declared completeness/uniqueness/freshness rules remain yours to own. (Ch. 22)
9. **[I] Who owns a data product in a mesh?** The domain—schema, quality, docs, SLAs; the central platform owns federated guardrails. (Ch. 22)
10. **[I] How does lineage help when a dashboard breaks?** Impact analysis runs upstream to the responsible source and owner—triage becomes a lookup. (Ch. 22)
11. **[F] What is an InfoType? Name two.** A sensitive-data category DLP recognizes—for example EMAIL_ADDRESS, CREDIT_CARD_NUMBER—via built-in, regex, or dictionary detectors. (Ch. 23)
12. **[I] Masking versus tokenization?** Tokenization is reversible—only by identities holding KMS decrypt permission, under audit; masking is irreversible by design. (Ch. 23)
13. **[I] Why a KMS-wrapped key for tokenization?** Key material is only handled encrypted; KMS unwraps under IAM control with per-use audit logging. (Ch. 23)
14. **[I] What does scheduled DLP scanning detect?** PII presence, new sensitive columns, and coverage gaps—findings land in BigQuery for dashboards and alerts. (Ch. 23)
15. **[A] How do you verify de-identified data is unreadable?** Negative detokenization tests, quasi-identifier risk analysis, and canary-subject game days proving per-layer unrecoverability. (Ch. 23)

G.7 DataOps and Career (Chapter 24)

1. **[I] Why must Terraform state live remotely for teams?** Local state diverges, causing conflicting applies and untracked resources; a locked GCS backend serializes change

and makes state auditable. (Ch. 24)

2. **[F] What does a Dataform assertion guarantee?** No row violates the assertion query at run time—failures block the run and dependents: quality as a gate, not a report. (Ch. 24)
3. **[I] What runs on every pull request?** Plan (plus Dataform compile and assertions); apply belongs post-merge behind approval—PRs propose, merges deploy. (Ch. 24)
4. **[I] Why is Workload Identity Federation safer than committed keys?** No long-lived secret exists to leak; short-lived tokens bound to workload identity, fully audited. (Ch. 24)
5. **[F] How do SQLX `ref()` calls order execution?** They build the dependency DAG; Dataform runs models in topological order—orchestration derived from the SQL. (Ch. 24)

G.8 Study Method for the Bank

Work domain by domain against the self-assessment checklist in the exam-mapping appendix. For each question, answer aloud in three sentences—requirement, choice, trade-off—then check the chapter reference. Tag any question you cannot answer fluently, rebuild that chapter’s Thursday lab from the lab-solutions appendix, and retest after forty-eight hours. Spaced, spoken recall plus one rebuild is worth several silent rereads.

Tip

In live interviews, the highest-scoring follow-up to any answer here is a cost or failure story from your own labs: “I hit this in the Chapter 13 project when...” Keep one such story per domain ready.

Glossary

This glossary grows with each chapter and collects the cross-cloud data-engineering terms introduced so far. Definitions are platform-neutral unless a platform is named explicitly.

Access tier A storage pricing class that trades retrieval cost and latency for lower at-rest price (for example Azure Hot, Cool, Cold, and Archive). See also *storage class*.

AlloyDB Google Cloud’s PostgreSQL-compatible engine with a columnar accelerator for analytical queries, positioned as the cloud-native tier above Cloud SQL.

Apache Spark A distributed data-processing engine; Microsoft Fabric runs it on Starter or Custom pools, with counterparts in AWS Glue/EMR, Azure Synapse/Databricks, and GCP Dataproc.

Aurora Amazon’s cloud-native relational engine with distributed storage, Global Databases, and a serverless (v2) compute option.

Availability SLA The contractual uptime a provider guarantees for a service, distinct from *durability*, which concerns data loss rather than reachability.

Bigtable Google Cloud’s wide-column, high-throughput, low-latency NoSQL store for time-series, IoT, and ad-tech workloads; exposes the HBase API and scales by nodes and clusters.

Bucket (storage) The top-level container for objects in AWS S3 and Google Cloud Storage; Azure calls the equivalent a *container*.

Bucket Pattern A MongoDB schema-design pattern that groups many small time-series readings into a single document to cut document count and metadata overhead.

Capped collection A fixed-size MongoDB collection that overwrites its oldest documents when full, useful for high-volume logs and bounded activity feeds.

Change Feed / Streams An ordered log of item-level changes used for event-driven processing; called Change Feed in Cosmos DB, DynamoDB Streams in DynamoDB, and change streams in MongoDB.

Checkpoint In WiredTiger, a periodic flush of a consistent snapshot of cached data to disk, bounding crash-recovery time together with the journal.

Clustering key A user-defined column set that Snowflake uses to co-locate related rows within micro-partitions, improving partition pruning on selective queries.

CMEK / CMK Customer-managed encryption keys (GCP CMEK; Azure customer-managed keys via Key Vault), giving the customer control over the key that encrypts data at rest.

Cloud Spanner Google Cloud’s horizontally scalable, globally distributed relational database offering external (strong) consistency backed by TrueTime.

- Consistency level** A tunable guarantee about read recency; Azure Cosmos DB offers five (Strong, Bounded Staleness, Session, Consistent Prefix, Eventual).
- Cosmos DB** Azure’s globally distributed multi-model NoSQL database, billed in Request Units (RU) and supporting multiple APIs (NoSQL, MongoDB, Cassandra).
- Cross-Region Replication (CRR)** Asynchronous copying of objects to a bucket in another region for disaster recovery; Azure’s equivalent user-configured feature is *object replication*.
- DynamoDB** Amazon’s managed key-value and document NoSQL database, with capacity measured in read/write capacity units (RCU/WCU) or on-demand, and Global Tables for multi-region.
- Durability** The probability that stored data is not lost, commonly quoted as “eleven nines” (99.999999999%) for cloud object storage.
- ElastiCache** Amazon’s managed in-memory cache (Redis- and Memcached-compatible) placed in front of a durable store to serve hot data at sub-millisecond latency; Azure’s analog is Cache for Redis and GCP’s is Memorystore.
- Embedding vs referencing** The MongoDB modeling choice between nesting related data in one document (embedding) or linking documents by reference (and joining with `lookup`).
- Fail-safe** Snowflake’s non-configurable seven-day recovery window, available only to Snowflake support, that follows the Time Travel retention period as a last-resort safety net.
- Global Secondary Index (GSI)** A DynamoDB index with its own partition/sort key enabling queries on non-primary attributes; a Local Secondary Index (LSI) shares the table’s partition key.
- Hierarchical namespace (HNS)** The Azure Data Lake Storage Gen2 feature that adds true directories and atomic directory operations on top of Blob storage.
- Journal (WAL)** The write-ahead log that records changes before they are checkpointed, enabling crash recovery in MongoDB’s WiredTiger engine.
- JSON Schema validation** MongoDB’s rule mechanism (via `jsonSchema` in `createCollection`) that enforces required fields, data types, and value constraints.
- Interleaved table** A Cloud Spanner layout that physically co-locates child rows with their parent row to make parent-child joins fast.
- Lakehouse** An architecture that stores open table formats (such as Delta) in a data lake while offering warehouse-like SQL and governance; the core storage abstraction in Microsoft Fabric.
- Lifecycle policy** A rule set that automatically transitions or expires objects across storage tiers based on age or other conditions.
- Medallion architecture** A progressive-refinement data layout with Bronze (raw), Silver (validated), and Gold (curated/enriched) layers.
- Materialized view** A precomputed, automatically maintained query result stored for fast reads, trading maintenance/storage cost for lower query latency (Snowflake, Redshift, Synapse, BigQuery).
- Memorystore** Google Cloud’s managed in-memory cache service (Redis and Memcached), the GCP counterpart to AWS ElastiCache and Azure Cache for Redis.

- Micro-partition** Snowflake's internal storage unit: an immutable, compressed, columnar file of roughly 50–500 MB of uncompressed data, annotated with per-column metadata for pruning.
- Multi-AZ** An AWS deployment spanning multiple Availability Zones for high availability; Azure's analog is zone-redundant HA and GCP's is regional HA.
- Multi-cluster warehouse** A Snowflake virtual warehouse that adds and removes compute clusters automatically to absorb concurrent query load.
- Object Lock** The AWS S3 WORM feature (governance mode, compliance mode, and legal hold) that prevents object deletion or overwrite for a retention period.
- OneLake** Microsoft Fabric's single, tenant-wide logical data lake shared by all workspaces.
- Partition pruning** Skipping storage units (micro-partitions, files, or partitions) that cannot match a query's predicates, reducing scanned data.
- Point-in-time recovery (PITR)** Restoring a database or table to any moment within a retention window, using continuous backups or transaction logs; realized as Snowflake Time Travel, RDS/SQL Database PITR, and Delta Lake time travel.
- Read replica** A read-only copy of a database that offloads read traffic and can serve as a failover or geo-local endpoint.
- Relational Migrator** MongoDB's tool for migrating relational schemas and data to document collections, applying embed-versus-reference decisions from schema-design patterns.
- Request Unit (RU)** Azure Cosmos DB's normalized currency for throughput, abstracting the CPU, memory, and IOPS consumed by a database operation.
- Search Optimization Service (SOS)** A Snowflake feature that builds a persistent search access path to accelerate selective point-lookup queries on high-cardinality columns.
- Shortcut (Fabric)** A OneLake reference that virtualizes external data (S3, ADLS Gen2, GCS) in place so it can be queried without copying.
- Storage class** The AWS/GCP term for a pricing tier of object storage (for example S3 Standard, Standard-IA, Glacier; GCP Standard, Nearline, Coldline, Archive).
- SSE-KMS** Server-side encryption in AWS S3 using keys managed in AWS Key Management Service.
- Time Travel** Snowflake's ability to query, clone, or restore data as it existed at an earlier point within a retention period (up to 90 days on higher editions); Delta Lake offers an analogous `VERSION AS OF / TIMESTAMP AS OF` capability.
- Transient table** A Snowflake table that persists across sessions but has no Fail-safe period, reducing storage cost for reproducible staging data; a temporary table lives only for one session.
- TrueTime** Google's globally synchronized clock (with bounded uncertainty) that lets Cloud Spanner assign commit timestamps and provide external consistency.
- TTL (Time to Live)** An expiry mechanism that automatically deletes items or documents after a set time (DynamoDB TTL, Cosmos DB TTL, MongoDB TTL index).
- Versioning** Retaining prior versions of an object or blob so that overwrites and deletes are recoverable.

Virtual warehouse Snowflake's unit of elastic compute, sized XS–6XL and billed in credits, fully decoupled from storage.

Wide-column store A NoSQL data model that stores rows as sparse, dynamically wide sets of column families keyed by a row key (Bigtable, HBase, Cassandra), optimized for high write throughput and range scans.

WiredTiger MongoDB's default storage engine, using a configurable internal cache alongside the operating-system page cache, with journaling and checkpointing.

WORM Write-once-read-many storage that prevents modification or deletion for a retention period; implemented as Object Lock (AWS), immutable blob storage (Azure), or Bucket Lock (GCP).

Z-ordering A Delta Lake optimization that co-locates related information in the same files by multi-dimensional clustering, improving data skipping; applied via `OPTIMIZE ... ZORDER BY` and complemented by `VACUUM` to reclaim obsolete files.

Zero-copy clone Snowflake's instant, storage-free copy of a table, schema, or database that shares underlying micro-partitions until either copy is modified (copy-on-write).

Bibliography

- [1] Tyler Akidau, Robert Bradshaw, Craig Chambers, Slava Chernyak, Rafael J. Fernández-Moctezuma, Reuven Lax, Sam McVeety, Daniel Mills, Frances Perry, Eric Schmidt, and Sam Whittle. The dataflow model: A practical approach to balancing correctness, latency, and cost in massive-scale, unbounded, out-of-order data processing. In *Proceedings of the VLDB Endowment*, volume 8, pages 1792–1803, 2015.
- [2] Fay Chang, Jeffrey Dean, Sanjay Ghemawat, Wilson C. Hsieh, Deborah A. Wallach, Mike Burrows, Tushar Chandra, Andrew Fikes, and Robert E. Gruber. Bigtable: A distributed storage system for structured data. In *ACM Transactions on Computer Systems*, volume 26, pages 1–26, 2008.
- [3] James C. Corbett, Jeffrey Dean, Michael Epstein, Andrew Fikes, Christopher Frost, J. J. Furman, Sanjay Ghemawat, Andrey Gubarev, Christopher Heiser, Peter Hochschild, et al. Spanner: Google’s globally-distributed database. In *Proceedings of the 10th USENIX Symposium on Operating Systems Design and Implementation*, pages 251–264, 2012.
- [4] Sanjay Ghemawat, Howard Gobioff, and Shun-Tak Leung. The google file system. *ACM SIGOPS Operating Systems Review*, 37(5):29–43, 2003.
- [5] Google Cloud. Alloydb ai documentation. <https://cloud.google.com/alloydb/docs/ai>, 2026. AlloyDB AI, vector search, and model integration documentation.
- [6] Google Cloud. Alloydb columnar engine documentation. <https://cloud.google.com/alloydb/docs/columnar-engine>, 2026. Columnar engine concepts and query acceleration guidance for AlloyDB.
- [7] Google Cloud. Alloydb for postgresql documentation. <https://cloud.google.com/alloydb/docs>, 2026. AlloyDB clusters, instances, backups, high availability, read pools, and PostgreSQL compatibility.
- [8] Google Cloud. Bi engine documentation. <https://cloud.google.com/bi-engine/docs>, 2026. In-memory analysis acceleration for BigQuery dashboards.
- [9] Google Cloud. Bigquery documentation. <https://cloud.google.com/bigquery/docs>, 2026. Serverless data warehouse documentation.
- [10] Google Cloud. Bigquery omni documentation. <https://cloud.google.com/bigquery/docs/omni-introduction>, 2026. Cross-cloud analytics over AWS and Azure data without movement.
- [11] Google Cloud. Bigquery pricing. <https://cloud.google.com/bigquery/pricing>, 2026. On-demand and capacity (Editions) pricing, slot commitments, and storage pricing.
- [12] Google Cloud. Cloud bigtable documentation. <https://cloud.google.com/bigtable/docs>, 2026. Cloud Bigtable product documentation for instances, clusters, tables, replication, backups, and operations.

- [13] Google Cloud. Cloud bigtable performance. <https://cloud.google.com/bigtable/docs/performance>, 2026. Performance guidance for nodes, workloads, latency, throughput, and client behavior.
- [14] Google Cloud. Cloud bigtable schema design. <https://cloud.google.com/bigtable/docs/schema-design>, 2026. Guidance for row keys, column families, time-series schemas, and avoiding hotspots.
- [15] Google Cloud. Cloud billing documentation. <https://cloud.google.com/billing/docs>, 2026. Billing accounts, budgets, alerts, billing export, and cost controls.
- [16] Google Cloud. Cloud build documentation. <https://cloud.google.com/build/docs>, 2026. CI/CD build triggers, build configurations, and Workload Identity Federation.
- [17] Google Cloud. Cloud composer documentation. <https://cloud.google.com/composer/docs>, 2026. Managed Apache Airflow environments, architecture, and operations.
- [18] Google Cloud. Cloud data fusion documentation. <https://cloud.google.com/data-fusion/docs>, 2026. Visual ETL, Wrangler, plugins, and pipeline scheduling documentation.
- [19] Google Cloud. Cloud dataproc documentation. <https://cloud.google.com/dataproc/docs>, 2026. Managed Hadoop and Spark clusters, serverless Spark, and autoscaling policies.
- [20] Google Cloud. Cloud kms documentation. <https://cloud.google.com/kms/docs>, 2026. Key rings, CMEK, key rotation, and key destruction workflows.
- [21] Google Cloud. Cloud monitoring documentation. <https://cloud.google.com/monitoring/docs>, 2026. Metrics, dashboards, alerting policies, and uptime checks.
- [22] Google Cloud. Cloud scheduler documentation. <https://cloud.google.com/scheduler/docs>, 2026. Cron-based managed job scheduling for HTTP and Pub/Sub targets.
- [23] Google Cloud. Cloud spanner documentation. <https://cloud.google.com/spanner/docs>, 2026. Cloud Spanner product documentation for instances, databases, schemas, transactions, backups, and operations.
- [24] Google Cloud. Cloud spanner query execution plans. <https://cloud.google.com/spanner/docs/query-execution-plans>, 2026. Documentation for reading query plans and understanding optimizer behavior.
- [25] Google Cloud. Cloud spanner schema design best practices. <https://cloud.google.com/spanner/docs/schema-design>, 2026. Guidance for primary keys, interleaved tables, indexes, and avoiding hot spots.
- [26] Google Cloud. Cloud sql documentation. <https://cloud.google.com/sql/docs>, 2026. Managed PostgreSQL, MySQL, and SQL Server documentation for instances, high availability, backups, replicas, and operations.
- [27] Google Cloud. Cloud sql private ip documentation. <https://cloud.google.com/sql/docs/private-ip>, 2026. Private services access and private IP connectivity guidance for Cloud SQL.
- [28] Google Cloud. Cloud storage documentation. <https://cloud.google.com/storage/docs>, 2026. Product documentation for buckets, objects, locations, classes, lifecycle management, retention, and transfer options.

- [29] Google Cloud. Database migration service documentation. <https://cloud.google.com/database-migration/docs>, 2026. Connection profiles, migration jobs, continuous replication, validation, and cutover guidance.
- [30] Google Cloud. Dataflow documentation. <https://cloud.google.com/dataflow/docs>, 2026. Apache Beam managed service documentation.
- [31] Google Cloud. Dataform documentation. <https://cloud.google.com/dataform/docs>, 2026. SQLX models, assertions, dependency trees, and scheduled runs.
- [32] Google Cloud. Dataplex documentation. <https://cloud.google.com/dataplex/docs>, 2026. Governance, metadata, data quality, and lake management documentation.
- [33] Google Cloud. Datastream documentation. <https://cloud.google.com/datastream/docs>, 2026. Change data capture sources, destinations, and schema drift handling.
- [34] Google Cloud. Google cloud ai apis. <https://cloud.google.com/products/ai>, 2026. Vision, Video Intelligence, Natural Language, and Translation API documentation.
- [35] Google Cloud. Google cloud architecture framework. <https://cloud.google.com/architecture/framework>, 2026. Reliability, security, operational excellence, performance, and cost guidance.
- [36] Google Cloud. Google cloud cli documentation. <https://cloud.google.com/sdk/docs>, 2026. Installation, initialization, authentication, and command reference.
- [37] Google Cloud. Google cloud free program. <https://cloud.google.com/free/docs/free-cloud-features>, 2026. Free trial, credits, Always Free products, and account upgrade behavior.
- [38] Google Cloud. Identity and access management documentation. <https://cloud.google.com/iam/docs>, 2026. Cloud IAM roles, service accounts, policies, and least-privilege guidance.
- [39] Google Cloud. Introduction to analytics hub. <https://cloud.google.com/bigquery/docs/analytics-hub-introduction>, 2026. Governed data sharing, listings, and data clean rooms.
- [40] Google Cloud. Introduction to biglake tables. <https://cloud.google.com/bigquery/docs/biglake-intro>, 2026. BigLake tables, object tables, and fine-grained lake security.
- [41] Google Cloud. Introduction to bigquery ml. <https://cloud.google.com/bigquery/docs/bqml-introduction>, 2026. SQL-based model training, evaluation, forecasting, and anomaly detection.
- [42] Google Cloud. Introduction to clustered tables. <https://cloud.google.com/bigquery/docs/clustered-tables>, 2026. Clustering guidance for selective filters and bytes-billed reduction.
- [43] Google Cloud. Introduction to geospatial analytics. <https://cloud.google.com/bigquery/docs/geospatial-intro>, 2026. BigQuery GEOGRAPHY type and geospatial function documentation.
- [44] Google Cloud. Introduction to materialized views. <https://cloud.google.com/bigquery/docs/materialized-views-intro>, 2026. Materialized view capabilities, restrictions, and smart tuning.

- [45] Google Cloud. Introduction to partitioned tables. <https://cloud.google.com/bigquery/docs/partitioned-tables>, 2026. Ingestion-time, time-unit column, and integer-range partitioning guidance.
- [46] Google Cloud. Key visualizer for cloud bigtable. <https://cloud.google.com/bigtable/docs/keyvis-overview>, 2026. Documentation for diagnosing Bigtable row-key access patterns, hot ranges, and heatmaps.
- [47] Google Cloud. Key visualizer for cloud spanner. <https://cloud.google.com/spanner/docs/key-visualizer>, 2026. Documentation for diagnosing key-range traffic, hot spots, and access patterns.
- [48] Google Cloud. Looker documentation. <https://cloud.google.com/looker/docs>, 2026. LookML, explores, dashboards, and governed BI documentation.
- [49] Google Cloud. Looker studio documentation. <https://cloud.google.com/looker-studio>, 2026. Self-service reporting and dashboarding documentation.
- [50] Google Cloud. Professional data engineer certification. <https://cloud.google.com/learn/certification/data-engineer>, 2026. Exam guide, domains, case studies, and registration information.
- [51] Google Cloud. Pub/sub documentation. <https://cloud.google.com/pubsub/docs>, 2026. Topics, subscriptions, schemas, dead-letter topics, and exactly-once delivery.
- [52] Google Cloud. Querying cloud storage data. <https://cloud.google.com/bigquery/docs/external-data-cloud-storage>, 2026. External tables over Cloud Storage objects and their limitations.
- [53] Google Cloud. Resource hierarchy documentation. <https://cloud.google.com/resource-manager/docs/cloud-platform-resource-hierarchy>, 2026. Organization, folder, project, and resource hierarchy guidance.
- [54] Google Cloud. Sensitive data protection documentation. <https://cloud.google.com/sensitive-data-protection/docs>, 2026. InfoTypes, inspection, de-identification, tokenization, and risk analysis.
- [55] Google Cloud. Storage transfer service documentation. <https://cloud.google.com/storage-transfer/docs>, 2026. Online and scheduled data transfer guidance.
- [56] Google Cloud. Transfer appliance documentation. <https://cloud.google.com/transfer-appliance/docs>, 2026. Offline transfer appliance guidance for large data migrations.
- [57] Google Cloud. Vertex ai documentation. <https://cloud.google.com/vertex-ai/docs>, 2026. AutoML, custom training, Model Registry, endpoints, and Explainable AI.
- [58] Google Cloud. Vertex ai feature store documentation. <https://cloud.google.com/vertex-ai/docs/featurestore>, 2026. Online and offline feature serving and point-in-time correctness.
- [59] Google Cloud. Vertex ai pipelines documentation. <https://cloud.google.com/vertex-ai/docs/pipelines>, 2026. Kubeflow-based pipeline orchestration, components, and continuous training.
- [60] Google Cloud. Vpc service controls documentation. <https://cloud.google.com/vpc-service-controls/docs>, 2026. Service perimeters, access levels, and ingress and egress rules.

- [61] HashiCorp. Terraform google provider documentation. <https://registry.terraform.io/providers/hashicorp/google/latest/docs>, 2026. Resource reference for provisioning Google Cloud infrastructure as code.
- [62] Ralph Kimball and Margy Ross. *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling*. Wiley, 3 edition, 2013.
- [63] Sergey Melnik, Andrey Gubarev, Jing Jing Long, Geoffrey Romer, Shiva Shivakumar, Matt Tolton, and Theo Vassilakis. Dremel: Interactive analysis of web-scale datasets. In *Proceedings of the VLDB Endowment*, volume 3, pages 330–339, 2010.
- [64] The Apache Software Foundation. Apache airflow documentation. <https://airflow.apache.org/docs/>, 2026. DAGs, operators, sensors, XComs, and scheduling semantics.
- [65] The Apache Software Foundation. Apache beam documentation. <https://beam.apache.org/documentation/>, 2026. Beam programming guide, windowing, triggers, and state and timers.
- [66] The Apache Software Foundation. Apache iceberg documentation. <https://iceberg.apache.org/>, 2026. Open table format for analytic datasets on object storage.
- [67] The Apache Software Foundation. Apache kafka documentation. <https://kafka.apache.org/documentation/>, 2026. Kafka concepts, producers, consumers, and delivery semantics.
- [68] The Apache Software Foundation. Apache spark documentation. <https://spark.apache.org/docs/latest/>, 2026. Spark programming guide, tuning, and configuration reference.

Index

- 2-Step Verification, 4
- 99.999 percent availability, 61
- access levels, 222
- acknowledgement, 157
- AI APIs
 - equivalence, 261
- Airflow
 - DAG, 147
 - operators, 147
 - XComs, 147
- allowed lateness, 170
 - configuration, 172
- AlloyDB, 37
 - capstone, 47
 - columnar engine, 42
 - ML integration, 42
 - PostgreSQL, 41
- Always Free tier, 2
- analyst enablement, 135
- Analytics Hub, 112
 - listing, 115
 - listings, 114
- anomaly detection, 190
- Apache Airflow, 146
- Apache Beam, 163
- Apache Spark, 123
 - equivalence, 254
- app profiles, 70
- Archive, 20
- ARIMA_PLUS, 188
 - options, 190
- arrays, 91
- artifacts and lineage, 204
- assertions, 243
- AutoDQ, 226, 228
- AutoML, 195
- AutoML Tabular, 198
- AutoML vs custom training, 196
- autoscaling policies, 125
- availability, 19
- backfill, 139
- BagStateSpec, 172
- banking reconciliation, 150
- batch ELT, 152
- batching
 - API calls, 211
- Beam model, 164
- BI Engine, 96, 99
- BigLake, 112
 - capstone, 119
 - connections, 113
- BigQuery, 87
 - architecture, 88
 - clustering, 96
 - equivalence, 255
 - partitioning, 96
 - pricing, 90
 - schema enforcement, 141
- BigQuery GIS, 104
- BigQuery ML, 188
 - equivalence, 260
- BigQuery Omni, 104, 107
- BigQuery Search, 104
- bigquery sink, 134
- BigQueryInsertJobOperator, 148
- Bigtable
 - app profiles, 72, 73
 - cbt, 76
 - clusters, 72
 - equivalence, 255
 - financial services, 78
 - garbage collection, 74
 - instances, 72
 - monitoring, 74
 - nodes, 72
 - Python client, 76
 - replication, 73
 - row keys, 70
 - row-key design, 73
 - tablets, 70, 72
- billing account, 2, 266
- billing budgets, 1, 6
- billing export, 6

- block pruning, 98
- bq CLI, 270
- BQML, 188
 - model types, 189
- bucket, 19
- Bucket Lock, 21, 31
- budgets, 268
- bytes billed, 96
 - measurement, 99
- caching
 - equivalence, 254
- capstone
 - Composer, 152
- capstone project, 31, 47, 64
- case studies, 13, 288
 - EHR Healthcare, 246
 - Helium Sports, 246
- catalog search, 228
- CDAP, 131
 - concepts, 132
- CDC
 - equivalence, 258
 - log-based, 139
- certification, 287
- change data capture, 45, 138
- checklists, 275
- churn prediction, 188
- CI/CD, 241
 - equivalence, 263
- clean rooms
 - equivalence, 256
- client libraries, 267
- cloning
 - equivalence, 255
- Cloud Bigtable, 70
 - architecture, 72
 - capstone, 80
 - performance, 74
 - security, 78
- Cloud Build, 241
 - triggers, 244
- Cloud Composer, 146
- Cloud Data Fusion, 131
- Cloud Dataproc, 123
- Cloud DLP, 233
- Cloud Function
 - reverse ETL, 181
- Cloud IAM, 1, 4
- Cloud KMS, 28, 219
- Cloud Shell, 1, 5, 266
- Cloud Spanner, 53
 - architecture, 55
 - capstone, 64, 80
 - indexes, 57
 - interleaving, 57
 - multi-region, 59
 - performance, 58
 - replicas, 55
 - schema design, 56
 - security, 62
- Cloud SQL, 37
 - architecture, 38
 - capstone, 47, 80
 - high availability, 39
 - PostgreSQL, 43
 - read replicas, 40
 - security, 46
- Cloud Storage, 17
 - capstone, 80
 - verification, 9
- Cloudera migration, 128
- clustering, 98
 - equivalence, 256
- CMEK, 27, 31, 62, 78, 219
 - equivalence, 262
 - vs CSEK, 222
- CoGroupByKey, 175
- Coldline, 20
- Colossus, 87, 88
- columnar engine, 41
- community resources, 13
- compliance architecture, 238
- Composer
 - equivalence, 258
- Composer 2, 148
- conformed dimensions, 91
- connection pooling, 46
- consistency, 19
- content moderation, 213
- continuous training, 202, 204
- cost estimation, 93
- cost governance, 28, 46, 62, 78
- cost guardrails, 266, 268
- cost optimization, 101
- cost vs operational overhead, 159
- CREATE MODEL, 189
- cross-cloud analytics, 107
 - equivalence, 256
- cross-cloud equivalence, 251
- cross-service orchestration, 148
- crypto-shredding, 233
- custom training, 195

- cutover, 45
- DAG, 146
- Data Catalog, 226
- data clean room, 112, 114
 - advertising, 116
- data contracts, 138
 - breaking changes, 141
- data drift, 202
- Data Fusion
 - architecture, 132
 - equivalence, 257
- Data Fusion vs Dataflow, 135
- data gravity, 109
- data lake, 17
- data lineage, 226
 - impact analysis, 228
- data mesh, 226
 - architecture, 230
 - capstone, 241
 - three projects, 248
- data migration, 23
- data preparation, 133
- data profiling, 228
- data quality
 - equivalence, 262
- data quality rules, 228
- data scientist workflow, 199
- data sharing, 114
 - equivalence, 256
- data warehouse, 87
- database migration, 40
- Database Migration Service, 37, 40
- Databricks
 - equivalence summary, 264
- Dataflow, 163
 - autoscaling, 165
 - equivalence, 259
 - external API calls, 211
- Dataform, 241
 - CI, 244
 - SQLX, 243
- DataOps
 - equivalence, 263
- Dataplex, 28, 226
 - assets, 227
 - equivalence, 262
 - lake, 229
 - lakes, 227
 - zones, 227
- Dataproc
 - architecture, 124
 - equivalence, 257
- Dataproc Serverless, 125
- DataprocSubmitJobOperator, 148
- Datastream, 138
 - architecture, 139
 - equivalence, 258
- de-identification, 233
 - equivalence, 263
 - lab, 236
 - pipeline, 167
- dead-letter topic, 156
 - configuration, 158
- decoupled storage, 125
- defense in depth, 223
- dependency trees, 243
- deterministic encryption, 235
- dimensions, 180
- disaster recovery, 17, 27
- distributed databases
 - equivalence, 253
- DLP
 - equivalence, 263
- document modeling
 - equivalence, 253
- DoFn, 164
- domain ownership, 230
- Dremel, 87, 88
- drift vs skew, 204
- drift-triggered retraining, 206
- dry run, 99
- dual-region bucket, 20
- durability, 19
- e-commerce backend, 80
- Editions, 90
 - recommendation, 93
- egress cost, 107
- elastic compute
 - equivalence, 252
- email domain extraction, 134
- endpoints, 195
 - online vs batch, 197
- environment setup, 266
- environment sizing, 148
- ephemeral clusters, 123
 - rationale, 124
- erasure map, 238
- ERP migration, 45
 - capstone, 47
- event time, 170
- event time vs processing time, 171
- exactly-once, 170

- end-to-end, 175
- exactly-once processing, 160
- exam
 - Professional Data Engineer, 287
- exam guide, 246
- executor memory, 125
- exercises
 - solutions, 286
- exfiltration prevention, 223
- expand-contract, 241
 - four stages, 244
- Explainable AI, 195, 197
- explores, 178
 - design, 180
- exponential backoff, 159
- external consistency, 53, 55
- external table, 107
- external tables, 104
 - Cloud Storage, 106
- fact table, 91
- fail-safe, 235
- failover, 38
- failover test, 43
- failure notification, 149
- feature attributions, 197
- Feature Store, 202
 - capstone, 216
 - offline serving, 203
 - online serving, 203
- feature store
 - equivalence, 260
- federated governance, 230
- federated queries, 104
- federation
 - equivalence, 256
 - limits, 106
- financial trading, 78
- financial transactions, 61
- fine-grained security, 113
- FinOps, 93
- fixed windows, 172
- Flex Templates, 163
- Folder, 3
- foreign keys, 56
- fraud detection, 184
- free trial, 2
 - commands, 270
 - installation, 266
- gcloud config list, 9
- gcloud init, 5
- gcloud storage, 5, 23
- GCS connector, 125
- GCSObjectExistenceSensor, 149
- GDPR, 233
- generative AI
 - equivalence, 261
- GEOGRAPHY, 104
 - functions, 105
- GKE
 - Composer, 148
- global ingestion, 160
- glossary, 298
- Google Analytics sample, 191
- Google Cloud
 - free trial, 1
 - getting started, 1
- Google Cloud account, 2
- Google Cloud CLI, 1, 5
- google-cloud-storage, 24
- governance
 - checklists, 275
 - equivalence, 262
- governed BI, 182
- GPU training, 199
- grain, 91
- gsutil, 5, 23, 270
- Hadoop migration, 123
- hands-on project, 24, 43, 59, 76, 92, 99, 107, 115, 126, 134, 141, 149, 159, 166, 173, 181, 191, 198, 205, 212, 222, 229, 236, 244
- hands-on projects
 - solutions, 279
- hashing, 73
- HDFS, 123
 - migration, 125
- healthcare anonymization, 167
- healthcare security, 223
- high availability, 37
- HL7, 167
- hot spots, 58
- hotspotting, 70, 73
- IAM, 28, 62, 78, 219
 - custom roles, 221
- idempotency, 147
- identity resolution, 116

- image pipeline, 213
- in-memory cache
 - equivalence, 254
- InfoTypes, 233
 - built-in, 234
- ingress and egress rules, 220
- inspection job
 - lab, 236
- inspection jobs, 234
- interleaved schema, 59
- interleaved tables, 53, 56
- interview questions, 290
- inventory forecasting, 193
- IoT telemetry, 76
- Jupiter network, 87, 88
- key destruction, 222
- key rotation, 222
- Key Visualizer, 58, 70, 75
 - Bigtable, 74
- Kimball dimensional modeling, 91
- Kubeflow, 202
 - components, 204
- lab solutions, 279
- labels, 6, 8
- lakehouse, 112
 - capstone, 119
 - equivalence, 256
- lakes and zones, 226
- leader replica, 55
- least privilege, 4
- lifecycle JSON, 24
- likelihood, 234
- location types, 20
- logical replication, 138
- logistic regression, 188
 - churn, 191
- Looker, 178
 - architecture, 179
 - equivalence, 260
- Looker Studio, 178
- Looker Studio dashboard, 181
- Looker Studio vs Looker, 180
- LookML, 178
 - semantic layer, 179
- low-code ETL
 - equivalence, 257
- machine learning, 41
- macros, 131
 - runtime arguments, 133
- masking, 235
- materialized views, 96, 99
 - equivalence, 254
- measures, 180
- medallion architecture
 - equivalence, 252
- media asset store, 31
- media streaming, 27
- message schema, 156
- messaging
 - equivalence, 259
- metadata, 19
- metric conflicts, 182
- migration strategy, 109, 128
- Milestone 1 capstone, 80
- Milestone 2 capstone, 119
- Milestone 3 capstone, 152
- Milestone 4 capstone, 184
- Milestone 5 capstone, 216
- Milestone 6 capstone, 248
- ML.EVALUATE, 188
 - metrics, 191
- MLOps, 202
 - BQML, 193
 - capstone, 216
 - equivalence, 261
- MLOps lifecycle, 206
- mobile telemetry, 160
- mock exams, 246
- Model Monitoring, 204
- model monitoring, 202
- Model Registry, 195
 - versioning, 197
- model registry
 - equivalence, 261
- monitoring, 28
- multi-cloud, 251
- multi-cloud architecture, 109
- multi-region architecture, 27
- multi-region bucket, 20
- multi-region databases, 53
- MySQL
 - binlog, 140
- naming conventions, 8
- Natural Language API, 209
 - entities, 211
- Nearline, 20
- negative test, 222
- NewSQL, 53
- NoSQL
 - equivalence, 253

- wide-column, 70
- Object Lifecycle Management, 21
- object storage, 17
 - equivalence, 251
 - fundamentals, 19
- Object Tables, 112
 - metadata, 114
- Object Versioning, 21
- OCR, 209
 - document text, 210
- offline transfer, 23
- on-demand pricing, 90
- online prediction
 - cURL, 198
- online transfer, 23
- open table formats, 113
- operations, 28
- operators, 146
- Oracle
 - LogMiner, 140
- Oracle migration, 143
- orchestration
 - equivalence, 258
- Organization, 3
- overfitting, 191
- ParDo, 163
- partitioning
 - equivalence, 256
 - ingestion-time, 97
 - integer-range, 97
 - time-unit column, 97
- Paxos, 53, 55
- payment ledger, 61
 - capstone, 64
- PCollection, 163
- PII protection, 116
- pipeline
 - compilation, 204
- pipeline run, 205
- pipeline scheduling, 133
- plugins, 131
 - Hub, 133
- point-in-time correctness, 203
- point-in-time recovery
 - equivalence, 255
- policy tags, 112, 115
 - salary, 229
- portfolio projects, 13
- PostgreSQL, 37
 - logical replication, 140
- PowerShell, 1
- precision and recall, 191
- preemptible VMs, 123
 - trade-offs, 124
- primary key, 56
- private connectivity, 80
- private IP, 37, 40
 - Data Fusion, 132
- Private Service Connect, 219
 - endpoints, 220
- private services access, 40, 41
- production readiness, 275
- Professional Data Engineer exam, 241
- Project, 3
- project setup, 266
- PTransform, 163
- Pub/Sub, 156
 - equivalence, 259
 - subscriptions, 157
 - topics, 157
- Pub/Sub Lite, 159
- public access prevention, 28
- public datasets, 92
- publication, 141
- publisher retry, 159
- pull request checks, 244
- PySpark, 126
- Python environment, 267
- query acceleration
 - equivalence, 254
- query execution plan, 58, 96, 99
- query optimizer, 58
- question bank, 290
- quick reference, 270
- quotas, 6, 268
- RAG
 - equivalence, 261
- rate limiting, 211
- re-architecture, 101
- read replica, 38
- read replicas, 37
- real-time BI, 143
- recommendation engine, 206
- regional bucket, 20
- regional HA, 38
- regions, 8
- regions and zones, 1
- relational databases
 - equivalence, 252
- replication lag, 40

- replication slot, 141
- reservations, 90
- Resource, 3
- resource hierarchy, 3
- Retention Policies, 21
- retry policy, 158
- reverse ETL, 178
 - identity resolution, 180
- reversed timestamp, 73, 76
- right to erasure, 235
- ROC curve, 191
- rolling average, 92
- row access policy, 112, 115
- row key
 - bucketing, 74
 - hashing, 74
 - salting, 74
- runbook, 28
- runners, 164
- salting, 73
- SCD Type 2, 91
- scheduled scanning, 235
- schema drift, 138
 - handling, 141
- schema evolution, 244
- SEARCH index, 105
- searchable metadata, 213
- secondary indexes, 56
- Secret Manager, 46
- secure ML, 199
- security logs, 101
- seek, 158
- self-healing pipelines, 150
- self-service analytics, 182
- semantic layer, 178
 - equivalence, 260
- Sensitive Data Protection, 233
- sensors, 146
- sentiment analysis, 209
- sentiment pipeline, 212
- sentiment score, 211
- service accounts, 4
 - least privilege, 221
- service mapping, 251
- serving layer, 78
- session windows, 172
 - lab, 173
- setup verification, 9
- Shapley, 197
- shuffle, 165
- signed URLs, 114
- skew, 99
- SLA, 150
- sliding windows, 172
 - fraud, 184
- slots, 90
- slowly changing dimensions, 87
- snapshot, 158
- snapshot and seek, 156
- soft delete, 21
- Spark
 - managed, 257
- Spark tuning, 125
- spatial join, 105
- SQL pipelines, 193
- ST_WITHIN, 105, 107
- Standard, 20
- star schema, 87
- state
 - remote backend, 242
- state and timers, 170
- state TTL, 172
- stock ticks, 78
- Storage Classes, 20
- storage classes
 - equivalence, 251
- Storage Transfer Service, 17
 - capstone, 31
- stream processing
 - equivalence, 259
- stream-stream join, 175
- Streaming Engine, 165
- strong consistency, 55
- structs, 91
- study strategy, 287
- subscriptions, 156
- synthetic data, 59
- tag templates, 226
 - fields, 228
- templates
 - classic, 166
 - flex, 166
- Terraform, 241
 - equivalence, 263
 - google provider, 242
 - setup, 267
- TestStream, 173
- third-party data, 238
- time series, 190
- time travel, 235
 - equivalence, 255
- tokenization, 233

- FFX, [235](#)
- tool requirements, [269](#)
- tool selection, [135](#)
- topic schema, [157](#)
- topics, [156](#)
- traffic splitting, [197](#)
- Transfer Appliance, [17](#)
- Translation API, [209](#), [211](#)
- triggers, [170](#)
 - early-on-time-late, [172](#)
- TrueTime, [53](#), [55](#)
 - external consistency, [56](#)
- uniform bucket-level access, [28](#)
- unit testing
 - Beam, [166](#)
- UNNEST, [92](#)
- unstructured data, [114](#)
- Vertex AI, [195](#)
 - ecosystem, [196](#)
 - equivalence, [261](#)
- Vertex AI Pipelines, [202](#)
 - lab, [205](#)
- Video Intelligence, [209](#)
 - features, [210](#)
- virtual warehouse
 - equivalence, [252](#)
- Vision API, [209](#)
 - features, [210](#)
- visual ETL, [131](#)
- VPC, [41](#)
- VPC SC
 - capstone, [248](#)
 - perimeters, [220](#)
- VPC Service Controls, [78](#), [219](#)
 - equivalence, [262](#)
- warehouse
 - equivalence, [255](#)
- watermark
 - estimate, [171](#)
- watermark lag, [175](#)
- watermarks, [170](#)
 - equivalence, [259](#)
- wide-column
 - equivalence, [255](#)
- wide-column NoSQL, [70](#)
- window functions, [91](#)
- windowing, [170](#)
- WordCount, [166](#)
- workflow template, [126](#)
- workload assessment, [128](#)
- Workload Identity, [219](#)
 - federation, [221](#)
- workspaces, [242](#)
- Wrangler, [131](#)
 - directives, [133](#)
- XComs, [146](#)
- zero data loss, [61](#), [64](#)
- zero-downtime migration, [45](#)
- zero-downtime replication, [143](#)
- zones, [8](#)